

VHDL

Very High Speed Integrated Circuit Hardware Description Language

RTL – Register Transfer Level Design

für synthesefähige Digitalschaltungen

Edmund Coersmeier

Hochschule Bochum

Version 0.0063

Wintersemester 2025/2026

Themen der Vorlesung

- Kapitel 1 Einleitung
- Kapitel 2 ASIC und FPGA Bausteine, Xilinx Vivado
- Kapitel 3 reservierte Wörter in VHDL, Entity, Architecture, Datentypen, Variablen, Signale, Packages
- Kapitel 4 Kontrollstrukturen, Operatoren, Funktionen, Prozeduren, Generics, Generate
- Kapitel 5 RTL – Register Transfer Level - Synchrone und asynchrone Prozesse, Komponenten
- Kapitel 6 Testbench, Simulation und Synthese
- Kapitel 7 Zähler, Synchronisationsschaltungen, Endliche Automaten
- Kapitel 8 CORDIC
- Kapitel 9 FIR und IIR Filter
- Kapitel 10 Künstliches Neuronales Netz
- Kapitel 11 Stromeffizienter VHDL Code
- Kapitel 12 UART
- Kapitel 13 ADC
- Kapitel 14 Frequenzmessung eines analogen, periodischen Signals

Kapitel 1

Einleitung

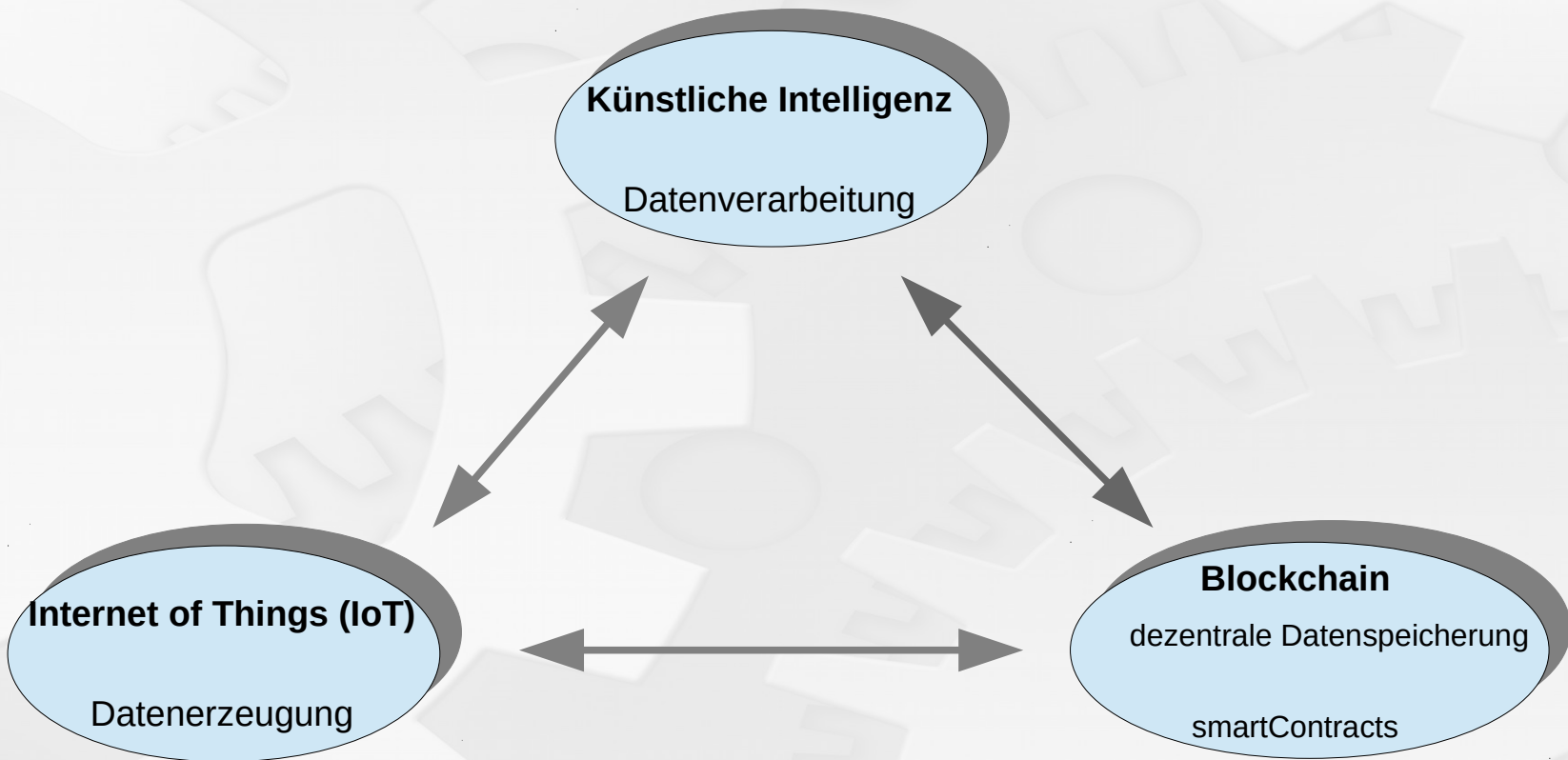
```
1  contract AWallet{
2      address owner;
3      mapping (address => uint) public outflow;
4
5      function AWallet(){ owner = msg.sender; }
```

Folien der Einleitung sind ab dem Jahr 2015 entstanden

und beziehen sich derzeit auf die Prognose bis zum Jahr 2030

```
7      function pay(uint amount, address recipient) returns (bool){
8          if (msg.sender != owner || msg.value != 0) throw;
9          if (amount > this.balance) return false;
10         outflow[recipient] += amount;
11         if (!recipient.send(amount)) throw;
12         return true;
13     }
14 }
```

Technologiedreieck der 2020iger



Bildmaterial auf dieser Seite:
© Gerd Altmann / pixelio.de

Copyright © Edmund Coersmeier 2008 - 2025

Mobile Phone

20 Mansfield News-Journal Thursday, April 18, 1963

You'll Be Able To Carry Phone In Pocket In Future

Some day, Mansfielders will carry their telephones in their pockets.

Don't expect it to be available tomorrow, though.

Frederick Huntsman, telephone company commercial manager, says, "This telephone is far in the future — commercially."

★

Right now, it's a laboratory development and it's workable, allowing the carrier to make and answer calls wherever he may be.

Other telephones of the future includes a kitchen loud speaking telephone, and a visual image telephone.

The kitchen instrument can be used as a regular telephone, a loudspeaking phone if the housewife happens to be busy preparing a meal, or as an intercom station for the home.

The visual image telephone allows the parties to converse by way of a microphone and loud speaker while a miniature television camera transmits the image. The "TV phone" also will have a writer signature transmission system and a conversation tape recorder.

The new phones are being displayed at the Home and Flower Show at the Coliseum.

HOW ABOUT THIS? — Mrs. Jean Conrad, commercial representative of Mansfield Telephone Co. holds up the pocket - sized, wireless telephone which Mansfielders will some day carry with them. The phone is still in the development stage and "far in the future."

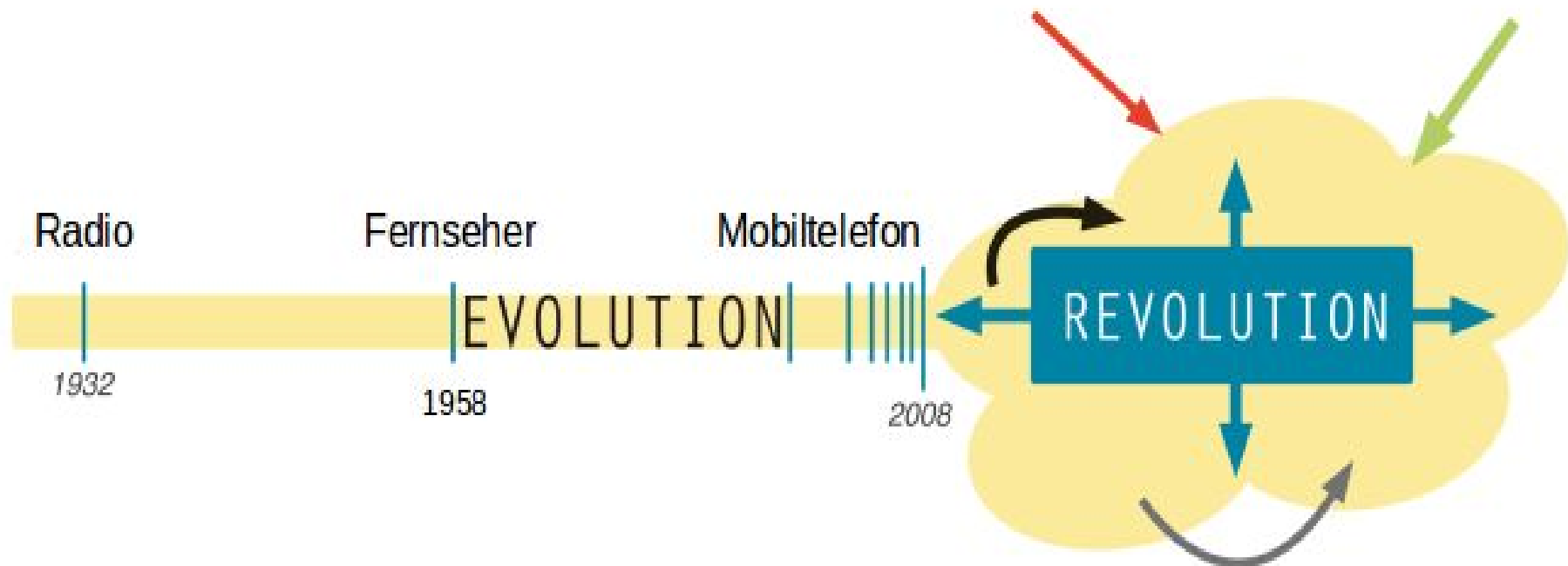


Price The sheriff ap...

\$200 M.H.

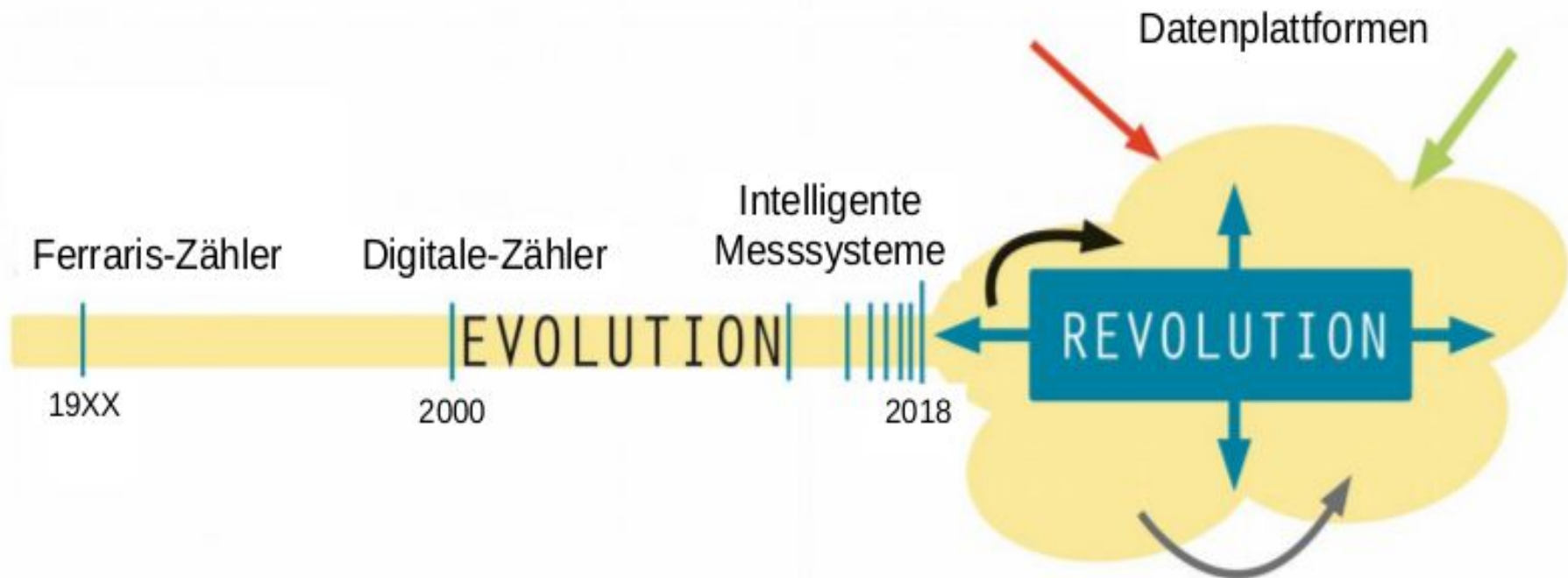
Von der Evolution zur Revolution

- Evolutionäre Schritte werden in der Zukunft durch revolutionäre Technologiekombinationen abgelöst
- Diese Entwicklung überrascht immer wieder ganze Branchen



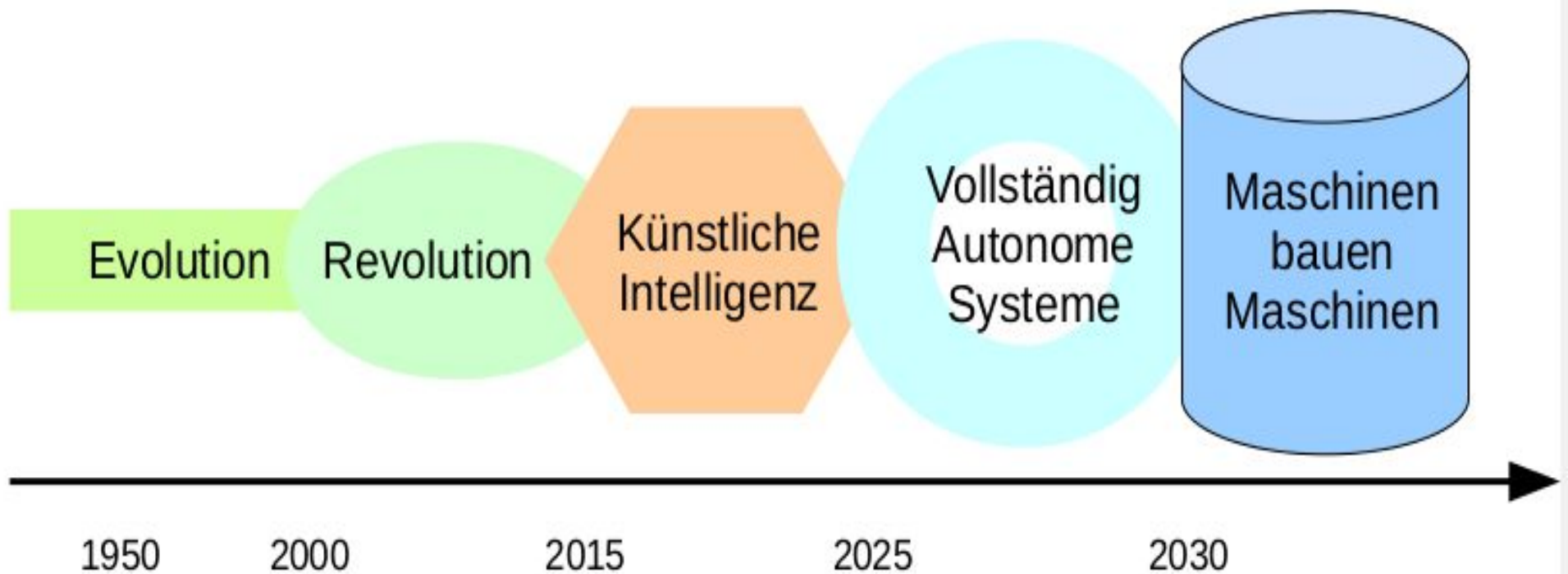
Von der Evolution zur Revolution: Energie

- Evolutionäre Schritte werden in der Zukunft durch revolutionäre Technologiekombinationen abgelöst



Evolution, Revolution, KI, Autonome Systeme, ...

- Die zeitlichen Abstände zwischen den einzelnen Themen werden immer kürzer



Industrielle Revolutionen – Industrie 4.0

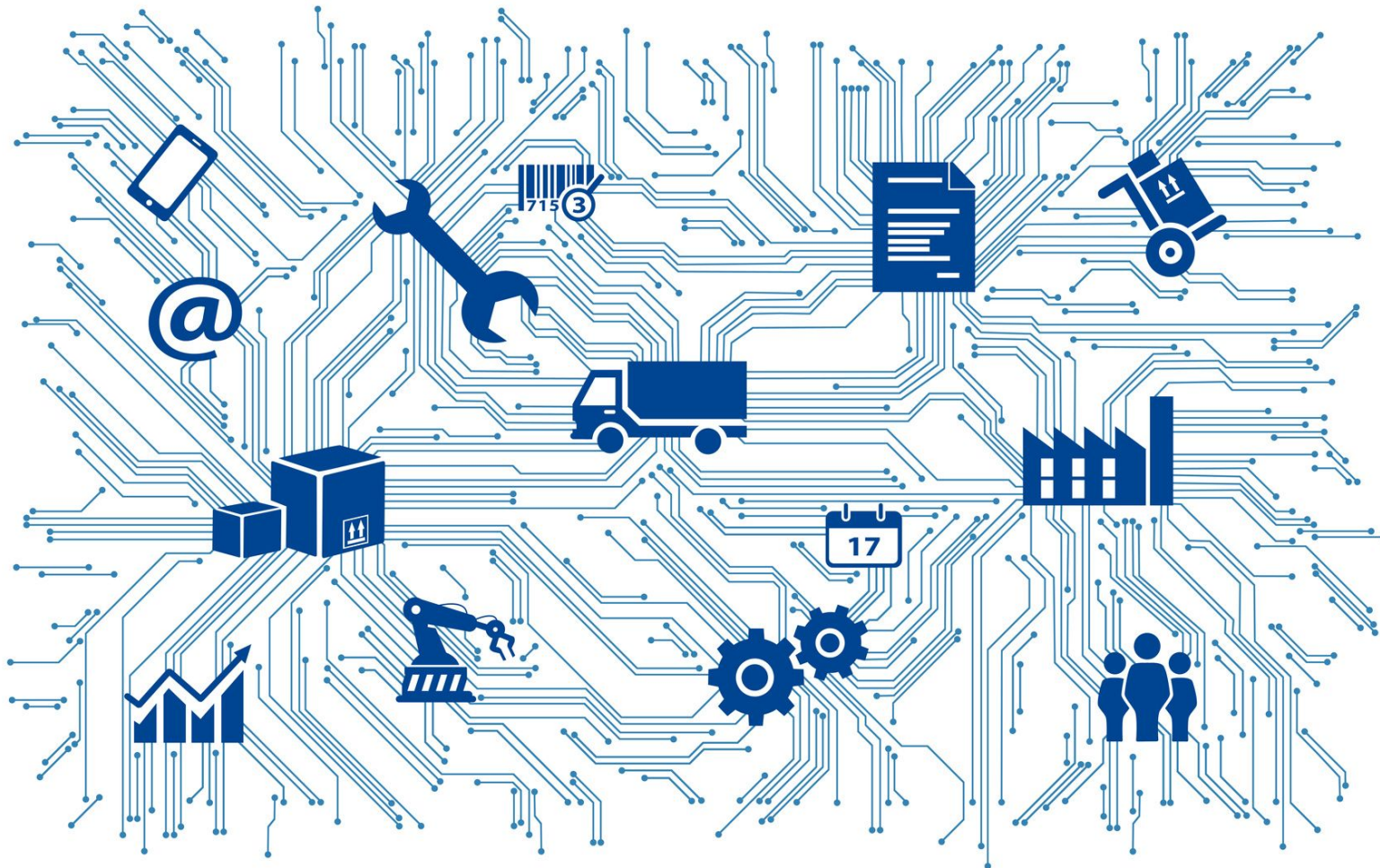
- Automatisierung von händischen Prozessen
 - 1. Industrielle Revolution Dampfmaschine
 - ca. 1712 Thomas Newcomer
 - ca. 1769 James Watt Verbesserung
 - Patent vom 05.01.1769 -> 7 Jahre, um verkaufsfähiges Produkt aus der Verbesserung abzuleiten (und viel Geld)
 - 2. Industrielle Revolution ca. 1900 Elektr. Strom, Fließband
 - 3. Industrielle Revolution ca. 1970 Mikroelektronik
 - 4. Industrielle Revolution ca. 2015 Künstliche Intelligenz, IoT, Blockchain
- Bisher: Zunahme Wohlstand, Gesundheitssysteme und Bildung
- Heute: Mehr persönliche Zeit /Grundsicherung
- Grund: Digitalisierung, IoT, Künstliche Intelligenz, Blockchain

Industrie 4.0



Was ist IoT

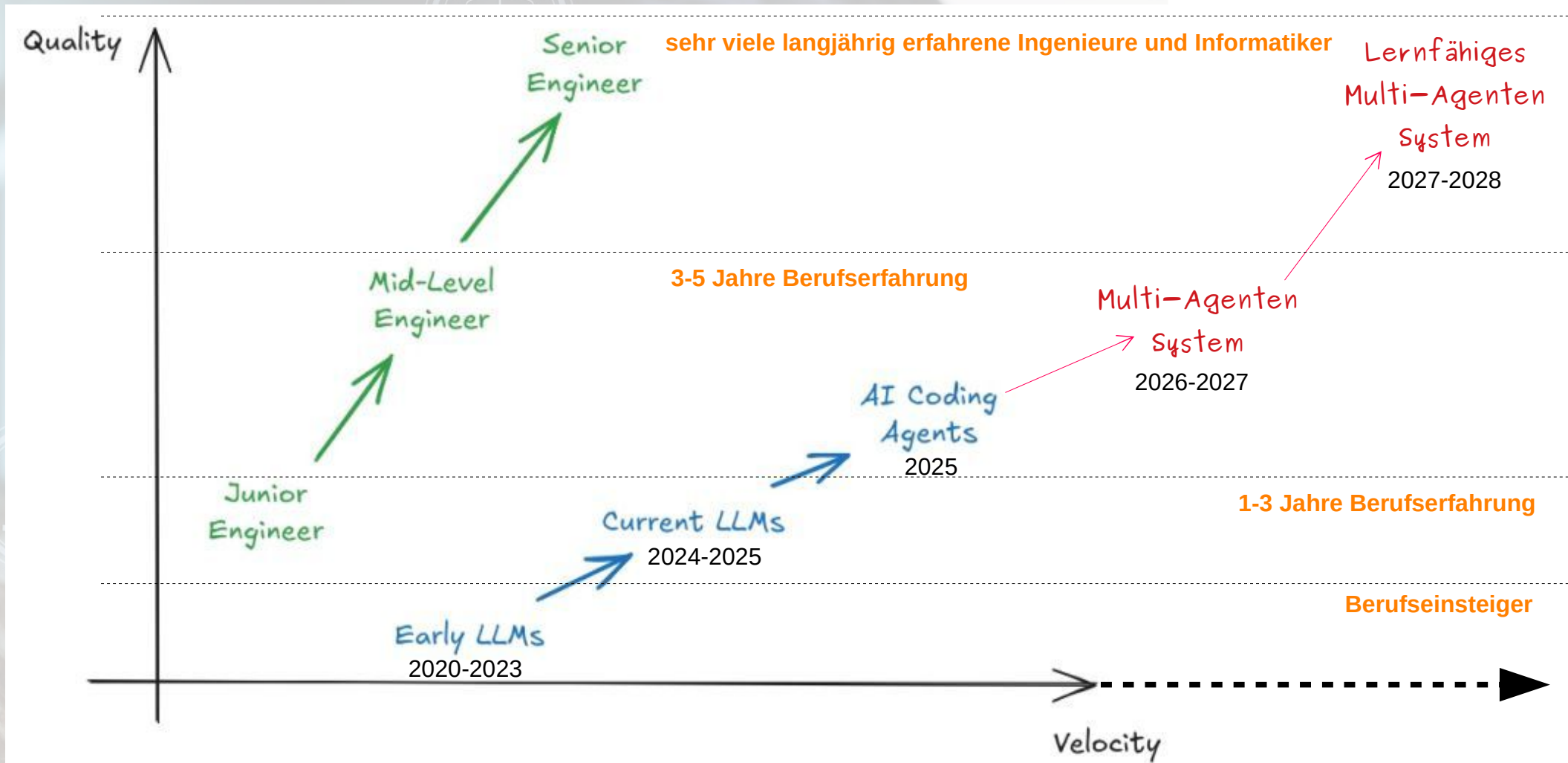
- Internet of Things
 - Alle Geräte werden miteinander vernetzt
 - Sensoren, Aktoren, Recheneinheiten, Datenspeicher, ...



Was wird in den nächsten 15 Jahren passieren?

- potentielle Prognose für die Jahre 2015 – 2030
- Große Mengen an gut und sehr gut ausgebildeten Akademikern / Berufsfeldern werden freigesetzt
 - Juristen, Politiker
 - Professoren, Banker
 - Manager, Programmierer
 - Zeichner, Autoren, ...
- die Industriellen Revolutionen 1 und 2 haben eher Handarbeitsjobs wegrationalisiert
- die Industriellen Revolutionen 3 und 4 verdrängen eher Akademikerjobs
- Folge: viel Zeit für soziales Engagement, Sport, Gesellschaft, ... falls man den technologischen Anschluss verpasst → gegebenenfalls Grundeinkommen
- (Sehr) viele der verwendeten Technologien beruhen heute und in Zukunft auf der Sprache Python
 - Python wird schnell für die finale Produktvariante durch komplexere, dann effizientere Implementierungssprachen abgelöst → komplexere Programmiersprachen sind wieder im Kommen, kombiniert mit anspruchsvollen Hardwareeinheiten

Nicht den technologischen Anschluss verpassen



~~4-Tage-Woche~~

~~5-Tage-Woche~~

6-Tage-Woche ✓

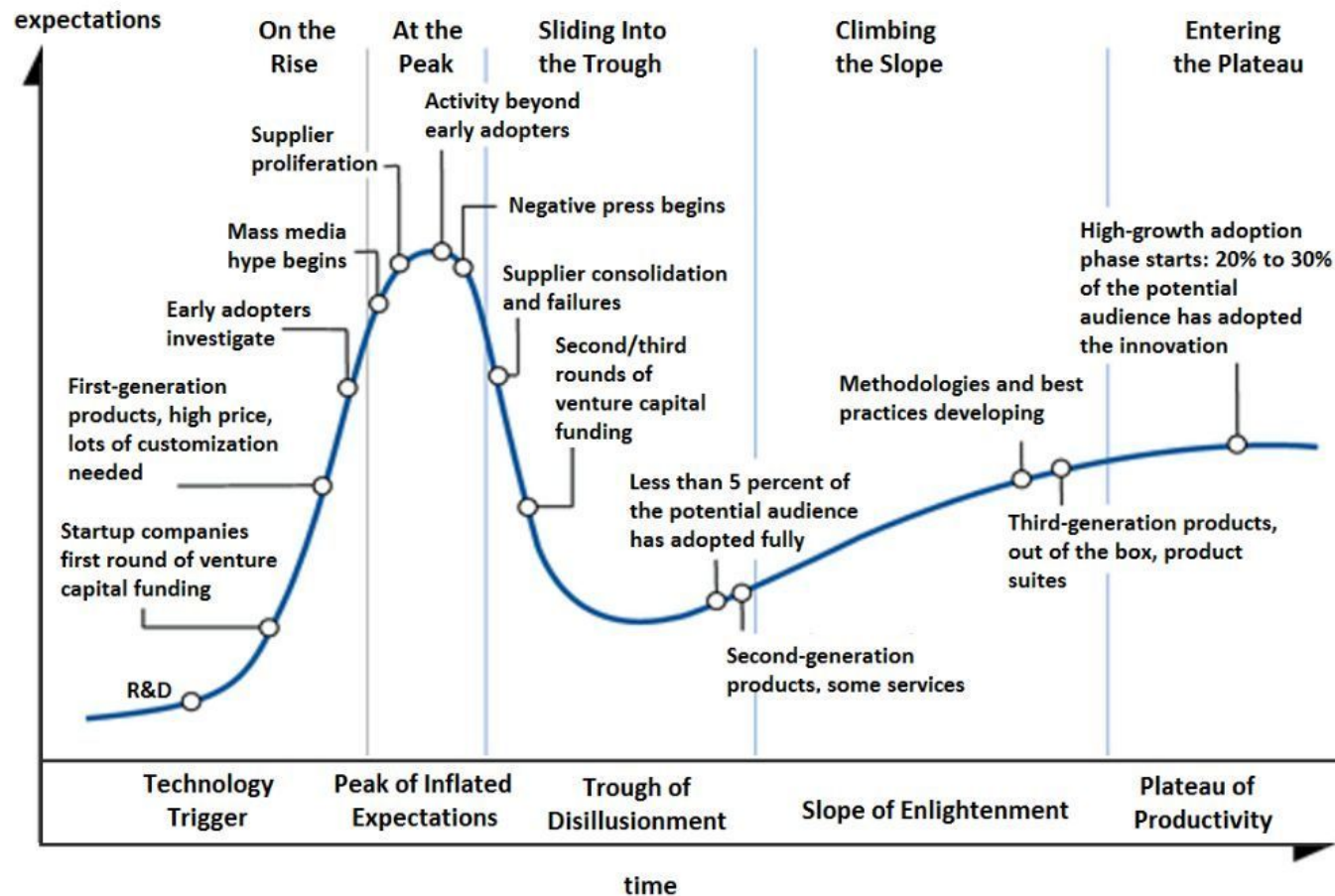
7-Tage-Woche ✓

FROM COMFORT ZONE TO GROWTH



@THEPRESENTPSYCHOLOGIST

The Emotional Cost Of Being A Startup Founder



Source: Gartner Hype Curve

Kapitel 2

ASIC und FPGA Bausteine

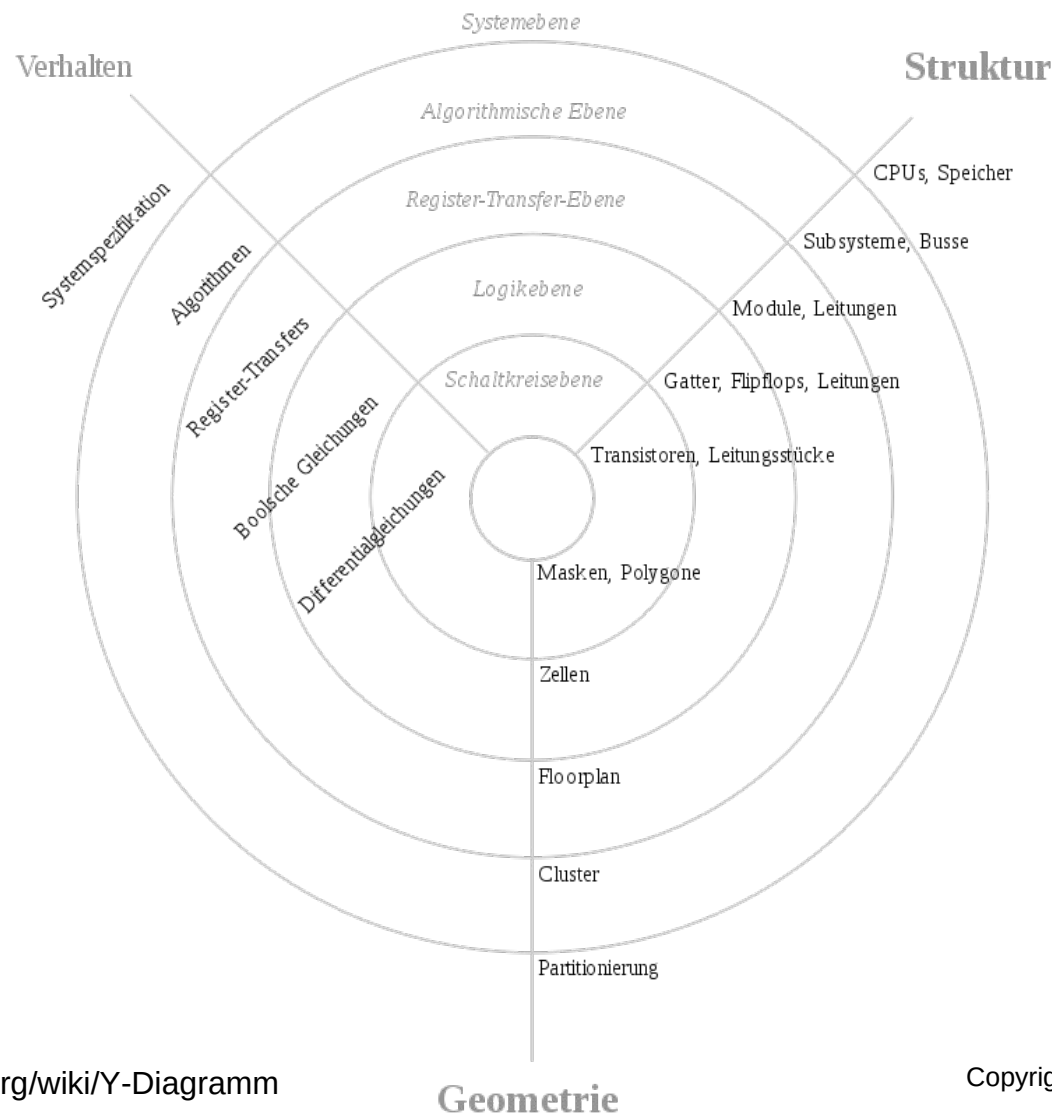
Digital Basics

Sequentielle und nebenläufige Logik

Xilinx Vivado

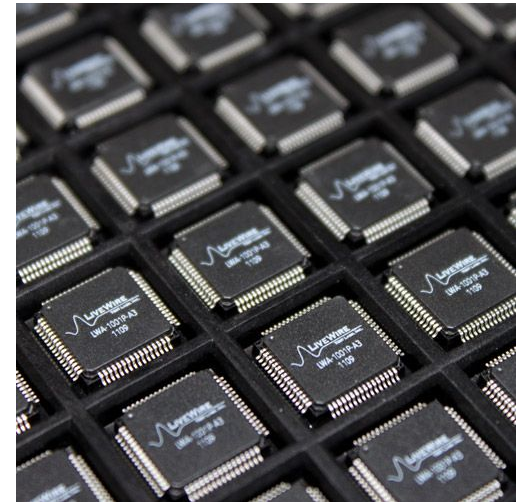
Y Diagramm

- Das Y-Diagramm oder Gajski-Diagramm beschreibt die Sichtweisen des Hardwareentwurfs
- 1983 entwickelten Daniel D. Gajski und Robert Kuhn das Entwurfsmodell
- 1985 wurde es von Robert Walker und Donald Thomas verfeinert.



ASIC Baustein

- Ein ASIC (Application Specific Integrated Circuit) umfasst eine vollständige Eigenimplementierung von Hardware Logikgattern als Integrierten Schaltkreis
- Es können selbst entworfene Funktionen und alternativ / zusätzlich IP (Intellectual Property) Blöcke von Drittanbietern integriert werden
- Ein ASIC Design ist sehr kostspielig, da alle Produktionsmasken nur für diesen Chiptyp erzeugt werden
 - Designern fehlt häufig das Geld und das Prozess-Design-Kit
- Google möchte das Problem gerade als Open Source Projekt lösen
 - <https://t3n.de/news/google-produziert-individuelle-1297831/>
 - <https://github.com/google/skywater-pdk>
 - Herstellung nur für Testchips, nicht für die kommerzielle Nutzung gedacht



Quelle: https://upload.wikimedia.org/wikipedia/commons/7/79/SSDTR-ASIC_technology.jpg

FPGA Baustein

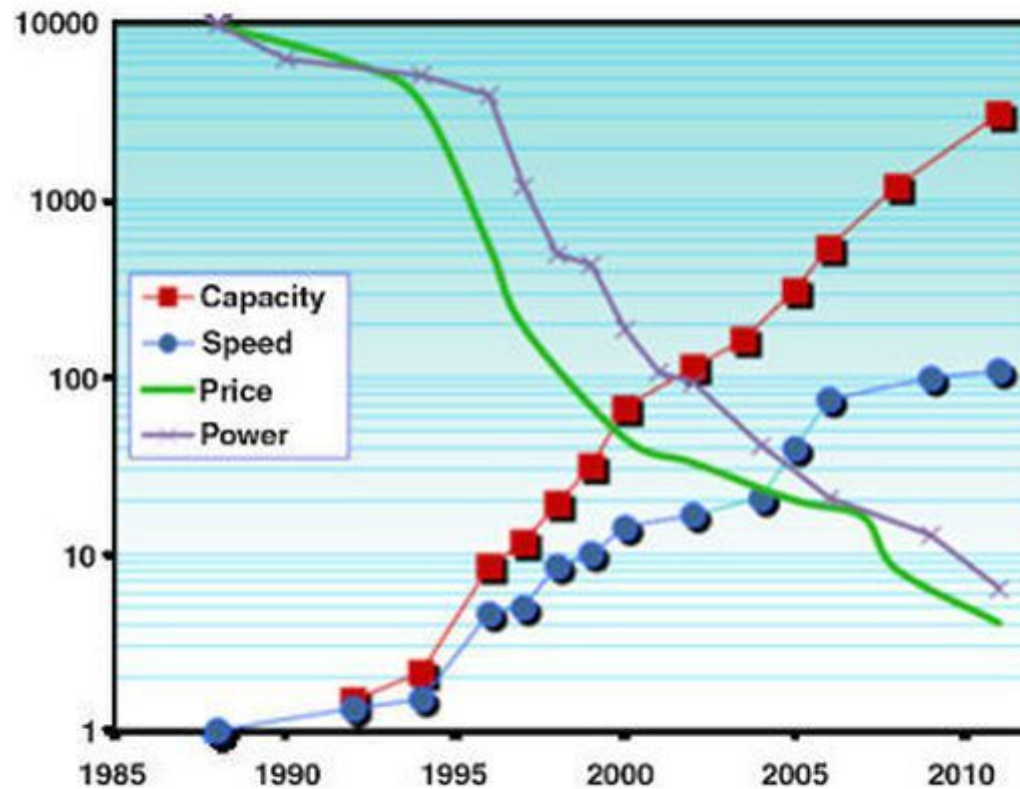
- Ein FPGA ist ein konfigurierbarer Hardwarechip, der verschiedene vorgefertigte Logikgatter zur Verfügung stellt
- Xilinx' erster FPGA-Baustein, der XC2064, bestand aus 64 Logikblöcken
- Jeder Logikblock umfasste zwei Look-up-Tabellen (LUTs) mit drei Eingängen und einem Register
- Nach heutiger Zählweise wären das etwa 64 Logikzellen – weniger als 1000 Gatter
- Trotz dieser bescheidenen Kapazität war der Chip des XC2064 geradezu riesig – größer als ein damaliger Mikroprozessor – und die 2,5-µm-Prozesstechnologie war kaum in der Lage, den Baustein zu produzieren



Quelle: <https://www.elektronikpraxis.vogel.de/die-fuenf-zeitalter-der-fpga-entwicklung-a-541059/>

FPGA Entwicklung

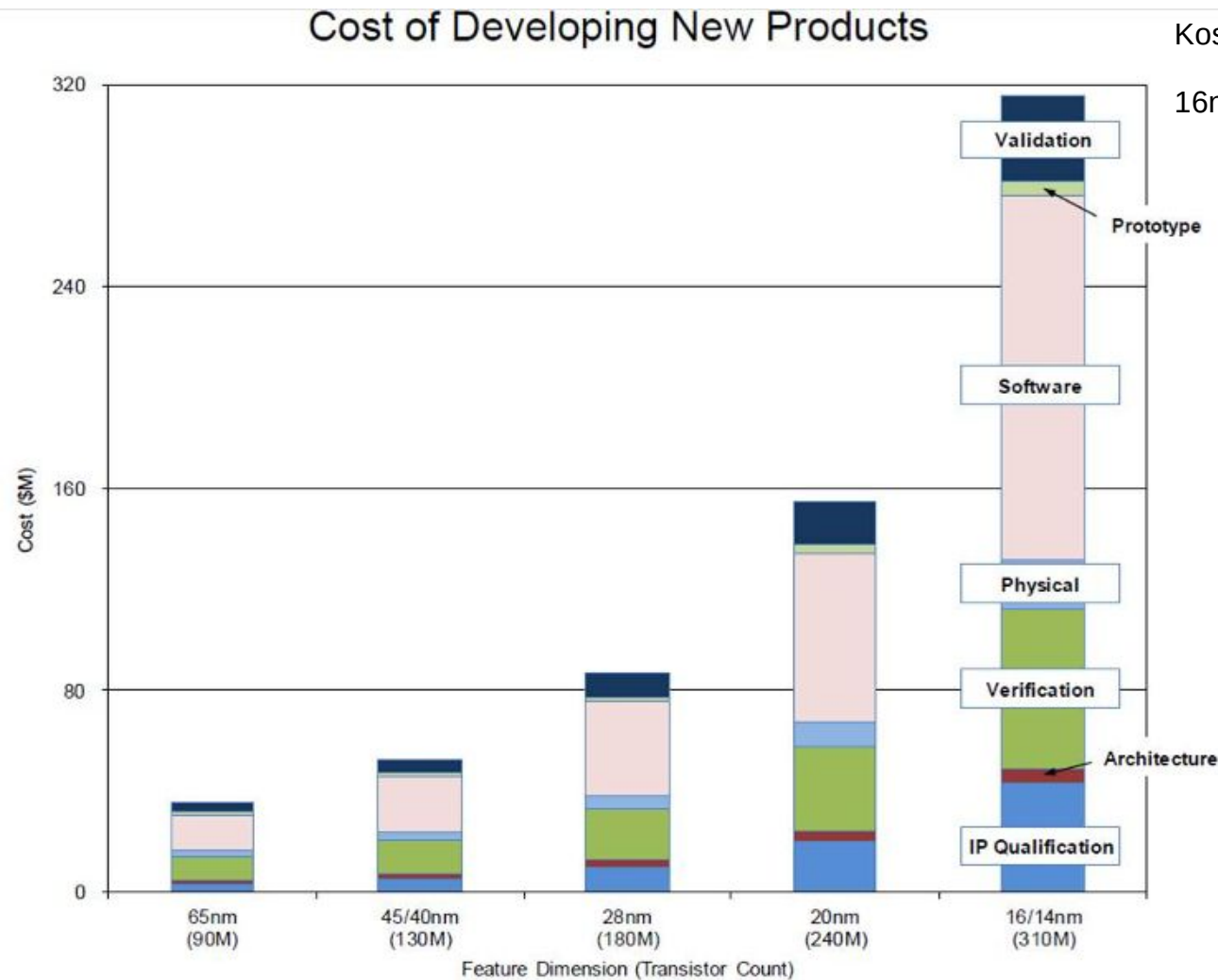
- Start Mitte der 1980iger



Quelle: <https://www.elektronikpraxis.vogel.de/die-fuenf-zeitalter-der-fpga-entwicklung-a-541059/>

ASIC Entwicklungskosten

- Kostenschätzung zur Herstellung eines eigenen Chips aus dem Jahr 2014



Kosten verringern sich kontinuierlich

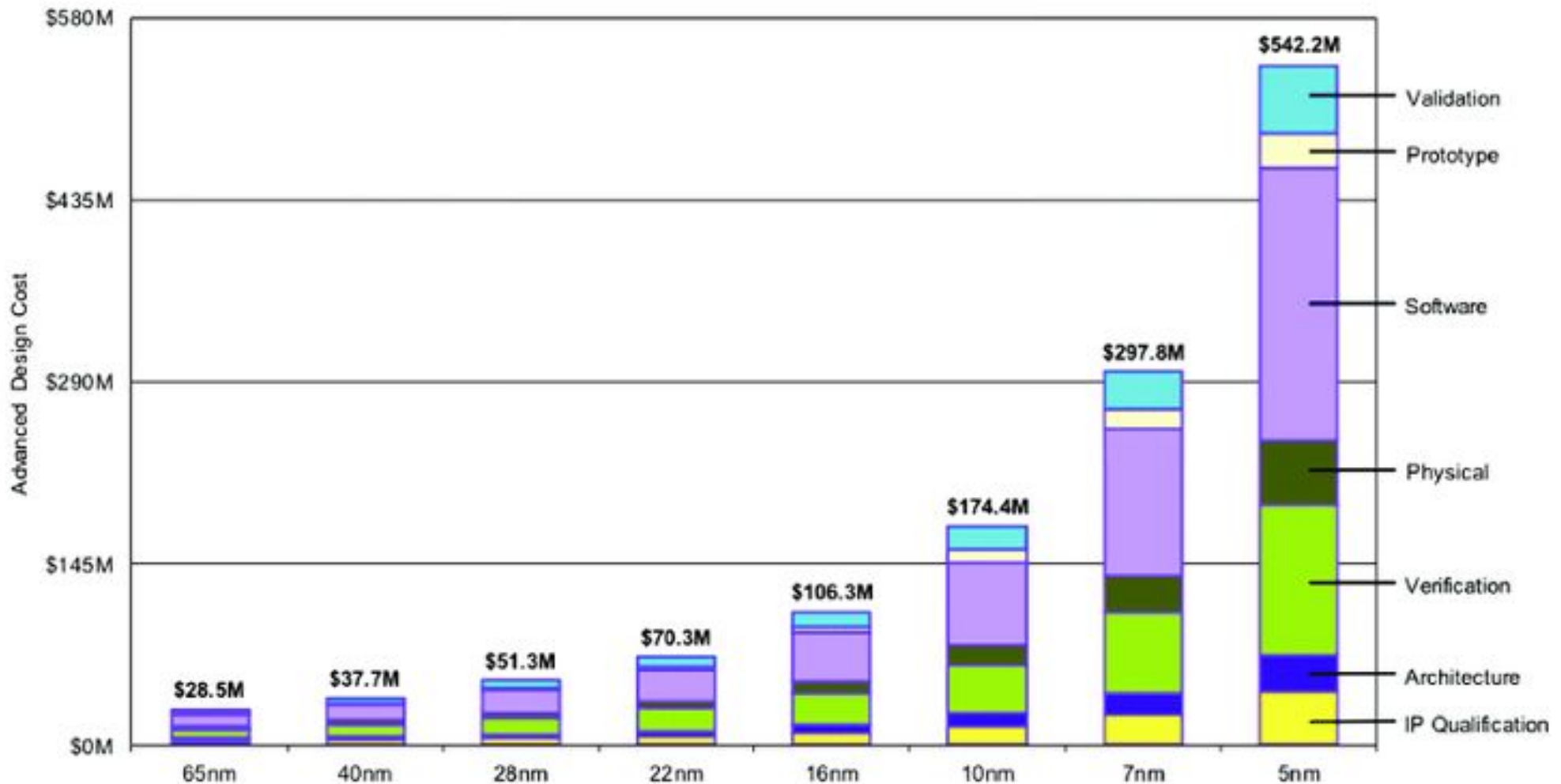
16nm/14nm (2014): \$310M → \$106M (2018)

28Nm (2014): \$85M → \$51M (2018)

Quelle: <https://semiengineering.com/what-will-that-chip-cost/>

ASIC Entwicklungskosten

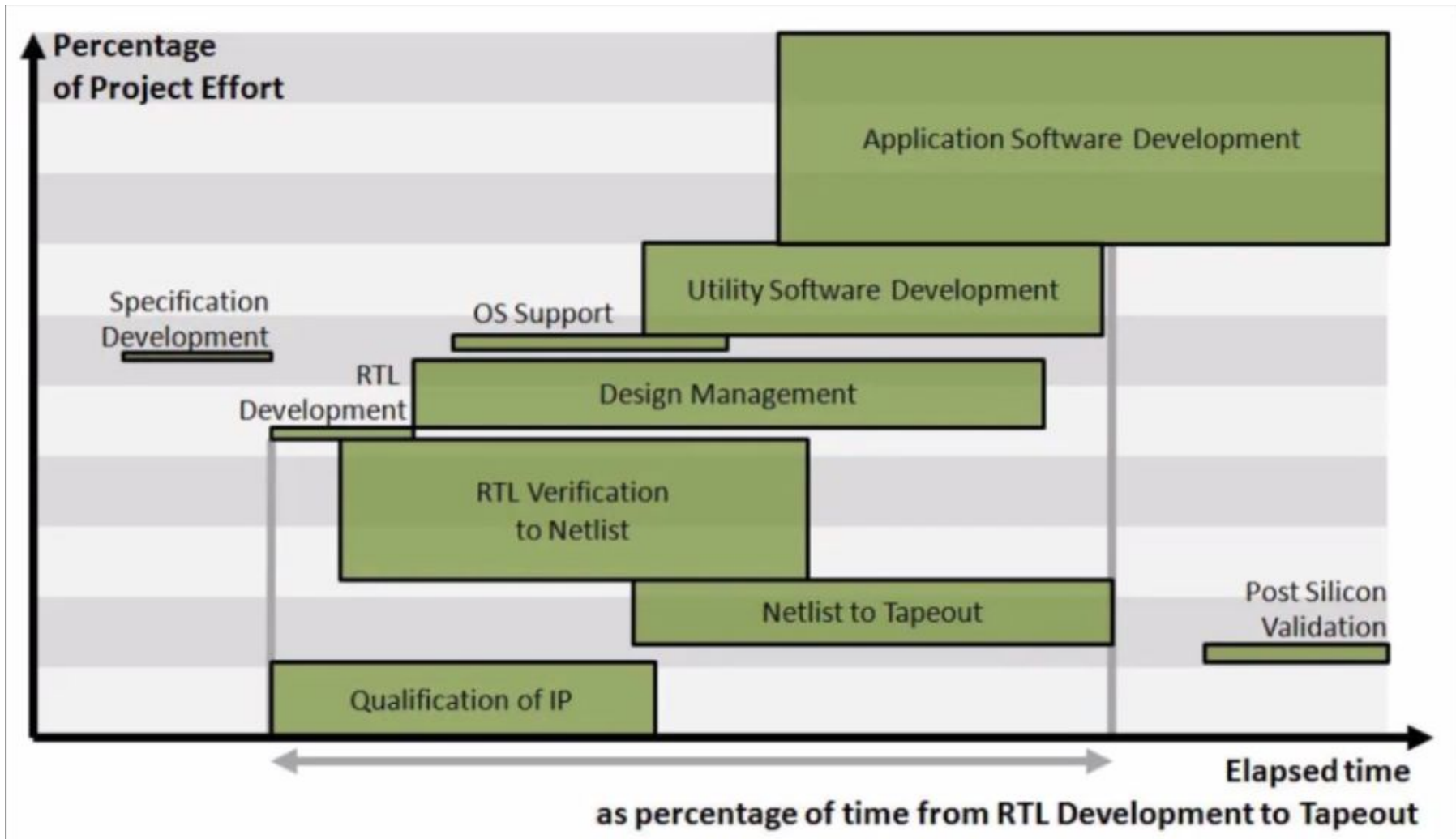
- Kostenschätzung zur Herstellung eines eigenen Chips aus dem Jahr 2018



Quelle: <https://semiengineering.com/what-will-that-chip-cost/>

ASIC Entwicklungsaufwand

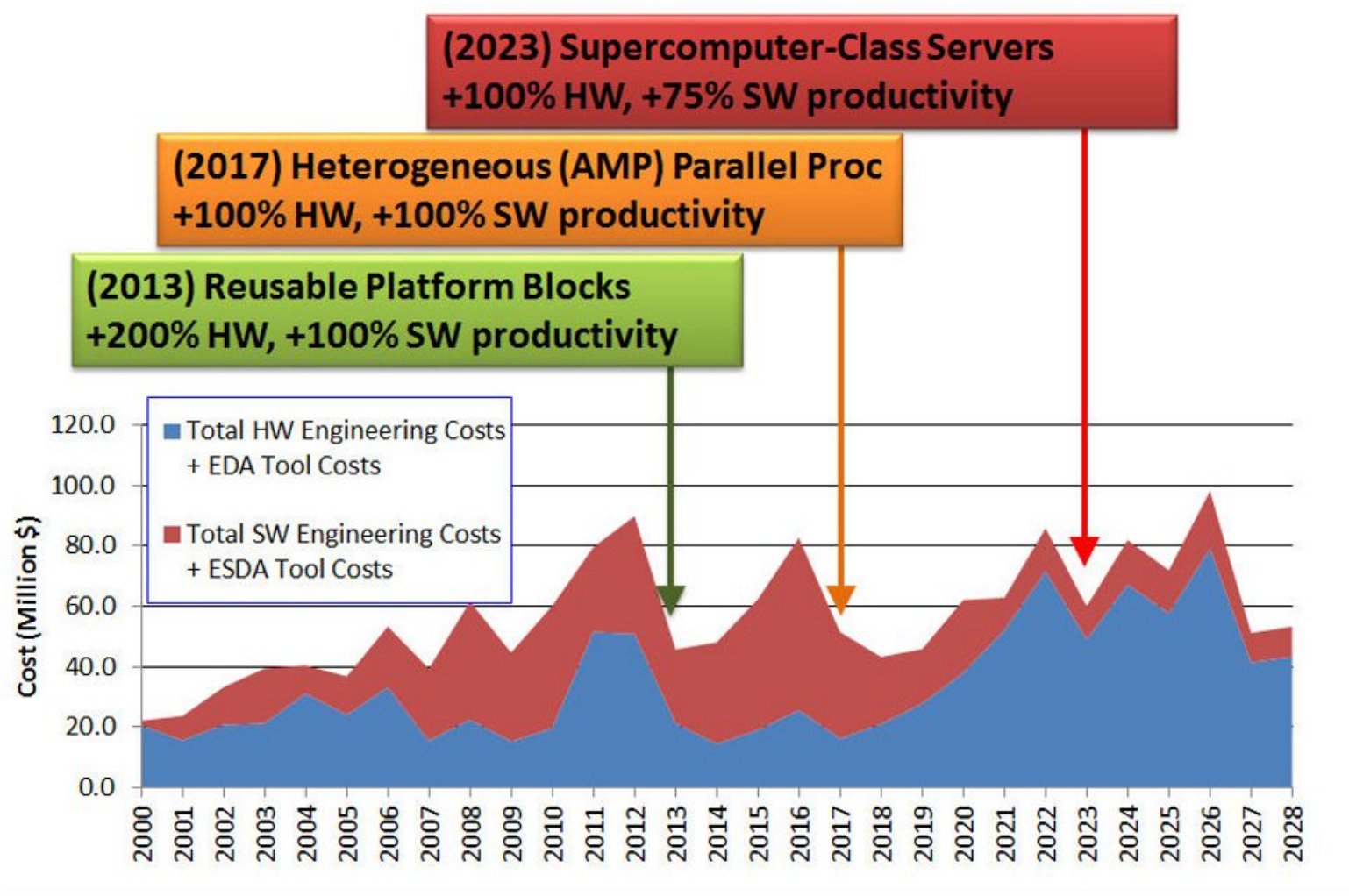
- Neben dem RTL Design und Verification bildet die Software den Hauptaufwand



Quelle: <https://semiengineering.com/what-will-that-chip-cost/>

ASIC SW vs. HW Entwicklungsaufwand

- Softwarekosten werden sinken, während die HW-Designs deutlich komplexer werden



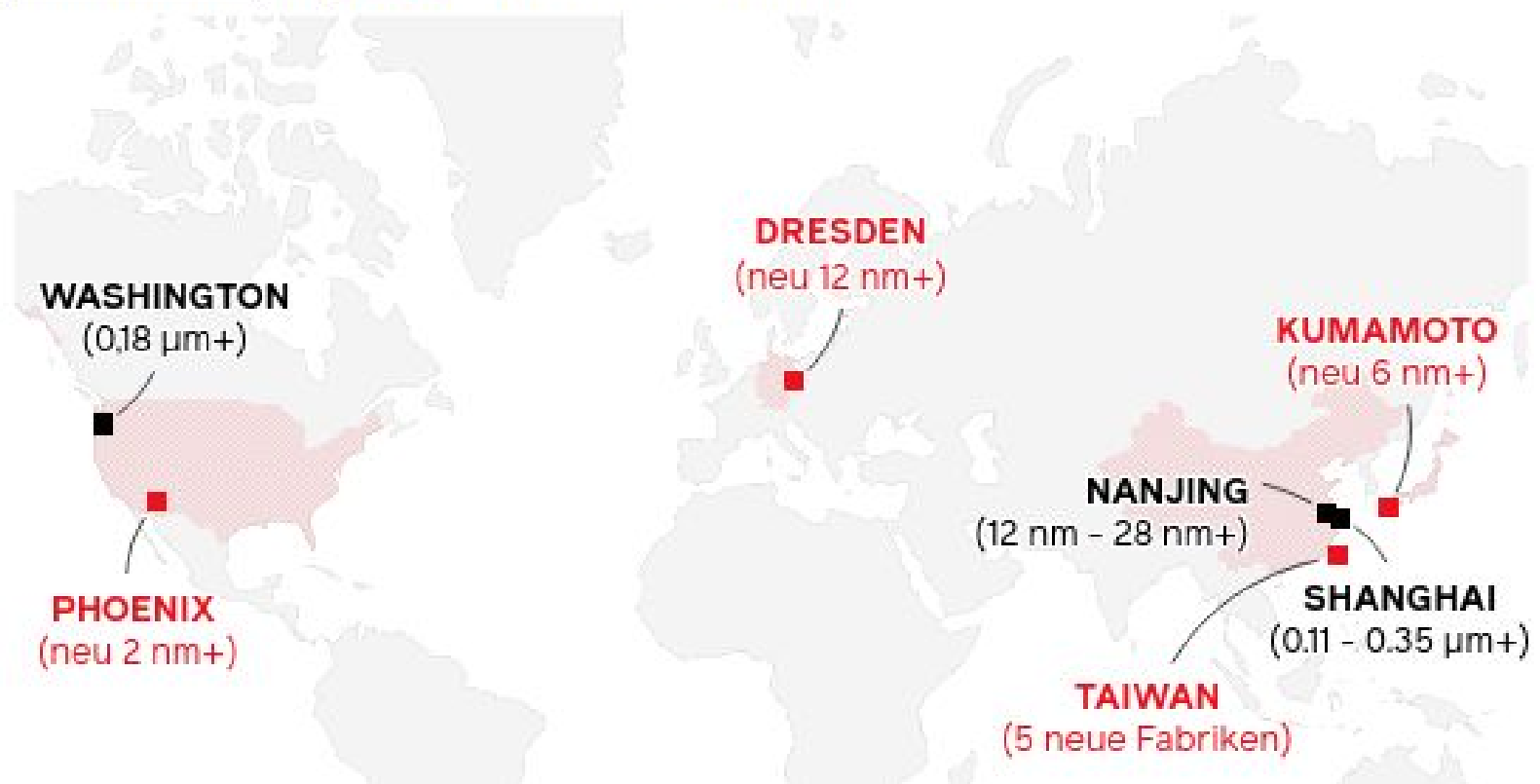
Quelle: <https://semiengineering.com/what-will-that-chip-cost/>

Chip Produktion von TSMC 2024++

- In Dresden wird „nur“ mit 12nm gefertigt → neueste TSMC Fabriken in Taiwan mit 2nm

CHIPPRODUKTION VON TSMC

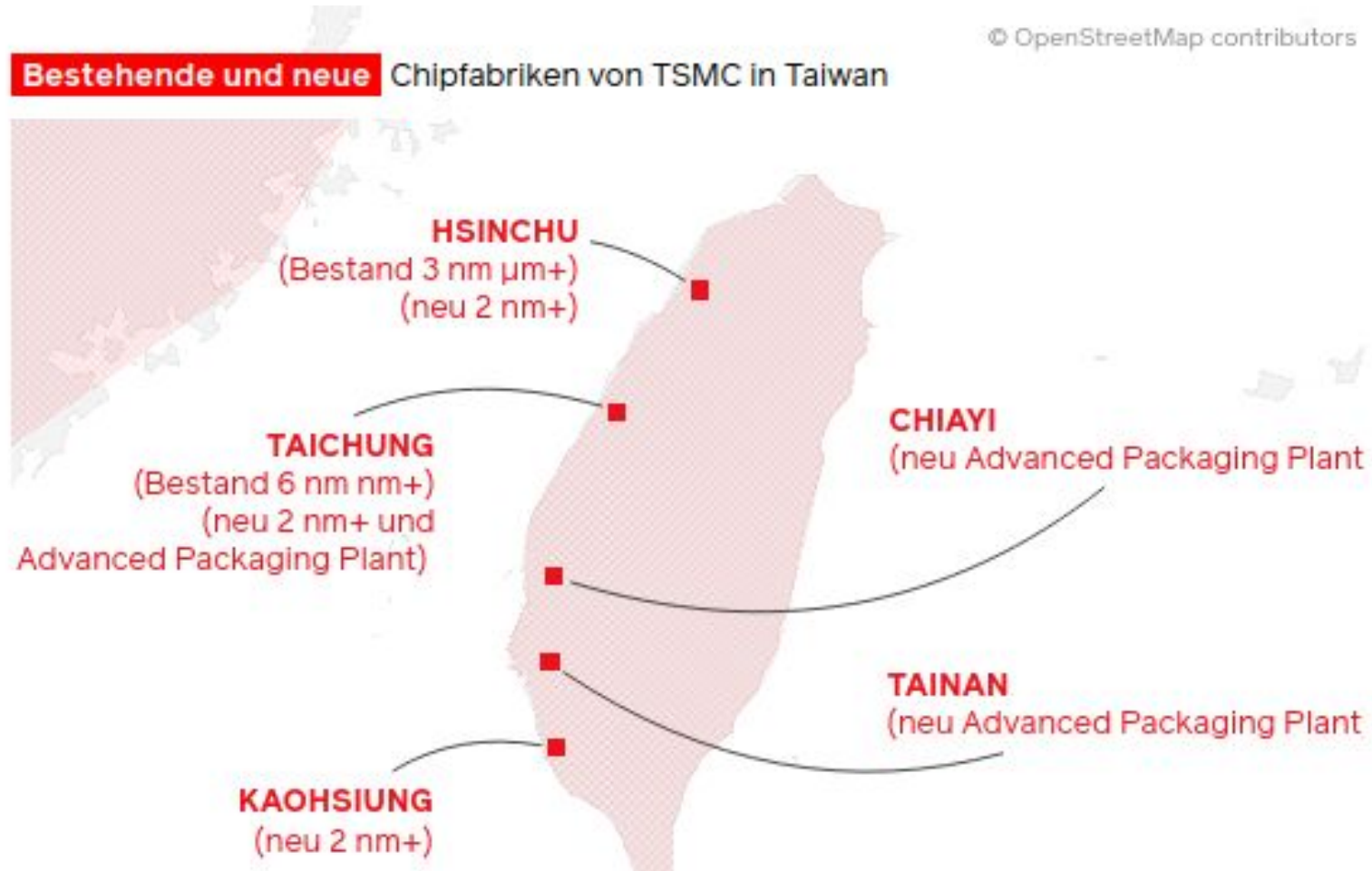
Bestehende und **neue** Chipfabriken von TSMC



Quelle: <https://www.wiwo.de/technologie/wirtschaft-von-oben/wirtschaft-von-oben-282-chipwerke-in-taiwan-hier-baut-taiwan-seinen-wirtschaftlichen-schutzwall-gegen-china-aus/30002302.html>

Chip Produktion von TSMC 2024++

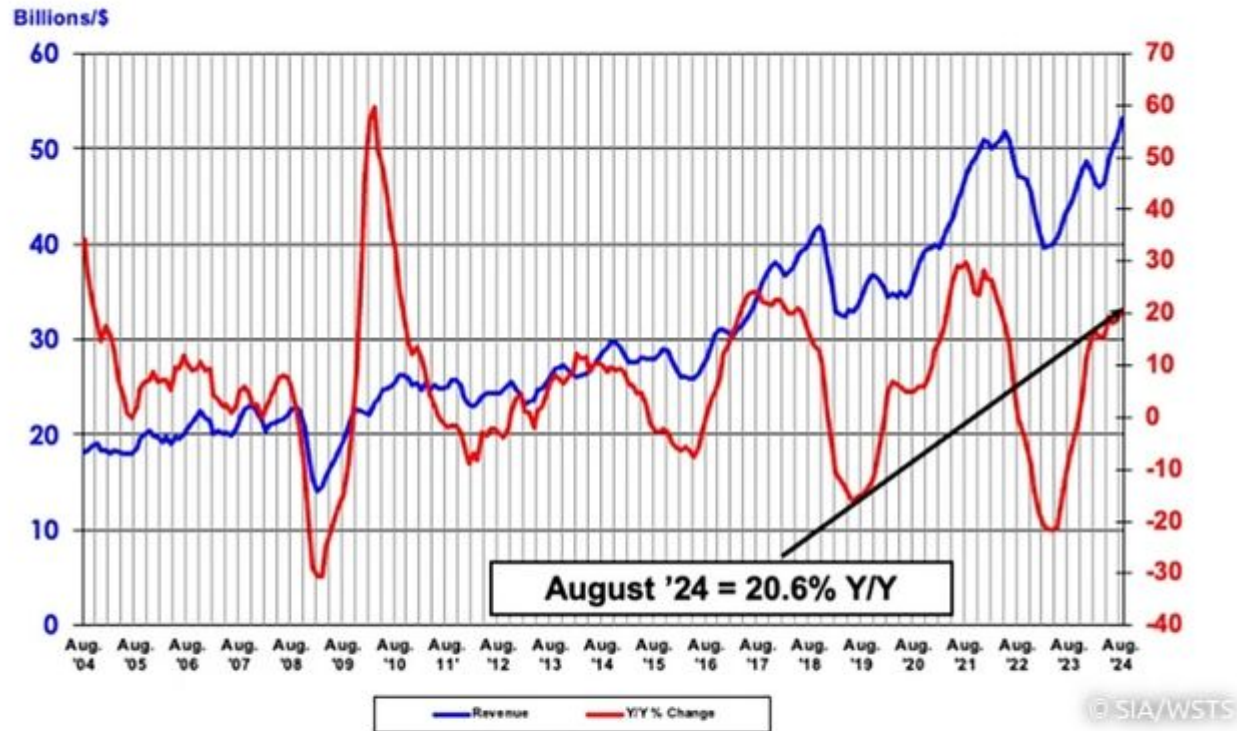
- In Dresden wird „nur“ mit 12nm gefertigt → neueste TSMC Fabriken in Taiwan mit 2nm



Quelle: <https://www.wiwo.de/technologie/wirtschaft-von-oben/wirtschaft-von-oben-282-chipwerke-in-taiwan-hier-baut-taiwan-seinen-wirtschaftlichen-schutzwall-gegen-china-aus/30002302.html>

Chip Umsatz August 2024

7. Oktober 2024, 7:36 Uhr | Heinz Arnold



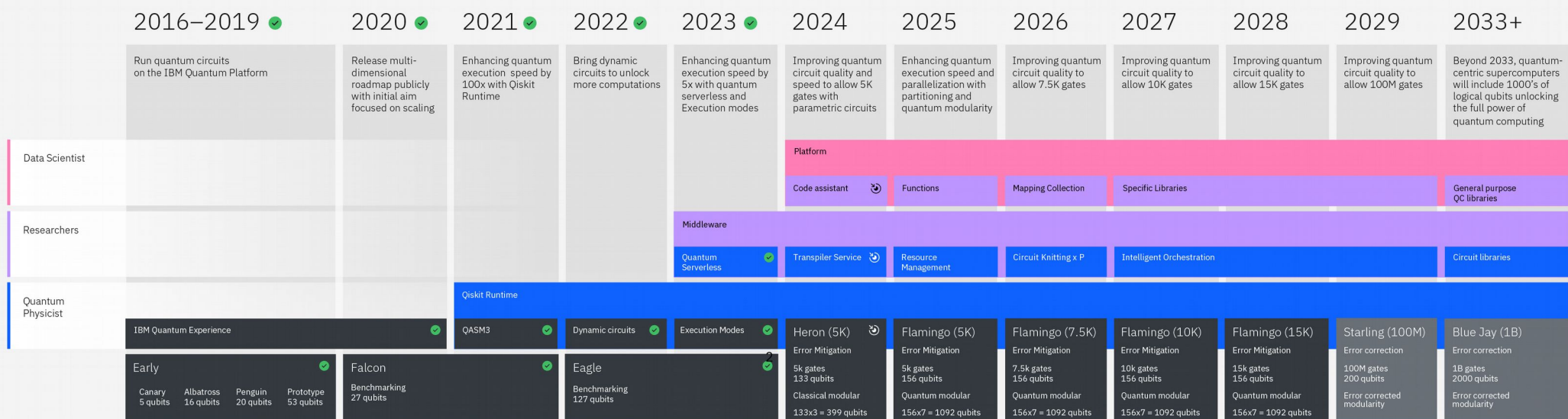
Im August 2024 erreichte der weltweite Umsatz mit Halbleitern 53,1 Mrd. Dollar, mehr als jemals zuvor in einem August. Gegenüber dem August 2023 entspricht das einer Steigerung von 20,6 Prozent, gegenüber dem Vormonat um 3,5 Prozent.

Quelle: https://www.elektroniknet.de/halbleiter/chip-umsatz-im-august-auf-allzeithoch.220858.html?utm_source=liana&utm_medium=email&utm_campaign=newsletter

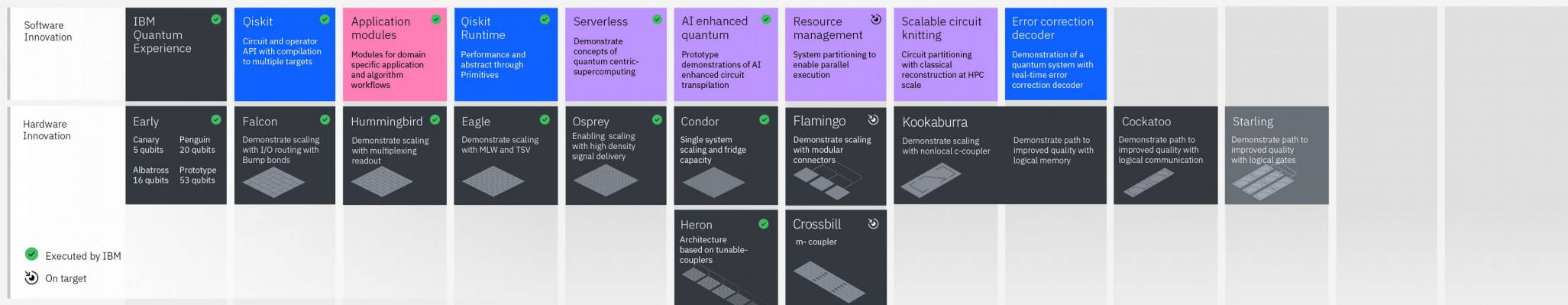
Quanten Computing

Development Roadmap

IBM Quantum



Innovation Roadmap



IBM Quantum / © 2023 IBM Corporation

Quelle: <https://newsroom.ibm.com/2023-12-04-IBM-Debuts-Next-Generation-Quantum-Processor-IBM-Quantum-System-Two,-Extends-Roadmap-to-Advance-Era-of-Quantum-Utility>

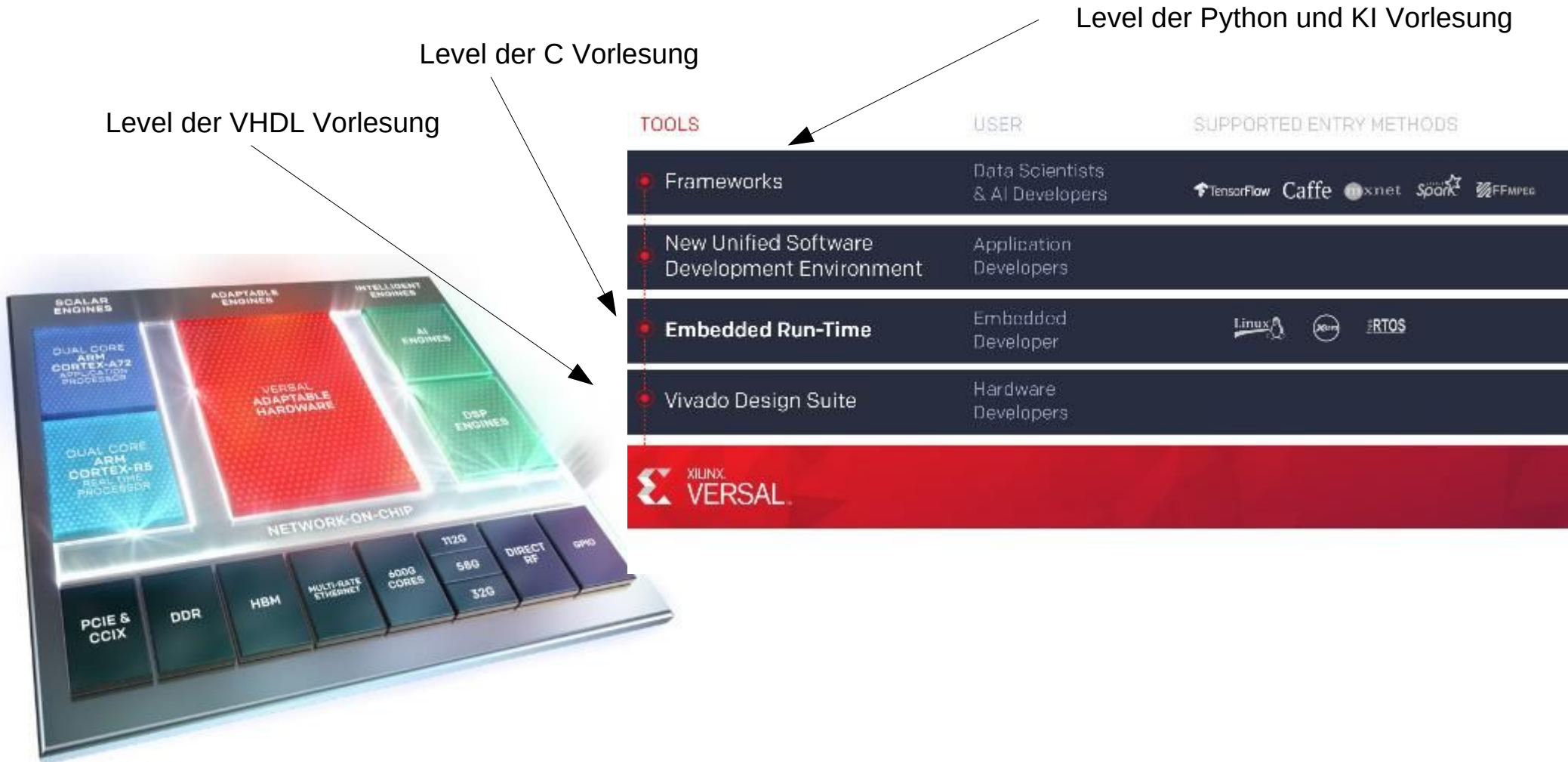
XILINX FPGA Lösungen

- Adaptive Compute Acceleration Platform (ACAP)
- ACAPs kombinieren
 - skalare Prozessor-Engines
 - adaptierbare Hardware-Engines
 - sogenannte Intelligent Engines mit fortschrittlichen Speicher- und Schnittstellentechnologien
- Hard- und Software der ACAPs programmierbar von Softwareentwicklern, Datenspezialisten und Hardwareentwicklern



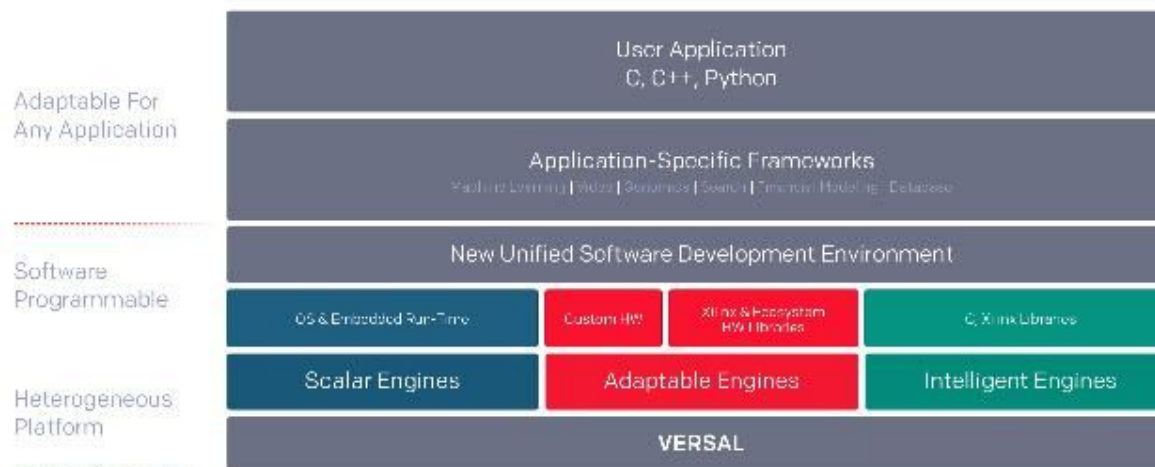
XILINX FPGA Lösungen

- Es stehen eine Vielzahl von Tools, Software, Bibliotheken, IP, Middleware und Frameworks zur Verfügung, die Designabläufe nach Industriestandard gestatten



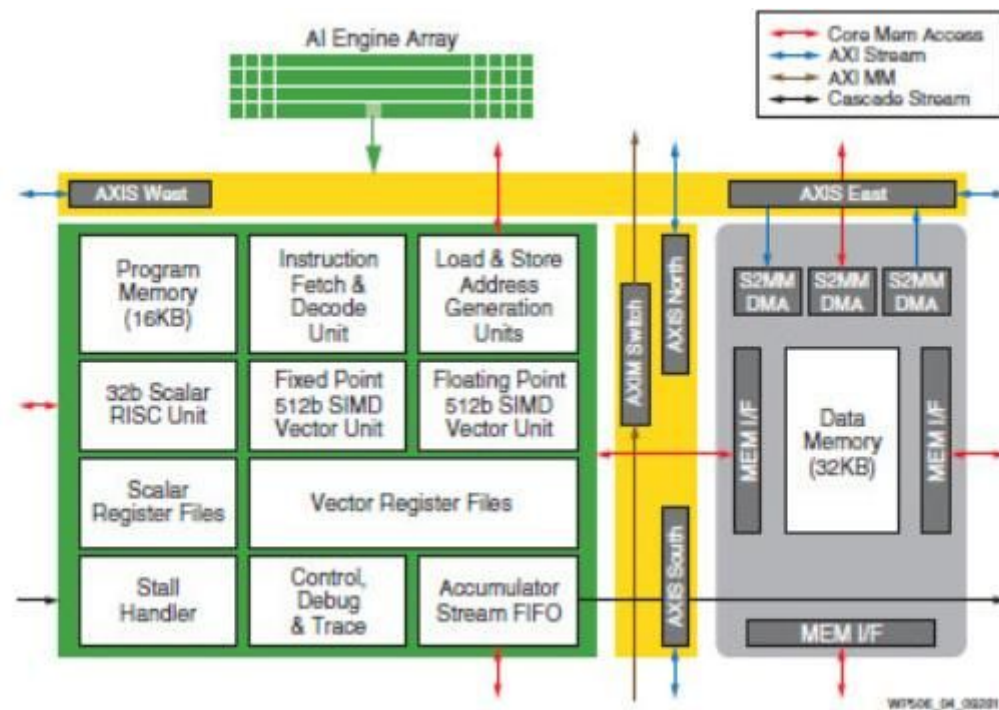
XILINX FPGA Lösungen

- Basierend auf der 7-nm-FinFET-Prozesstechnologie von TSMC
- Die Versal-Roadmap sieht zunächst zwei Familienstränge vor:
 - Prime Serie - drei Familien Prime, Premium und HBM (High Bandwidth Memory)
 - AI Core Serie - AI Core, AI Edge und AI RF
- AI-Core: deckt Bedarf an Low-Latency-KI-Inferenz für eine Vielzahl von Anwendungen



XILINX FPGA Lösungen

- Die AI Engine basiert auf mit mehr als 1 GHz getakteten VLIW/SIMD-Vektor-Prozessorkernen, mit enger Kopplung an lokale Speicher



XILINX FPGA Familie

- 16nm - UltraScale+ Artix, Kintex, Virtex
- 20nm - UltraScale Kintex, Vitex
- 28nm - 7-Familie Spartan, Artix, Kintex, Virtex
- 45nm - 6-Famile Spartan



16nm

The combination of TSMC's 16nm FinFET process with new UltraRAM and SmartConnect technologies enables AMD to continue delivering 'More than Moore's Law' value to the market.







20nm

AMD enhanced FPGA architecture that contains a step-function increase in both the amount of connectivity resources and the associated inter-die bandwidth in this second-generation 3D IC architecture.






28nm

Continuous innovation on the process node enables new devices with optimal performance at the lower power across product families to meet requirements for key applications.








45nm

AMD offers exceptional connectivity features such as high logic-to-pin ratios, small form-factor packaging, *MicroBlaze* soft processors, and a diverse number of supported I/O protocols.



Quelle: <https://www.xilinx.com/products/silicon-devices/fpga.html>

Spartan-7

Spartan-7 FPGAs

				I/O Optimization at the Lowest Cost and Highest Performance-per-Watt (1.0V, 0.95V)					
				XC7S6	XC7S15	XC7S25	XC7S50	XC7S75	XC7S100
Logic Resources	Part Number								
	Logic Cells			6,000	12,800	23,360	52,160	76,800	102,400
	Slices			938	2,000	3,650	8,150	12,000	16,000
CLB Flip-Flops				7,500	16,000	29,200	65,200	96,000	128,000
Memory Resources	Max. Distributed RAM (Kb)			70	150	313	600	832	1,100
	Block RAM/FIFO w/ ECC (36 Kb each)			5	10	45	75	90	120
	Total Block RAM (Kb)			180	360	1,620	2,700	3,240	4,320
Clock Resources	Clock Mgmt Tiles (1 MMCM + 1 PLL)			2	2	3	5	8	8
I/O Resources	Max. Single-Ended I/O Pins			100	100	150	250	400	400
	Max. Differential I/O Pairs			48	48	72	120	192	192
Embedded Hard IP Resources	DSP Slices			10	20	80	120	140	160
	Analog Mixed Signal (AMS) / XADC			0	0	1	1	1	1
	Configuration AES / HMAC Blocks			0	0	1	1	1	1
Speed Grades	Commercial Temp (C)			-1,-2	-1,-2	-1,-2	-1,-2	-1,-2	-1,-2
	Industrial Temp (I)			-1,-2,-1L	-1,-2,-1L	-1,-2,-1L	-1,-2,-1L	-1,-2,-1L	-1,-2,-1L
	Expanded Temp (Q)			-1	-1	-1	-1	-1	-1
				Available User I/O: 3.3V SelectIO™ HR I/O					
	Package ⁽¹⁾	Body Area (mm)	Ball Pitch (mm)						
	CPGA196	8x8	0.5	100	100				
	CSGA225	13x13	0.8	100	100	150			
	CSGA324	15x15	0.8			150	210		
	FTGB196	15x15	1.0	100	100	100	100		
	FGGA484	23x23	1.0				250	338	338
	FGGA676	27x27	1.0					400	400

Quelle: <https://docs.xilinx.com/v/u/en-US/7-series-product-selection-guide>

Artix-7

Artix-7 FPGAs

Transceiver Optimization at the Lowest Cost and Highest DSP Bandwidth
(1.0V, 0.95V, 0.9V)

	Part Number	XC7A12T	XC7A15T	XC7A25T	XC7A35T	XC7A50T	XC7A75T	XC7A100T	XC7A200T
Logic Resources	Logic Cells	12,800	16,640	23,360	33,280	52,160	75,520	101,440	215,360
	Slices	2,000	2,600	3,650	5,200	8,150	11,800	15,850	33,650
	CLB Flip-Flops	16,000	20,800	29,200	41,600	65,200	94,400	126,800	269,200
Memory Resources	Maximum Distributed RAM (Kb)	171	200	313	400	600	892	1,188	2,888
	Block RAM/FIFO w/ ECC (36 Kb each)	20	25	45	50	75	105	135	365
	Total Block RAM (Kb)	720	900	1,620	1,800	2,700	3,780	4,860	13,140
Clock Resources	CMTs (1 MMCM + 1 PLL)	3	5	3	5	5	6	6	10
I/O Resources	Maximum Single-Ended I/O	150	250	150	250	250	300	300	500
	Maximum Differential I/O Pairs	72	120	72	120	120	144	144	240
Embedded Hard IP Resources	DSP Slices	40	45	80	90	120	180	240	740
	PCIe® Gen2 ⁽¹⁾	1	1	1	1	1	1	1	1
	Analog Mixed Signal (AMS) / XADC	1	1	1	1	1	1	1	1
	Configuration AES / HMAC Blocks	1	1	1	1	1	1	1	1
	GTP Transceivers (6.6 Gb/s Max Rate) ⁽²⁾	2	4	4	4	4	8	8	16
Speed Grades	Commercial Temp (C)	-1, -2	-1, -2	-1, -2	-1, -2	-1, -2	-1, -2	-1, -2	-1, -2
	Extended Temp (E)	-2L, -3	-2L, -3	-2L, -3	-2L, -3	-2L, -3	-2L, -3	-2L, -3	-2L, -3
	Industrial Temp (I)	-1, -2, -1L	-1, -2, -1L	-1, -2, -1L	-1, -2, -1L	-1, -2, -1L	-1, -2, -1L	-1, -2, -1L	-1, -2, -1L
	Package ^{(3), (4)}	Dimensions (mm)	Ball Pitch (mm)	Available User I/O: 3.3V SelectIO™ HR I/O (GTP Transceivers)					
	CPG236	10 x 10	0.5		106 (2)		106 (2)		
	CPG238	10 x 10	0.5	112 (2)		112 (2)			
	CSG324	15 x 15	0.8		210 (0)		210 (0)	210 (0)	210 (0)
	CSG325	15 x 15	0.8	150 (2)	150 (4)	150 (4)	150 (4)		
	FTG256	17 x 17	1.0		170 (0)		170 (0)	170 (0)	170 (0)
	SBG484	19 x 19	0.8						285 (4)
Footprint Compatible	FGG484 ⁽⁵⁾	23 x 23	1.0		250 (4)		250 (4)	285 (4)	285 (4)
	FBG484 ⁽³⁾	23 x 23	1.0						285 (4)
Footprint Compatible	FGG676 ⁽⁶⁾	27 x 27	1.0				300 (8)	300 (8)	
	FBG676 ⁽⁶⁾	27 x 27	1.0						400 (8)
	FFG1156	35 x 35	1.0						500 (16)

Quelle: <https://docs.xilinx.com/v/u/en-US/7-series-product-selection-guide>

Kintex-7

Kintex-7 FPGAs

Optimized for Best Price-Performance
(1.0V, 0.95V, 0.9V)

	Part Number	XC7K70T	XC7K160T	XC7K325T	XC7K355T	XC7K410T	XC7K420T	XC7K480T
Logic Resources	Slices	10,250	25,350	50,950	55,650	63,550	65,150	74,650
	Logic Cells	65,600	162,240	326,080	356,160	406,720	416,960	477,760
	CLB Flip-Flops	82,000	202,800	407,600	445,200	508,400	521,200	597,200
Memory Resources	Maximum Distributed RAM (Kb)	838	2,188	4,000	5,088	5,663	5,938	6,788
	Block RAM/FIFO w/ ECC (36 Kb each)	135	325	445	715	795	835	955
	Total Block RAM (Kb)	4,860	11,700	16,020	25,740	28,620	30,060	34,380
Clock Resources	CMTs (1 MMCM + 1 PLL)	6	8	10	6	10	8	8
I/O Resources	Maximum Single-Ended I/O	300	400	500	300	500	400	400
	Maximum Differential I/O Pairs	144	192	240	144	240	192	192
Integrated IP Resources	DSP48 Slices	240	600	840	1,440	1,540	1,680	1,920
	PCIe® Gen2 ⁽¹⁾	1	1	1	1	1	1	1
	Analog Mixed Signal (AMS) / XADC	1	1	1	1	1	1	1
	Configuration AES / HMAC Blocks	1	1	1	1	1	1	1
	GTX Transceivers (12.5 Gb/s Max Rate)	8	8	16	24	16	32	32
Speed Grades	Commercial Temp (C)	-1, -2	-1, -2	-1, -2	-1, -2	-1, -2	-1, -2	-1, -2
	Extended Temp (E)	-2L, -3	-2L, -3	-2L, -3	-2L, -3	-2L, -3	-2L, -3	-2L, -3
	Industrial Temp (I)	-1, -2	-1, -2, -2L	-1, -2, -2L	-1, -2, -2L	-1, -2, -2L	-1, -2, -2L	-1, -2, -2L
Package ⁽²⁾		Dimensions (mm)	Ball Pitch (mm)	Available User I/O: 3.3V HR I/O, 1.8V HP I/Os (GTX)				
Footprint Compatible	FBG484 ⁽³⁾	23 x 23	1.0	185, 100 (4)	185, 100 (4)			
	FBG676 ⁽³⁾	27 x 27	1.0	200, 100 (8)	250, 150 (8)	250, 150 (8)	250, 150 (8)	
	FFG676	27 x 27	1.0		250, 150 (8)	250, 150 (8)	250, 150 (8)	
Footprint Compatible	FBG900 ⁽³⁾	31 x 31	1.0		350, 150 (16)		350, 150 (16)	
	FFG900	31 x 31	1.0		350, 150 (16)		350, 150 (16)	
	FFG901	31 x 31	1.0			300, 0 (24)	380, 0 (28)	380, 0 (28)
	FFG1156	35 x 35	1.0				400, 0 (32)	400, 0 (32)

Quelle: <https://docs.xilinx.com/v/u/en-US/7-series-product-selection-guide>

Virtex-7

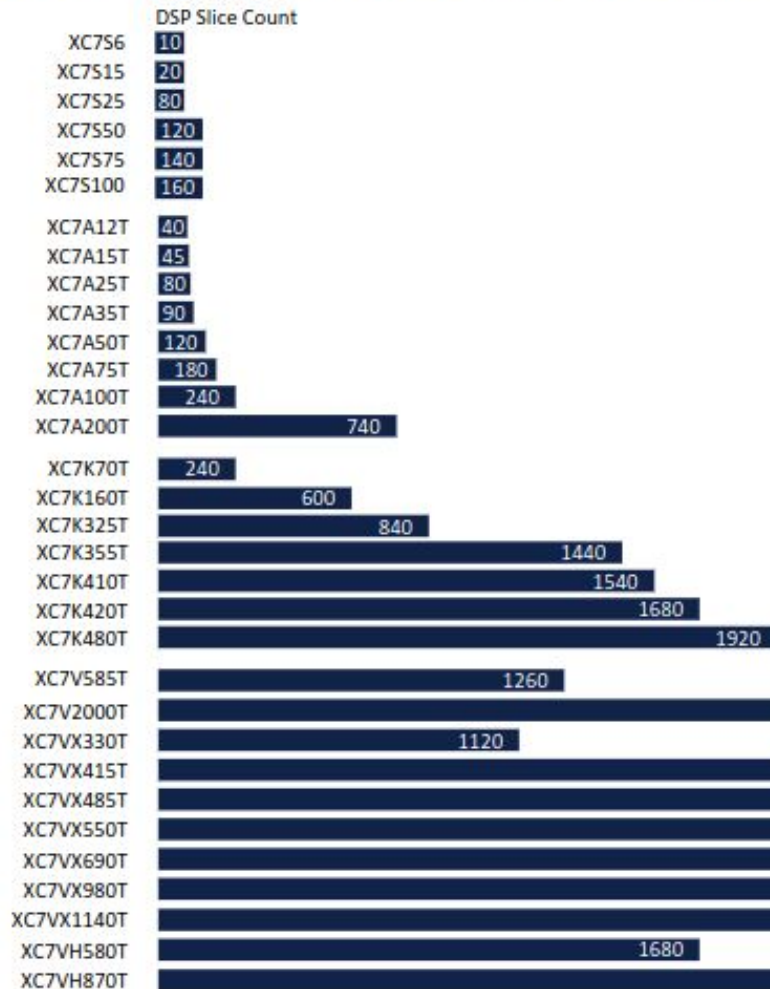
Virtex-7 FPGAs

Optimized for Highest System Performance and Capacity (1.0V)												
Logic Resources	Part Number	XC7V585T	XC7V2000T	XC7VX330T	XC7VX415T	XC7VX485T	XC7VX550T	XC7VX690T	XC7VX980T	XC7VX1140T	XC7VH380T	XC7VH870T
	Slices	91,030	305,400	51,000	64,400	73,900	86,600	108,300	153,000	178,000	90,700	136,900
	Logic Cells	582,720	1,954,560	326,400	412,160	483,700	554,240	693,120	979,200	1,139,200	580,480	876,160
Memory Resources	CLB Flip-Flops	728,400	2,443,200	408,000	515,200	607,200	692,800	866,400	1,224,000	1,424,000	725,600	1,095,200
	Maximum Distributed RAM (Kb)	6,938	21,550	4,388	6,525	8,175	8,725	10,888	13,838	17,700	8,850	13,275
	Block RAM/FIFO w/ ECC (36 Kb each)	795	1,292	750	880	1,030	1,180	1,470	1,500	1,880	940	1,410
Clocking	Total Block RAM (Kb)	28,620	46,512	27,000	31,680	37,080	42,480	52,920	54,000	67,680	33,840	50,760
	CMTs (1 MMCM + 1 PLL)	18	24	14	12	14	20	20	18	24	12	18
	Maximum Single-Ended I/O	850	1,200	700	600	700	600	1,000	900	1,100	600	300
I/O Resources	Maximum Differential I/O Pairs	408	576	336	288	336	288	480	432	528	288	144
	DSP Slices	1,260	2,160	1,120	2,160	2,800	2,880	3,600	3,600	3,360	1,680	2,520
	PCIe® Gen2 ⁽¹⁾	3	4	—	—	4	—	—	—	—	—	—
Integrated IP Resources	PCIe Gen3	—	—	2	2	—	2	3	3	4	2	3
	Analog Mixed Signal (AMS) / XADC	1	1	1	1	1	1	1	1	1	1	1
	Configuration AES / HMAC Blocks	1	1	1	1	1	1	1	1	1	1	1
	GTX Transceivers (12.5 Gb/s Max Rate) ⁽²⁾	36	36	—	—	56	—	—	—	—	—	—
	GTH Transceivers (13.1 Gb/s Max Rate) ⁽³⁾	—	—	28	48	—	80	80	72	96	48	72
	GTZ Transceivers (28.05 Gb/s Max Rate)	—	—	—	—	—	—	—	—	—	8	16
	Commercial Temp (C)	-1, -2	-1, -2	-1, -2	-1, -2	-1, -2	-1, -2	-1, -2	-1, -2	-1, -2	-1, -2	-1, -2
Speed Grades	Extended Temp (E) ⁽⁴⁾	-2L, -3	-2L, -2G	-2L, -3	-2L, -3	-2L, -3	-2L, -3	-2L, -3	-2L	-2L, -2G	-2L, -2G	-2L, -2G
	Industrial Temp (I)	-1, -2	-1	-1, -2	-1, -2	-1, -2	-1, -2	-1, -2	-1	-1	—	—
Package ⁽⁵⁾		Dimensions (mm)	Ball Pitch (mm)	Available User I/O: 3.3V HR I/O, 1.8V HP I/Os (GTX, GTH)								1.8V HP I/O (GTH, GTZ)
Footprint Compatible	FFG1157 ⁽⁶⁾	35 x 35	1.0	0, 600 (20, 0)	0, 600 (0, 20)	0, 600 (0, 20)	0, 600 (20, 0)	0, 600 (0, 20)				
	FFG1761 ⁽⁶⁾	42.5 x 42.5	1.0	100, 750 (36, 0)	50, 650 (0, 28)		0, 700 (28, 0)	0, 850 (0, 36)				
	FHG1761	45 x 45	1.0									
	FLG1925	45 x 45	1.0									
Footprint Compatible	FFG1158 ⁽⁶⁾	35 x 35	1.0			0, 350 (0, 48)	0, 350 (48, 0)	0, 350 (0, 48)	0, 350 (0, 48)			
	FFG1926	45 x 45	1.0						0, 720 (0, 64)	0, 720 (0, 64)		
	FLG1926	45 x 45	1.0							0, 720 (0, 64)		
	FFG1927 ⁽⁶⁾	45 x 45	1.0			0, 600 (0, 48)	0, 600 (56, 0)	0, 600 (0, 80)	0, 600 (0, 80)			
Footprint Compatible	FFG1928 ⁽⁷⁾	45 x 45	1.0						0, 480 (0, 72)			
	FLG1928	45 x 45	1.0						0, 480 (0, 96)			
Footprint Compatible	FFG1930	45 x 45	1.0				0, 700 (24, 0)		0, 1000 (0, 24)	0, 900 (0, 24)		
	FLG1930	45 x 45	1.0							0, 1100 (0, 24)		
	FLG1155 ⁽⁷⁾	35 x 35	1.0								400 (24, 8)	
	FLG1931 ⁽⁷⁾	45 x 45	1.0								600 (48, 8)	
	FLG1932 ⁽⁷⁾	45 x 45	1.0									300 (72, 16)

Quelle: <https://docs.xilinx.com/v/u/en-US/7-series-product-selection-guide>

DSP Vergleich

Digital Signal Processing Metrics



Spartan®-7 FPGAs

Speed grade	-1	-2
F _{MAX} [MHz]	464	550
Max GMAC/s	148	176

Artix®-7 FPGAs

Speed grade	-1	-2	-3
F _{MAX} [MHz]	464	550	628
Max GMAC/s	686	814	929

Kintex®-7 FPGAs

Speed grade	-1	-2	-3
F _{MAX} [MHz]	464	550	741
Max GMAC/s	1,781	2,112	2,845

Virtex®-7 FPGAs

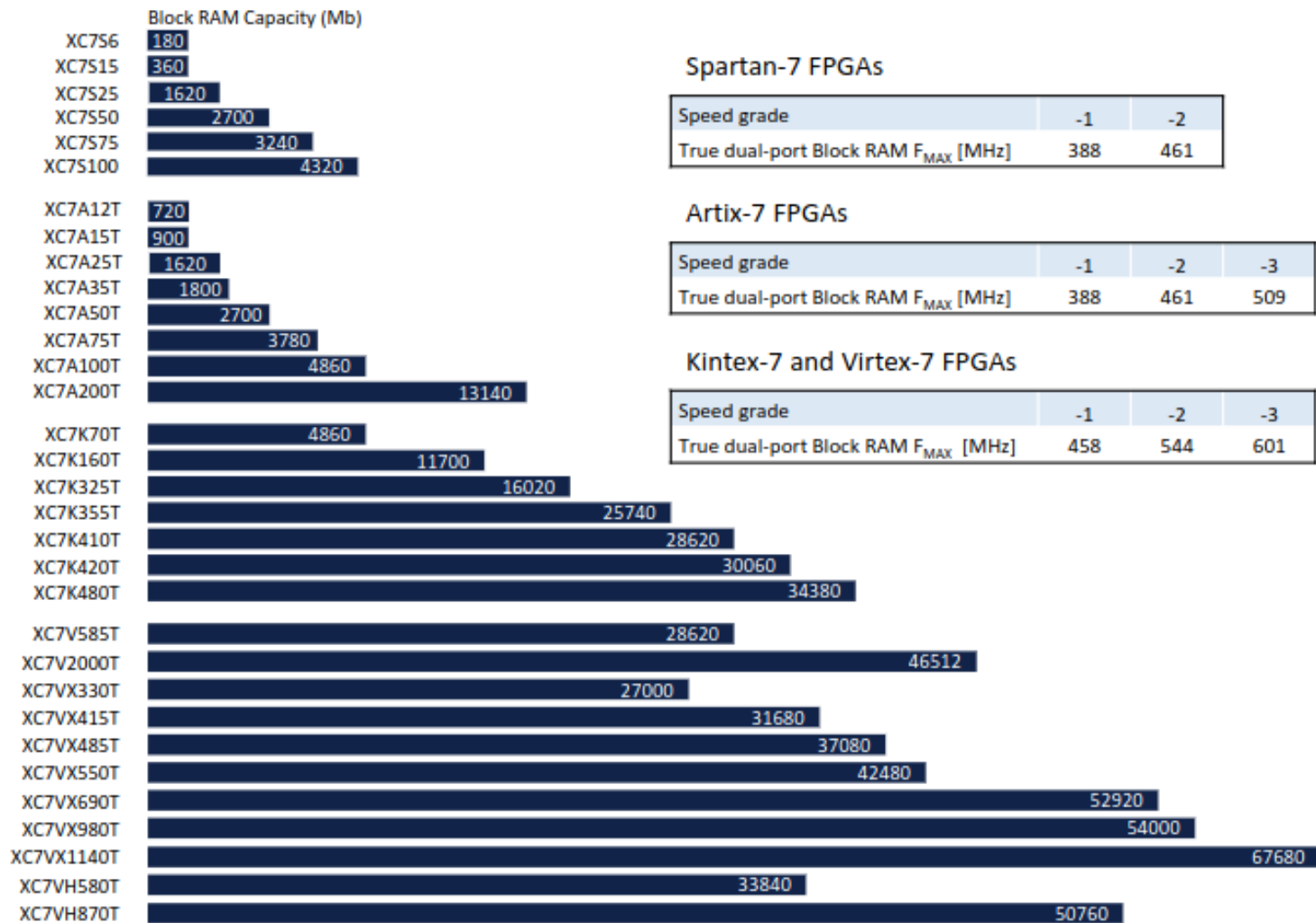
Speed grade	-1	-2	-3
F _{MAX} [MHz]	547	650	741
Max GMAC/s	2,756	3,276	3,734

For more information, refer to: [UG479, 7 Series FPGAs DSP48E1 Slice User Guide](#)

Quelle: <https://docs.xilinx.com/v/u/en-US/7-series-product-selection-guide>

Speicher Vergleich

Block RAM Metrics

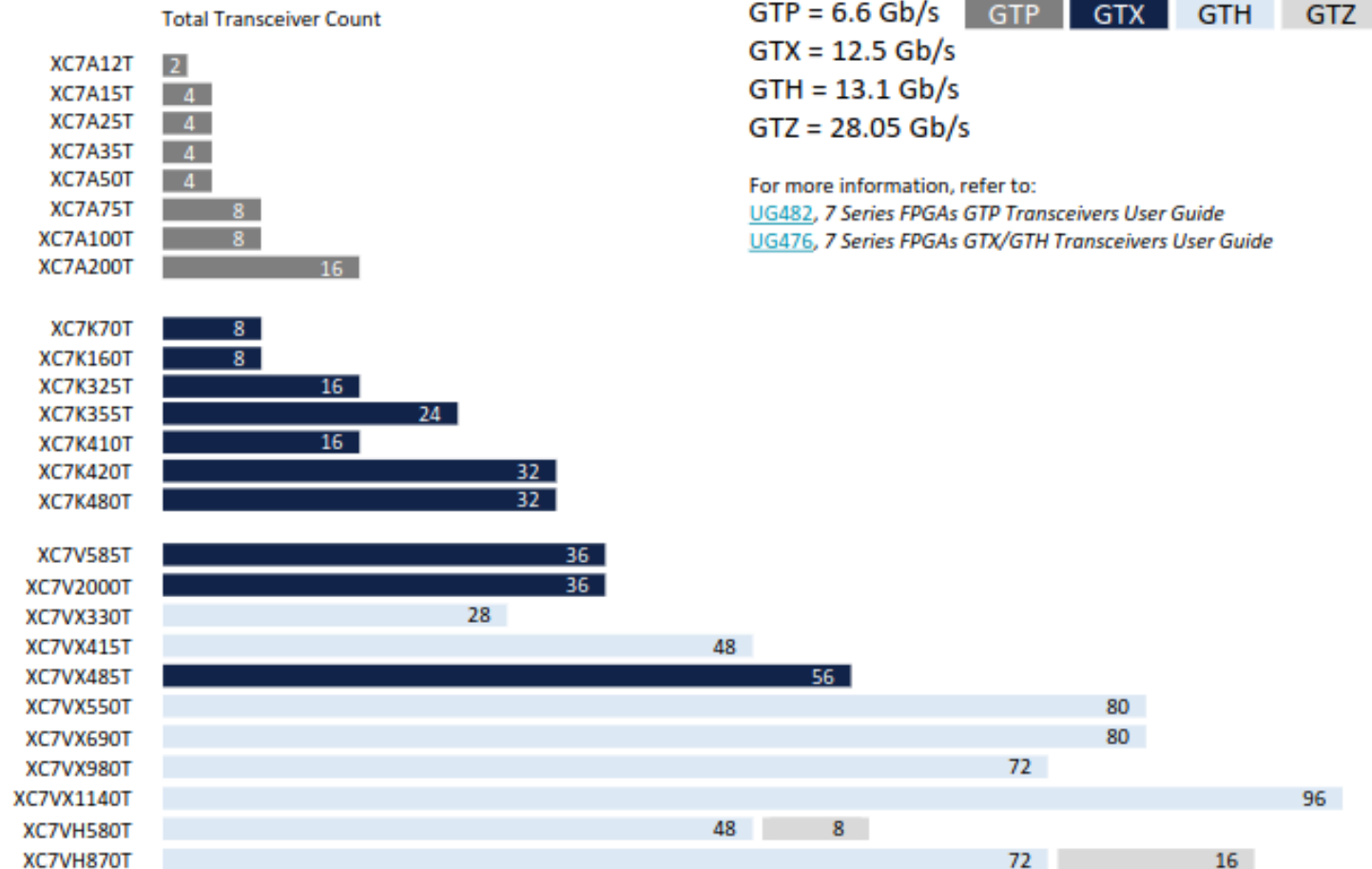


Quelle: <https://docs.xilinx.com/v/u/en-US/7-series-product-selection-guide>

High Speed Vergleich

High-Speed Serial Transceivers

7 series devices provide a broad portfolio of transceivers for applications ranging from low-cost consumer products to high-end networking systems.

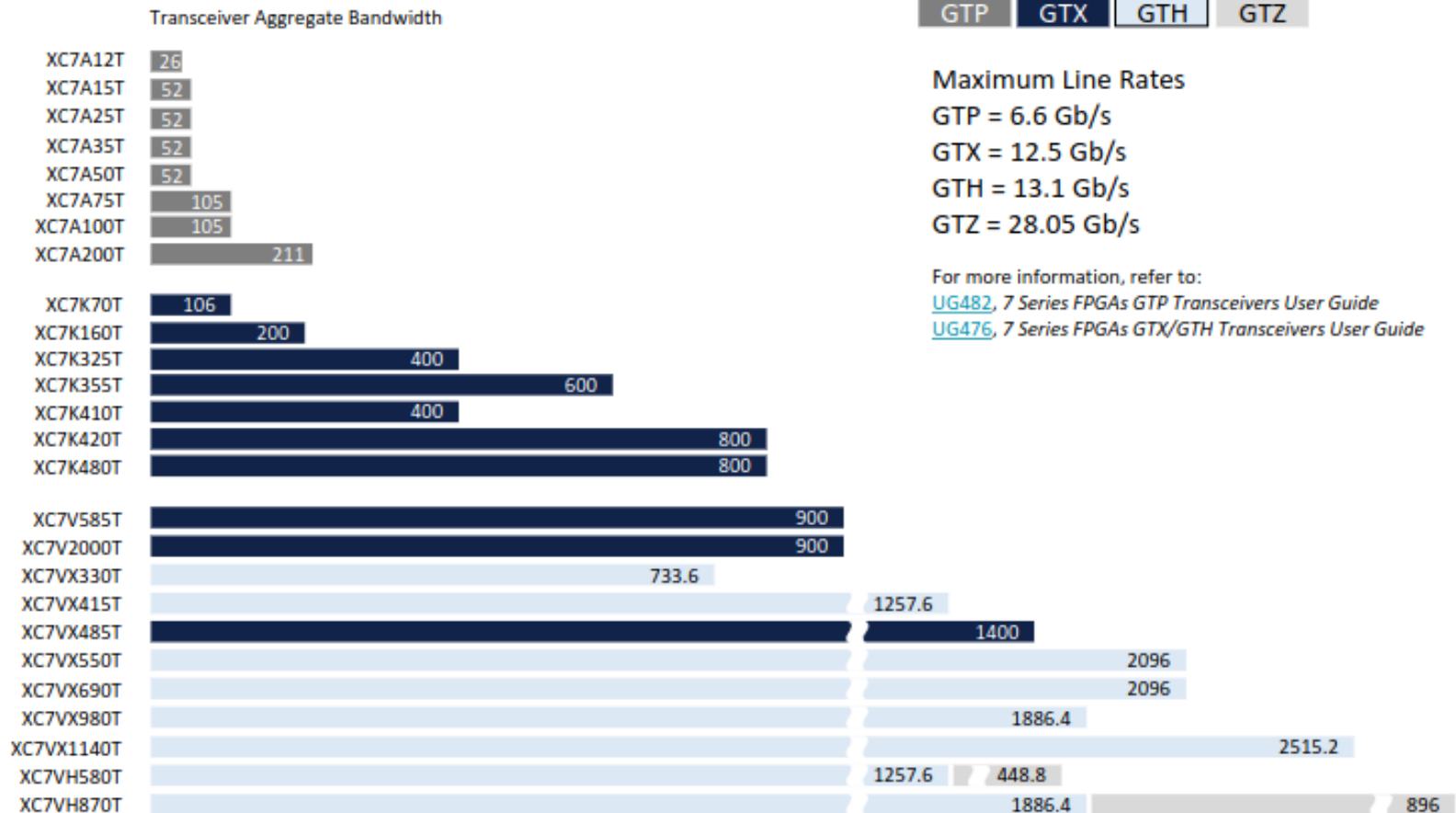


Quelle: <https://docs.xilinx.com/v/u/en-US/7-series-product-selection-guide>

Bandbreite Vergleich

Transceiver Aggregate Bandwidth

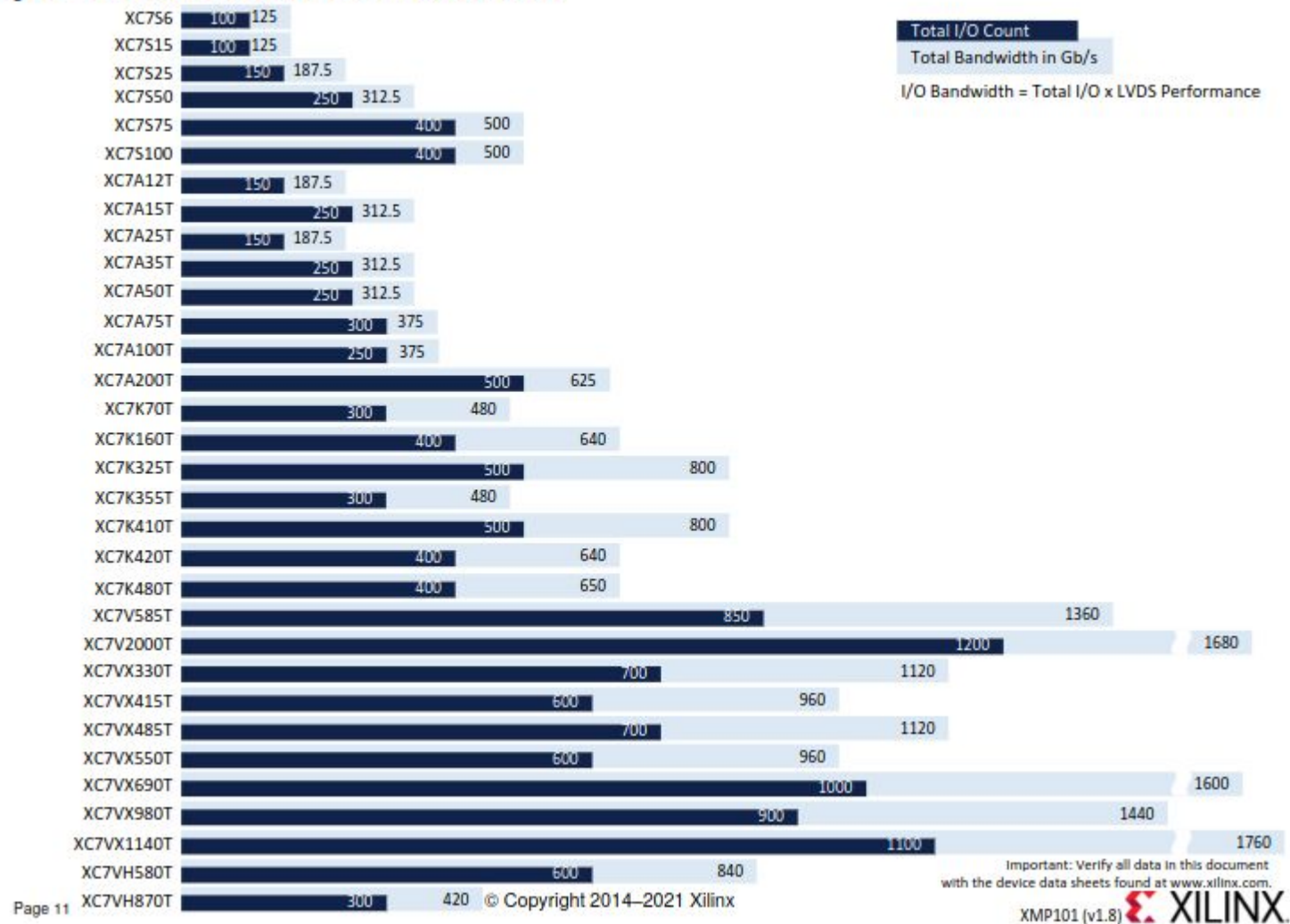
7 series devices provide a broad portfolio of transceivers for applications ranging from low-cost consumer products to high-end networking systems.



Quelle: <https://docs.xilinx.com/v/u/en-US/7-series-product-selection-guide>

I/O Bandbreite Vergleich

I/O Count and Bandwidth



Quelle: <https://docs.xilinx.com/v/u/en-US/7-series-product-selection-guide>

Literatur

- Cord, E., FPGAs für Maker: Eine praktische Einführung in die programmierbare Logik, Dpunkt Verlag, 2016
- Hamblen, J.O., Hall, T.S., Furman, M.D., Rapid Prototyping of Digital Systems, Springer Verlag, 2006
- Fehn, H.-G., Einführung in die Digitaltechnik, Schlembach Verlag, 2011
- Molitor, P., Ritter, J., Kompaktkurs VHDL, Oldenbourg Verlag, 2013

Video Links

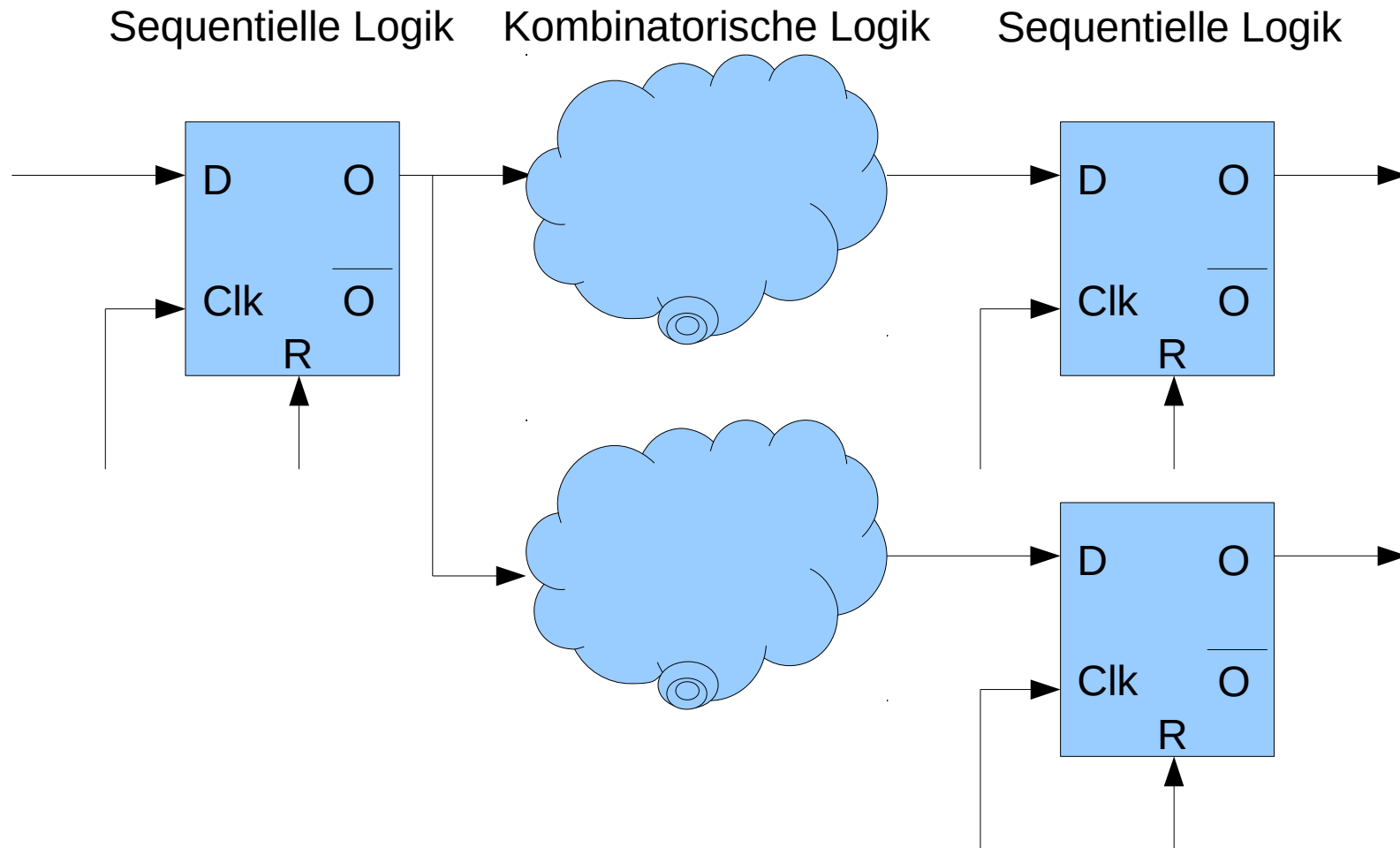
- <https://www.youtube.com/watch?v=6ruLtdD5dUE>
- <https://www.youtube.com/watch?v=zpOioOiKYp4>
- <https://www.youtube.com/watch?v=c9arR8T0Qts>

Ziel von VHDL für ASIC und FPGA Designs

- Entwurf von
 - kombinatorischer
 - sequentieller

Logik für die Synthese in ASIC oder FPGA Bausteinen

wird als RTL Design bezeichnet – Register Transfer Level



Video zum Thema Chipherstellung/ASIC/FPGA

TSMC – Taiwan Semiconductor Manufacturing Company

<https://www.youtube.com/watch?v=Hb1WDxSoSec>

FPGA Beschreibung

<https://www.youtube.com/watch?v=ePwo3P1iZO4>

https://www.youtube.com/watch?v=ILg1AgA2Xoo&list=PLEBQazB0HUyT1WmMONxRZn9NmQ_9CIKhb

Intel hat den FPGA Hersteller Altera gekauft, AMD überlegt, Xilinx zu kaufen (hat es schon längst getan)

<https://www.youtube.com/watch?v=2yklU69Xiuo>

Arty FPGA Evaluation Board von Digilent

<https://www.youtube.com/watch?v=h2IE8Y5xd5k>

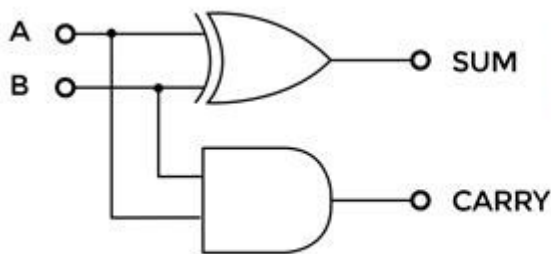
Nebenläufige und Sequentielle Abläufe

- C ist typischerweise eine sequentielle Programmiersprache, deren Ablauf in einem Prozessorkern nacheinander, sequentiell organisiert wird
 - Maschinenbefehl für Maschinenbefehl wird nacheinander abgearbeitet
- Anweisungen in VHDL werden in der Regel nebenläufig, also parallel in der Hardware abgearbeitet
 - einzelne Logikgatter sind von anderen Logikgattern völlig unabhängig und agieren eigenständig autonom und damit parallel zueinander
 - Sequentielle Gatter wie Flip Flops arbeiten takt-synchron und organisieren einen Datenfluss durch den Chip, indem sie die Signale aus der kombinatorischen, parallelen Logik an ihrem Dateneingang aufnehmen und mit dem Clock-Takt an ihren Ausgang weiter geben
- Es ist daher wichtig, beim VHDL Entwurf nicht in einer Programmiersprache zu denken, sondern in einer Hardwarebeschreibungssprache, die verschiedene Blöcke / Bereiche / Umgebungen nebenläufig organisiert und durch sequentielle Komponenten zueinander synchronisiert

Beispiele kombinatorischer

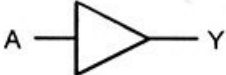
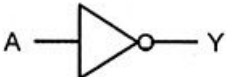
Logikgatter

- Typische Logikgatter sind
 - Und,
 - Oder,
 - Exklusiv-Oder, ... Gatter
- Look Up Table (LUT)
- Addierer, Subtrahierer, Multiplizierer, Dividierer
können rein kombinatorisch oder auch eine Mischung aus Kombinatorik und sequentieller Logik darstellen, IC 7480, 7482, 7483

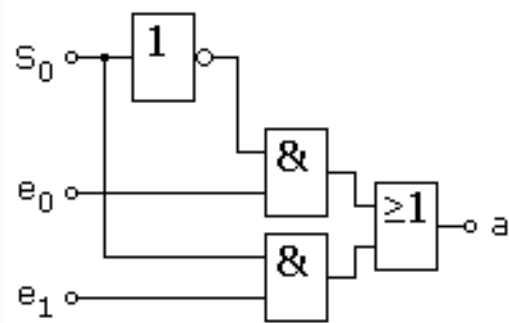


A	B	Sum	Carry
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

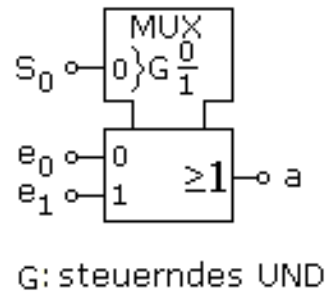
Quelle: <https://projects-static.raspberrypi.org/projects/halfadder/fbd927fdbca5dcb6631fad44fa49ec03feafd80c/en/images/fig1.png>

Logic function	Logic symbol	Truth table	Boolean expression															
Buffer		<table><tr><th>A</th><th>Y</th></tr><tr><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td></tr></table>	A	Y	0	0	1	1	$Y = A$									
A	Y																	
0	0																	
1	1																	
Inverter (NOT gate)		<table><tr><th>A</th><th>Y</th></tr><tr><td>0</td><td>1</td></tr><tr><td>1</td><td>0</td></tr></table>	A	Y	0	1	1	0	$Y = \bar{A}$									
A	Y																	
0	1																	
1	0																	
2-input AND gate		<table><tr><th>A</th><th>B</th><th>Y</th></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td><td>1</td></tr></table>	A	B	Y	0	0	0	0	1	0	1	0	0	1	1	1	$Y = A \cdot B$
A	B	Y																
0	0	0																
0	1	0																
1	0	0																
1	1	1																
2-input NAND gate		<table><tr><th>A</th><th>B</th><th>Y</th></tr><tr><td>0</td><td>0</td><td>1</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>0</td></tr></table>	A	B	Y	0	0	1	0	1	1	1	0	1	1	1	0	$Y = \overline{A \cdot B}$
A	B	Y																
0	0	1																
0	1	1																
1	0	1																
1	1	0																
2-input OR gate		<table><tr><th>A</th><th>B</th><th>Y</th></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>1</td></tr></table>	A	B	Y	0	0	0	0	1	1	1	0	1	1	1	1	$Y = A + B$
A	B	Y																
0	0	0																
0	1	1																
1	0	1																
1	1	1																
2-input NOR gate		<table><tr><th>A</th><th>B</th><th>Y</th></tr><tr><td>0</td><td>0</td><td>1</td></tr><tr><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td><td>0</td></tr></table>	A	B	Y	0	0	1	0	1	0	1	0	0	1	1	0	$Y = \overline{A + B}$
A	B	Y																
0	0	1																
0	1	0																
1	0	0																
1	1	0																
2-input EX-OR gate		<table><tr><th>A</th><th>B</th><th>Y</th></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>0</td></tr></table>	A	B	Y	0	0	0	0	1	1	1	0	1	1	1	0	$Y = A \oplus B$
A	B	Y																
0	0	0																
0	1	1																
1	0	1																
1	1	0																
2-input EX-NOR gate		<table><tr><th>A</th><th>B</th><th>Y</th></tr><tr><td>0</td><td>0</td><td>1</td></tr><tr><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td><td>1</td></tr></table>	A	B	Y	0	0	1	0	1	0	1	0	0	1	1	1	$Y = \overline{A \oplus B}$
A	B	Y																
0	0	1																
0	1	0																
1	0	0																
1	1	1																

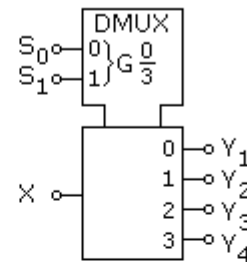
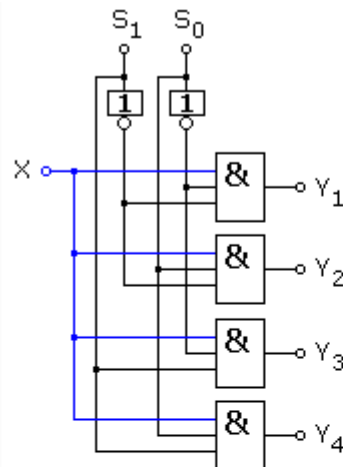
Multiplexer und Demultiplexer



1-MUX 2-Bit zu 1-Bit Datenselektor



S_0	e_1	e_0	a
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1



S_1	S_0	X	Y_1	Y_2	Y_3	Y_4
0	0	0	0	*	*	*
0	0	1	1	*	*	*
0	1	0	*	0	*	*
0	1	1	*	1	*	*
1	0	0	*	*	0	*
1	0	1	*	*	1	*
1	1	0	*	*	*	0
1	1	1	*	*	*	1



S_1	S_0	Y_1	Y_2	Y_3	Y_4
0	0	X	*	*	*
0	1	*	X	*	*
1	0	*	*	X	*
1	1	*	*	*	X

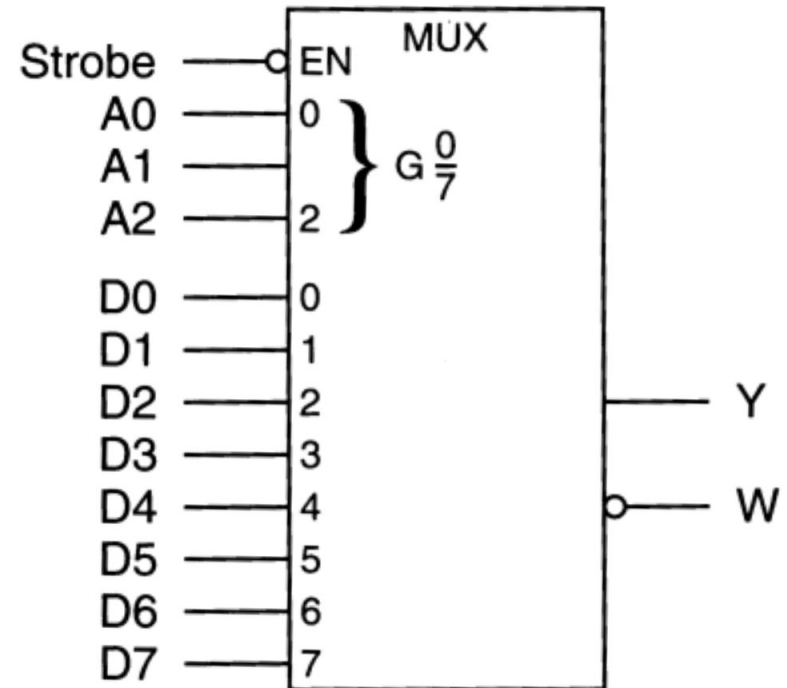
links die ausführliche und oben die verkürzte Schalttable des 1:4 Demultiplexers

Quelle: <https://www.elektroniktutor.de/digitaltechnik/mux.html>

Copyright © Edmund Coersmeier 2008 - 2025

Multiplexer IC 74151A

- Multiplexer 74151A
 - 8 Dateneingängen
 - A0 – A2 Adresseingänge
 - D0 – D7 Dateneingänge
 - Y Ausgang
 - $W = !Y$
- Gruppenübung: Was ist der Sinn des Strobe Eingangs?



Flip-Flop

- Speicherbausteine werden als Flip-Flop (FF) bezeichnet
 - Schaltungen, die einen von zwei möglichen Zuständen für eine unbegrenzte Zeit annehmen können
- Flip-Flops wurden früher auch als Multivibratoren bezeichnet
- Je nach Aufbau und Funktion unterscheidet man verschiedene Flip-Flop Typen
 - RS FF
 - D FF
 - JK FF

Überblick Speicherbausteine

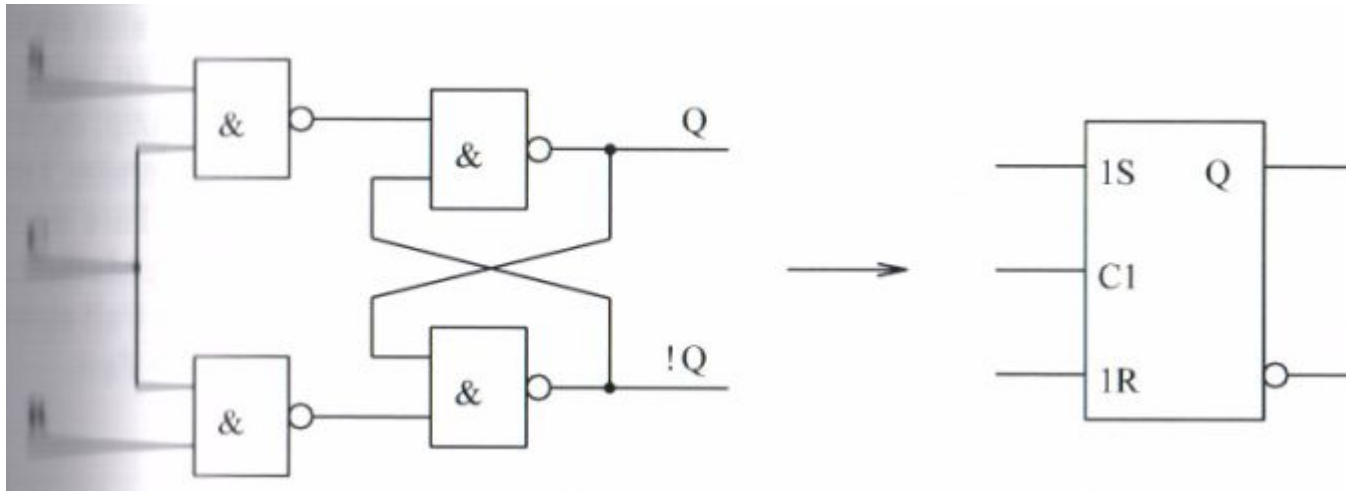
- RS-Flip-Flop
 - Ungetaktet
 - Taktzustandgesteuert
 - Master-Slave RS-Flip-Flop
 - Flankengetriggertes RS-Flip-Flop
- D-Flip-Flop
- JK-Flip-Flop
- Zusätze
 - Preset und Clear
 - Takt

Unterschiedliche Flip-Flops

- Je nach Aufbau und Funktion unterscheidet man verschiedene Flip-Flop Typen
 - Art der Steuereingänge
 - RS FF
 - D FF
 - JK FF
 - Art der Takteingänge
 - Ungetaktet
 - Taktzustandgesteuert
 - Flankengetriggert
 - Master-Slave
- In der Praxis kommen nur wenige Typen vor

Taktzustandsgesteuerte RS FF (Latch)

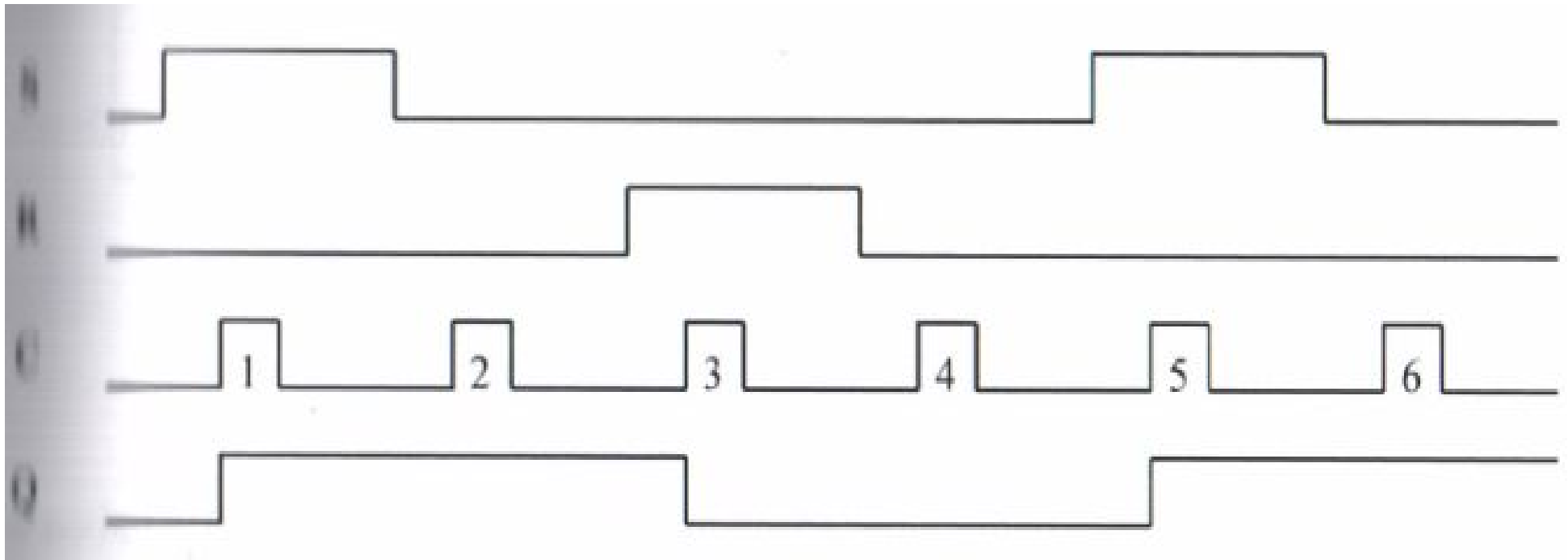
- Beim ungetakteten RS FF ändert sich der Ausgang sofort nach einer Änderung an den Eingängen R und S
- Schema eines Taktzustandsgesteuerten RS FF



- Ein 0-Signal am Eingang C bewirkt, dass die Ausgänge beider NAND-Glieder auf 1 liegen, unabhängig von den Eingangssignalen S und R → Speicherzustand
- Für $C = 1$ verhält sich die Schaltung genau gleich wie das ungetaktete RS FF

Schaltverhalten

- Schaltverhalten des Taktzustandsgesteuerten RS FF



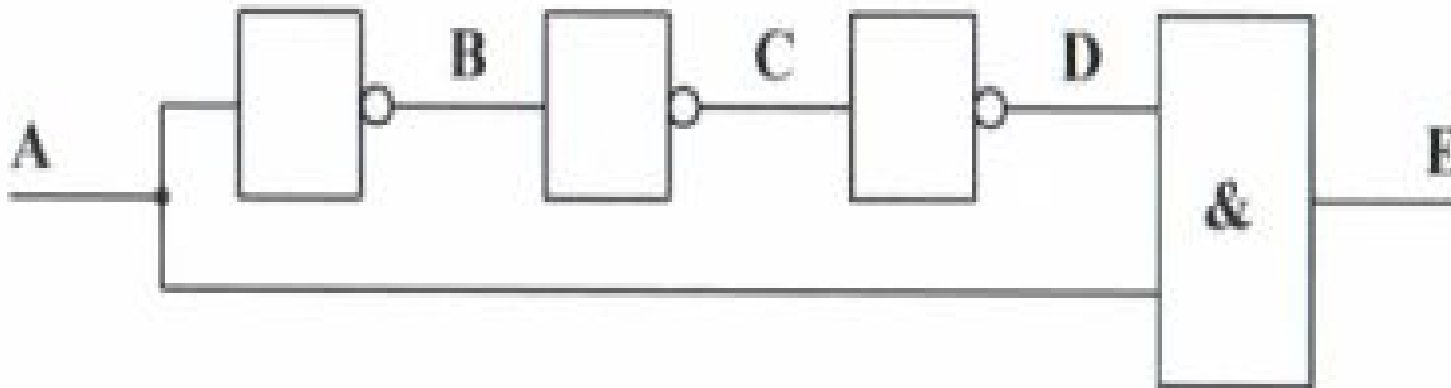
- Der Setzimpuls wirkt nicht sofort auf den Ausgang Q
 - Erst beim Auftreten des Taktimpulses 1

Definition Transparenz

- Das ungetaktete RS FF und das taktzustandsgesteuerte RS FF mit $C = 1$ ändern unmittelbar sofort die Ausgänge
- Diese FF werde als transparent bezeichnet
- Der Effekt ist unerwünscht
 - Transparente FFs können z.B. keinen Zähler aufbauen

Gruppenübung

- Versuchen Sie die Funktion dieser Schaltung zu erklären!



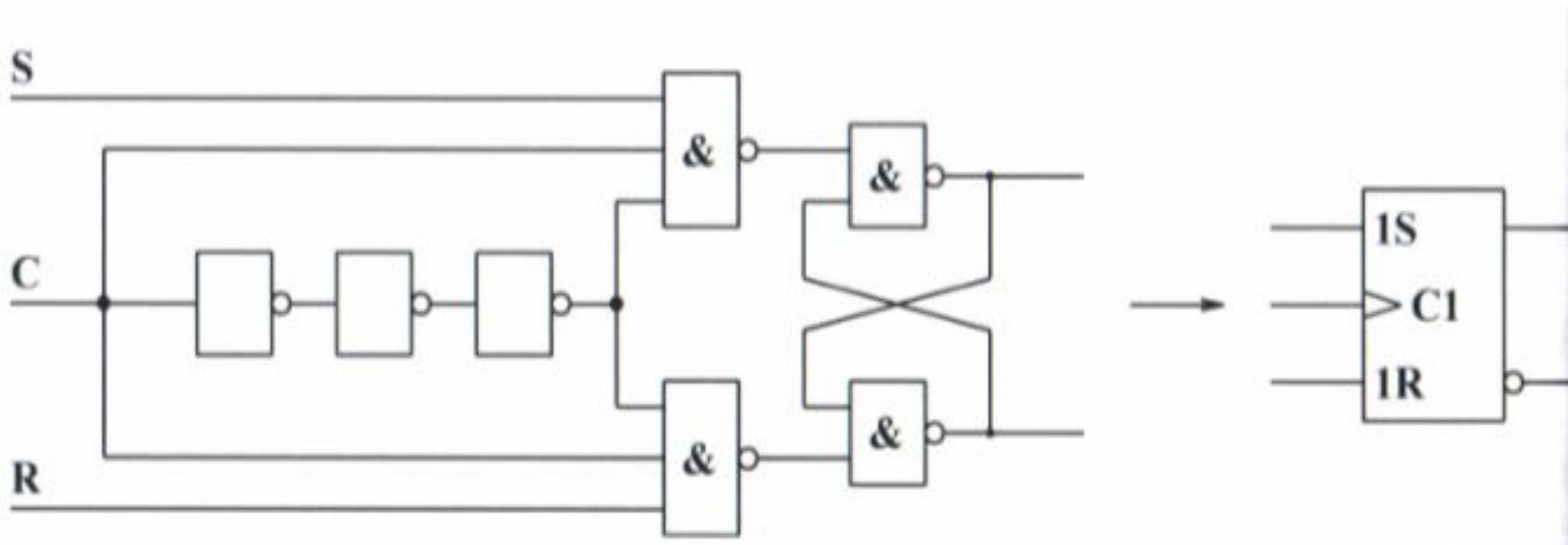
Flankengetriggertes RS FF

- Impulserzeugung durch Verzögerungskette
- Schaltung erzeugt am Ausgang E einen Impuls der Länge $3 t_{pd}$, ausgelöst durch die steigende Flanke des Eingangssignals A
- Die fallende Flanke von A bewirkt keine Änderung am Ausgang



Realisierung

- Der erzeugte Impuls ist gerade lang genug, um als Taktsignal für ein taktzustandsgesteuertes RS FF zu dienen



- Im Symbol wird die Flankentriggerung durch ein Dreieck beim Takteingang dargestellt
- Falls das FF auf die fallende Flanke triggern soll, wird das zusätzlich durch einen Negationskreis auf der Außenseite des Takteingangs gekennzeichnet

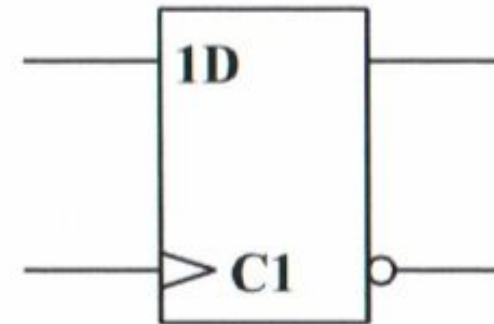
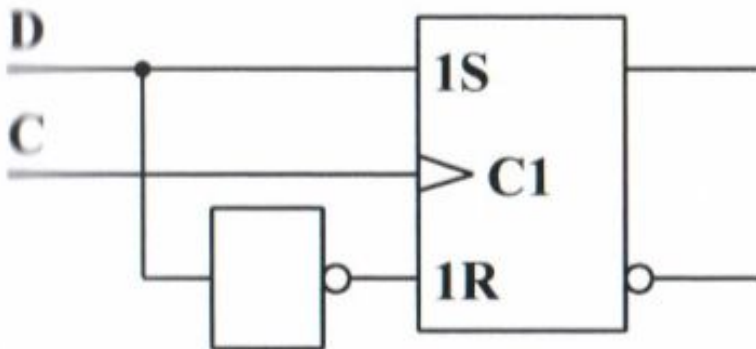
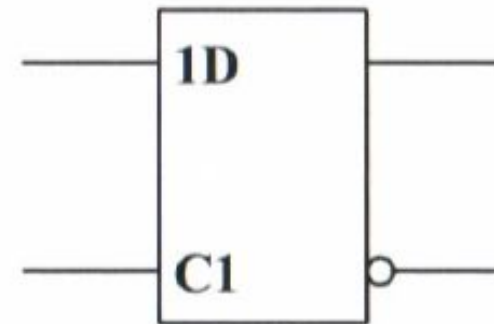
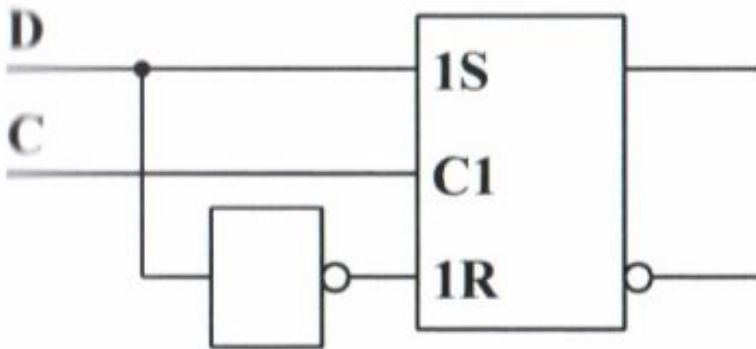
D FF

- Entscheidender Nachteil des RS FF
 - Eingangszustand $R = S = 1$ führt zu einem unsicheren Ausgangszustand
- Wie kann man garantieren, dass dieser Zustand nie eintritt?
 - Schaltungstechnische Maßnahme indem man $R = !S$ erzwingt
- Einschalten eine Inverters zwischen die Eingänge S und R
 - FF mit nur einem einzigen Dateneingang D

Realisierung D FF

- Oben: taktzustandsgesteuert
- Unten: flankengetriggert

- $Q_{n+1} = D_n$



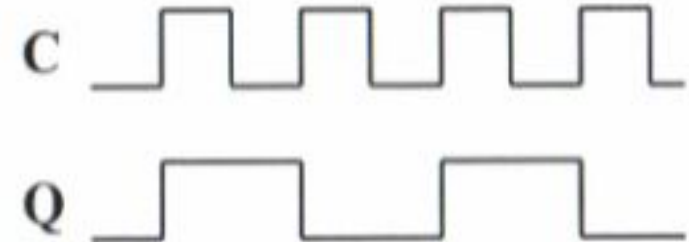
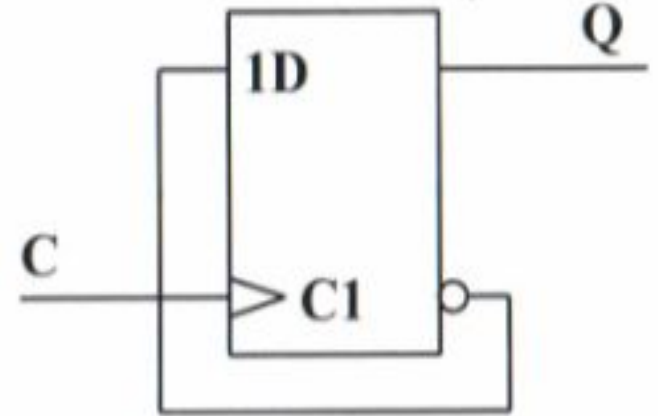
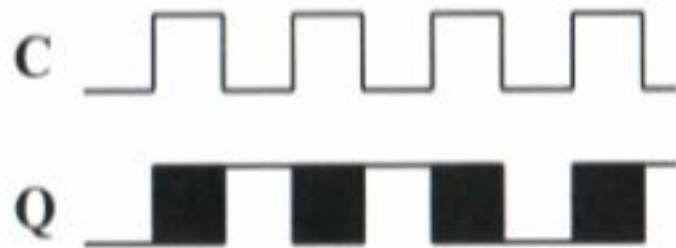
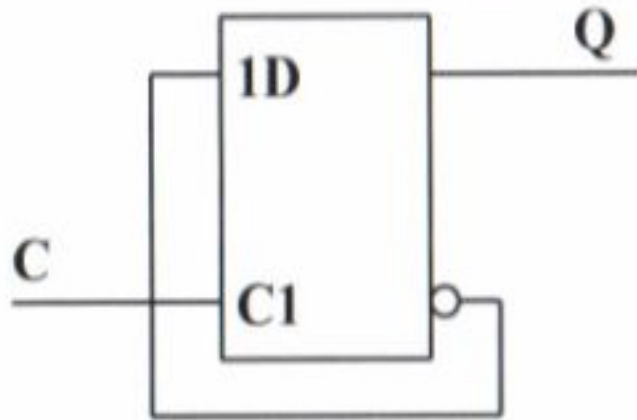
Zustand- und flankengetriggerte FF

- D FF existieren als
 - Taktzustandgesteuerte FF → D-Latch → transparent
 - Taktflankengesteuertes FF → → nicht transparent

Gruppenübung

- Ausgang !Q wird auf den D-Eingang zurückgeführt
- An dem Eingang C wird ein Taktsignal angelegt
- Erklären Sie bitte das Verhalten an Ausgang Q
 - Transparent
 - Nicht transparent

Lösung

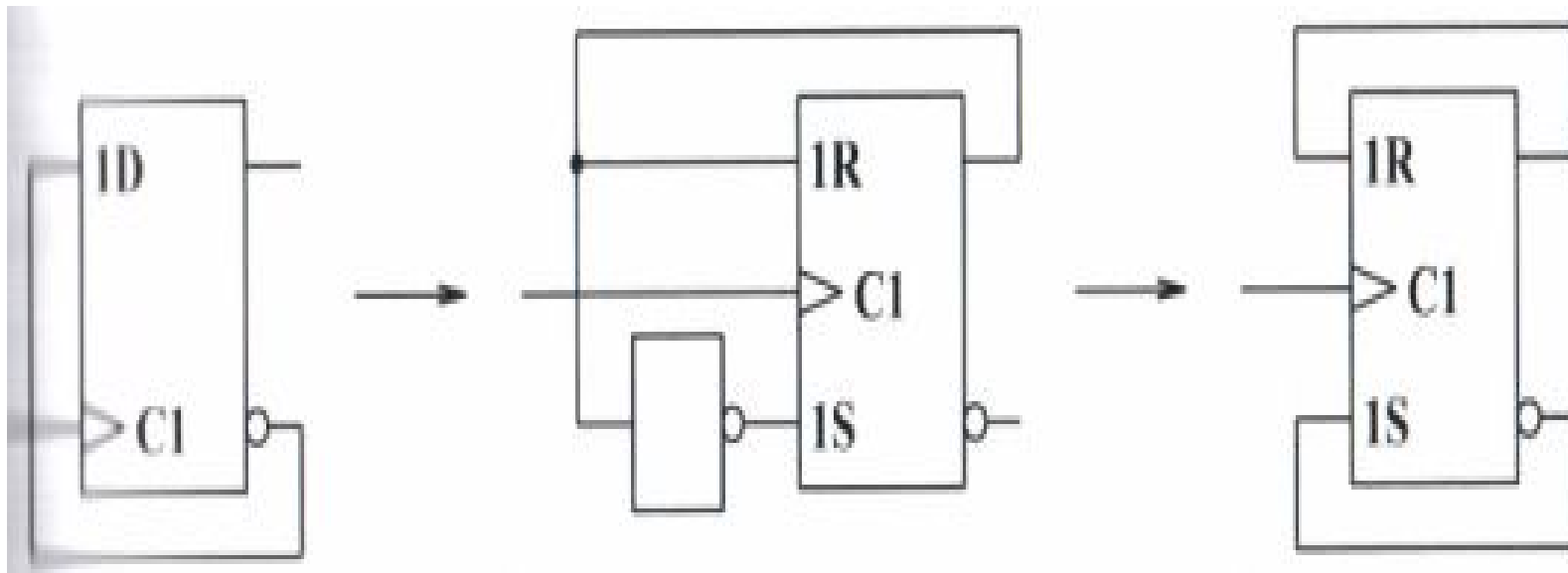


Lösung

- Beim taktzustandsgesteuerten D FF wirkt sich eine Änderung am D Eingang sofort auf den Ausgang aus, solange das Taktsignal $C = 1$
 - Ausgang Q des FF
 - oszilliert solange C gleich bleibt
 - Zufälliger Wert 0 oder 1 bleibt, sobald $C = 0$
- Flankengetriggertes D FF
 - Mit steigender Flanke des Taktsignals wird der Wert vom Eingang an den Ausgang geschrieben
 - Und das FF sofort wieder in den internen Speicherzustand gebracht
 - Erst bei der nächsten steigenden Taktflanke kann sich das Ausgangsdatum ändern

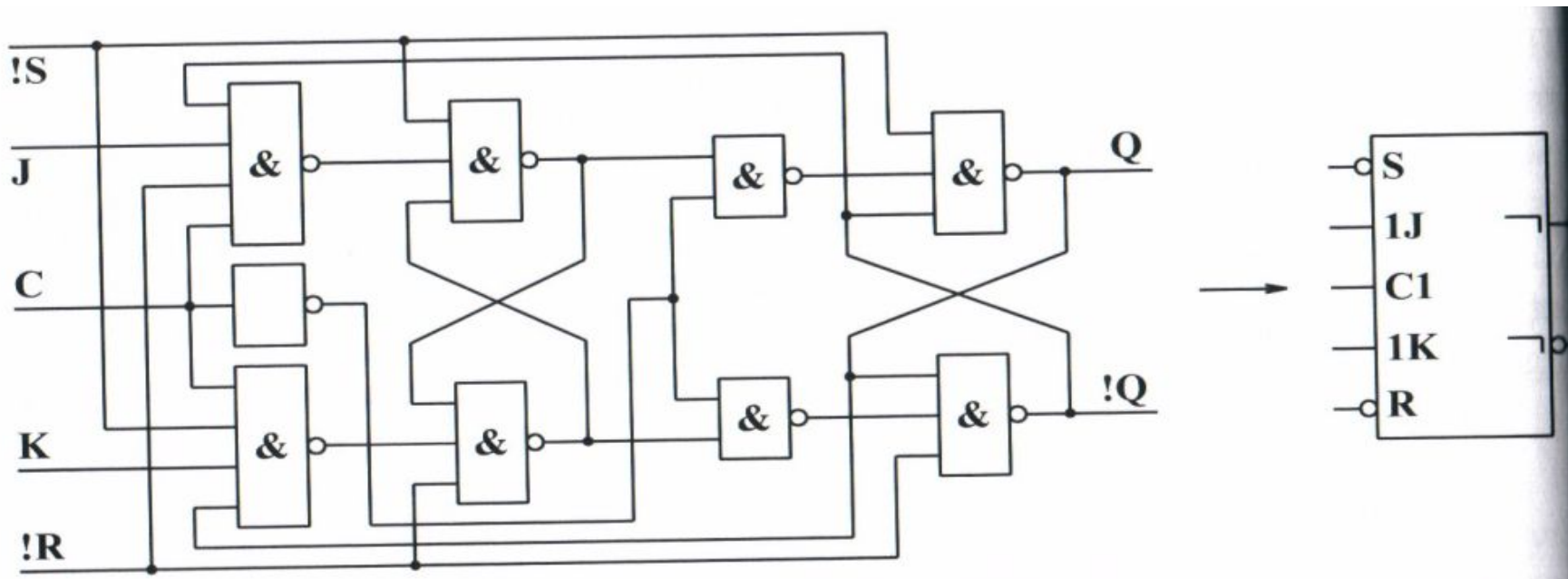
Alternative Form eines D FF

- Alternative Form eines D FF mit $D = !Q$



Asynchroner Setz- und Rücksetz-Eingang

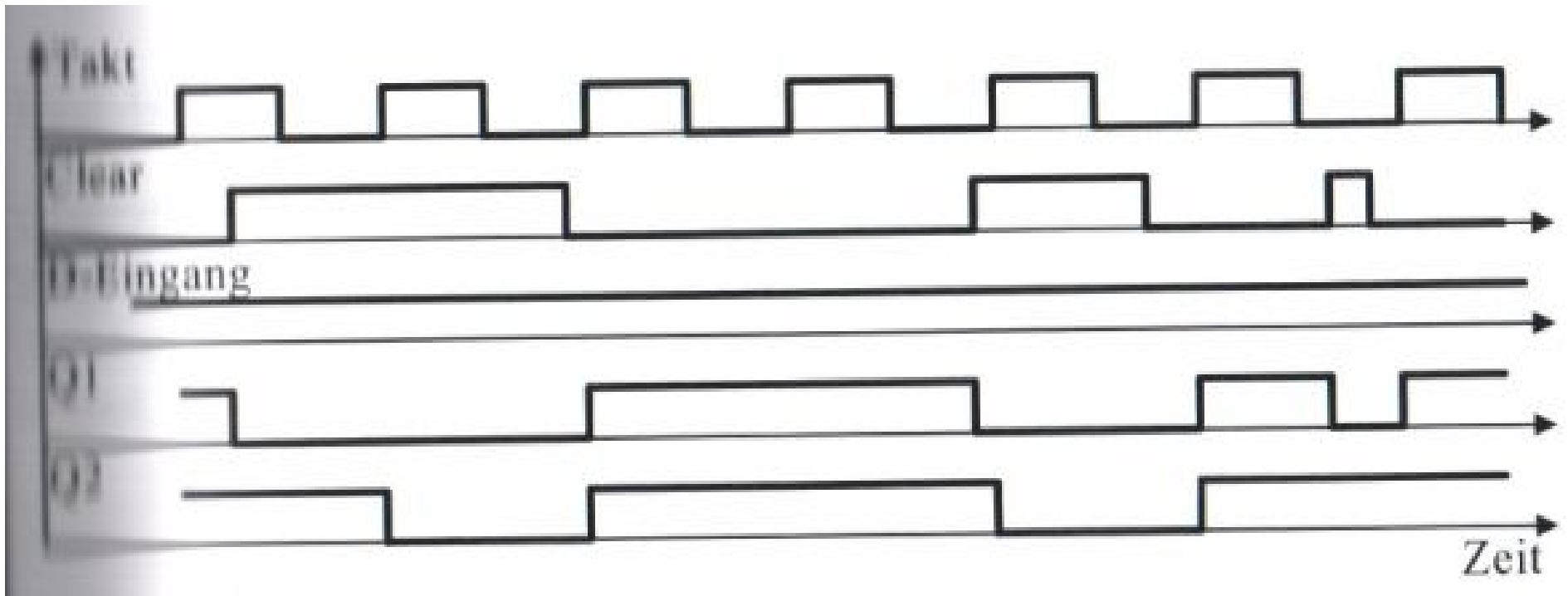
- Preset $\rightarrow Q = 1$
- Clear $\rightarrow Q = 0$



- Man erkennt, dass die Signale S bzw. R direkt auf die NAND-Glieder am Ausgang wirken, also unabhängig vom Takt sind

Synchroner Setz- und Rücksetz-Eingang

- Es gibt auch Anwendungen, die das Setzen und zurücksetzen eines FF erst bei aktivem Takt zulassen
 - Synchrone Initialisierung



- Asynchroner Clear Ausgang Q1
- Synchroner Clear Ausgang Q2

Takt (1/2)

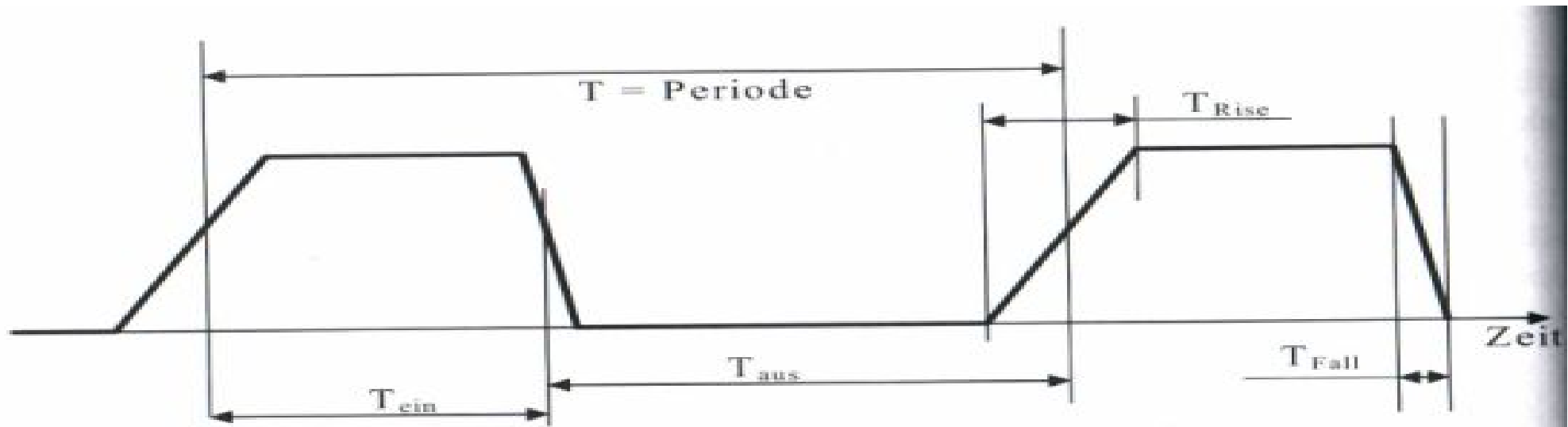
- Das Taktsignal oszilliert zwischen Logik 0 und Logik 1

$$DC = \frac{T_{EIN}}{T_{EIN} + T_{aus}}$$

- Wichtige Merkmale
 - Frequenz: Die Taktfrequenz beschreibt, wie oft das Signal sich pro Sekunde wiederholt
 - Periode: Die Taktperiode wird in Sekunden gemessen
 - sie ist die Umkehrung der Frequenz und setzt sich aus der Summe von Impulsdauer T_{EIN} und Pausendauer T_{AUS} zusammen
 - $T = T_{EIN} + T_{AUS}$
 - Tastgrad = Duty Cycle: Ein Takt besteht aus einem Teil, in dem das Signal auf Logik 1 ist T_{EIN} und einem Teil, in dem das Signal auf Logik 0 ist
 - der Duty Cycle beschreibt das Verhältnis

Takt (2/2)

- Wichtige Merkmale
 - Anstiegszeit (Rising Time): beschreibt die Geschwindigkeit des Wechsels von 1 → 0
 - Abfallzeit (Falling Time): beschreibt die Geschwindigkeit des Wechsels von 0 → 1



Design-Tool Vivado von AMD

Vivado

- Vivado ist ein Hardware Design Tool des FPGA Herstellers Xilinx, heute AMD
 - Es gibt aber durchaus weitere alternative FPGA Hersteller
 - In der Vorlesung nutzen wir die kostenfreie Vivado HLx Edition
- Download unter: <https://www.xilinx.com/products/design-tools/vivado.html>
 - Achten Sie bitte darauf, dass Sie keine kostenpflichtige Version nutzen
 - Man benötigt einen Account bei Xilinx bzw. AMD, um das Tool herunter zu laden
 - Sie können auch jedes andere beliebige VHDL Tool nutzen
- Unterstützte Betriebssysteme

Operating Systems

Xilinx® supports the following operating systems on x86 and x86-64 processor architectures.

Microsoft Windows Support

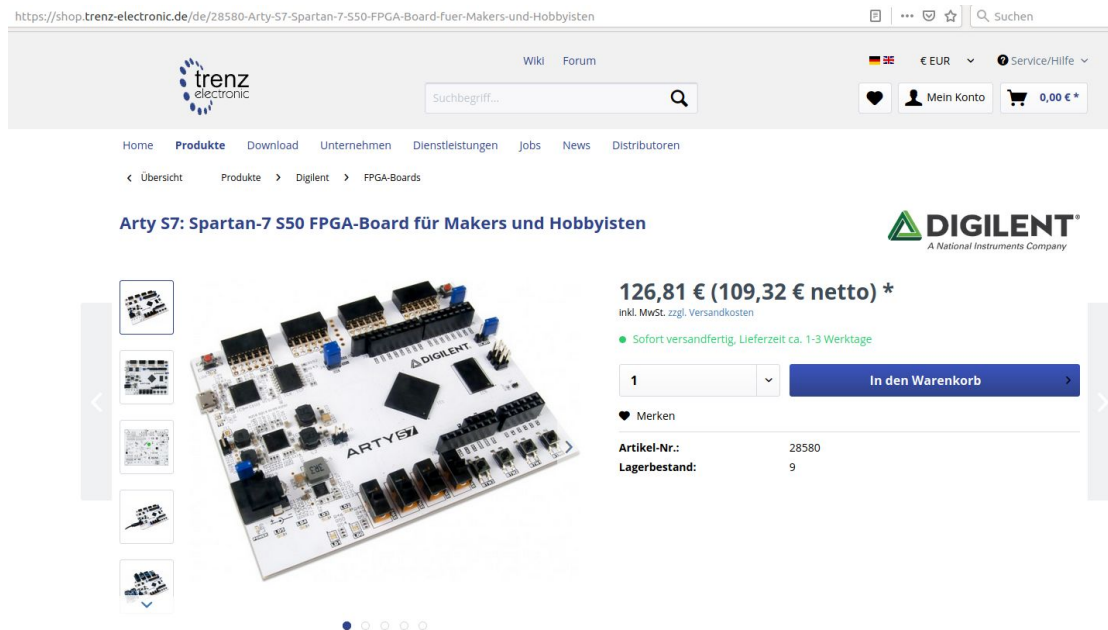
- Windows 7 SP1 Professional (64-bit), English/Japanese.
- Windows 10.0 Fall Creators update (version 1709), 10.0 Version 1803 (64-bit), English/Japanese.

Linux Support

- Red Hat Enterprise Workstation/Server 7.2, 7.3, and 7.4 (64-bit)
- Red Hat Enterprise Workstation 6.6, 6.7, 6.8, and 6.9 (64-bit)
- SUSE Linux Enterprise 11.4 and 12.3 (64-bit)
- CentOS 7.2, 7.3, and 7.4 (64-bit)
- CentOS 6.7, 6.8, and 6.9 (64-bit)
- Ubuntu Linux 16.04.3 LTS (64-bit)

Das ausgewählte FPGA und korrespondierendes Board (1/2)

- Es wird kein real existierendes FPGA für die Vorlesung benötigt
- Beispielsweise kann ein Spartan-7 S50 auf einem Arty S7 Board genutzt werden
<https://shop.trenz-electronic.de/de/28580-Arty-S7-Spartan-7-S50-FPGA-Board-fuer-Makers-und-Hobbyisten>
- Die mögliche Anschaffung ist aber absolut nicht notwendig, um der Vorlesung folgen zu können
 - nur, falls Sie gerne mit echter Hardware arbeiten möchten, dann ist ein echtes FPGA mit Board notwendig
- Es gibt auch alternative, kostengünstigere Boards – hier soll nur ein Beispiel gegeben werden
- Produkt-Video: https://www.youtube.com/watch?v=w_DIAbrQPeg



Das ausgewählte FPGA und korrespondierendes Board (2/2)

- Konkretes FPGA XC7S50-CSGA324 FPGA

Eigenschaften

- XC7S50-CSGA324 FPGA
- Interne Taktung über 450 MHz
- 1 MSPS On-chip Analog-zu-Digital Konverter (XADC)
- Programmierbar über JTAG und Quad-SPI Flash
- 256 MB DDR3L mit 16-Bit Bus @ 667 MHz
- 128 MB Quad-SPI Flash
- 100 MHz External Clock
- Vier Pmod-Anschlüsse
- Zwei Standard 12-Pin Pmod-Stecker
- Zwei High-speed 12-Pin Pmod-Stecker
- USB-UART Bridge
- Arduino/chipKIT Shield Stecker
- Mit USB oder jeder anderen 7V-15 V externen Stromquellen betrieben.
- USB-JTAG Programmierung
- Vier Schalter
- Vier Buttons, 1 Board Reset-Button, 1 FPGA Reset-Button (insgesamt sechs)
- Vier grüne LEDs, 2 RGB LEDs (insgesamt sechs)

Zielanwendungen

- Sensorfusion
- Motorsteuerung - Einzelachse
- Bildverarbeitung
- Allgemeiner Zweck

Vivado starten

- Vivado ist ein Hardware Design Tool des FPGA Herstellers Xilinx
 - Es gibt aber durchaus weitere alternative FPGA Hersteller
 - In der Vorlesung nutzen wir die kostenfreie Vivado HLx Edition

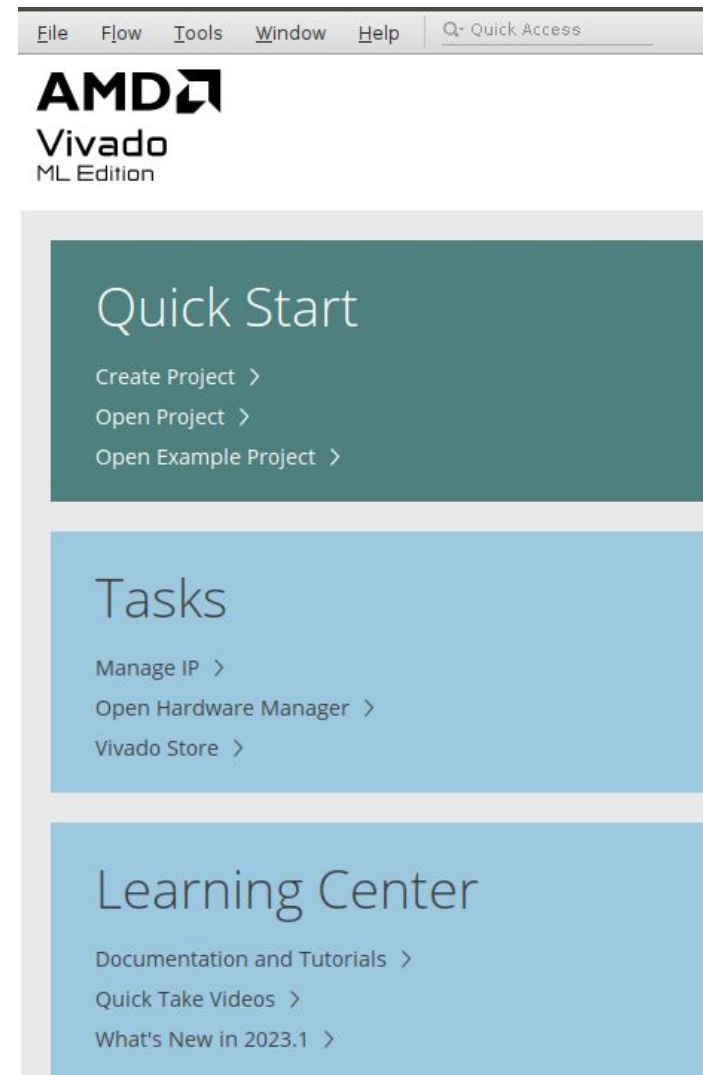
- Download unter: <https://www.xilinx.com/products/design-tools/vivado.html>

```
:-$ vivado

***** Vivado v2020.1.1 (64-bit)
**** SW Build 2960000 on Wed Aug  5 22:57:21 MDT 2020
**** IP Build 2956692 on Thu Aug  6 01:41:30 MDT 2020
** Copyright 1986-2020 Xilinx, Inc. All Rights Reserved.

start_gui
```

- Es sollten folgende Pakete installiert sein
`sudo apt-get install libncurses5 libtinfo5`
- Die fehlenden Pakete führen dazu, dass die Installation im 3. Schritt hängen bleibt und nicht zu Ende geführt wird



Neues Projekt einrichten – Create Project

File Flow Tools Window Help Q- Quick Access



Quick Start

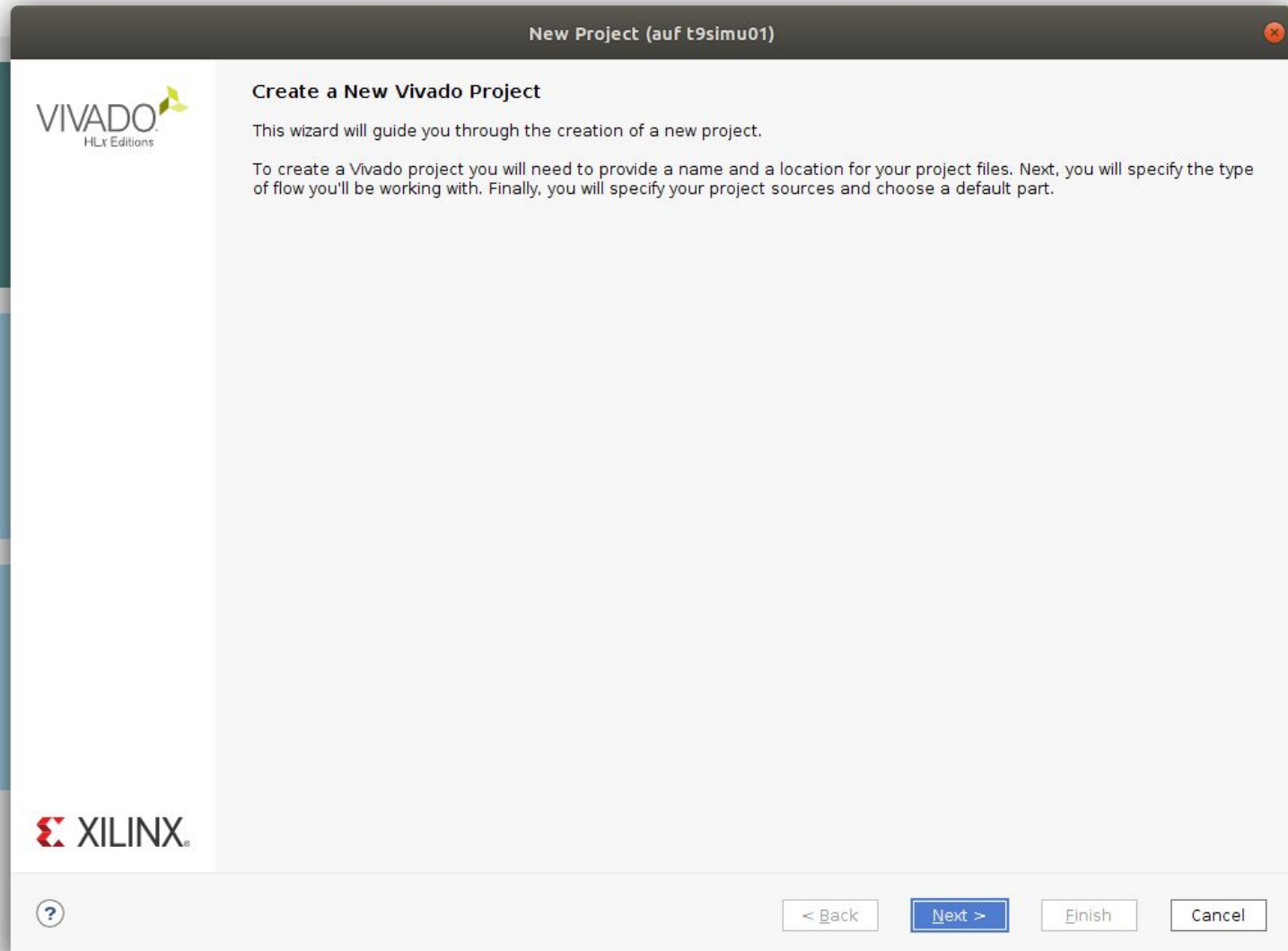
[Create Project >](#)
[Open Project >](#)
[Open Example Project >](#)

Tasks

[Manage IP >](#)
[Open Hardware Manager >](#)
[XHub Stores >](#)

Learning Center

[Documentation and Tutorials >](#)
[Quick Take Videos >](#)
[Release Notes Guide >](#)



XOR Projekt einrichten

- Als Testprojekt bauen wir ein XOR Design auf
- Verzeichnis erzeugen
- Projektart als RTL aus wählen – Register Transfer Level

Vorlesung/VHDL/Codes/VHDL/XOR

Project Type

Specify the type of project to create.



- ☒ **RTL Project**
You will be able to add sources, create block designs in IP Integrator, generate IP, run RTL analysis, synthesis, implementation, design planning and analysis.
☐ Do not specify sources at this time
- ☐ **Post-synthesis Project**
You will be able to add sources, view device resources, run design analysis, planning and implementation.
☐ Do not specify sources at this time
- ☐ **I/O Planning Project**
Do not specify design sources. You will be able to view part/package resources.
- ☐ **Imported Project**
Create a Vivado project from a Synplify, XST or ISE Project File.
- ☐ **Example Project**
Create a new Vivado project from a predefined template.



< Back

Next >

Finish

Cancel

HDL Sprache auswählen - VHDL

- Unten VHDL als Hardware Beschreibungssprache auswählen

New Project (auf t9sinu01)

Add Sources

Specify HDL, netlist, Block Design, and IP files, or directories containing those files, to add to your project. Create a new source file on disk and add it to your project. You can also add and create sources later.

Use Add Files, Add Directories or Create File buttons below

☐ Scan and add RTL include files into project
☐ Copy sources into project
☒ Add sources from subdirectories

Target language: **VHDL** Simulator language: **VHDL**

FPGA Baustein auswählen

- In der Vorlesung wird der Baustein XC7S50-CSGA324 verwendet

Default Part

Choose a default Xilinx part or board for your project.



Parts | Boards

[Reset All Filters](#)

Category:

Package:

Temperature:

Family:

Speed:

Static power:

Search:

Part	I/O Pin Count	Available IOBs	LUT Elements	FlipFlops	Block RAMs	Ultra RAMs	DSPs	Gb Transceivers	GTPE2 Transceivers	GTXE2 Transceivers	GTHE2 Trans
xc7s25ftgb196-2	196	100	14600	29200	45	0	80	0	0	0	0
xc7s25ftgb196-1	196	100	14600	29200	45	0	80	0	0	0	0
xc7s25ftgb196-1IL	196	100	14600	29200	45	0	80	0	0	0	0
xc7s25ftgb196-1Q	196	100	14600	29200	45	0	80	0	0	0	0
xc7s50csga324-2	324	210	32600	65200	75	0	120	0	0	0	0
xc7s50csga324-1	324	210	32600	65200	75	0	120	0	0	0	0
xc7s50csga324-1IL	324	210	32600	65200	75	0	120	0	0	0	0
xc7s50csga324-1Q	324	210	32600	65200	75	0	120	0	0	0	0
xc7s50fgga484-2	484	250	32600	65200	75	0	120	0	0	0	0
xc7s50fgga484-1	484	250	32600	65200	75	0	120	0	0	0	0
xc7s50fgga484-1IL	484	250	32600	65200	75	0	120	0	0	0	0
xc7s50fgga484-1Q	484	250	32600	65200	75	0	120	0	0	0	0
xc7s50ftgb196-2	196	100	32600	65200	75	0	120	0	0	0	0
xc7s50ftgb196-1	196	100	32600	65200	75	0	120	0	0	0	0
xc7s50ftgb196-1IL	196	100	32600	65200	75	0	120	0	0	0	0
xc7s50ftgb196-1Q	196	100	32600	65200	75	0	120	0	0	0	0
xc7s75fgga484-2	484	338	48000	96000	90	0	140	0	0	0	0
xc7s75fgga484-1	484	338	48000	96000	90	0	140	0	0	0	0
xc7s75fgga484-1IL	484	338	48000	96000	90	0	140	0	0	0	0
xc7s75fgga484-1Q	484	338	48000	96000	90	0	140	0	0	0	0

?

< Back

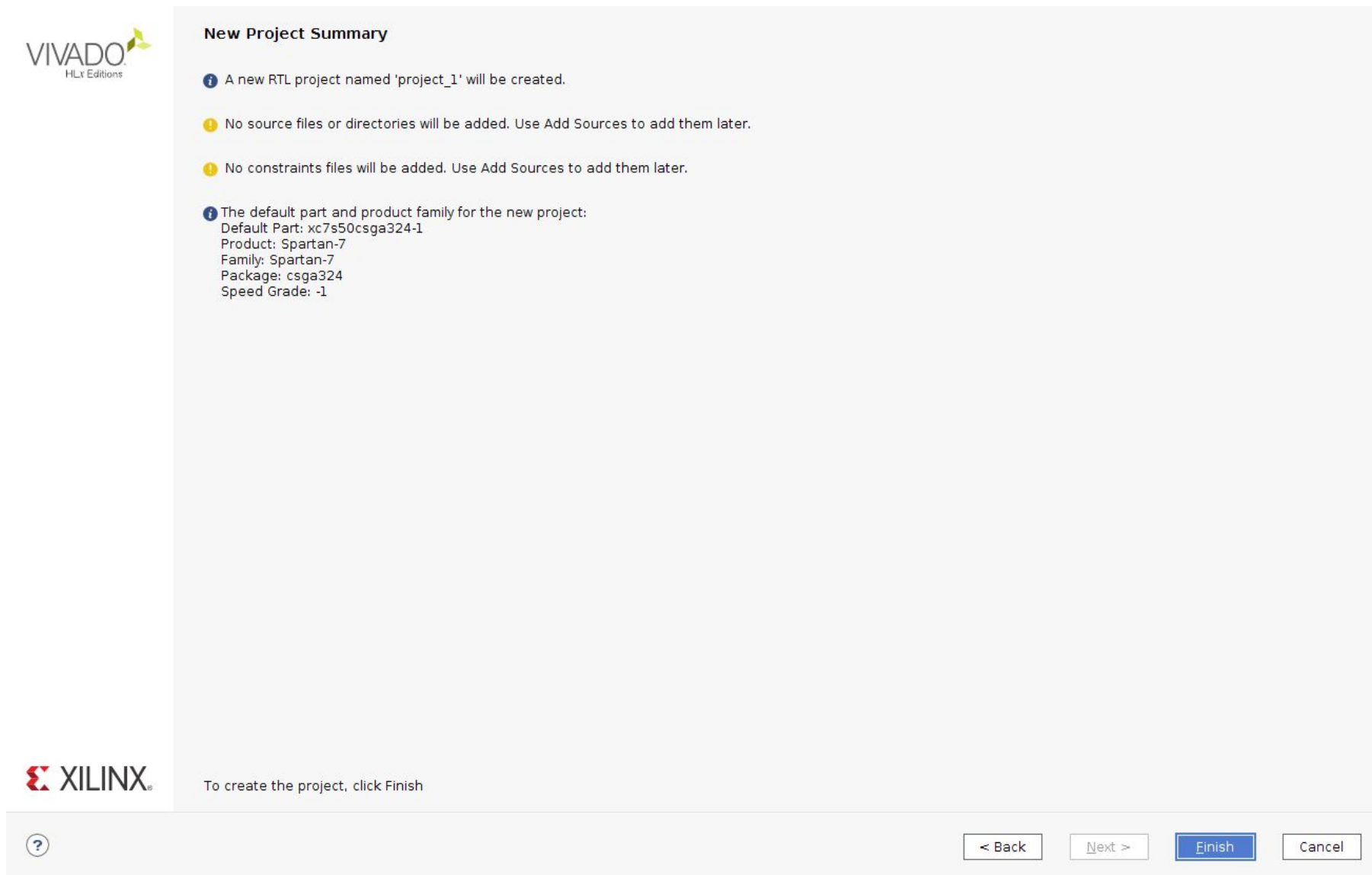
Next >

Finish

Cancel

Projekt bestätigen

- Das Projekt kann erzeugt werden



Projekt ist aktiv

- Das Projekt kann erzeugt werden

The screenshot displays the Vivado Project Manager interface for a project named 'project_1'. The left sidebar contains a 'Flow Navigator' with sections for PROJECT MANAGER, IP INTEGRATOR, SIMULATION, RTL ANALYSIS, SYNTHESIS, IMPLEMENTATION, and PROGRAM AND DEBUG. The main area is divided into several panels: 'Sources' (showing Design, Simulation, and Utility sources), 'Properties' (empty), 'Project Summary' (Overview and Dashboard tabs), 'Synthesis', 'Implementation', 'DRC Violations', 'Timing', 'Utilization', and 'Power'. The 'Project Summary' panel shows details for 'project_1', including its location, product family (Spartan-7), and target language (VHDL). The 'Synthesis' and 'Implementation' panels show that no synthesis or implementation has been performed yet. The 'DRC Violations' and 'Timing' panels provide links to run implementation and view results. The 'Utilization' and 'Power' panels are currently empty.

Project Summary

Overview | Dashboard

Settings [Edit](#)

Project name: project_1
Project location: /home/edmund/ /Hochschule-Bochum/Vorlesung/VHDL/Codes/VHDL/XOR/project_1
Product family: Spartan-7
Project part: xc7s50csga324-1
Top module name: Not defined
Target language: VHDL
Simulator language: VHDL

Synthesis

Status: Not started
Messages: No errors or warnings
Part: xc7s50csga324-1
Strategy: Vivado Synthesis Defaults
Report Strategy: Vivado Synthesis Default Reports
Incremental synthesis: None

Implementation

Status: Not started
Messages: No errors or warnings
Part: xc7s50csga324-1
Strategy: Vivado Implementation Defaults
Report Strategy: Vivado Implementation Default Reports
Incremental implementation: None

DRC Violations

[Run Implementation](#) to see DRC results

Timing

[Run Implementation](#) to see timing results

Utilization

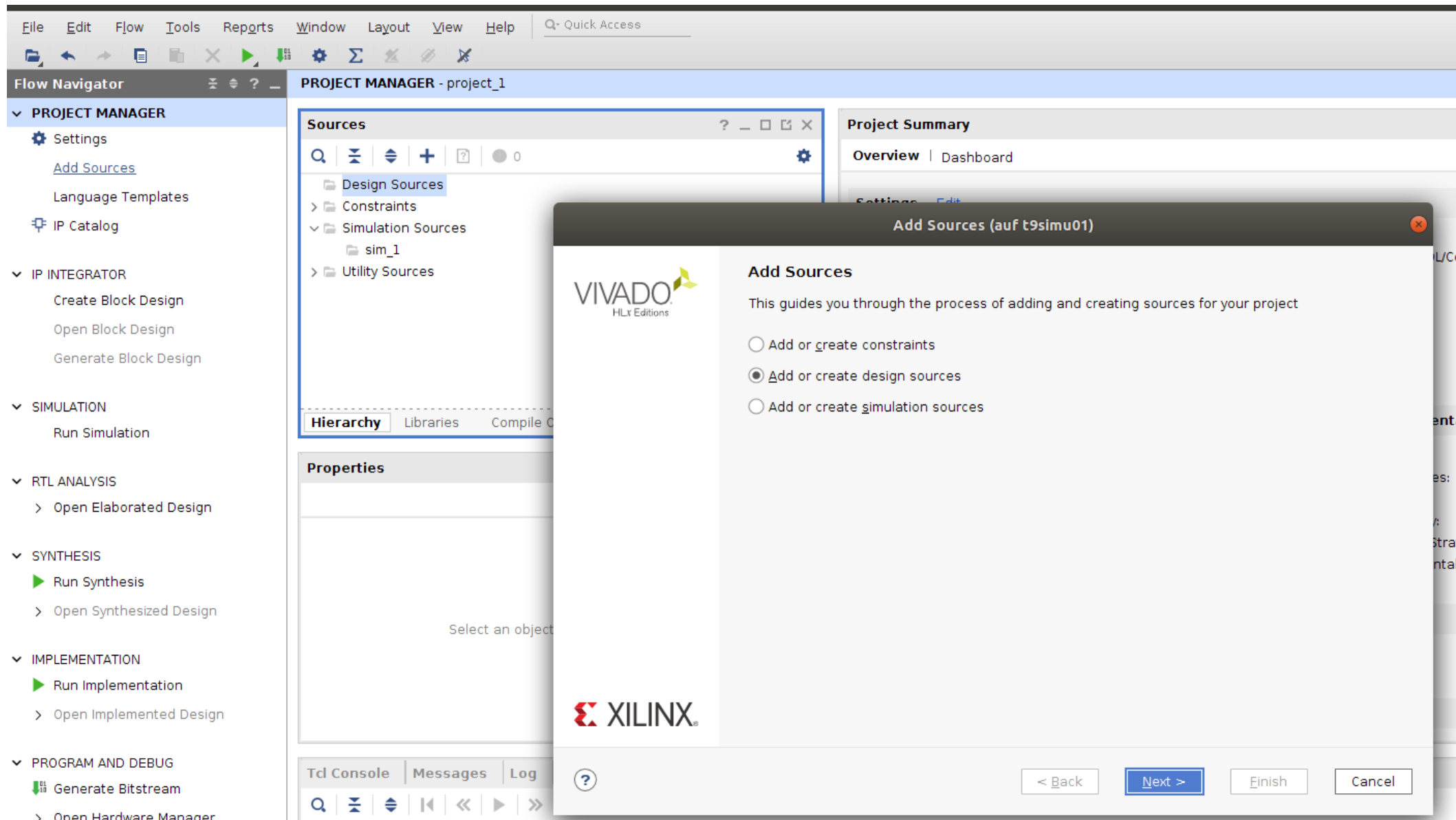
Power

Design Runs

Name	Constraints	Status	WNS	TNS	WHS	THS	TPWS	Total Power	Failed Routes	LUT	FF	BRAM	URAM	DSP	Start	Elapsed	Run Strategy	Report Strategy
synth_1	constrs_1	Not started															Vivado Synthesis Defaults (Vivado Synthesis 2020)	Vivado Synthesis Defau
impl_1	constrs_1	Not started															Vivado Implementation Defaults (Vivado Implementation 2020)	Vivado Implementation

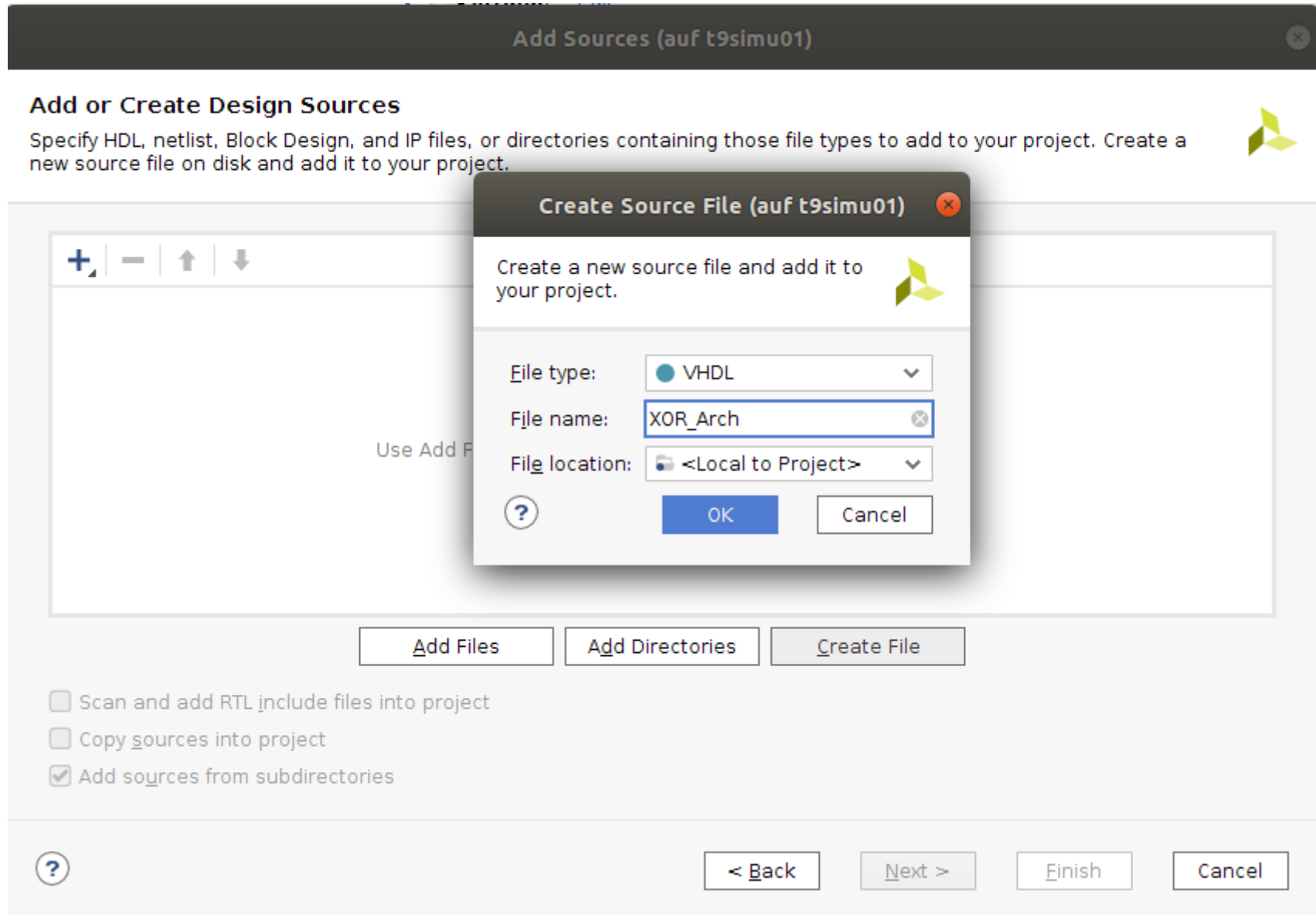
Dateien hinzufügen

- *Add Source* auswählen – dann *Design Sources* hinzufügen



Datei benennen

- XOR_Arch steht für Architecture, obwohl die Entity auch mit in der Datei aufgestellt wird



Entity definieren

- Es gibt drei Ports
 - A, B sind In-Ports
 - C ist eine Out-Port

Define a module and specify I/O Ports to add to your source file.
For each port specified:
MSB and LSB values will be ignored unless its Bus column is checked.
Ports with blank names will not be written.



Module Definition

Entity name:

Architecture name:

I/O Port Definitions

Port Name	Direction	Bus	MSB	LSB
A	in	☐	0	0
B	in	☐	0	0
C	out	☐	0	0

Datei editieren

The screenshot displays the Xilinx Vivado IDE interface. The top menu bar includes File, Edit, Flow, Tools, Reports, Window, Layout, View, and Help. The left-hand side contains the Project Manager and Flow Navigator. The Project Manager shows the project hierarchy for 'project_1', including Sources (Design Sources, Constraints, Simulation Sources, Utility Sources), IP Integrator, Simulation, RTL Analysis, Synthesis, Implementation, and Program and Debug. The Flow Navigator shows the project flow, including Run Synthesis, Run Implementation, and Open Hardware Manager. The central editor window displays the 'XOR_Arch.vhd' file, which is a VHDL file. The file content includes a header section with metadata (Company, Engineer, Create Date, Design Name, Module Name, Project Name, Target Devices, Tool Versions, Description) and a library declaration section (library IEEE, use IEEE.STD_LOGIC_1164.ALL;). The bottom Design Runs console shows the project runs, including synth_1 and impl_1, with columns for Name, Constraints, Status, WNS, TNS, WHS, THS, TPWS, Total Power, Failed Routes, LUT, FF, BRAM, URAM, DSP, Start, Elapsed, Run Strategy, and Report Strategy.

Project Manager - project_1

Sources

- Design Sources (1)
 - XOR_Arch_Entity(Behavioral) (XOR_Arch.vhd)
- Constraints
- Simulation Sources (1)
 - sim_1 (1)
- Utility Sources

Source File Properties

XOR_Arch.vhd

- ☒ Enabled
- Location: /home/edmund/Task9-Docu/Hochschule-Bochum/Vorlesun...
- Type: VHDL
- Library: xil_defaultlib
- Size: 1.0 KB
- Modified: Today at 18:28:26 PM

General Properties

Design Runs

Name	Constraints	Status	WNS	TNS	WHS	THS	TPWS	Total Power	Failed Routes	LUT	FF	BRAM	URAM	DSP	Start	Elapsed	Run Strategy	Report Strategy
synth_1	constrs_1	Not started															Vivado Synthesis Defaults (Vivado Synthesis 2020)	Vivado Synthesis Default
impl_1	constrs_1	Not started															Vivado Implementation Defaults (Vivado Implementation 2020)	Vivado Implementation Default

Source Code

Bibliotheken

Entity

Architecture

Prozess Sensitivity Liste

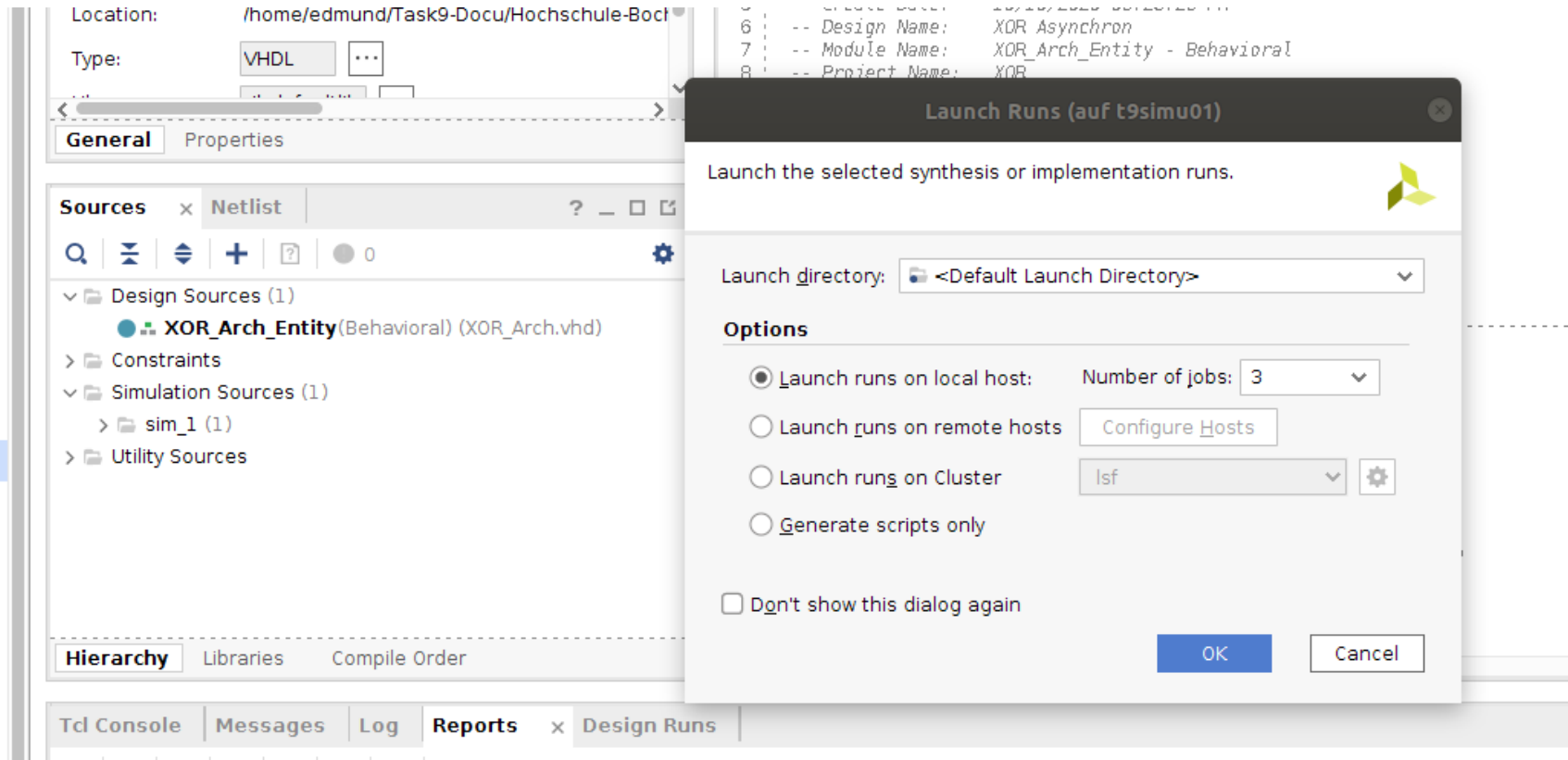
```
5  -- Create Date:      10/10/2020 06:28:26 PM
6  -- Design Name:     XOR Asynchron
7  -- Module Name:     XOR_Arch_Entity - Behavioral
8  -- Project Name:    XOR
9  -- Target Devices:  SPartan 7 XC7S50
10 -- Tool Versions:   Vivado 2020
11 -- Description:     XOR Funktion als asynchroner Prozess
12 --
13 -- Dependencies:
14 --
15 -- Revision:
16 -- Revision 0.01 - File Created
17 -- Additional Comments:
18 --
19 -----
20
21
22 library IEEE;
23 use IEEE.STD_LOGIC_1164.ALL;
24
25 -- Uncomment the following library declaration if using
26 -- arithmetic functions with Signed or Unsigned values
27 --use IEEE.NUMERIC_STD.ALL;
28
29 -- Uncomment the following library declaration if instantiating
30 -- any Xilinx leaf cells in this code.
31 --library UNISIM;
32 --use UNISIM.VComponents.all;
33
34 entity XOR_Arch_Entity is
35     Port ( A : in  STD_LOGIC;
36           B : in  STD_LOGIC;
37           C : out STD_LOGIC);
38 end XOR_Arch_Entity;
39
40 architecture Behavioral of XOR_Arch_Entity is
41
42 begin
43
44     P1: process(A,B)
45     begin
46
47         C <= A xor B;
48
49     end Process P1;
50
51
52 end Behavioral;
```

Signalzuweisung

Design Synthese starten

Start der Synthese

- ✓ IP INTEGRATOR
 - Create Block Design
 - Open Block Design
 - Generate Block Design
- ✓ SIMULATION
 - Run Simulation
- ✓ RTL ANALYSIS
 - > Open Elaborated Design
- ✓ **SYNTHESIS**
 - ▶ Run Synthesis
- ✓ Open Synthesized Design
 - Constraints Wizard
 - Edit Timing Constraints
 - ⚙ Set Up Debug
 - 🕒 Report Timing Summary
 - Report Clock Networks



Design Synthese öffnen

Open Block Design

Generate Block Design

SIMULATION

Run Simulation

RTL ANALYSIS

Open Elaborated Design

SYNTHESIS

Run Synthesis

Open Synthesized Design

Constraints Wizard

Edit Timing Constraints

Set Up Debug

General

Properties

Sources

Netlist

Design Sources (1)

XOR_Arch_Entity(Behavioral) (XOR_Arch.vhd)

Constraints

Simulation Sources (1)

sim_1 (1)

Utility Sources

Hierarchy

Libraries

Compile Order

Synthesis Completed (auf t9simu01)

Synthesis successfully completed.

Next

Run Implementation

☒ Open Synthesized Design

View Reports

Don't show this dialog again

OK

Cancel

10 -- Tool Versions: Vivado 2020

11 -- Description: XOR Funktion als asynchroner Proze

12

13

14

15

16

17

18

19

20

21

22 lib

23 use

24

25

26

27 --US

28

29

30

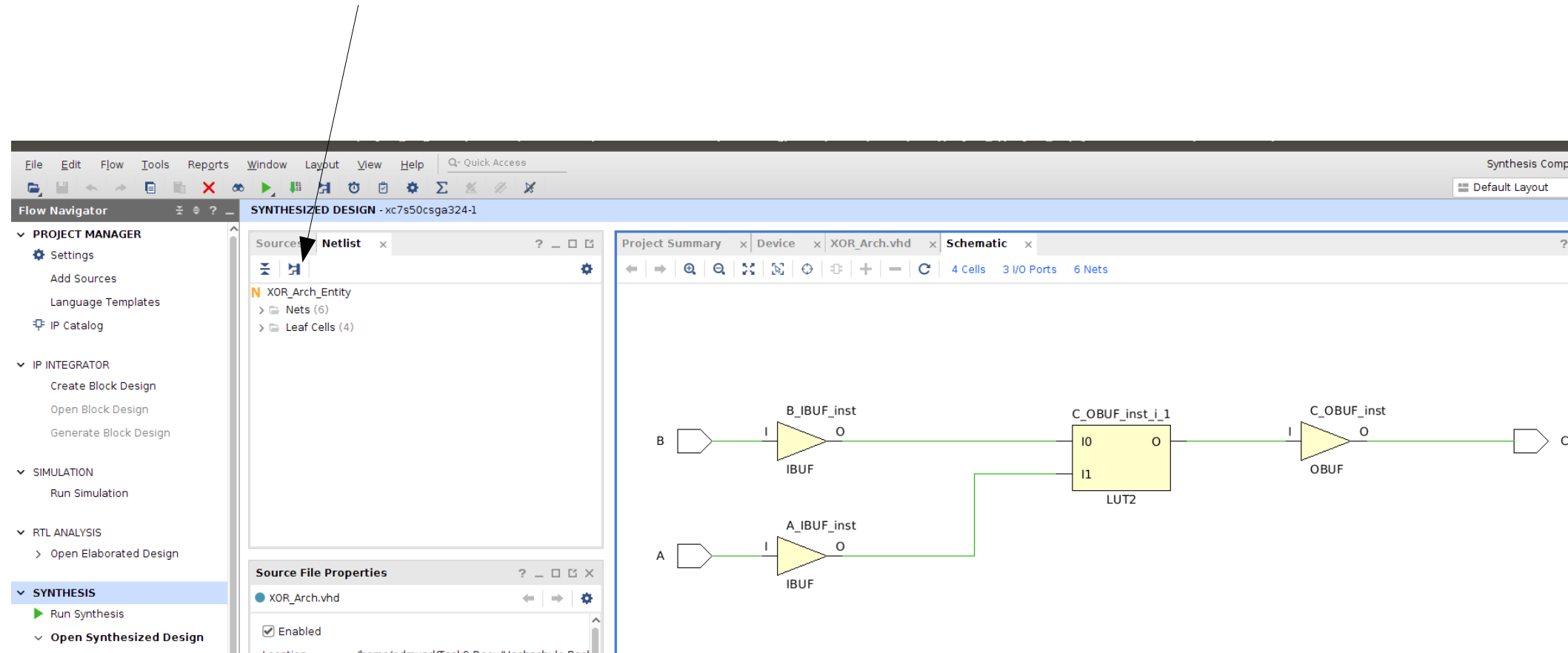
31 --I

32 --USE UNISIM.VCOMPONENTS.BC

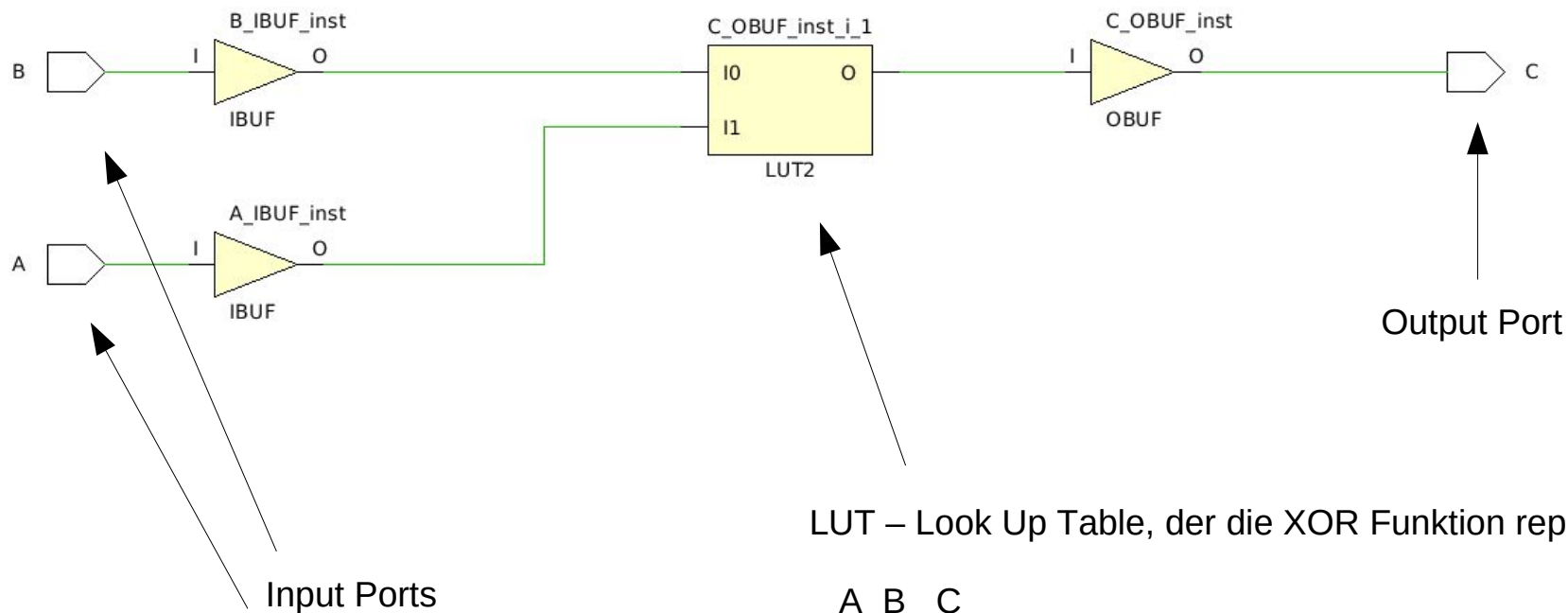
33

Design Schematic anschauen - F4 drücken

Um den synthetisierten Schematic anzusehen, hier klicken oder F4 drücken



Design Schematic verstehen



LUT – Look Up Table, der die XOR Funktion repräsentiert

A	B	C
0	0	0
0	1	1
1	0	1
1	1	0

Iterativer Design Ablauf

- Jetzt beginnt ein Kreislauf aus
 - Neue VHDL Funktionalitäten integrieren
 - Simulation und Überprüfung auf korrekte Funktionalität
 - Synthetisieren
 - Laufzeiten und Gatterkomplexität verstehen und analysieren
 - VHDL Code optimieren oder direkt von vorne beginnen und neue VHDL Funktionalitäten integrieren

Kapitel 3

VDHL Basics,

reservierte Wörter in VHDL

Entity, Architecture, Reset Strategie

Prozesse, Datentypen,

Variablen, Signale, Packages

Warum entstand VHDL?

- ASIC Schaltungen wurden in den 70iger Jahren per Hand entworfen → Layout gezeichnet
- Komplexität wurde dabei immer größer, sodass die Entwicklung digitaler Systeme zu weiten Teilen nur noch mit Computerunterstützung durchführbar ist
- Zur Beschreibung von Hardware existieren Hardwarebeschreibungssprachen, wie etwa
 - VHDL
 - Verilogdie Architektur, Struktur und Verhalten von digitalen Systemen beschreiben

Was ist VHDL?

- Very High Speed Integrated Circuit Hardware Description Language
- Spezifikation: Anfang der 80er Jahre
- Standardisierung: 1987, 1993 von IEEE
- Wichtige Schritte des Hardwaredesign-Prozesses, die unterstützt werden
 - Simulation und Verifikation des modellierten Systems
 - Optimierung
 - Synthese der beschriebenen Hardware für eine bestimmte Hardware Technologie

Reservierte VHDL Wörter (1/7)

- **abs** operator, absolute value of right operand. No () needed.
- **access** used to define an access type, pointer
- **after** specifies a time after NOW
- **alias** create another name for an existing identifier
- **all** dereferences what precedes the .all
- **and** operator, logical "and" of left and right operands
- **architecture** a secondary design unit
- **array** used to define an array, vector or matrix
- **assert** used to have a program check on itself
- **attribute** used to declare attribute functions
- **begin** start of a begin end pair
- **block** start of a block structure
- **body** designates a procedure body rather than declaration
- **buffer** a mode of a signal, holds a value
- **bus** a mode of a signal, can have multiple drivers
- 96• **case** part of a case statement

Reservierte VHDL Wörter (2/7)

- component starts the definition of a component
- configuration a primary design unit
- constant declares an identifier to be read only
- disconnect signal driver condition
- downto middle of a range 31 downto 0
- else part of "if" statement, if cond then ... else ... end if;
- elsif part of "if" statement, if cond then ... elsif cond ...
- end part of many statements, may be followed by word and id
- entity a primary design unit
- exit sequential statement, used in loops
- file used to declare a file type
- for start of a for type loop statement
- function starts declaration and body of a function
- generate make copies, possibly using a parameter
- generic introduces generic part of a declaration
- 97• group collection of types that can get an attribute

Reservierte VHDL Wörter (3/7)

- guarded causes a wait until a signal changes from False to True
- if used in "if" statements
- impure an impure function is assumed to have side effects
- in indicates a parameter in only input, not changed
- inertial signal characteristic, holds a value
- inout indicates a parameter is used and computed in and out
- is used as a connective in various statements
- label used in attribute statement as entity specification
- library context clause, designates a simple library name
- linkage a mode for a port, used like buffer and inout
- literal used in attribute statement as entity specification
- loop sequential statement, loop ... end loop;
- map used to map actual parameters, as in port map
- mod operator, left operand modulo right operand
- nand operator, "nand" of left and right operands
- 98• new allocates memory and returns access pointer

Reservierte VHDL Wörter (4/7)

- next sequential statement, used in loops
- nor operator, "nor" of left and right operands
- not operator, complement of right operand
- null sequential statement and a value
- of used in type declarations, of Real ;
- on used as a connective in various statements
- open initial file characteristic
- or operator, logical "or" of left and right operands
- others fill in missing, possibly all, data
- out indicates a parameter is computed and output
- package a design unit, also package body
- port interface definition, also port map
- postponed make process wait for all non postponed process to suspend
- procedure typical programming procedure
- process sequential or concurrent code to be executed
- 99• pure a pure function may not have side effects

Reservierte VHDL Wörter (5/7)

- range used in type definitions, range 1 to 10;
- record used to define a new record type
- register signal parameter modifier
- reject clause in delay mechanism, followed by a time
- rem operator, remainder of left operand divided by right op
- report statement and clause in assert statement, string output
- return statement in procedure or function
- rol operator, left operand rotated left by right operand
- ror operator, left operand rotated right by right operand
- select used in selected signal assignment statement
- severity used in assertion and reporting, followed by a severity
- signal declaration that an object is a signal
- shared used to declare shared objects
- sla operator, left operand shifted left arithmetic by right op
- sll operator, left operand shifted left logical by right op
- sra operator, left operand shifted right arithmetic by right

Reservierte VHDL Wörter (6/7)

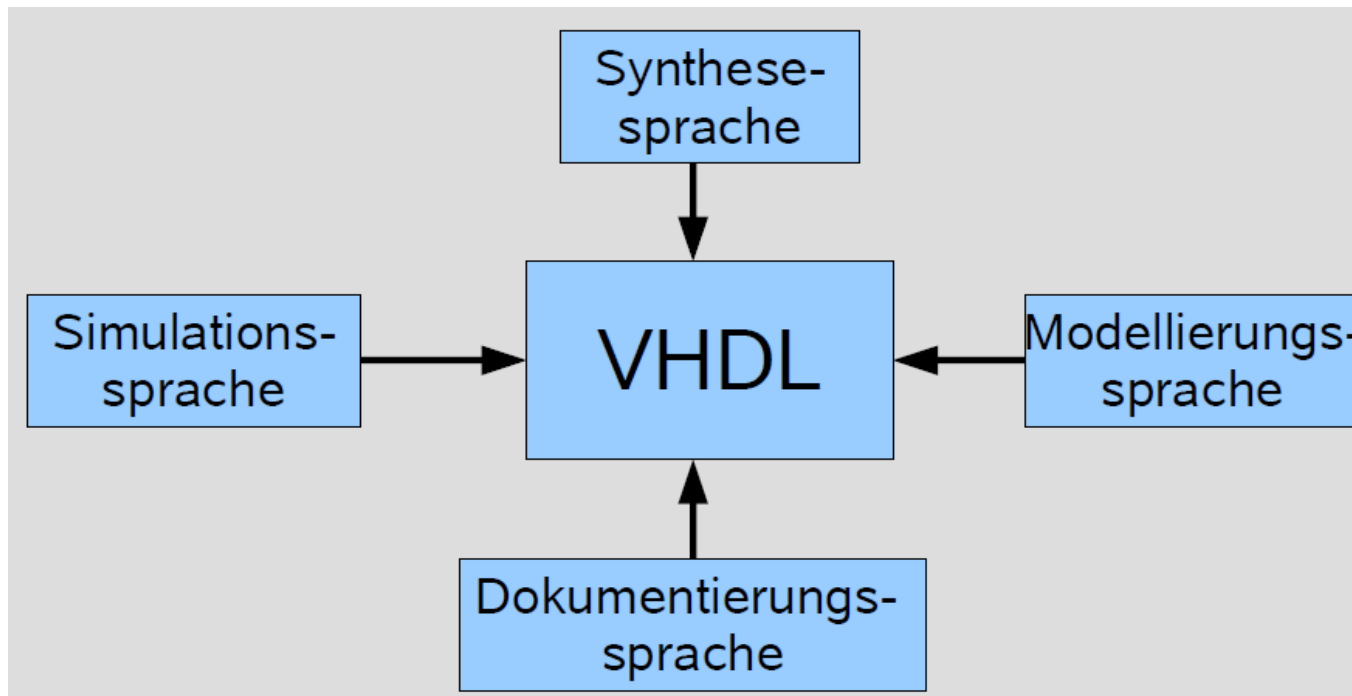
- srl operator, left operand shifted right logical by right op
- subtype declaration to restrict an existing type
- then part of if condition then ...
- to middle of a range 1 to 10
- transport signal characteristic
- type declaration to create a new type
- unaffected used in signal waveform
- units used to define new types of units
- until used in wait statement
- use make a package available to this design unit
- variable declaration that an object is a variable

Reservierte VHDL Wörter (7/7)

- wait sequential statement, also used in case statement
- when used for choices in case and other statements
- while kind of loop statement
- with used in selected signal assignment statement
- xnor operator, exclusive "nor" of left and right operands
- xor operator, exclusive "or" of left and right operands

Idee von VHDL

- Im Gegensatz zu „normalen“ Programmiersprachen wie C oder Python entsteht durch die Compilierung / Interpretation von VHDL Code kein direkt ausführbares Programm
 - es gibt weder eine Compilierung noch eine Interpretation
- Eine VHDL-Beschreibung ist nur im Zusammenhang mit einem Logiksimulator (=Testumgebung) verifizierbar und mit einem Synthetool und FPGA Hardware realisierbar

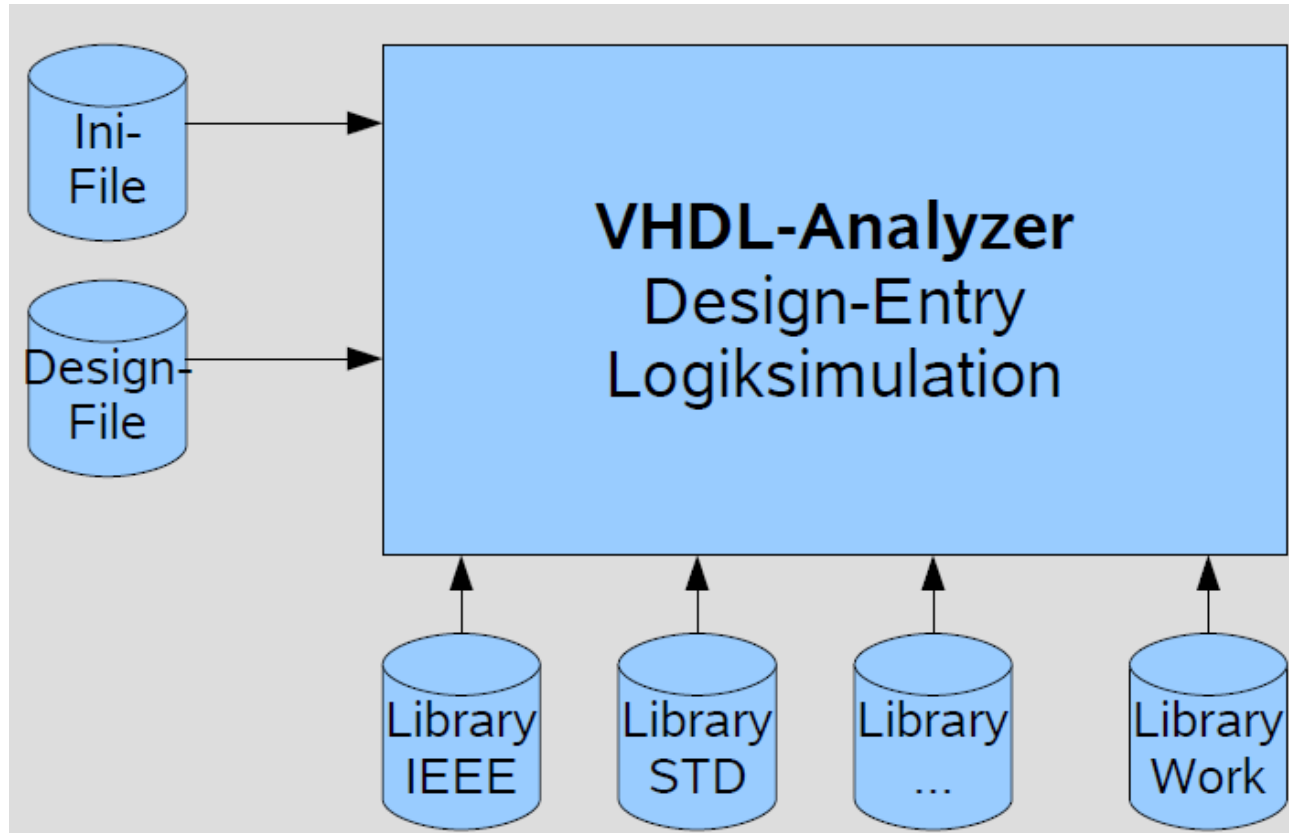


VHDL Eigenschaften

- Enthält neben den aus „normalen“ Programmiersprachen bekannten
 - Datentypen
 - Kontrollstrukturen
 - Prozeduren
 - ...

auch einige Konzepte, welche die speziellen Eigenschaften von Hardware reflektieren
- Syntax ähnlich ADA
- Kann Parallelität sehr gut darstellen

Aufbau



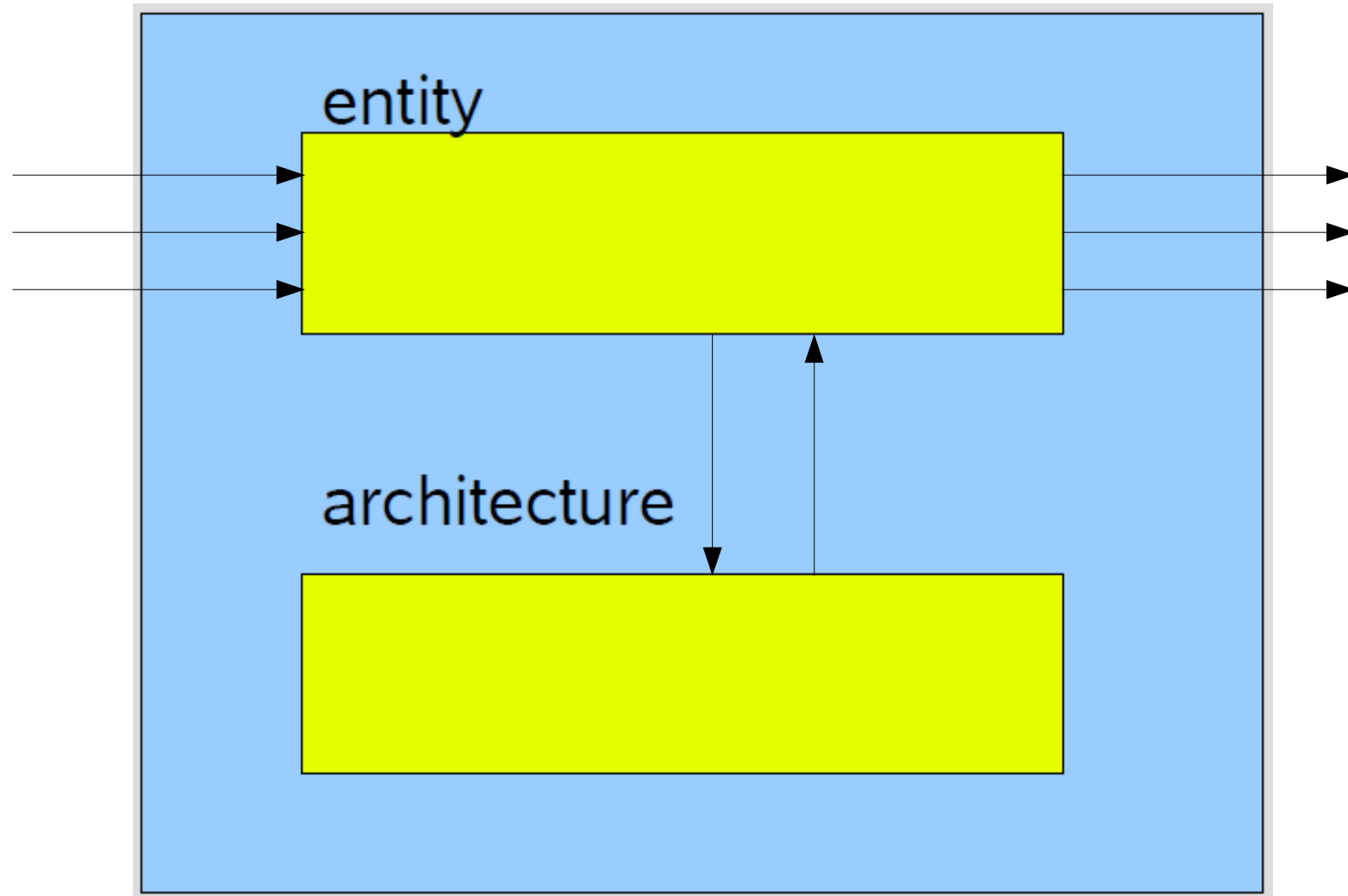
Hello World

```
-- Hello world program.  
use std.textio.all; -- Imports the standard textio package.  
  
-- Defines a design entity, without any ports.  
entity hello_world is  
end hello_world;  
  
architecture behaviour of hello_world is  
begin  
  process  
    variable l : line;  
  begin  
    l := new String("Hello world!");  
    writeline (output, l);  
    wait;  
  end process;  
end behaviour;
```

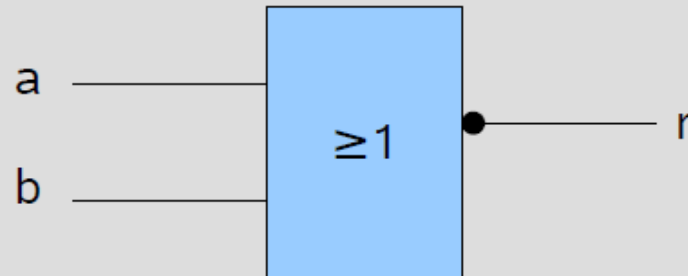
Entity & Architecture

- Reales System verfügt üblicherweise über
 - Schnittstellen nach Außen
 - interne Realisierung
 - logische Gatter
 - FlipFlops
 - ...
- Schnittstelle
 - Entity
- Interne Realisierung
 - Architecture

Entity & Architecture



Entity NOR Gatter



```
entity mnor is
  port
  (
    a, b:  in bit;
    r:     out bit;
  );
end mnor2;
```

Bit Datentypen

- Bit Typ ist zwei-wertig

- Bit
- bit_vector

```
1
2 -----
3 --
4 -- Project      : Synchrone und asynchrone Prozesse
5 --
6 -- Tool         : Xilinx, Vivado
7 --
8 -- FPGA Device  : S50
9 --
10 -- Copyright    : Edmund Coersmeier
11 --
12 -----
13
14 library IEEE;
15 use IEEE.STD_LOGIC_1164.ALL;      -- Bibliothek fuer Basisoperation mit STD_LOGIC Signalen
16 use IEEE.STD_LOGIC_UNSIGNED.ALL;  -- Bibliothek fuer Basisoperation mit STD_LOGIC Unsigned Signalen
17 use IEEE.NUMERIC_STD.ALL;         -- Bibliothek fuer numrische Berechnungen
18
```

- Auch sehr häufig verwendet - wir benutzen IMMER bei Bit Typen std_logic und std_logic_vector
 - std_logic
 - std_logic_vector
- Der Typ std_logic erfordert das Einbinden der
 - Library IEEE vor der Entity-Deklaration durch
 - Library IEEE;
 - use IEEE.std_logic_1164.all;
 - use IEEE.std_logic_unsigned.all;

Signal Informationen

- Nach IEEE 1164 kann ein Signal folgende Zustände annehmen
 - U : Uninitialized
 - X : Forcing Unknown
 - 0 : Forcing 0
 - 1 : Forcing 1
 - Z : High Impedance
 - W : Weak Unknown
 - L : Weak 0
 - H : Weak 1
 - D : Don't Care

Hinweis

- Mit `std_logic` wird ein Logik-Signal als 9-wertig definiert
- Diese 9-wertige Logik ist heute faktisch Industrie-Standard
- Durch das Einbinden der
 - Library `IEEE.std_logic_1164.all`
stehen bestimmte Funktionen zur Verfügung
- Das gilt auch für die Library
 - `IEEE.std_logic_unsigned.all`

Beispiel - std_logic NOT

U	X	0	1	Z	W	L	H	D
U	X	1	0	X	X	1	0	x

Zurück zu Entity und Architecture

- Architekturbeschreibung
 - Eigentliche Realisierung der Entity
 - Schlüsselwort architecture
 - Zu jeder Architektur gehört eine Entity,
- Beispiel (NOR-Gatter)

architecture a of mnor2 is

begin

r <= not (a or b);

end architecture a;

Formale Unterscheidung Flip Flop vs. Register

- Ein Flip Flop ist ein Bauelement, welches 1 Bit speichern kann
- Ein Register ist typischerweise eine Ansammlung von mehreren, mindestens 2 parallelen Flip Flops, um mehr Bits als nur 1 Bit gleichzeitig speichern zu können
- ABER in der Praxis wird zwischen diesen beiden Begriffen bei 1 Bit Wortbreite NICHT unterschieden!
 - ein Flip Flop ist gleich einem 1-Bit Register
 - den Begriff 1-Bit Register benutzt aber niemand
 - ein Synthesetool bezeichnet ein Flip Flop aber durchaus auch als Register
- Es gilt hier in der Vorlesung
 - ein Flip Flop speichert eine Wortbreite von $= 1$
 - ein Register speichert eine Wortbreite von ≥ 1
 - ein Flip Flop betrachten wir als 1-Bit Register $\rightarrow$ ein Flip Flop bezeichnen wir also auch als ein Register, ein Register ist deswegen aber nicht automatisch nur ein 1Bit Flip Flop
- auch der Umkehrschluss ist nicht gültig: falls es eine Wortbreite von 1 Bit ist, dann ist es kein Register - dieser Umkehrschluss stimmt NICHT!

Prozesse (1/5)

- Entity mit 4 Eingangsports und 1 Ausgangsport
- Reset und Clk kommen in jedem Design vor, falls es sich um ein synchrones Design handelt

```
1 :
2 ⊞ -----
3 --
4 -- Project      : Synchrone und asynchrone Prozesse
5 --
6 -- Tool         : Xilinx, Vivado
7 --
8 -- FPGA Device  : S50
9 --
10 -- Copyright   : Edmund Coersmeier
11 --
12 ⊞ -----
13
14 library IEEE;
15 use IEEE.STD_LOGIC_1164.ALL;      -- Bibliothek fuer Basisoperation mit STD_LOGIC Signalen
16 use IEEE.STD_LOGIC_UNSIGNED.ALL;  -- Bibliothek fuer Basisoperation mit STD_LOGIC Unsigned Signalen
17 use IEEE.NUMERIC_STD.ALL;         -- Bibliothek fuer numrische Berechnungen
18
19 ⊞ -----
20 -- Entity
21 -- Eingangs- und Ausgangsports deklarieren
22 ⊞ -----
23 ⊞ entity selection is
24
25     port (
26         Reset      : in  STD_LOGIC;      -- Input Port
27         Clk         : in  STD_LOGIC;      -- Input Port
28         A           : in  STD_LOGIC;      -- Input Port
29         B           : in  STD_LOGIC;      -- Input Port
30         C           : out STD_LOGIC       -- Output Port
31     );
32
33 ⊞ end entity selection;|
34
```

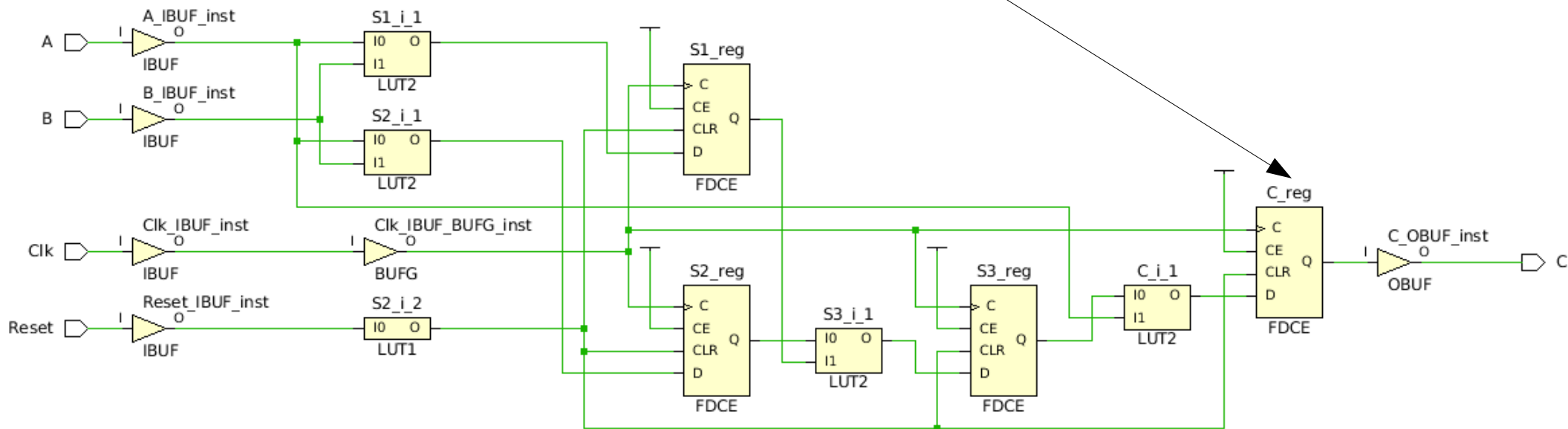
Prozesse (2/5)

- Der Prozess P1 wird von Ereignissen der Signale, die in der „Sensitivity-List“ (Reset, Clk) enthalten sind, getriggert
- Jedes Ereignis für ein Signal der „Sensitivity“-List verursacht die Ausführung des Prozesses
- Die Reihenfolge der Signal ist dabei völlig irrelevant, solange Signale verwendet werden
- Reset vor dem Clock-Event / rising_edge

```
35
36 -- Architecture
37 -- Funktionalitaet in synchronen und asynchronen Prozessen definieren
38 -- synchrone Prozesse : Clk und Reset in der Sensitivity List
39 -- asynchrone Prozesse: Prozesseingangssignale im der Sensitivity List
40
41 architecture arch_selection of selection is
42
43     signal S1 : STD_LOGIC; -- Ergebnis einer ODER Verknuepfung
44     signal S2 : STD_LOGIC; -- Ergebnis einer UND Verknuepfung
45     signal S3 : STD_LOGIC; -- Ergebnis einer XOR Verknuepfung
46
47
48 begin -- architecture
49
50     -- synchroner Prozess
51     P1: process(Reset, Clk)
52
53     begin
54
55         -- Reset
56         -- allen Signalen und Ausgaengen einen Anfangswertzuwesien
57         -- nicht den Eingangssignalen
58
59         if Reset = '0' then
60             C <= '0'; -- Ausgang
61             S1 <= '0'; -- internes Flip Flop
62             S2 <= '0'; -- internes Flip Flop
63             S3 <= '0'; -- internes Flip Flop
64
65
66         -- aktiver Prozessbereich
67         -- Sensitivity List reagiert nur auf eine steigende Flanke
68         elsif Clk'event and Clk = '1' then
69             S1 <= A or B; -- Signalzuweisung im Clk'Event Zweig erzeugt ein FlipFlop/Register;
70             S2 <= A and B; -- Signalzuweisung im Clk'Event Zweig erzeugt ein FlipFlop/Register;
71             S3 <= S1 xor S2; -- Signalzuweisung im Clk'Event Zweig erzeugt ein FlipFlop/Register;
72             C <= S3 and A; -- Das Flip Flop S3 liegt im Signalfluss hinter den Flip Flops S1 und S2
73
74         end if; -- rising edge
75
76     end process P1;
77
78 end arch_selection;
79
```

Prozesse (3/5) - Syntheseergebnis

- Jede Signalzuweisung in einem synchronen Prozess ergibt ein FlipFlop / 1-Bit Register (1Bit) oder ein Register (mehr als 1 Bit) → bei uns gilt: Register ≥ 1 Bit, denn das Synthesetool unterscheidet nicht zwischen FF/Register
- Der Reset ist in dieser Variante asynchron und aktiv low → Eingang ,0' erzeugt einen Reset
- Der Reset Pin geht immer nur an ein jeweiliges Flip Flop mit Bezeichnung Clear, zuvor nach dem Durchlauf eines Inverters → aktiv Low von Außen



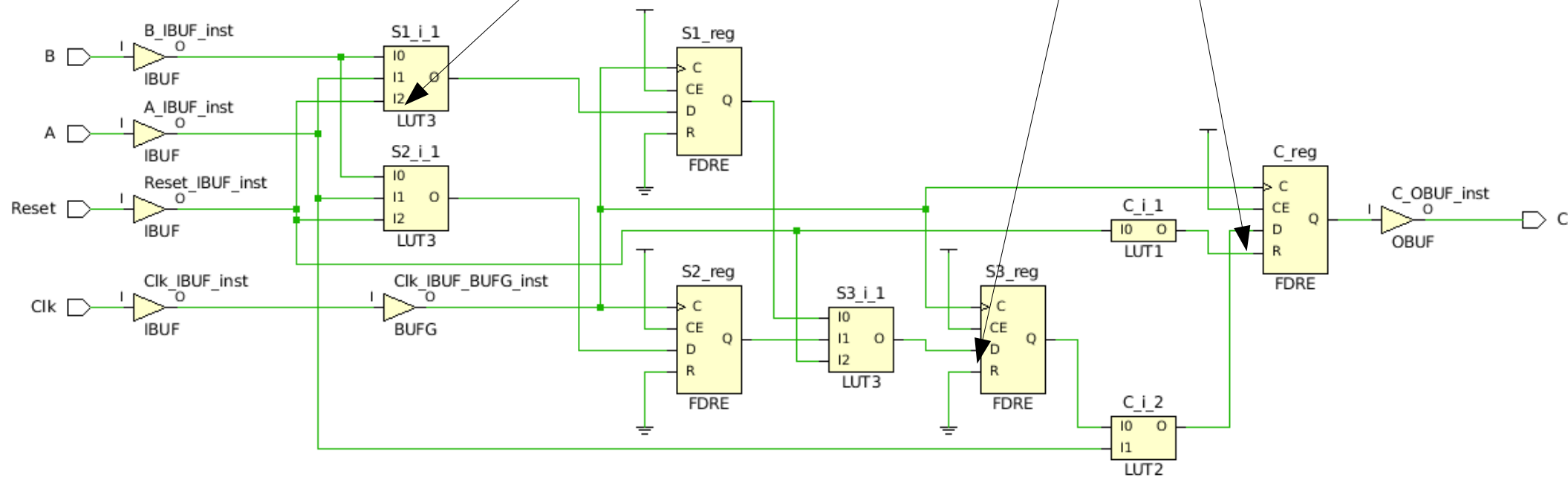
Prozesse (4/5) – Synchroner Reset

- Bei einem synchronen Reset wird der Reset immer erst NACH einem Clock-Event ausführbar
- Achtung: geänderte Sensitivity Liste → nur noch das Clk-Signal in der Sensitivity Liste vorhanden

```
47
48   begin -- architecture
49
50
51   -- synchroner Prozess
52   P1: process(Clk)
53
54   begin
55
56   -- aktiver Prozessbereich
57   -- Sensitivity List reagiert nur auf eine steigende Flanke
58   if Clk'event and Clk = '1' then
59
60   -- Reset
61   -- allen Signalen und Ausgängen einen Anfangswert zuweisen
62   -- nicht den Eingangssignalen
63
64   if Reset = '0' then
65       C <= '0'; -- Ausgang
66       S1 <= '0'; -- internes Flip Flop
67       S2 <= '0'; -- internes Flip Flop
68       S3 <= '0'; -- internes Flip Flop
69
70   else
71       S1 <= A or B; -- Signalzuweisung im Clk'Event Zweig erzeugt ein FlipFlop/Register;
72       S2 <= A and B; -- Signalzuweisung im Clk'Event Zweig erzeugt ein FlipFlop/Register;
73       S3 <= S1 xor S2; -- Signalzuweisung im Clk'Event Zweig erzeugt ein FlipFlop/Register;
74       C <= S3 and A; -- Das Flip Flop S3 liegt im Signalfloss hinter den Flip Flops S1 und S2
75
76   end if; -- reset
77
78   end if; -- rising edge
79
80   end process P1;
81
82   end arch_selection;
83
```

Prozesse (5/5) – Synchroner Reset

- Das Reset Signal nimmt auch Einfluss auf die Kombinatorische Logik und die Flip Flops
 - jetzt heißt der Flip Flop Eingang für den Reset tatsächlich „R“ anstatt Clear „CLR“
- Auch wenn sehr sehr viele Designs ausschließlich synchrone Reset Eingaben nutzen, wird in dieser Vorlesung ausschließlich ein asynchroner Reset favorisiert



Moderne Clock Flanken Abtastung (1/2)

- Es gibt verschiedene Schreibweisen für die steigende Flanke der Clock
 - `Clk'event and Clk = '1'` → traditionell seit den 1990igern
 - `rising_edge(Clk)` → modern seit ca. 2000sender

```
64 |  
65 |  
66 | -- aktiver Prozessbereich  
67 | -- Sensitivity List reagiert nur auf eine steigende Flanke  
68 |  
69 |  
70 |  
71 |  
72 |  
73 |  
74 |  
75 |  
76 |  
77 |  
78 |  
79 |  
80 |  
81 |  
82 |  
83 |  
84 |  
85 |  
86 |  
87 |  
88 |  
89 |  
90 |  
91 |  
92 |  
93 |  
94 |  
95 |  
96 |  
97 |  
98 |  
99 |  
100 |  
101 |  
102 |  
103 |  
104 |  
105 |  
106 |  
107 |  
108 |  
109 |  
110 |  
111 |  
112 |  
113 |  
114 |  
115 |  
116 |  
117 |  
118 |  
119 |  
120 |  
121 |  
122 |  
123 |  
124 |  
125 |  
126 |  
127 |  
128 |  
129 |  
130 |  
131 |  
132 |  
133 |  
134 |  
135 |  
136 |  
137 |  
138 |  
139 |  
140 |  
141 |  
142 |  
143 |  
144 |  
145 |  
146 |  
147 |  
148 |  
149 |  
150 |  
151 |  
152 |  
153 |  
154 |  
155 |  
156 |  
157 |  
158 |  
159 |  
160 |  
161 |  
162 |  
163 |  
164 |  
165 |  
166 |  
167 |  
168 |  
169 |  
170 |  
171 |  
172 |  
173 |  
174 |  
175 |  
176 |  
177 |  
178 |  
179 |  
180 |  
181 |  
182 |  
183 |  
184 |  
185 |  
186 |  
187 |  
188 |  
189 |  
190 |  
191 |  
192 |  
193 |  
194 |  
195 |  
196 |  
197 |  
198 |  
199 |  
200 |  
201 |  
202 |  
203 |  
204 |  
205 |  
206 |  
207 |  
208 |  
209 |  
210 |  
211 |  
212 |  
213 |  
214 |  
215 |  
216 |  
217 |  
218 |  
219 |  
220 |  
221 |  
222 |  
223 |  
224 |  
225 |  
226 |  
227 |  
228 |  
229 |  
230 |  
231 |  
232 |  
233 |  
234 |  
235 |  
236 |  
237 |  
238 |  
239 |  
240 |  
241 |  
242 |  
243 |  
244 |  
245 |  
246 |  
247 |  
248 |  
249 |  
250 |  
251 |  
252 |  
253 |  
254 |  
255 |  
256 |  
257 |  
258 |  
259 |  
260 |  
261 |  
262 |  
263 |  
264 |  
265 |  
266 |  
267 |  
268 |  
269 |  
270 |  
271 |  
272 |  
273 |  
274 |  
275 |  
276 |  
277 |  
278 |  
279 |  
280 |  
281 |  
282 |  
283 |  
284 |  
285 |  
286 |  
287 |  
288 |  
289 |  
290 |  
291 |  
292 |  
293 |  
294 |  
295 |  
296 |  
297 |  
298 |  
299 |  
300 |  
301 |  
302 |  
303 |  
304 |  
305 |  
306 |  
307 |  
308 |  
309 |  
310 |  
311 |  
312 |  
313 |  
314 |  
315 |  
316 |  
317 |  
318 |  
319 |  
320 |  
321 |  
322 |  
323 |  
324 |  
325 |  
326 |  
327 |  
328 |  
329 |  
330 |  
331 |  
332 |  
333 |  
334 |  
335 |  
336 |  
337 |  
338 |  
339 |  
340 |  
341 |  
342 |  
343 |  
344 |  
345 |  
346 |  
347 |  
348 |  
349 |  
350 |  
351 |  
352 |  
353 |  
354 |  
355 |  
356 |  
357 |  
358 |  
359 |  
360 |  
361 |  
362 |  
363 |  
364 |  
365 |  
366 |  
367 |  
368 |  
369 |  
370 |  
371 |  
372 |  
373 |  
374 |  
375 |  
376 |  
377 |  
378 |  
379 |  
380 |  
381 |  
382 |  
383 |  
384 |  
385 |  
386 |  
387 |  
388 |  
389 |  
390 |  
391 |  
392 |  
393 |  
394 |  
395 |  
396 |  
397 |  
398 |  
399 |  
400 |  
401 |  
402 |  
403 |  
404 |  
405 |  
406 |  
407 |  
408 |  
409 |  
410 |  
411 |  
412 |  
413 |  
414 |  
415 |  
416 |  
417 |  
418 |  
419 |  
420 |  
421 |  
422 |  
423 |  
424 |  
425 |  
426 |  
427 |  
428 |  
429 |  
430 |  
431 |  
432 |  
433 |  
434 |  
435 |  
436 |  
437 |  
438 |  
439 |  
440 |  
441 |  
442 |  
443 |  
444 |  
445 |  
446 |  
447 |  
448 |  
449 |  
450 |  
451 |  
452 |  
453 |  
454 |  
455 |  
456 |  
457 |  
458 |  
459 |  
460 |  
461 |  
462 |  
463 |  
464 |  
465 |  
466 |  
467 |  
468 |  
469 |  
470 |  
471 |  
472 |  
473 |  
474 |  
475 |  
476 |  
477 |  
478 |  
479 |  
480 |  
481 |  
482 |  
483 |  
484 |  
485 |  
486 |  
487 |  
488 |  
489 |  
490 |  
491 |  
492 |  
493 |  
494 |  
495 |  
496 |  
497 |  
498 |  
499 |  
500 |  
501 |  
502 |  
503 |  
504 |  
505 |  
506 |  
507 |  
508 |  
509 |  
510 |  
511 |  
512 |  
513 |  
514 |  
515 |  
516 |  
517 |  
518 |  
519 |  
520 |  
521 |  
522 |  
523 |  
524 |  
525 |  
526 |  
527 |  
528 |  
529 |  
530 |  
531 |  
532 |  
533 |  
534 |  
535 |  
536 |  
537 |  
538 |  
539 |  
540 |  
541 |  
542 |  
543 |  
544 |  
545 |  
546 |  
547 |  
548 |  
549 |  
550 |  
551 |  
552 |  
553 |  
554 |  
555 |  
556 |  
557 |  
558 |  
559 |  
560 |  
561 |  
562 |  
563 |  
564 |  
565 |  
566 |  
567 |  
568 |  
569 |  
570 |  
571 |  
572 |  
573 |  
574 |  
575 |  
576 |  
577 |  
578 |  
579 |  
580 |  
581 |  
582 |  
583 |  
584 |  
585 |  
586 |  
587 |  
588 |  
589 |  
590 |  
591 |  
592 |  
593 |  
594 |  
595 |  
596 |  
597 |  
598 |  
599 |  
600 |  
601 |  
602 |  
603 |  
604 |  
605 |  
606 |  
607 |  
608 |  
609 |  
610 |  
611 |  
612 |  
613 |  
614 |  
615 |  
616 |  
617 |  
618 |  
619 |  
620 |  
621 |  
622 |  
623 |  
624 |  
625 |  
626 |  
627 |  
628 |  
629 |  
630 |  
631 |  
632 |  
633 |  
634 |  
635 |  
636 |  
637 |  
638 |  
639 |  
640 |  
641 |  
642 |  
643 |  
644 |  
645 |  
646 |  
647 |  
648 |  
649 |  
650 |  
651 |  
652 |  
653 |  
654 |  
655 |  
656 |  
657 |  
658 |  
659 |  
660 |  
661 |  
662 |  
663 |  
664 |  
665 |  
666 |  
667 |  
668 |  
669 |  
670 |  
671 |  
672 |  
673 |  
674 |  
675 |  
676 |  
677 |  
678 |  
679 |  
680 |  
681 |  
682 |  
683 |  
684 |  
685 |  
686 |  
687 |  
688 |  
689 |  
690 |  
691 |  
692 |  
693 |  
694 |  
695 |  
696 |  
697 |  
698 |  
699 |  
700 |  
701 |  
702 |  
703 |  
704 |  
705 |  
706 |  
707 |  
708 |  
709 |  
710 |  
711 |  
712 |  
713 |  
714 |  
715 |  
716 |  
717 |  
718 |  
719 |  
720 |  
721 |  
722 |  
723 |  
724 |  
725 |  
726 |  
727 |  
728 |  
729 |  
730 |  
731 |  
732 |  
733 |  
734 |  
735 |  
736 |  
737 |  
738 |  
739 |  
740 |  
741 |  
742 |  
743 |  
744 |  
745 |  
746 |  
747 |  
748 |  
749 |  
750 |  
751 |  
752 |  
753 |  
754 |  
755 |  
756 |  
757 |  
758 |  
759 |  
760 |  
761 |  
762 |  
763 |  
764 |  
765 |  
766 |  
767 |  
768 |  
769 |  
770 |  
771 |  
772 |  
773 |  
774 |  
775 |  
776 |  
777 |  
778 |  
779 |  
780 |  
781 |  
782 |  
783 |  
784 |  
785 |  
786 |  
787 |  
788 |  
789 |  
790 |  
791 |  
792 |  
793 |  
794 |  
795 |  
796 |  
797 |  
798 |  
799 |  
800 |  
801 |  
802 |  
803 |  
804 |  
805 |  
806 |  
807 |  
808 |  
809 |  
810 |  
811 |  
812 |  
813 |  
814 |  
815 |  
816 |  
817 |  
818 |  
819 |  
820 |  
821 |  
822 |  
823 |  
824 |  
825 |  
826 |  
827 |  
828 |  
829 |  
830 |  
831 |  
832 |  
833 |  
834 |  
835 |  
836 |  
837 |  
838 |  
839 |  
840 |  
841 |  
842 |  
843 |  
844 |  
845 |  
846 |  
847 |  
848 |  
849 |  
850 |  
851 |  
852 |  
853 |  
854 |  
855 |  
856 |  
857 |  
858 |  
859 |  
860 |  
861 |  
862 |  
863 |  
864 |  
865 |  
866 |  
867 |  
868 |  
869 |  
870 |  
871 |  
872 |  
873 |  
874 |  
875 |  
876 |  
877 |  
878 |  
879 |  
880 |  
881 |  
882 |  
883 |  
884 |  
885 |  
886 |  
887 |  
888 |  
889 |  
890 |  
891 |  
892 |  
893 |  
894 |  
895 |  
896 |  
897 |  
898 |  
899 |  
900 |  
901 |  
902 |  
903 |  
904 |  
905 |  
906 |  
907 |  
908 |  
909 |  
910 |  
911 |  
912 |  
913 |  
914 |  
915 |  
916 |  
917 |  
918 |  
919 |  
920 |  
921 |  
922 |  
923 |  
924 |  
925 |  
926 |  
927 |  
928 |  
929 |  
930 |  
931 |  
932 |  
933 |  
934 |  
935 |  
936 |  
937 |  
938 |  
939 |  
940 |  
941 |  
942 |  
943 |  
944 |  
945 |  
946 |  
947 |  
948 |  
949 |  
950 |  
951 |  
952 |  
953 |  
954 |  
955 |  
956 |  
957 |  
958 |  
959 |  
960 |  
961 |  
962 |  
963 |  
964 |  
965 |  
966 |  
967 |  
968 |  
969 |  
970 |  
971 |  
972 |  
973 |  
974 |  
975 |  
976 |  
977 |  
978 |  
979 |  
980 |  
981 |  
982 |  
983 |  
984 |  
985 |  
986 |  
987 |  
988 |  
989 |  
990 |  
991 |  
992 |  
993 |  
994 |  
995 |  
996 |  
997 |  
998 |  
999 |  
1000 |  
1001 |  
1002 |  
1003 |  
1004 |  
1005 |  
1006 |  
1007 |  
1008 |  
1009 |  
1010 |  
1011 |  
1012 |  
1013 |  
1014 |  
1015 |  
1016 |  
1017 |  
1018 |  
1019 |  
1020 |  
1021 |  
1022 |  
1023 |  
1024 |  
1025 |  
1026 |  
1027 |  
1028 |  
1029 |  
1030 |  
1031 |  
1032 |  
1033 |  
1034 |  
1035 |  
1036 |  
1037 |  
1038 |  
1039 |  
1040 |  
1041 |  
1042 |  
1043 |  
1044 |  
1045 |  
1046 |  
1047 |  
1048 |  
1049 |  
1050 |  
1051 |  
1052 |  
1053 |  
1054 |  
1055 |  
1056 |  
1057 |  
1058 |  
1059 |  
1060 |  
1061 |  
1062 |  
1063 |  
1064 |  
1065 |  
1066 |  
1067 |  
1068 |  
1069 |  
1070 |  
1071 |  
1072 |  
1073 |  
1074 |  
1075 |  
1076 |  
1077 |  
1078 |  
1079 |  
1080 |  
1081 |  
1082 |  
1083 |  
1084 |  
1085 |  
1086 |  
1087 |  
1088 |  
1089 |  
1090 |  
1091 |  
1092 |  
1093 |  
1094 |  
1095 |  
1096 |  
1097 |  
1098 |  
1099 |  
1100 |  
1101 |  
1102 |  
1103 |  
1104 |  
1105 |  
1106 |  
1107 |  
1108 |  
1109 |  
1110 |  
1111 |  
1112 |  
1113 |  
1114 |  
1115 |  
1116 |  
1117 |  
1118 |  
1119 |  
1120 |  
1121 |  
1122 |  
1123 |  
1124 |  
1125 |  
1126 |  
1127 |  
1128 |  
1129 |  
1130 |  
1131 |  
1132 |  
1133 |  
1134 |  
1135 |  
1136 |  
1137 |  
1138 |  
1139 |  
1140 |  
1141 |  
1142 |  
1143 |  
1144 |  
1145 |  
1146 |  
1147 |  
1148 |  
1149 |  
1150 |  
1151 |  
1152 |  
1153 |  
1154 |  
1155 |  
1156 |  
1157 |  
1158 |  
1159 |  
1160 |  
1161 |  
1162 |  
1163 |  
1164 |  
1165 |  
1166 |  
1167 |  
1168 |  
1169 |  
1170 |  
1171 |  
1172 |  
1173 |  
1174 |  
1175 |  
1176 |  
1177 |  
1178 |  
1179 |  
1180 |  
1181 |  
1182 |  
1183 |  
1184 |  
1185 |  
1186 |  
1187 |  
1188 |  
1189 |  
1190 |  
1191 |  
1192 |  
1193 |  
1194 |  
1195 |  
1196 |  
1197 |  
1198 |  
1199 |  
1200 |  
1201 |  
1202 |  
1203 |  
1204 |  
1205 |  
1206 |  
1207 |  
1208 |  
1209 |  
1210 |  
1211 |  
1212 |  
1213 |  
1214 |  
1215 |  
1216 |  
1217 |  
1218 |  
1219 |  
1220 |  
1221 |  
1222 |  
1223 |  
1224 |  
1225 |  
1226 |  
1227 |  
1228 |  
1229 |  
1230 |  
1231 |  
1232 |  
1233 |  
1234 |  
1235 |  
1236 |  
1237 |  
1238 |  
1239 |  
1240 |  
1241 |  
1242 |  
1243 |  
1244 |  
1245 |  
1246 |  
1247 |  
1248 |  
1249 |  
1250 |  
1251 |  
1252 |  
1253 |  
1254 |  
1255 |  
1256 |  
1257 |  
1258 |  
1259 |  
1260 |  
1261 |  
1262 |  
1263 |  
1264 |  
1265 |  
1266 |  
1267 |  
1268 |  
1269 |  
1270 |  
1271 |  
1272 |  
1273 |  
1274 |  
1275 |  
1276 |  
1277 |  
1278 |  
1279 |  
1280 |  
1281 |  
1282 |  
1283 |  
1284 |  
1285 |  
1286 |  
1287 |  
1288 |  
1289 |  
1290 |  
1291 |  
1292 |  
1293 |  
1294 |  
1295 |  
1296 |  
1297 |  
1298 |  
1299 |  
1300 |  
1301 |  
1302 |  
1303 |  
1304 |  
1305 |  
1306 |  
1307 |  
1308 |  
1309 |  
1310 |  
1311 |  
1312 |  
1313 |  
1314 |  
1315 |  
1316 |  
1317 |  
1318 |  
1319 |  
1320 |  
1321 |  
1322 |  
1323 |  
1324 |  
1325 |  
1326 |  
1327 |  
1328 |  
1329 |  
1330 |  
1331 |  
1332 |  
1333 |  
1334 |  
1335 |  
1336 |  
1337 |  
1338 |  
1339 |  
1340 |  
1341 |  
1342 |  
1343 |  
1344 |  
1345 |  
1346 |  
1347 |  
1348 |  
1349 |  
1350 |  
1351 |  
1352 |  
1353 |  
1354 |  
1355 |  
1356 |  
1357 |  
1358 |  
1359 |  
1360 |  
1361 |  
1362 |  
1363 |  
1364 |  
1365 |  
1366 |  
1367 |  
1368 |  
1369 |  
1370 |  
1371 |  
1372 |  
1373 |  
1374 |  
1375 |  
1376 |  
1377 |  
1378 |  
1379 |  
1380 |  
1381 |  
1382 |  
1383 |  
1384 |  
1385 |  
1386 |  
1387 |  
1388 |  
1389 |  
1390 |  
1391 |  
1392 |  
1393 |  
1394 |  
1395 |  
1396 |  
1397 |  
1398 |  
1399 |  
1400 |  
1401 |  
1402 |  
1403 |  
1404 |  
1405 |  
1406 |  
1407 |  
1408 |  
1409 |  
1410 |  
1411 |  
1412 |  
1413 |  
1414 |  
1415 |  
1416 |  
1417 |  
1418 |  
1419 |  
1420 |  
1421 |  
1422 |  
1423 |  
1424 |  
1425 |  
1426 |  
1427 |  
1428 |  
1429 |  
1430 |  
1431 |  
1432 |  
1433 |  
1434 |  
1435 |  
1436 |  
1437 |  
1438 |  
1439 |  
1440 |  
1441 |  
1442 |  
1443 |  
1444 |  
1445 |  
1446 |  
1447 |  
1448 |  
1449 |  
1450 |  
1451 |  
1452 |  
1453 |  
1454 |  
1455 |  
1456 |  
1457 |  
1458 |  
1459 |  
1460 |  
1461 |  
1462 |  
1463 |  
1464 |  
1465 |  
1466 |  
1467 |  
1468 |  
1469 |  
1470 |  
1471 |  
1472 |  
1473 |  
1474 |  
1475 |  
1476 |  
1477 |  
1478 |  
1479 |  
1480 |  
1481 |  
1482 |  
1483 |  
1484 |  
1485 |  
1486 |  
1487 |  
1488 |  
1489 |  
1490 |  
1491 |  
1492 |  
1493 |  
1494 |  
1495 |  
1496 |  
1497 |  
1498 |  
1499 |  
1500 |  
1501 |  
1502 |  
1503 |  
1504 |  
1505 |  
1506 |  
1507 |  
1508 |  
1509 |  
1510 |  
1511 |  
1512 |  
1513 |  
1514 |  
1515 |  
1516 |  
1517 |  
1518 |  
1519 |  
1520 |  
1521 |  
1522 |  
1523 |  
1524 |  
1525 |  
1526 |  
1527 |  
1528 |  
1529 |  
1530 |  
1531 |  
1532 |  
1533 |  
1534 |  
1535 |  
1536 |  
1537 |  
1538 |  
1539 |  
1540 |  
1541 |  
1542 |  
1543 |  
1544 |  
1545 |  
1546 |  
1547 |  
1548 |  
1549 |  
1550 |  
1551 |  
1552 |  
1553 |  
1554 |  
1555 |  
1556 |  
1557 |  
1558 |  
1559 |  
1560 |  
1561 |  
1562 |  
1563 |  
1564 |  
1565 |  
1566 |  
1567 |  
1568 |  
1569 |  
1570 |  
1571 |  
1572 |  
1573 |  
1574 |  
1575 |  
1576 |  
1577 |  
1578 |  
1579 |  
1580 |  
1581 |  
1582 |  
1583 |  
1584 |  
1585 |  
1586 |  
1587 |  
1588 |  
1589 |  
1590 |  
1591 |  
1592 |  
1593 |  
1594 |  
1595 |  
1596 |  
1597 |  
1598 |  
1599 |  
1600 |  
1601 |  
1602 |  
1603 |  
1604 |  
1605 |  
1606 |  
1607 |  
1608 |  
1609 |  
1610 |  
1611 |  
1612 |  
1613 |  
1614 |  
1615 |  
1616 |  
1617 |  
1618 |  
1619 |  
1620 |  
1621 |  
1622 |  
1623 |  
1624 |  
1625 |  
1626 |  
1627 |  
1628 |  
1629 |  
1630 |  
1631 |  
1632 |  
1633 |  
1634 |  
1635 |  
1636 |  
1637 |  
1638 |  
1639 |  
1640 |  
1641 |  
1642 |  
1643 |  
1644 |  
1645 |  
1646 |  
1647 |  
1648 |  
1649 |  
1650 |  
1651 |  
1652 |  
1653 |  
1654 |  
1655 |  
1656 |  
1657 |  
1658 |  
1659 |  
1660 |  
1661 |  
1662 |  
1663 |  
1664 |  
1665 |  
1666 |  
1667 |  
1668 |  
1669 |  
1670 |  
1671 |  
1672 |  
1673 |  
1674 |  
1675 |  
1676 |  
1677 |  
1678 |  
1679 |  
1680 |  
1681 |  
1682 |  
1683 |  
1684 |  
1685 |  
1686 |  
1687 |  
1688 |  
1689 |  
1690 |  
1691 |  
1692 |  
1693 |  
1694 |  
1695 |  
1696 |  
1697 |  
1698 |  
1699 |  
1700 |  
1701 |  
1702 |  
1703 |  
1704 |  
1705 |  
1706 |  
1707 |<
```

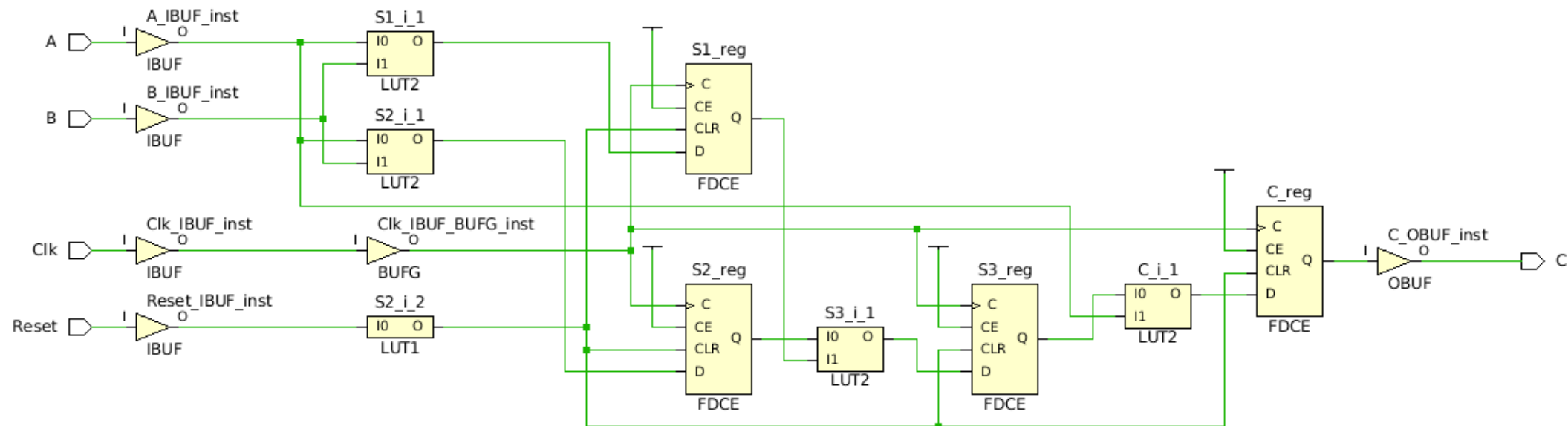
Moderne Clock Flanken Abtastung (2/2)

- Es gibt verschiedene Schreibweisen für die steigende Flanke der Clock
 - Clk'event and Clk = ,1' → traditionell seit den 1990igern
 - rising_edge(Clk) → modern seit ca. 2000sender

```

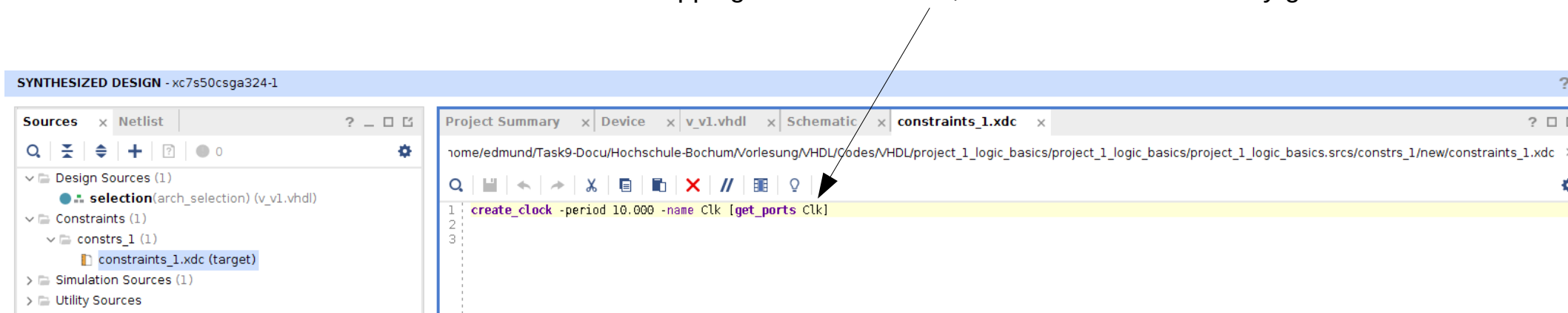
64
65 -- aktiver Prozessbereich
66 -- Sensitivity List reagiert nur auf eine steigende Flanke
67     elsif rising_edge(Clk) then
68
69         S1 <= A or B;      -- Signalzuweisung im Clk'Event Zweig erzeugt ein FlipFlop/Register;
70         S2 <= A and B;    -- Signalzuweisung im Clk'Event Zweig erzeugt ein FlipFlop/Register;
71         S3 <= S1 xor S2;  -- Signalzuweisung im Clk'Event Zweig erzeugt ein FlipFlop/Register;
72         C <= S3 and A;    -- Das Flip Flip S3 liegt im Signalfluss hinter den Flip Flops S1 und S2
73

```



Clock Signal mit dem FPGA verbinden

- Damit das FPGA Design Tool (Vivado) versteht, welches Clock Signal die Clock Information übertragen soll, muss ein Mapping von der Physik auf den VHDL durchgeführt werden
- Mapping im Tool Vivado im Constraint File
- Definition einer Clock Periodendauer und Mapping auf den Port Clk, der im VHDL Code Entity genutzt wird



- Das Reset Signal wird übrigens von dem Synthesetool dann automatisch erkannt

Signale

- Eine Signalzuweisung (SIGNAL Assignment) ist ein 'Concurrent' - Statement
- Signalzuweisungen sind das wichtigste 'Behavioral' - Statement von VHDL
- Signale übertragen Informationen zwischen Prozessen
- Syntax: `signal_name <= wert [after time expression];`
- Die Signalzuweisung ändert links nur dann den Wert, wenn rechts ein Ereignis auftritt
- Der zugewiesene Wert ist ein Folgeereignis

Beispiel Signalzuweisung über zwei Prozesse

```
43 signal S1 : STD_LOGIC; -- Ergebnis einer ODER Verknuepfung
44 signal S2 : STD_LOGIC; -- Ergebnis einer UND Verknuepfung
45 signal S3 : STD_LOGIC; -- Ergebnis einer XOR Verknuepfung
46 signal S4 : STD_LOGIC; -- Ergebnis einer XOR Verknuepfung
47
48 begin -- architecture
49
50     -- synchroner Prozess
51     P1: process(Reset, Clk)
52
53     begin
54         -- Reset
55         -- allen Signalen und Ausgaengen einen Anfangswertzuwesien
56         -- nicht den Eingangssignalen
57         if Reset = '0' then
58             C <= '0'; -- Ausgang
59             S1 <= '0'; -- internes Flip Flop
60             S2 <= '0'; -- internes Flip Flop
61             S3 <= '0'; -- internes Flip Flop
62
63         -- aktiver Prozessbereich
64         -- Sensitivity List reagiert nur auf eine steigende Flanke
65         elsif rising_edge(Clk) then
66             S1 <= A or B; -- Signalzuweisung im Clk'Event Zweig erzeugt ein FlipFlop/Register;
67             S2 <= A and B; -- Signalzuweisung im Clk'Event Zweig erzeugt ein FlipFlop/Register;
68             S3 <= S1 xor S2; -- Signalzuweisung im Clk'Event Zweig erzeugt ein FlipFlop/Register;
69             C <= S4 and A; -- Das Flip Flop S3 liegt im Signalfuss hinter den Flip Flops S1 und S2
70
71         end if; -- reset
72
73     end process P1;
74
75     -- asynchroner Prozess
76     P2: process(B, S3)
77
78     begin
79         S4 <= B xor S3;
80
81     end process P2;
```

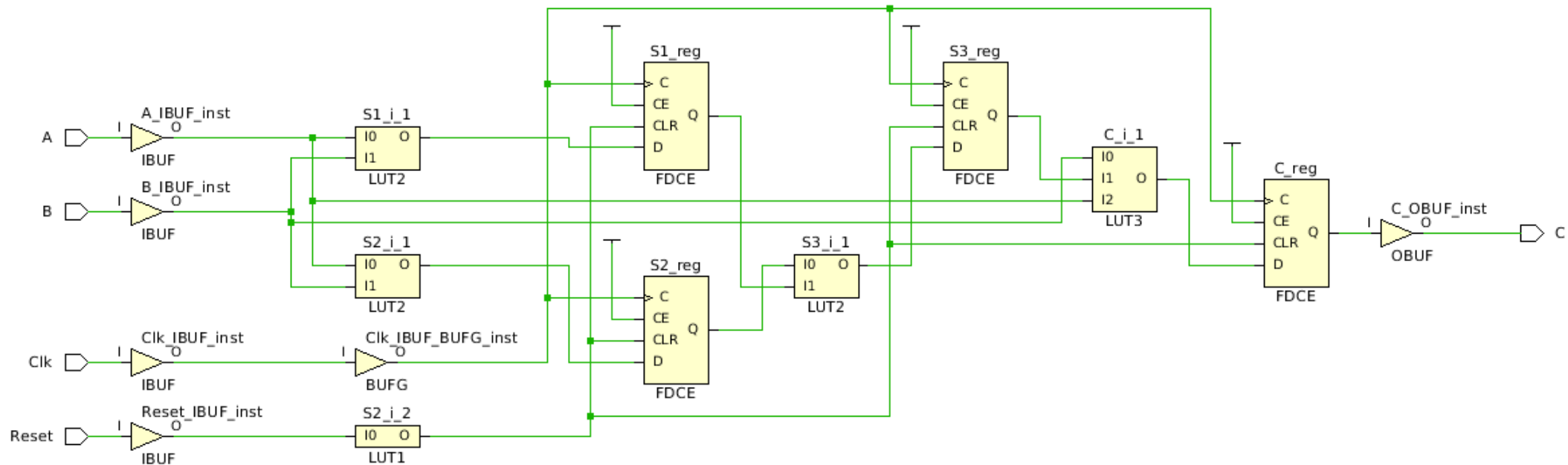
Neues Signal S4

S4 steht nicht im Reset Zweig

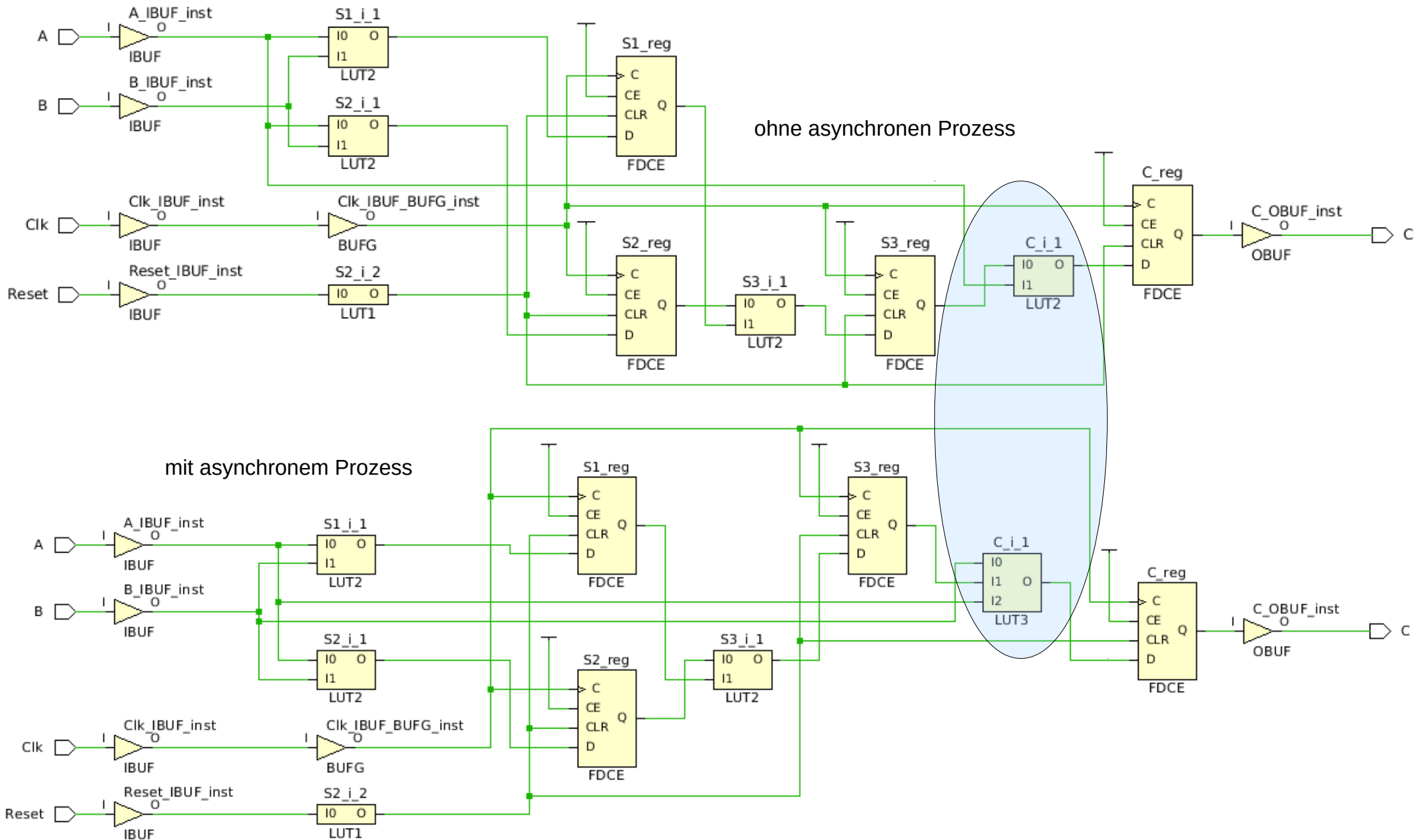
Sensitivity Liste ohne Clk und Reset

Nur die Signale rechts von der Zuweisung

Syntheseergebnis mit asynchronem Prozess



Syntheseergebnis ohne / mit asynchronen Prozess



Bibliotheken und Einheiten

- Die Entwürfe sind in Bibliotheken organisiert
- Bibliotheken können folgende vier Teile enthalten
 - package : globale Deklarationen
 - entity : Design Sicht von Außen (black box)
 - architecture : Design Implementation
 - configuration : Festlegung einer Design-Version
- Neben herstellereigenen- und benutzerdefinierten Bibliotheken gibt es zwei Standardbibliotheken
 - WORK : default-Bibliothek des Benutzers
 - Wenn nichts anderes angegeben wird, ist WORK die Bibliothek, mit der der VHDL-Compiler und -Simulator arbeiten
 - STD : enthält die beiden Packages STANDARD und TEXTIO mit vordefinierten Datentypen und Funktionen.

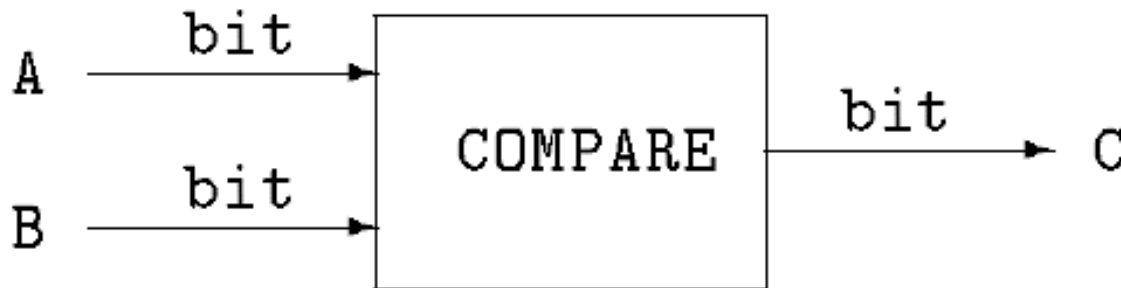
Entity Syntax

Syntax: `entity entity_name is`
 `[generics]`
 `[ports]`
 `[declarations (constants, ports, signals)]`

 `[begin`
 `statements]`
 `end [entity_name];`

Entity Beispiel

Beispiel: entity COMPARE is
 port (A, B: in bit;
 C: out bit);
 end COMPARE;



Architecture Syntax

Syntax: architecture architecture_name of entity_name is
 [arch_declarative_part]

begin
 [arch_statement_part]
end [architecture_name];

Prozess – interner Aufbau

- Das Grundkonstrukt der Verhaltensbeschreibung ist der Prozess
- interner Aufbau: Ein VHDL-Prozess ist einem Programmablauf einer Programmiersprache vergleichbar, mit den üblichen Konzepten
 - { sequentielle Abarbeitung der Anweisungen }
 - { Kontrollanweisungen zur Steuerung des Ablaufs }
 - { Verwendung lokaler Variablen, -Datentypen }
 - { Unterprogrammtechniken (Prozeduren und Funktionen) }

Prozess - Aktivierung

- Spezielle Konstrukte, die festlegen, wann der Prozess aktiv wird
- Im Gegensatz zu Programmen in herkömmlichen Sinne sind Hardwareelemente immer gleichzeitig aktiv
- Ein Prozess hat deshalb entweder eine
 - sensitivity list → wir arbeiten im RTL Design NUR mit sensitivity list
 - oder eine
 - Wait-Anweisungen → wait-Anweisungen gerne in Testbenches verwenden
- Beide Methoden bewirken, dass bei der Änderung von Eingangswerten der Architektur der Prozess von Simulator aktiviert wird, die Anweisungen sequentiell abgearbeitet werden und dadurch neue Ausgangswerte erzeugt werden

Prozess - Ein/Ausgabe

- Nur über Signale können Prozesse nach aussen hin aktiv werden
- Diese Signale können
 - Ein- und Ausgänge der Schaltung sein
 - mehrere Prozesse über interne Signale verbinden
- In Gegensatz zu den Signalen können die Variablen des Prozesses nur prozessintern verwendet werden und stellen so etwas wie lokalen Speicher dar

Prozess - Beispiel

```
architecture ARCH_BEHAV of COMPARE is
begin
    process (A, B)
    begin
        if (A = B) then
            C <= '1' after 1 ns;
        else
            C <= '0' after 2 ns;
        end if;
    end process;
end ARCH_BEHAV;
```

Asynchroner Prozess



Synchroner und Asynchroner Reset

- Es gibt zwei Arten von Reset Verarbeitungen
 - Synchroner Reset
 - Asynchroner Reset

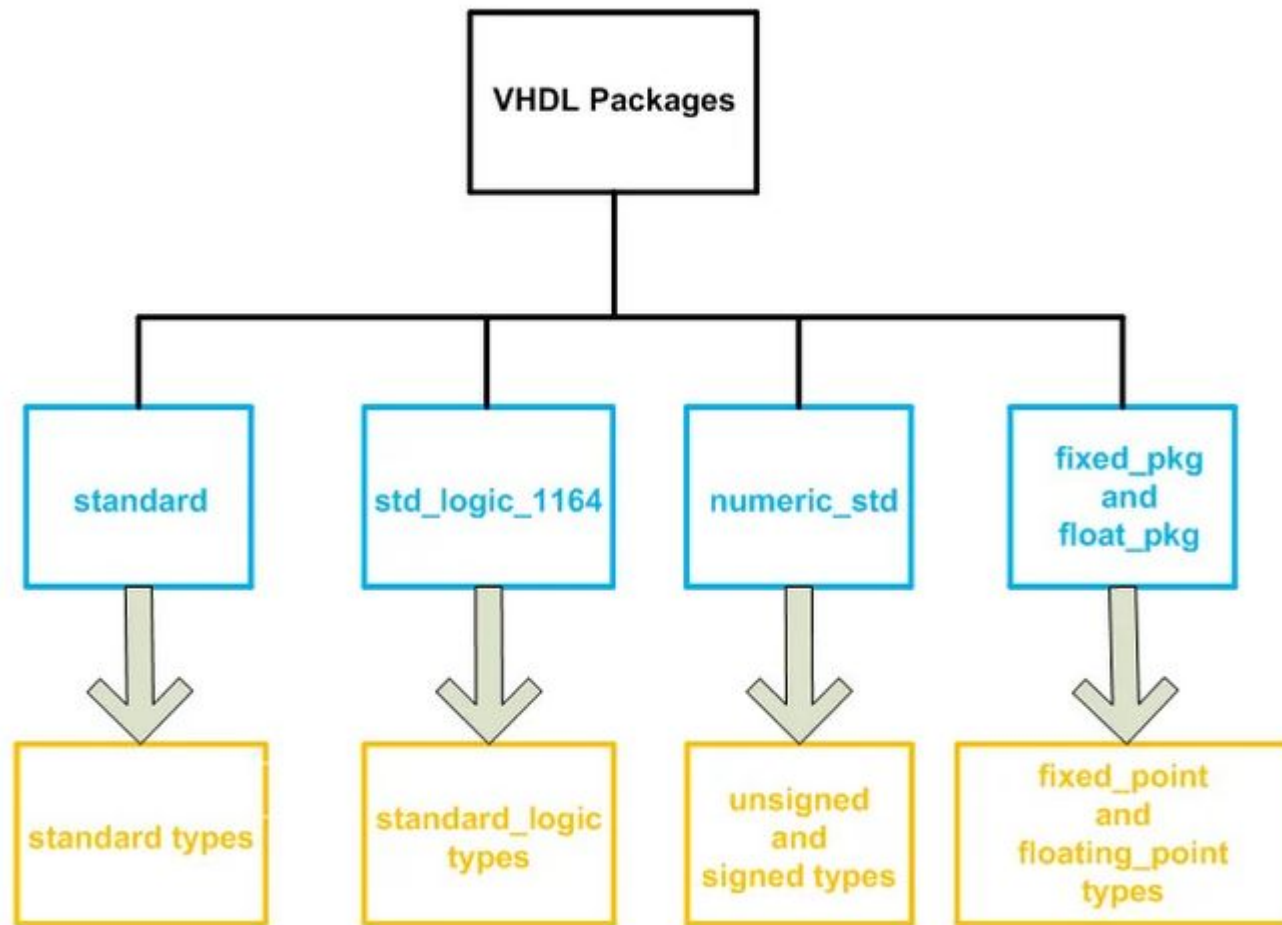
Datentypen

Datentypen für die Synthese oder Simulation

- Es gibt eine Vielzahl von verschiedenen Datentypen
- Synthesefähig
 - integer
 - std_logic
 - std_logic_vector
 - time
- Nicht Synthesefähig → nur für die Simulation im VHDL Code einsetzbar
 - real
 - string

Datentypen

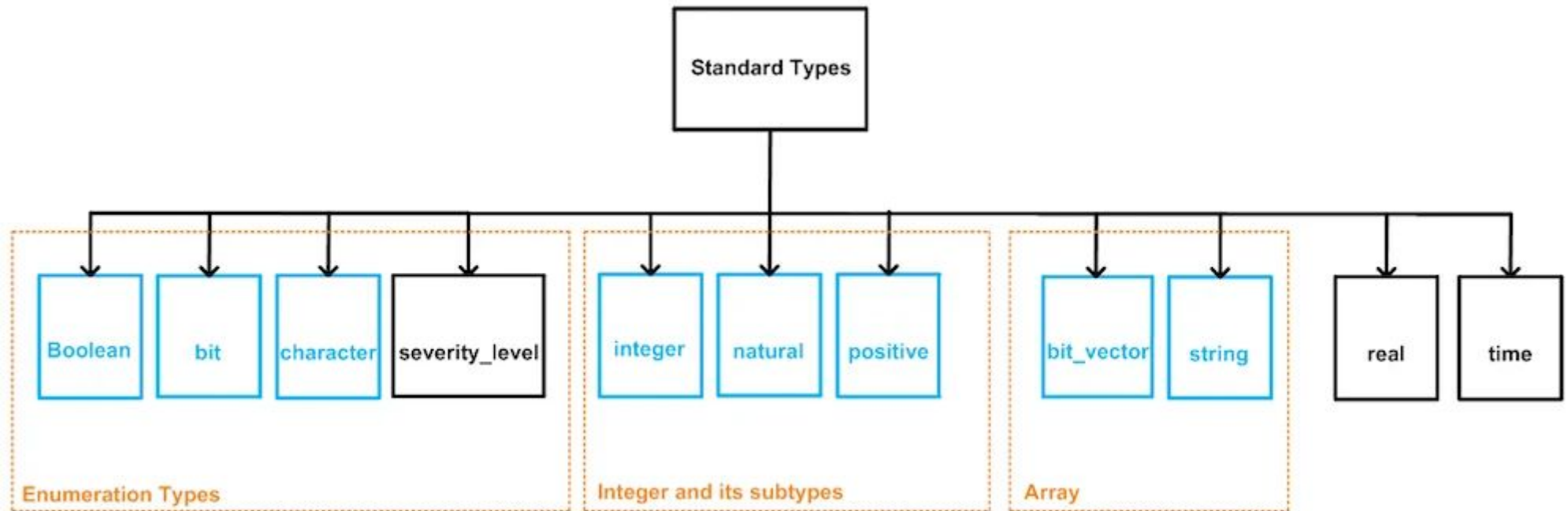
- VHDL Packages - blau
- Definierte Datentypen in dem jeweiligen Package - gelb



Quelle: <https://www.allaboutcircuits.com/technical-articles/integer-and-its-subtypes-in-vhdl/>

Integer Datentyp aus Paket Standard

- VHDL Packages in blau
- Definierte Datentypen in dem jeweiligen Package in gelb



Quelle: <https://www.allaboutcircuits.com/technical-articles/integer-and-its-subtypes-in-vhdl/>

Integer

- Integertypen verarbeiten nur ganze Zahlen
- Range Constraint liefert den Wertebereich
- In VHDL deklariert man typischerweise die passenden Integertypen für die jeweilige Anwendung

```
type short_int      is range 0 to 7;
```

```
type day_of_month   is range 0 to 31;
```

```
type liste          is range 15 downto -16;
```

- Operatoren auf Integertypen sind

- + - * /

- mod - modulo

- rem - remainder

- abs - absolute value

Integer Beispiele (1/6)

- Um synthetisierbaren VHDL Code für mathematische Berechnungen zu implementieren, sollte vornehmlich mit Integerzahlen gerechnet werden
- `std_logic` und `std_logic_vektor` sehr gerne für Ports und digitale Bussysteme nutzen, nicht aber für mathematische Berechnung
- Konvertierungen von `std_logic` in Integer und zurück sind anzuwenden
- `std_logic_unsigned` und `std_logic_signed` sind nicht in IEEE-standardisierten Packages enthalten, aber häufig von Synthestools unterstützt
- Besser: Anwendung von `numeric_std`
- Package `numeric_std` definiert die Typen
 - `signed`
 - `unsigned`

als Array von `std_logic` und definiert alle gebräuchlichen Rechenoperatoren, auch zusammen mit Integern

```
1  -----
2  -- Company:
3  -- Engineer:
4  --
5  -- Create Date: 10/18/2020 03:37:57 PM
6  -- Design Name: Integer Demo Design
7  -- Module Name: v_integer - Behavioral
8  -- Project Name: Addieren und Multiplizieren mit Konvertierungsfunktionen
9  -- Target Devices:
10 -- Tool Versions:
11 -- Description:
12 --
13 -- Dependencies:
14 --
15 -- Revision:
16 -- Revision 0.01 - File Created
17 -- Additional Comments:
18 --
19  -----
20
21
22 library IEEE;
23 use IEEE.STD_LOGIC_1164.ALL;
24 use IEEE.std_logic_signed.all;
25 use IEEE.std_logic_unsigned.all;
26 use IEEE.NUMERIC_STD.ALL;
27 -- use IEEE.STD_LOGIC_ARITH.ALL;
28
29
30 -- Uncomment the following library declaration if using
31 -- arithmetic functions with Signed or Unsigned values
32 --use IEEE.NUMERIC_STD.ALL;
33
34 -- Uncomment the following library declaration if instantiating
35 -- any Xilinx leaf cells in this code.
36 --library UNISIM;
37 --use UNISIM.VComponents.all;
```

Integer Beispiele (2/6)

- Ports immer mit std_logic und std_logic_vector definieren
- Für die interne Rechnung soll der Typ
 - Integer;
 - Integer range x to y;

genutzt werden

```
39 entity v_integer is
40     Port ( Reset    : in  STD_LOGIC;
41           Clk       : in  STD_LOGIC;
42           A         : in  STD_LOGIC_VECTOR (1 downto 0); -- von 0 ... 3
43           B         : in  STD_LOGIC_VECTOR (1 downto 0); -- von 0 ... 3
44           C         : in  STD_LOGIC_VECTOR (3 downto 0); -- von 0 ... 15
45           Result    : out STD_LOGIC_VECTOR (4 downto 0)); -- von 0 ... 31
46 end v_integer;
47
48 architecture Behavioral of v_integer is
49
50     signal Add1 : integer range 0 to 7;
51     signal Add2 : integer range 0 to 31;
52     signal Mul3 : integer range 0 to 255;
53     signal Fin4 : integer range 0 to 31;
54
```

Integer Beispiele (3/6)

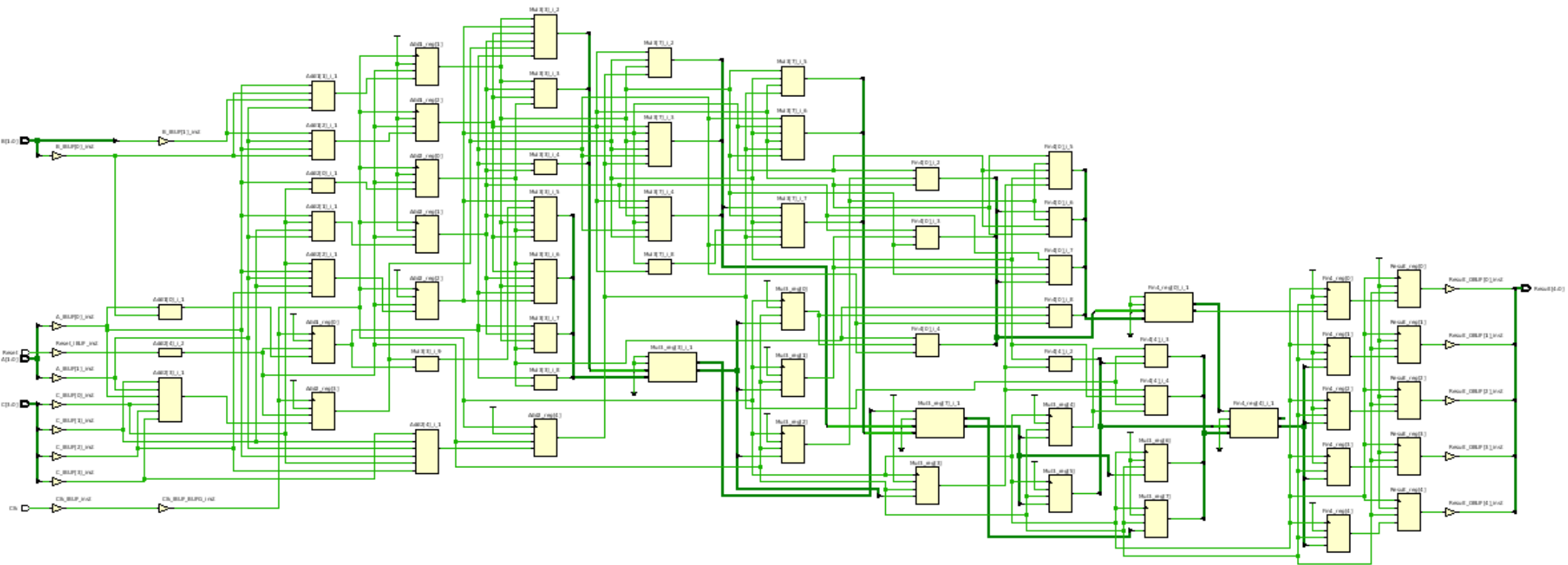
- Um synthetisierbaren VHDL Code für mathematische Berechnungen zu implementieren, sollte vornehmlich mit Integerzahlen gerechnet werden
- `std_logic` und `std_logic_vektor` sehr gerne für Ports und digitale Bussysteme nutzen, nicht aber für mathematische Berechnung
- Konvertierungen von `std_logic` in Integer und zurück sind anzuwenden
- `std_logic_unsigned` und `std_logic_signed` sind nicht in IEEE-standardisierten Packages enthalten, aber häufig von Synthesetools unterstützt
- Besser: Anwendung von `numeric_std`
- Package `numeric_std` definiert die Typen
 - `signed`
 - `unsigned`

als Array von `std_logic` und definiert alle gebräuchlichen Rechenoperatoren, auch zusammen mit Integern

```
48 architecture Behavioral of v_integer is
49
50     signal Add1 : integer range 0 to 7;
51     signal Add2 : integer range 0 to 31;
52     signal Mul3 : integer range 0 to 255;
53     signal Fin4 : integer range 0 to 31;
54
55     begin -- achitecture
56
57     P1: process(Reset, Clk) is
58
59         begin -- process P1
60
61             if Reset = '0' then
62
63                 Add1 <= 0;
64                 Add2 <= 0;
65                 Mul3 <= 0;
66                 Fin4 <= 0;
67                 Result <= (others => '0'); -- meint "00000";
68
69             elsif rising_edge(Clk) then
70
71                 Add1 <= to_integer(unsigned(A)) + to_integer(unsigned(B));
72                 Add2 <= to_integer(unsigned(A)) + to_integer(unsigned(C));
73                 Mul3 <= Add1 * Add2;
74                 Fin4 <= (Add1 + Add2 + Mul3) / 8;
75
76                 Result <= std_logic_vector(to_unsigned(Fin4, Result'length));
77
78             end if; -- Reset
79
80         end process P1;
81
82     end Behavioral;
```

Integer Beispiele (4/6) – schnell komplexe Designs

- Es entstehen sehr schnell sehr komplexe Design
- Besser das Design in einfache Komponenten aufteilen bzw. Schritt für Schritt synthetisieren



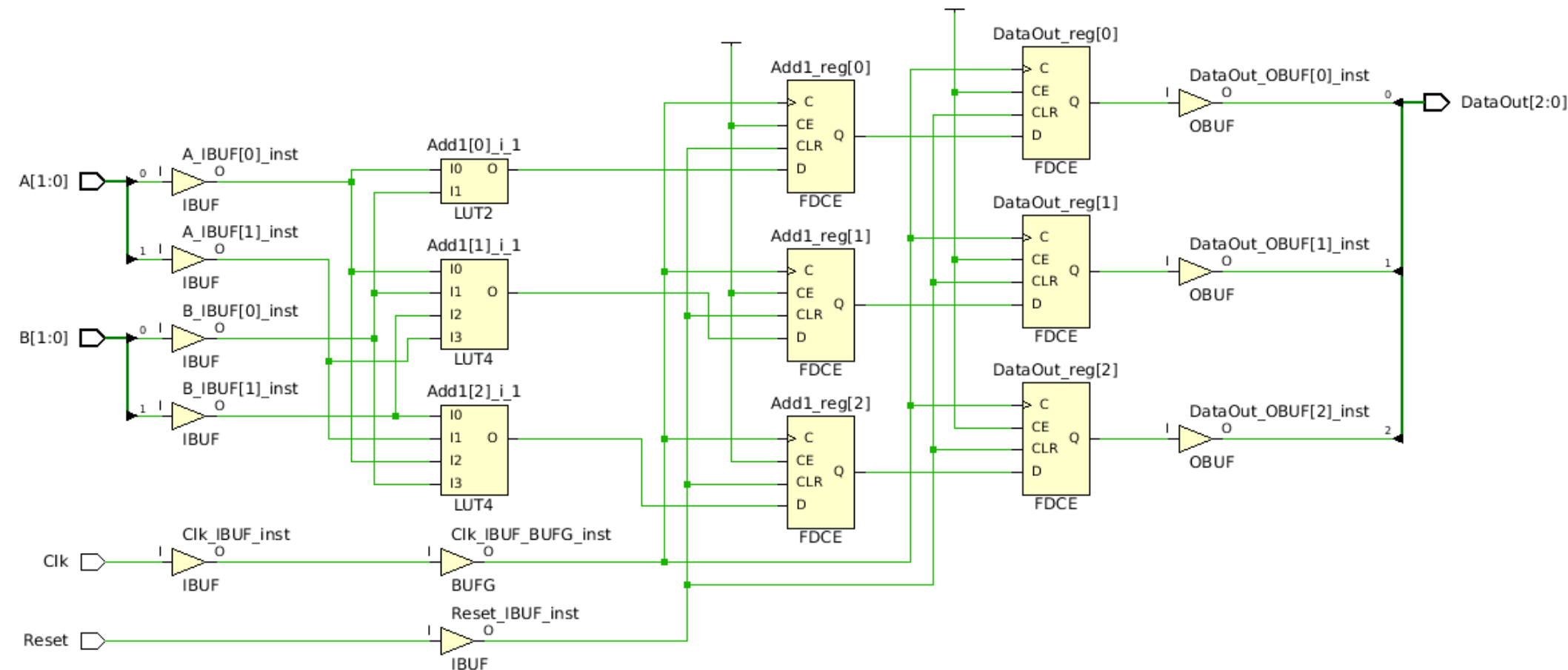
Integer Beispiele (5/6) – 2 Bit Addierer

- 2x 2-Bit Eingänge werden zu einem 3-Bit Ausgang aufaddiert

```
--
21 library IEEE;
22 use IEEE.STD_LOGIC_1164.ALL;
23 use IEEE.STD_LOGIC_signed.ALL;
24 use IEEE.STD_LOGIC_unsigned.ALL;
25 use IEEE.NUMERIC_STD.ALL;
26
27 entity v_integer_short is
28   Port ( Reset      : in STD_LOGIC;
29         Clk         : in STD_LOGIC;
30         A           : in STD_LOGIC_VECTOR (1 downto 0);
31         B           : in STD_LOGIC_VECTOR (1 downto 0);
32         DataOut      : out STD_LOGIC_VECTOR (2 downto 0));
33 end v_integer_short;
34
35 architecture Behavioral of v_integer_short is
36
37   signal Add1 : integer range 0 to 7;
38
39   begin -- architecture
40
41     P1 : process(Reset, Clk) is
42
43       begin --process P1
44
45         if Reset = '1' then
46
47           Add1 <= 0;
48           DataOut <= (others => '0');
49
50         elsif rising_edge(Clk) then
51
52           Add1 <= to_integer(unsigned(A)) + to_integer(unsigned(B));
53
54           DataOut <= std_logic_vector(to_unsigned(Add1, DataOut'length));
55
56         end if; -- reset / Clk
57
58       end process P1;
59
60     end Behavioral;
```

Integer Beispiele (6/6) – 2 Bit Addierer

- 2x 2-Bit Eingänge werden zu einem 3-Bit Ausgang aufaddiert
- Nachteil: unnötige Latenz: 2 Registerstufen hintereinander: Add1_reg und DataOutReg
 - eine der beiden Registerstufen ist nicht nötig und führt nur einen unnötigen Takt Verzögerung ein



Konvertierungsfunktionen (1/4)

- Integertypen lassen sich gut konvertieren
- Dabei ist zu beachten, dass ein Integer nicht direkt in `std_logic_vector` gewandelt werden kann
- Mit dem Package `IEEE.NUMERIC_STD.ALL` können diverse Konvertierungsfunktionen importiert werden

- INTEGER to SIGNED

```
signal input   : integer;  
signal output  : signed(3 downto 0);  
output <= to_signed(input, output'length);
```

- INTEGER to UNSIGNED

```
signal input   : integer;  
signal output  : unsigned(3 downto 0);  
output <= to_unsigned(input, output'length);
```

Konvertierungsfunktionen (2/4)

- STD_LOGIC_VECTOR to SIGNED

```
signal input_6 : std_logic_vector(3 downto 0);
```

```
signal output_6 : signed(3 downto 0);
```

```
output_6 <= signed(input_6);
```

- STD_LOGIC_VECTOR to UNSIGNED

```
signal input_5 : std_logic_vector(3 downto 0);
```

```
signal output_5 : unsigned(3 downto 0);
```

```
output_5 <= unsigned(input_5);
```


Konvertierungsfunktionen (3/4)

- SIGNED to INTEGER

```
signal input_10 : signed(3 downto 0);
```

```
signal output_10 : integer;
```

```
output_10 <= to_integer(input_10);
```

- SIGNED to STD_LOGIC_VECTOR

```
signal input_11 : signed(3 downto 0);
```

```
signal output_11 : std_logic_vector(3 downto 0);
```

```
output_11 <= std_logic_vector(input_11);
```

Konvertierungsfunktionen (4/4)

- INTEGER to STD_LOGIC_VECTOR

```
signal input_1 : integer;  
  
signal output_1a : std_logic_vector(3 downto 0);  
  
signal output_1b : std_logic_vector(3 downto 0);  
  
output_1a <= std_logic_vector(to_unsigned(input_1, output_1a'length));  
output_1b <= std_logic_vector(to_signed(input_1, output_1b'length));
```

- STD_LOGIC_VECTOR to INTEGER

```
signal input_4 : std_logic_vector(3 downto 0);  
  
signal output_4a : integer;  
  
signal output_4b : integer;  
  
output_4a <= to_integer(unsigned(input_4));  
output_4b <= to_integer(signed(input_4));
```

Variablen und Signale

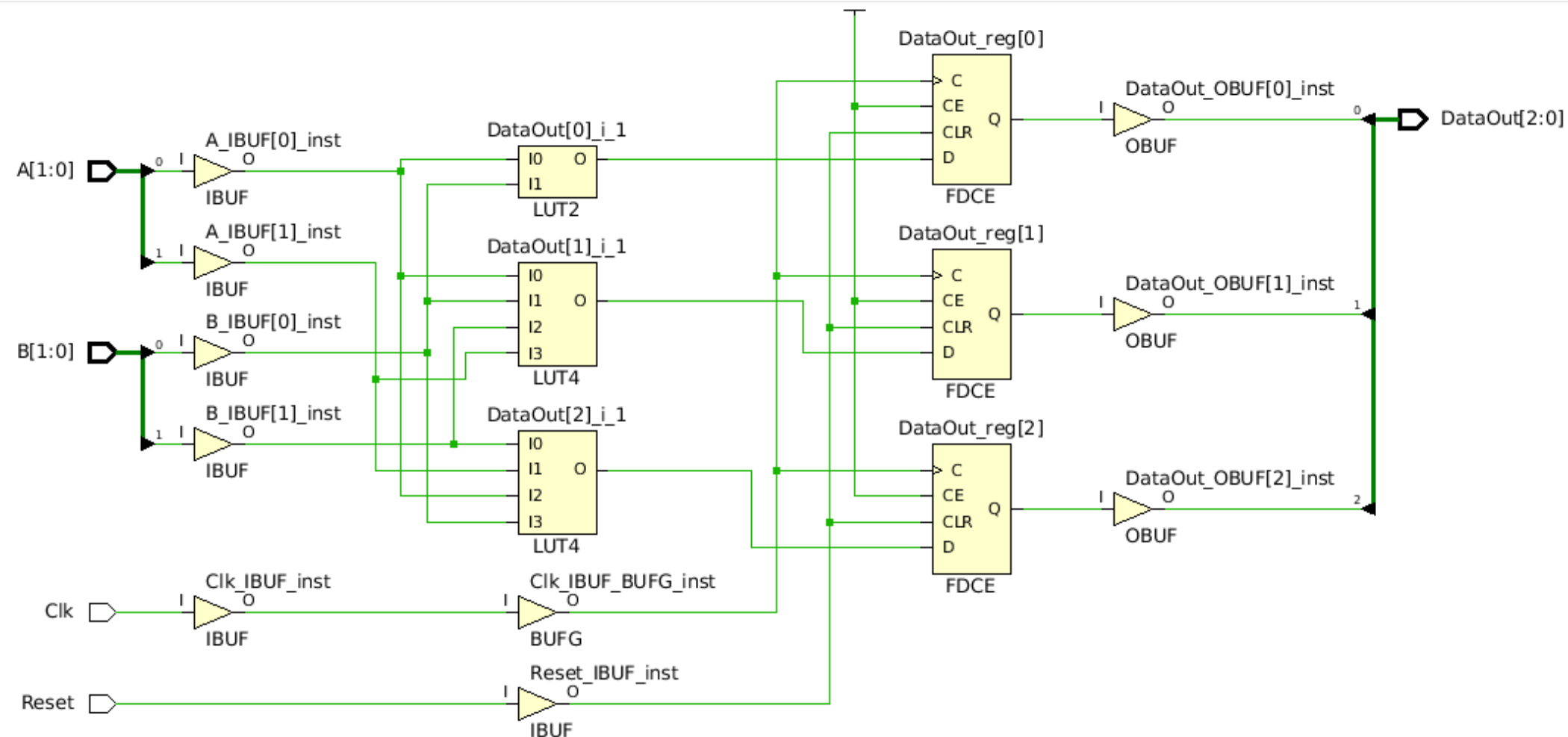
Synchrone Signale gegen asynchrone Signale tauschen

- Für jede Signalzuweisung entsteht in einem synchronen Prozess ein Register/Flip Flop
- Berechnungen / Zuweisungen, die kein Register / Flip Flop erzeugen sollen, werden dann in einem asynchronen Prozess durchgeführt
- Dann entsteht für die asynchronen Signale kein Flip Flop / Register
- Es wird diesen asynchronen Signalen auch kein Reset Wert zugeordnet

```
27 entity v_integer_short is
28     Port ( Reset      : in STD_LOGIC;
29           Clk         : in STD_LOGIC;
30           A           : in STD_LOGIC_VECTOR (1 downto 0);
31           B           : in STD_LOGIC_VECTOR (1 downto 0);
32           DataOut      : out STD_LOGIC_VECTOR (2 downto 0));
33 end v_integer_short;
34
35 architecture Behavioral of v_integer_short is
36
37     signal Add1 : integer range 0 to 7;
38
39     begin -- architecture
40
41         P1 : process(Reset, Clk) is
42
43             begin --process P1
44
45                 if Reset = '1' then
46                     DataOut <= (others => '0');
47
48                 elsif rising_edge(Clk) then
49
50                     DataOut <= std_logic_vector(to_unsigned(Add1, DataOut'length));
51
52                 end if; -- reset / Clk
53
54             end process P1;
55
56         P2 : process(A,B) is
57
58             begin --process P2
59
60                 Add1 <= to_integer(unsigned(A)) + to_integer(unsigned(B));
61
62             end process P2;
63
64     end Behavioral;
```

Synchrone Signale gegen asynchrone Signale tauschen

- Nur eine Register - Ebene zum Schluss bei DataOut



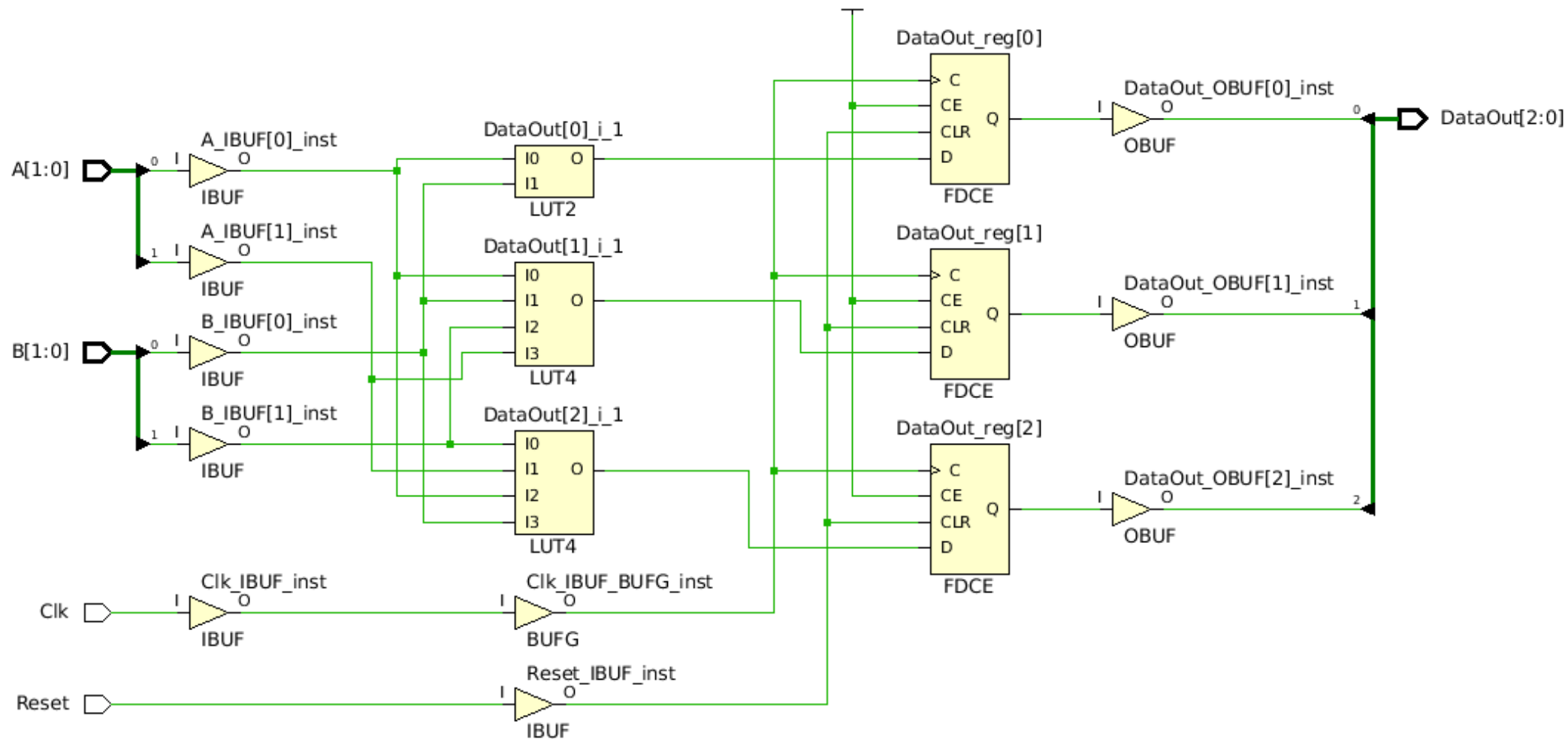
Synchrone Signale gegen asynchrone Variablen tauschen

- Für jede Signalzuweisung entsteht im synchronen Prozess ein Register/Flip Flop
- Um lokale Berechnungen in einem synchronen Prozess durchzuführen, kann man besser Variablen nutzen

```
21 library IEEE;
22 use IEEE.STD_LOGIC_1164.ALL;
23 use IEEE.STD_LOGIC_signed.ALL;
24 use IEEE.STD_LOGIC_unsigned.ALL;
25 use IEEE.NUMERIC_STD.ALL;
26
27 entity v_integer_short is
28     Port ( Reset      : in STD_LOGIC;
29           Clk         : in STD_LOGIC;
30           A           : in STD_LOGIC_VECTOR (1 downto 0);
31           B           : in STD_LOGIC_VECTOR (1 downto 0);
32           DataOut     : out STD_LOGIC_VECTOR (2 downto 0));
33 end v_integer_short;
34
35 architecture Behavioral of v_integer_short is
36
37     begin -- architecture
38
39         P1 : process(Reset, Clk) is
40
41             variable Add1 : integer range 0 to 7 := 0;
42
43             begin --process P1
44
45                 if Reset = '1' then
46                     DataOut <= (others => '0');
47
48                 elsif rising_edge(Clk) then
49
50                     Add1 := to_integer(unsigned(A)) + to_integer(unsigned(B));
51
52                     DataOut <= std_logic_vector(to_unsigned(Add1, DataOut'length));
53
54                 end if; -- reset / Clk
55
56             end process P1;
57
58         end process P1;
59
60 end Behavioral;
```

Synchrone Signale gegen asynchrone Variablen tauschen

- Für jede Signalzuweisung entsteht im synchronen Prozess ein Register/Flip Flop
- Um lokale Berechnungen in einem synchronen Prozess durchzuführen, kann man besser Variablen nutzen



Variablen

- Variablen erzeugen typischerweise keine Register / Flip Flops
 - Falls doch, bzw. falls ein Latch entsteht, ist dies eher nicht so gewollt
 - Es entstehen dann Latches / Register bei Variablen, falls die Variable vor der ersten Zuweisung auf der rechten Seite verwendet wird → dies ist aber nicht unbedingt ein guter CodeingStyle
- Variablen erzeugen typischerweise kombinatorische Logik
 - sowohl in synchronen Prozessen
 - als auch in asynchronen Prozessen
- Die Position, an denen die Variablen im VHDL Code eingesetzt werden, hat durchaus einen Einfluss auf den Aufbau der Logik
- Die Position der VHDL Code Zeilen für Register / Flip Flops im synchronen Prozess hat bei Signalen keinen Einfluss auf die Art, wie die Logik aufgebaut wird

Signale außerhalb des synchronen Prozesses

- Signale außerhalb des synchronen Prozesses dienen in diesem Falle nur als elektrische Verbindungen
- Gruppenübung
 - Wieviele FlipFlops erzeugt der folgende Code?

```

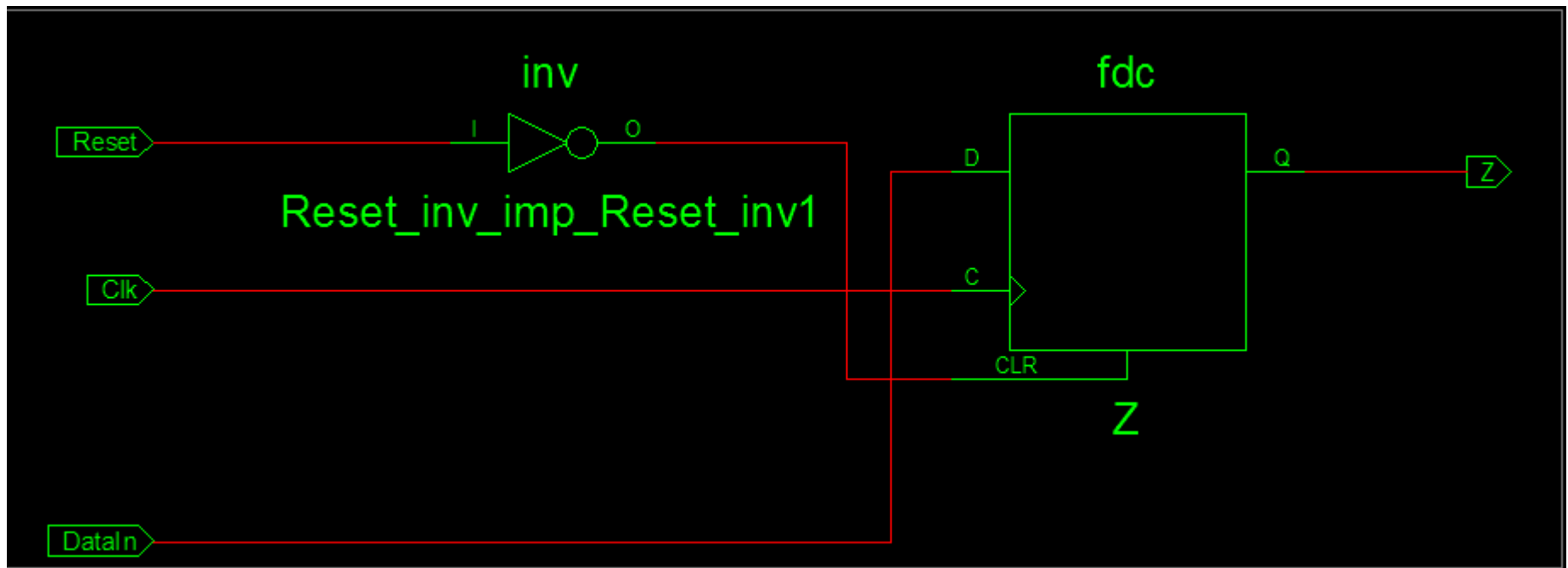
30 entity FourBitSHR is
31     Port ( DataIn    : in  STD_LOGIC;
32           Reset      : in  STD_LOGIC;
33           Clk         : in  STD_LOGIC;
34           Z           : out STD_LOGIC
35     );
36 end FourBitSHR;
37
38 architecture Behavioral of FourBitSHR is
39
40     signal Tmp0 : std_logic;
41     signal Tmp1 : std_logic; -- _vector(3 downto 0);
42     signal Tmp2 : std_logic;
43
44 begin
45
46     Tmp0 <= DataIn;
47     Tmp1 <= Tmp0;
48     Tmp2 <= Tmp1;
49
50     P1Synch: process(Clk,Reset)
51
52     begin
53
54         if Reset = '0' then
55
56             Z <= '0';
57
58             elsif Clk'event and Clk = '1' then
59
60                 Z <= Tmp2;|
61
62             end if;
63
64         end process P1Synch;
65
66 end Behavioral;

```

Handschematic

- Bitte hier schematic von den tatsächlichen Signalverläufen aufzeichnen
- Also DataIn and Tmp0 Tmp0 and Tmp1 ... Signalnamen werden wegoptimiert

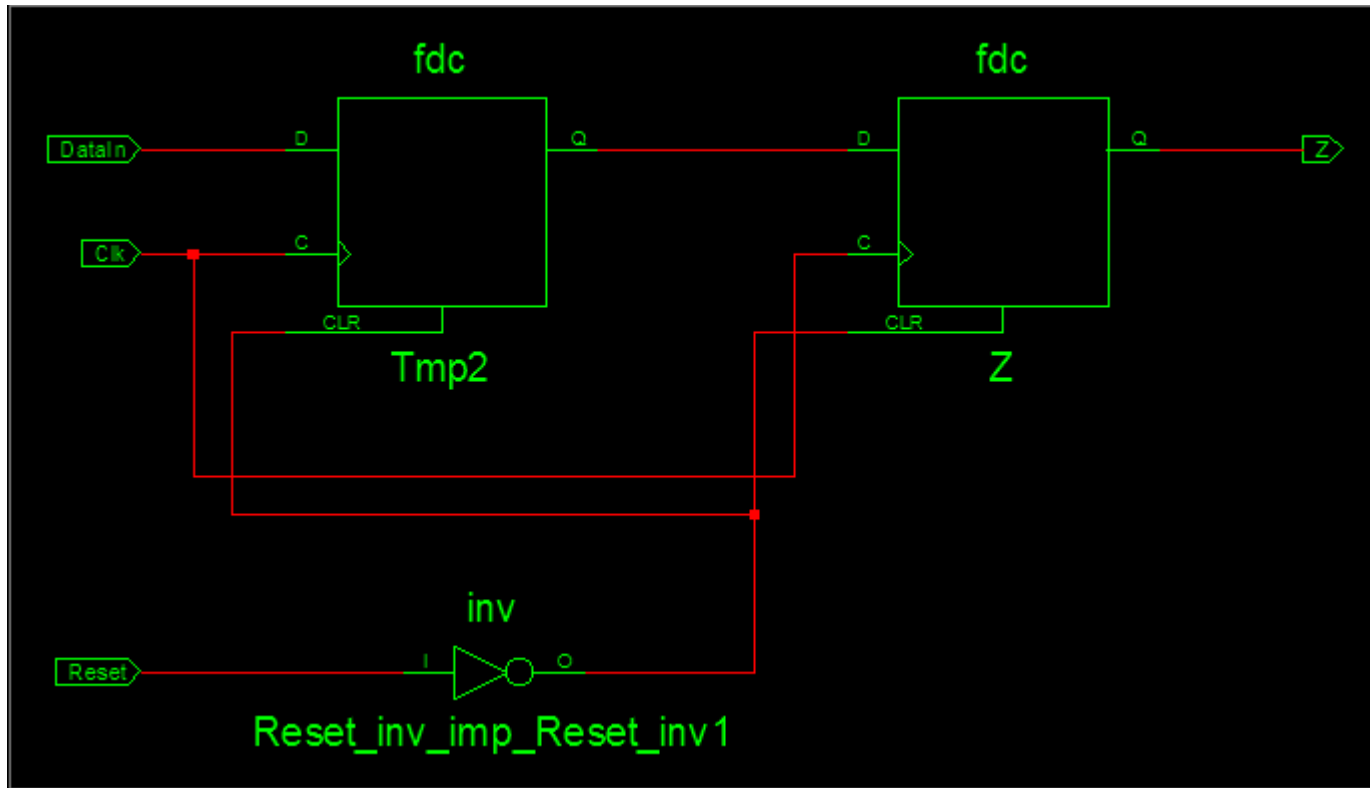
Syntheseergebnis



Ein synchrones Signal mehr

```
30 entity FourBitSHR is
31     Port ( DataIn   : in  STD_LOGIC;
32           Reset     : in  STD_LOGIC;
33           Clk        : in  STD_LOGIC;
34           Z          : out STD_LOGIC
35     );
36 end FourBitSHR;
37
38 architecture Behavioral of FourBitSHR is
39
40     signal Tmp0 : std_logic;
41     signal Tmp1 : std_logic; -- _vector(3 downto 0);
42     signal Tmp2 : std_logic;
43
44 begin
45
46     Tmp0 <= DataIn;
47     Tmp1 <= Tmp0;
48
49     P1Synch: process(Clk,Reset)
50
51     begin
52
53         if Reset = '0' then
54
55             Z <= '0';
56             Tmp2 <= '0';
57
58         elsif Clk'event and Clk = '1' then
59
60             Z <= Tmp2;
61             Tmp2 <= Tmp1;
62
63         end if;
64
65     end process P1Synch;
66
67 end Behavioral;
```

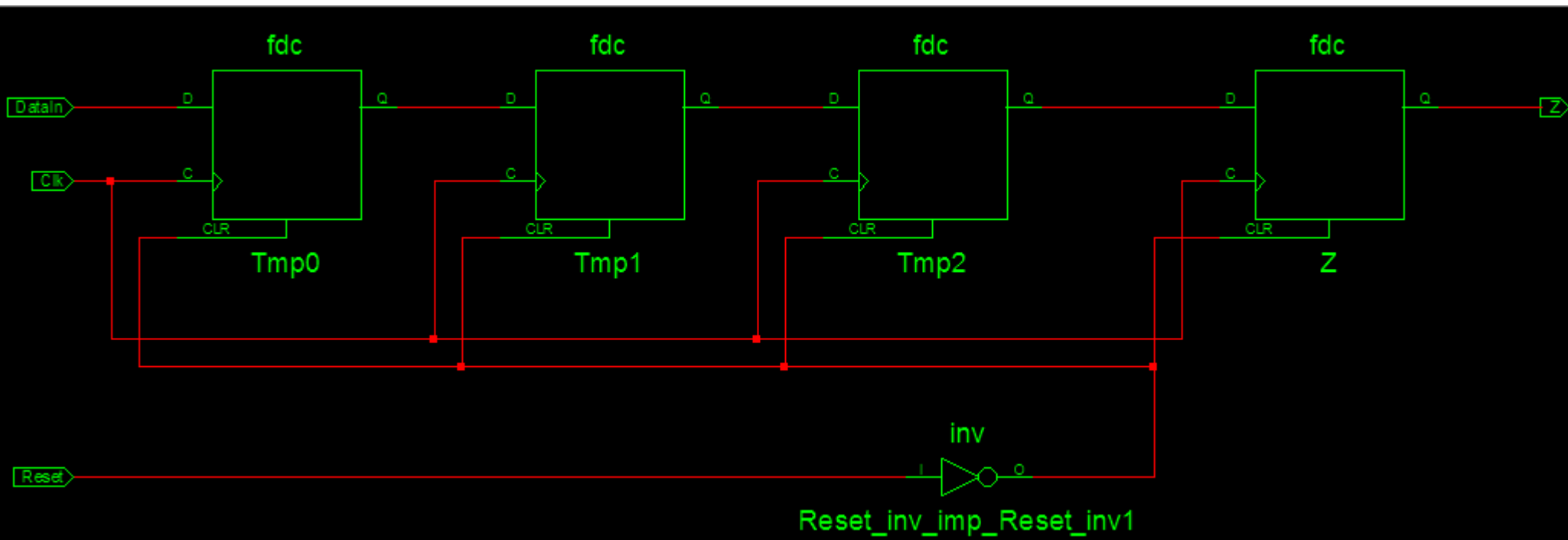
Syntheseresult



Mehrere Prozesse parallel

```
48 P1Synch: process (Clk, Reset)
49
50 begin
51
52     if Reset = '0' then
53
54         Z <= '0';
55         Tmp2 <= '0';
56
57     elsif Clk'event and Clk = '1' then
58
59         Z <= Tmp2;
60         Tmp2 <= Tmp1;
61
62     end if;
63
64 end process P1Synch;
65
66
67 P2Synch: process (Clk, Reset)
68
69 begin
70
71     if Reset = '0' then
72
73         Tmp0 <= '0';
74         Tmp1 <= '0';
75
76     elsif Clk'event and Clk = '1' then
77
78         Tmp0 <= DataIn;
79         Tmp1 <= Tmp0;
80
81     end if;
82
83 end process P2Synch;
84
85
86
87 end Behavioral;
```

Syntheseergebnis



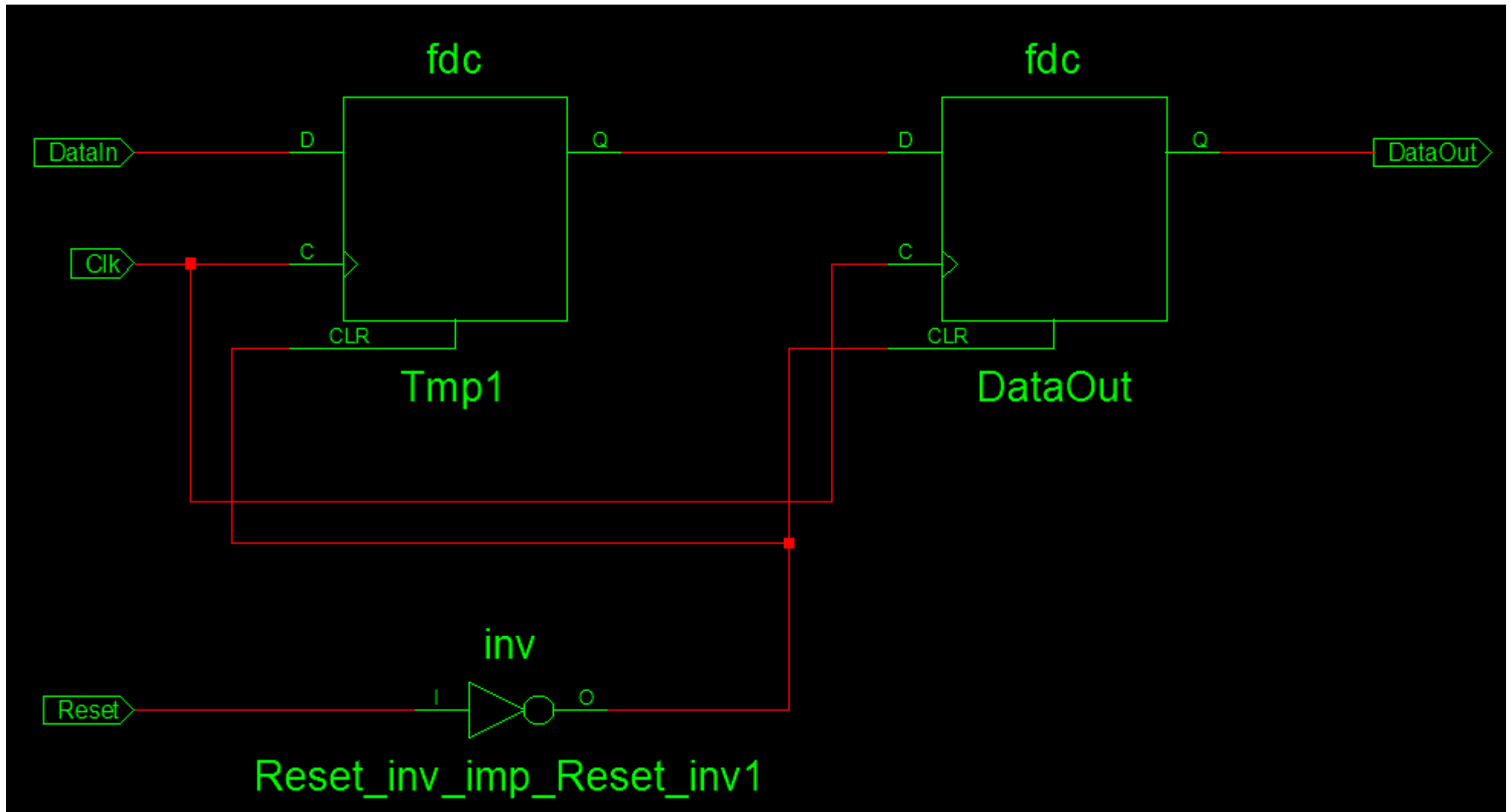
Variablen

- Alle Signale erhalten am Ende eines Prozesses gleichzeitig die korrespondierende Information
- Variablen erhalten sofort die Information
- Zuweisungsoperator
 - Signal $\leq$
 - Variable $:=$

Wie viele FlipFlops erzeugt dieser Code?

```
30 entity ShiftVariables is
31     Port ( DataIn   : in   STD_LOGIC;
32           Reset    : in   STD_LOGIC;
33           Clk       : in   STD_LOGIC;
34           DataOut   : out  STD_LOGIC
35           );
36 end ShiftVariables;
37
38 architecture Behavioral of ShiftVariables is
39
40     signal Tmp1 : std_logic;
41
42 begin
43
44     P1Synch: process(Clk,Reset)
45
46     variable Tmp0 : std_logic;
47     variable Tmp2 : std_logic;
48
49     begin
50
51         if Reset = '0' then
52
53             DataOut <= '0';
54
55         elsif Clk'event and Clk = '1' then
56
57             Tmp2 := Tmp1;
58             DataOut <= Tmp2;
59
60         end if;
61
62     end process P1Synch;
63
64
65
66     P2Synch: process(Clk,Reset)
67
68     variable Tmp0 : std_logic;
69
70     begin
71
72         if Reset = '0' then
73
74             Tmp1 <= '0';
75
76         elsif Clk'event and Clk = '1' then
77
78             Tmp0 := DataIn;
79             Tmp1 <= Tmp0;
80
81         end if;
82
83     end process P2Synch;
84
85 end Behavioral;
```

Syntheseergebnis

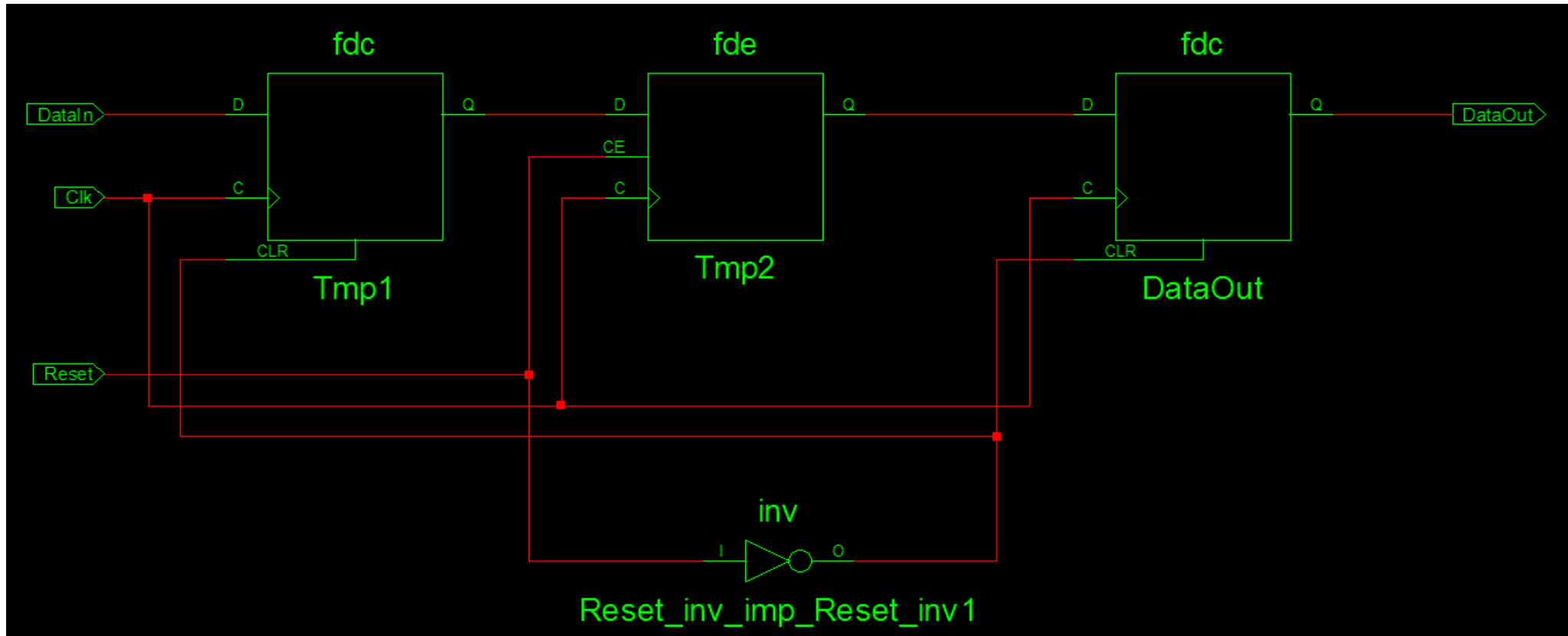


Wie viele FlipFlops erzeugt dieser Code?

```
30 entity ShiftVariables is
31     Port ( DataIn      : in  STD_LOGIC;
32           Reset       : in  STD_LOGIC;
33           Clk          : in  STD_LOGIC;
34           DataOut      : out STD_LOGIC
35     );
36 end ShiftVariables;
37
38 architecture Behavioral of ShiftVariables is
39
40     signal Tmp1 : std_logic;
41
42     begin
43
44         P1Synch: process(Clk,Reset)
45
46             variable Tmp0 : std_logic;
47             variable Tmp2 : std_logic;
48
49             begin
50
51                 if Reset = '0' then
52
53                     DataOut <= '0';
54
55                 elsif Clk'event and Clk = '1' then
56
57                     DataOut <= Tmp2;
58                     Tmp2 := Tmp1;
59
60                 end if;
61
62             end process P1Synch;
```

```
63
64
65         P2Synch: process(Clk,Reset)
66
67             variable Tmp0 : std_logic;
68
69             begin
70
71                 if Reset = '0' then
72
73                     Tmp1 <= '0';
74
75                 elsif Clk'event and Clk = '1' then
76
77                     Tmp0 := DataIn;
78                     Tmp1 <= Tmp0;
79
80                 end if;
81
82             end process P2Synch;
83
84 end Behavioral;
```

Syntheseergebnis

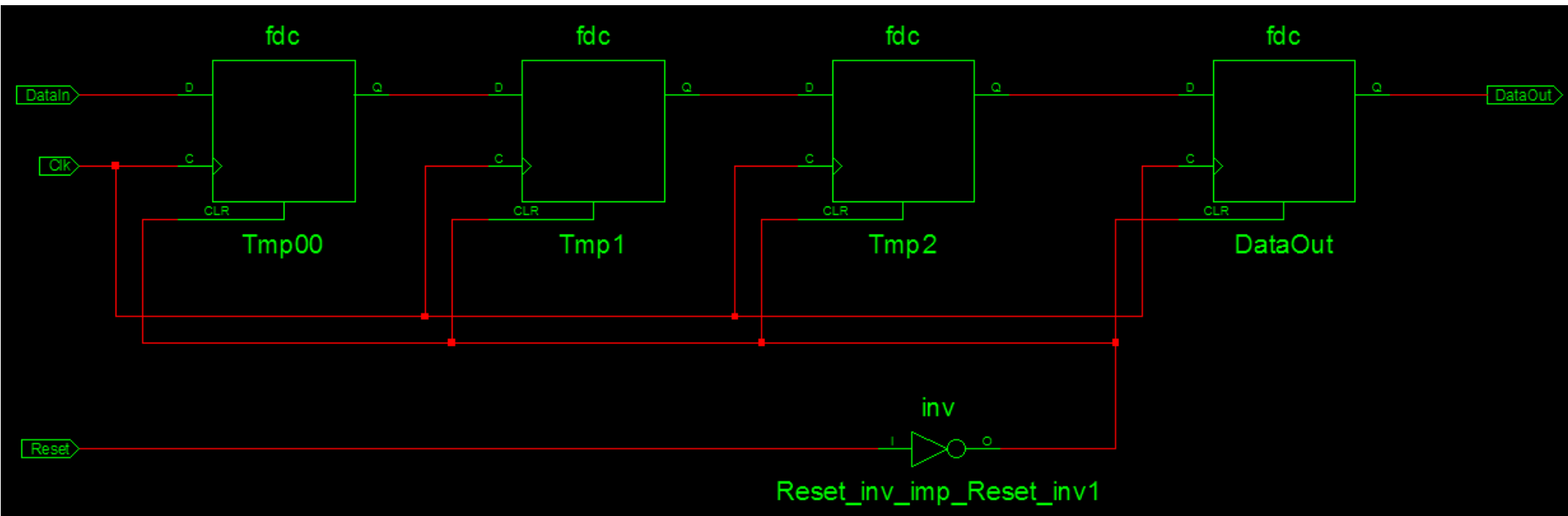


Wie viele FlipFlops erzeugt dieser Code?

```
30 entity ShiftVariables is
31     Port ( DataIn   : in   STD_LOGIC;
32           Reset     : in   STD_LOGIC;
33           Clk        : in   STD_LOGIC;
34           DataOut    : out  STD_LOGIC
35     );
36 end ShiftVariables;
37
38 architecture Behavioral of ShiftVariables is
39
40     signal Tmp1 : std_logic;
41
42     begin
43
44         P1Synch: process(Clk,Reset)
45
46             variable Tmp0 : std_logic;
47             variable Tmp2 : std_logic;
48
49             begin
50
51                 if Reset = '0' then
52
53                     DataOut <= '0';
54                     Tmp2 := '0';
55
56                 elsif Clk'event and Clk = '1' then
57
58                     DataOut <= Tmp2;
59                     Tmp2 := Tmp1;
60
61                 end if;
62
63             end process P1Synch;
64
```

```
64
65
66         P2Synch: process(Clk,Reset)
67
68             variable Tmp0 : std_logic;
69
70             begin
71
72                 if Reset = '0' then
73
74                     Tmp1 <= '0';
75                     Tmp0 := '0';
76
77                 elsif Clk'event and Clk = '1' then
78
79                     Tmp1 <= Tmp0;
80                     Tmp0 := DataIn;
81
82                 end if;
83
84             end process P2Synch;
85
86         end Behavioral;
87
```

Syntheseergebnis



Variablen als Zwischenspeicher

- Wird hier eine Variable vor der ersten Zuweisung verwendet, muss sie speichernd sein
 - Wenn Variablen als Zwischenspeicher dienen, erzeugen Sie ebenfalls FlipFlops bzw. Register
 - Ansonsten bilden Variablen kombinatorische Logik
- Tipp:
 - Versuche, die Aufgabe mit Signalen zu lösen
 - Wenn es tatsächlich nicht anders geht, nimm Variablen
 - Was mit Signalen nicht synthetisierbar ist, das geht auch mit Variablen nicht

Warum dann überhaupt Variablen?

- Unübersichtlich, wenn man komplexe Rechnung nur mit Signalzuweisung und Klammerung beschreibt

```
Result1 <= (A + B) - (C + D);  
Result2 <= (A + B) - (E + F);
```

- Besser, die Werte nach und nach über Variablen berechnen und am Schluss auf ein Signal zuzuweisen

```
Sig1 := A + B;  
Sig2 := C + D;  
Sig3 := E + F;  
Result1 <= Sig1 - Sig2;  
Result2 <= Sig1 - Sig3;
```

Gruppenübung

- Verhalten sich die Codes unterschiedlich?

```
signal a: std_logic;  
  
process (clk)  
begin  
    if rising_edge(clk) then  
        a <= a and b;  
        a <= a and c;  
    end if;  
end process;
```

```
process (clk)  
    variable a: std_logic;  
begin  
    if rising_edge(clk) then  
        a := a and b;  
        a := a and c;  
    end if;  
end process;
```

Lösung

- Links: Das Ergebnis lautet

- $a \leq a \text{ and } c$

denn die Zeile zuvor wird ignoriert

- Rechts: Das Ergebnis lautet

- $a := a \text{ and } b \text{ and } c$

a erzeugt ein Flip-Flop in den Eingang der Logik,
aus dem Rückkopplungspfad, der Ausgang a ist
aber nicht mit einem Ausgangsregister ausgestattet

Einzelübung

- Schreiben Sie VHDL code für die folgende Funktion

```
A1 := A1 xor DataIn;
```

```
DataOut <= A1;
```

und

```
A1 <= A1 xor DataIn;
```

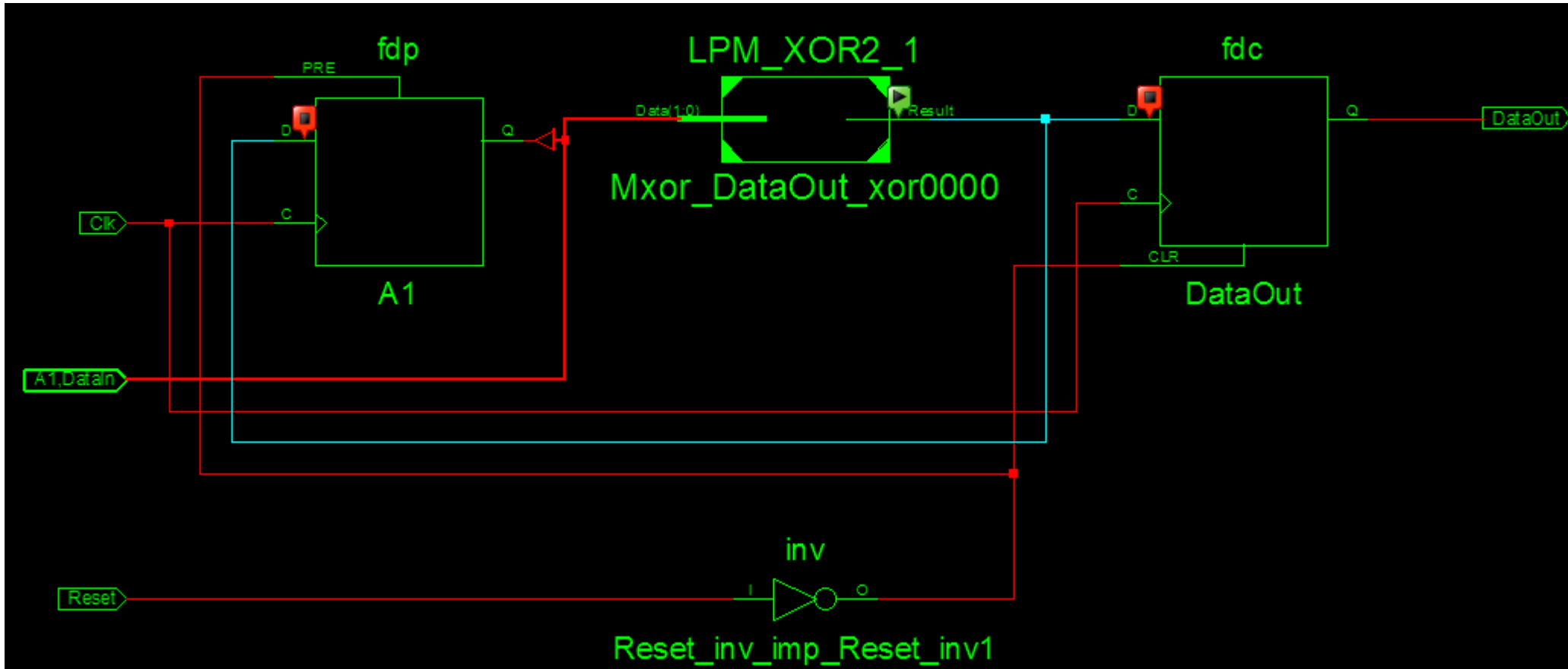
```
DataOut <= A1;
```

- Sind die Syntheseergebnisse gleich oder unterschiedlich?

Variablenbasiert

```
30 entity SameVarArch is
31     Port ( DataIn    : in  STD_LOGIC;
32           Clk        : in  STD_LOGIC;
33           Reset      : in  STD_LOGIC;
34           DataOut    : out STD_LOGIC);
35 end SameVarArch;
36
37 architecture Behavioral of SameVarArch is
38
39 begin
40
41     P1 : process(Clk,Reset)
42
43         variable A1 : std_logic;
44
45         begin
46
47             if Reset = '0' then
48
49                 A1 := '1';
50                 DataOut <= '0';
51
52             elsif Clk'event and Clk = '1' then
53
54                 A1 := A1 xor DataIn;
55
56                 DataOut <= A1;
57
58             end if;
59         |
60     end process P1;
61
62 end Behavioral;
```

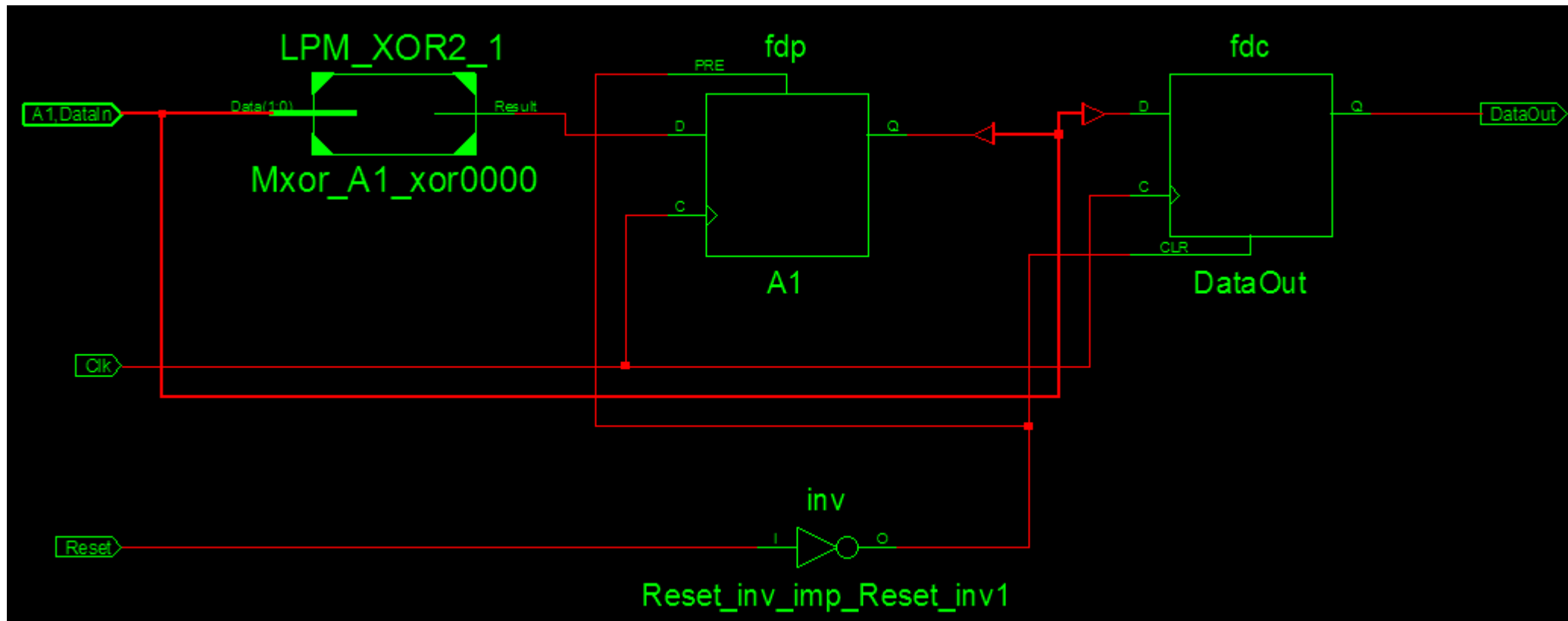
Syntheseergebnis



Signalbasiert

```
30 entity SameSignals is
31     Port ( DataIn : in  STD_LOGIC;
32           Clk : in  STD_LOGIC;
33           Reset : in  STD_LOGIC;
34           DataOut : out  STD_LOGIC);
35 end SameSignals;
36
37 architecture Behavioral of SameSignals is
38
39     signal A1 : std_logic;
40
41     begin
42
43
44     P1 : process (Clk, Reset)
45
46         begin
47
48             if Reset = '0' then
49
50                 A1 <= '1';
51                 DataOut <= '0';
52
53             elsif Clk'event and Clk = '1' then
54
55                 A1 <= A1 xor DataIn;
56
57                 DataOut <= A1;
58
59             end if;
60
61         end process P1;
62
63
64 end Behavioral;
```

Syntheseergebnis



Kapitel 4

Kontrollstrukturen, Operatoren,

Funktionen, Prozeduren,

Kontrollstrukturen, Operatoren, Funktionen,

Prozeduren, Generics, Generate

Selektive Signalzuweisung

- Die Anweisung wird aktiviert, wenn sich eines der Signale der Auswahlbedingung oder der Ausdrücke auf der rechten Seite der Zuweisung ändert

Syntax: [label:] with expression select
signal_name <= expression when value{,
expression when value};

Beispiel: with MYSEL select
Z <= A when 15,
B when 22,
C when 28;

4x1 Mux

```
20 library IEEE;
21 use IEEE.STD_LOGIC_1164.ALL;
22 use IEEE.STD_LOGIC_ARITH.ALL;
23 use IEEE.STD_LOGIC_UNSIGNED.ALL;
24
25 ---- Uncomment the following library declaration if instantiating
26 ---- any Xilinx primitives in this code.
27 --library UNISIM;
28 --use UNISIM.VComponents.all;
29
30 entity mux4x1_2_arc is
31     Port ( S : in  STD_LOGIC_VECTOR (1 downto 0);
32           E : in  STD_LOGIC_VECTOR (3 downto 0);
33           Y : out STD_LOGIC);
34 end mux4x1_2_arc;
35
36
37 architecture Behavioral of mux4x1_2_arc is
38
39 begin
40
41     with S select
42     Y <=      E(0)  when "00",
43              E(1)  when "01",
44              E(2)  when "10",
45              E(3)  when others;
46
47
48
49 end Behavioral;
50
```

Lösung – Testbench (1/3)

```
28  LIBRARY ieee;
29  USE ieee.std_logic_1164.ALL;
30  USE ieee.std_logic_unsigned.all;
31  USE ieee.numeric_std.ALL;
32
33  ENTITY mux4x1_2_tb IS
34  END mux4x1_2_tb;
35
36  ARCHITECTURE behavior OF mux4x1_2_tb IS
37
38      -- Component Declaration for the Unit Under Test (UUT)
39
40      COMPONENT mux4x1_2_arc
41      PORT(
42          S : IN  std_logic_vector(1 downto 0);
43          E : IN  std_logic_vector(3 downto 0);
44          Y : OUT std_logic
45      );
46      END COMPONENT;
47
48
49      --Inputs
50      signal S : std_logic_vector(1 downto 0) := (others => '0');
51      signal E : std_logic_vector(3 downto 0) := (others => '0');
52
53      --Outputs
54      signal Y : std_logic;
55
56      signal clk : std_logic;
57
```

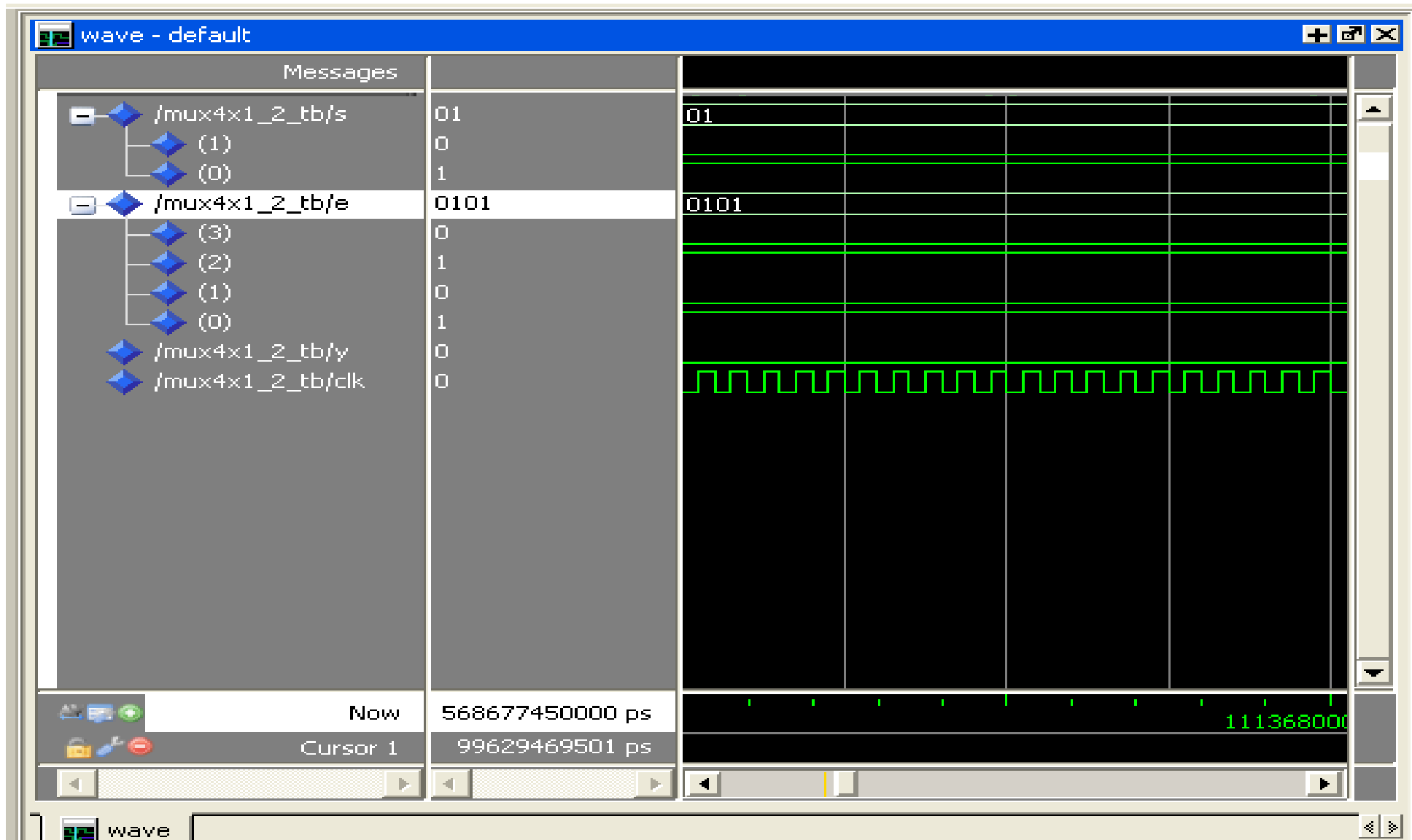
Lösung – Testbench (2/3)

```
57
58 BEGIN
59
60     -- Instantiate the Unit Under Test (UUT)
61     uut: mux4x1_2_arc PORT MAP (
62         S => S,
63         E => E,
64         Y => Y
65     );
```

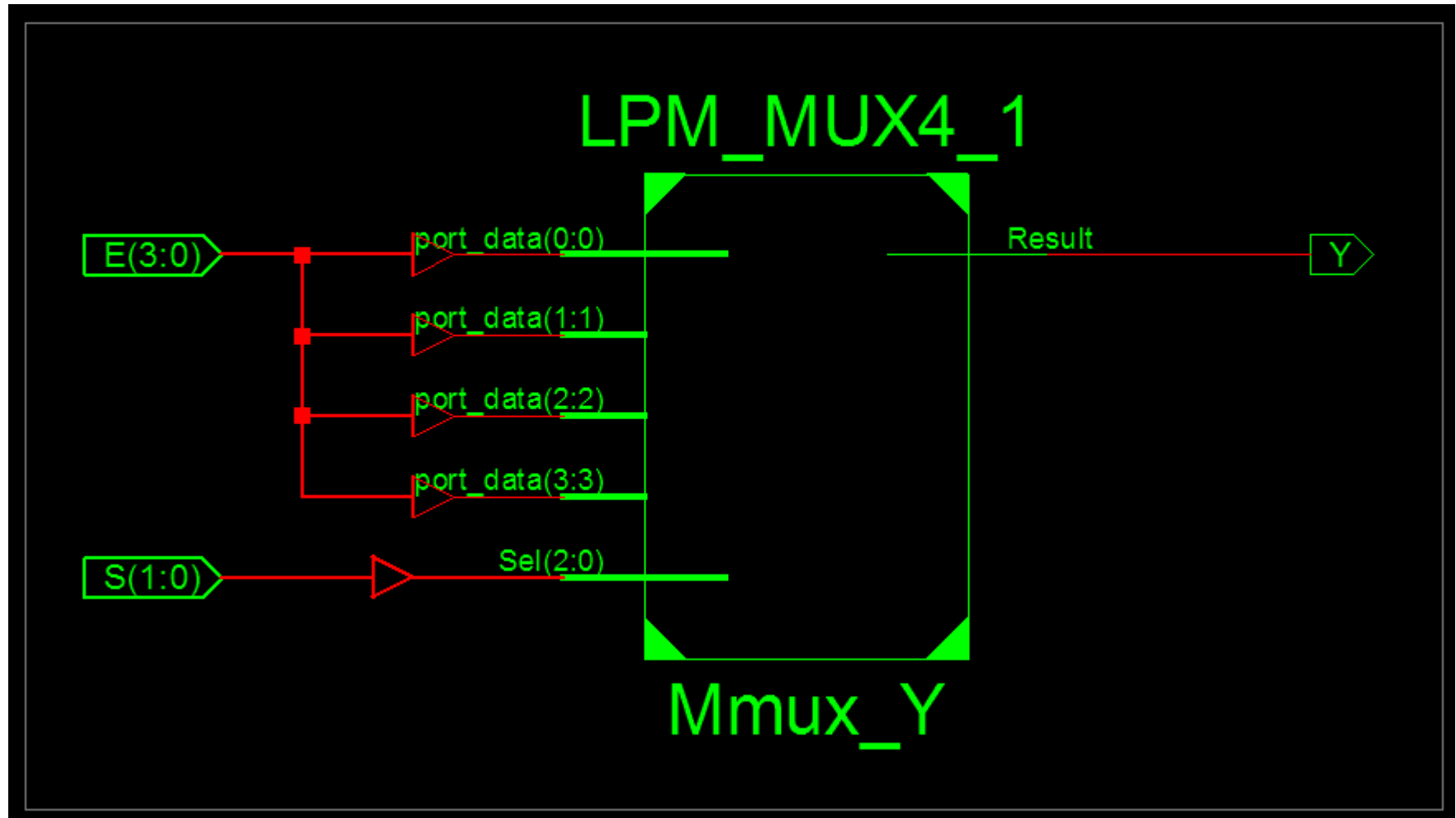
Lösung – Testbench (3/3)

```
66
67  -- No clocks detected in port list. Replace <clock> below with
68  -- appropriate port name
69
70  -- constant clk_period := 10 ns;
71
72  clk_process :process
73  begin
74      clk <= '0';
75      -- wait for clk_period/2;
76      wait for 50 ns;
77      clk <= '1';
78      -- wait for clk_period/2;
79      wait for 50 ns;
80  end process;
81
82
83  -- Stimulus process
84  stim_proc: process
85  begin
86      -- hold reset state for 100ms.
87      wait for 100ms;
88
89      -- wait for clk_period*10;
90      wait for 500 ns;
91
92      -- insert stimulus here
93
94      E <= "1010", "0101" after 400 ns;
95      S <=  "00",   "01" after 100 ns ;
96
97      wait;
98
99  end process;
100
101  END;
102
```

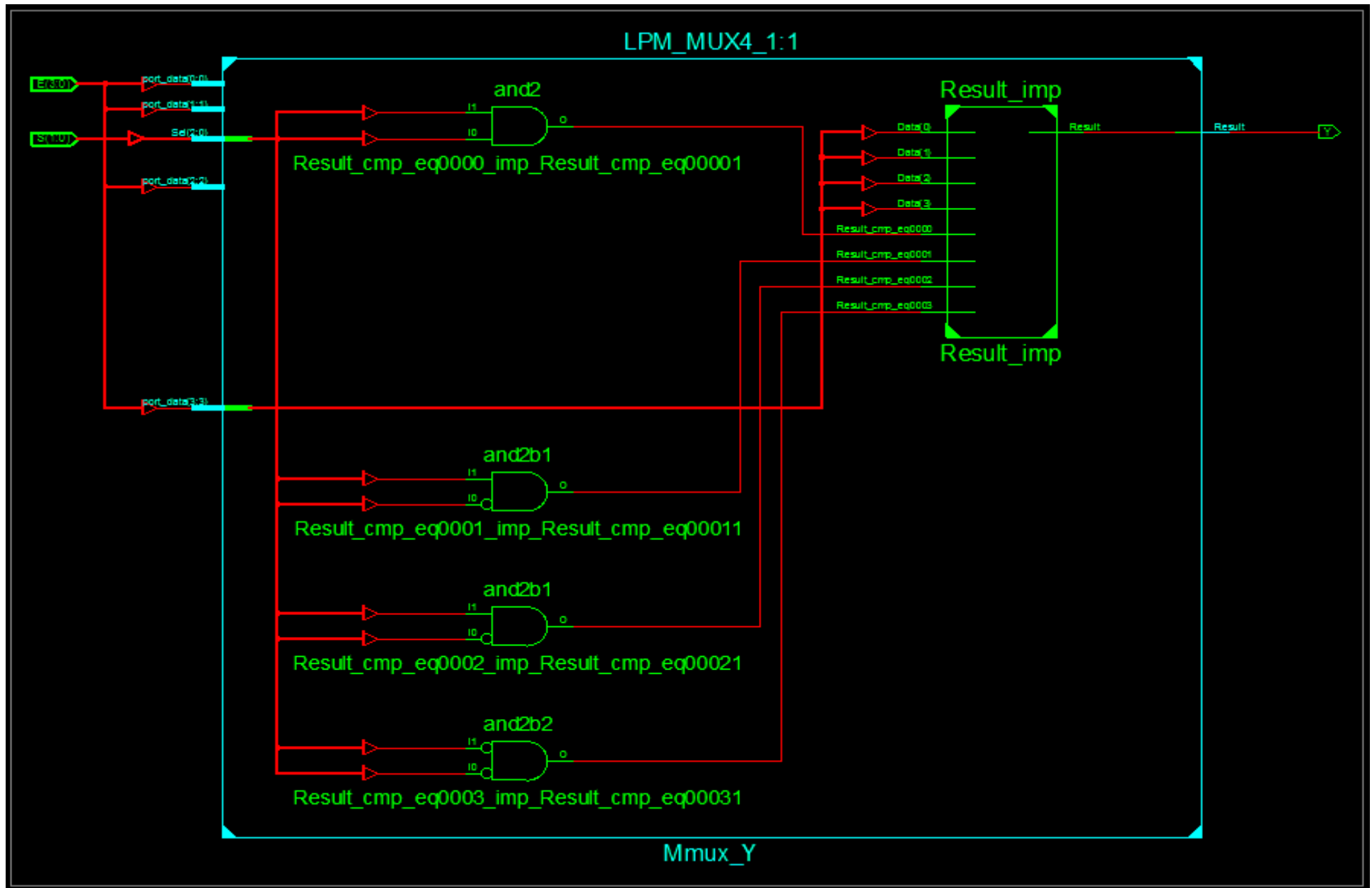
Simulationsergebnis



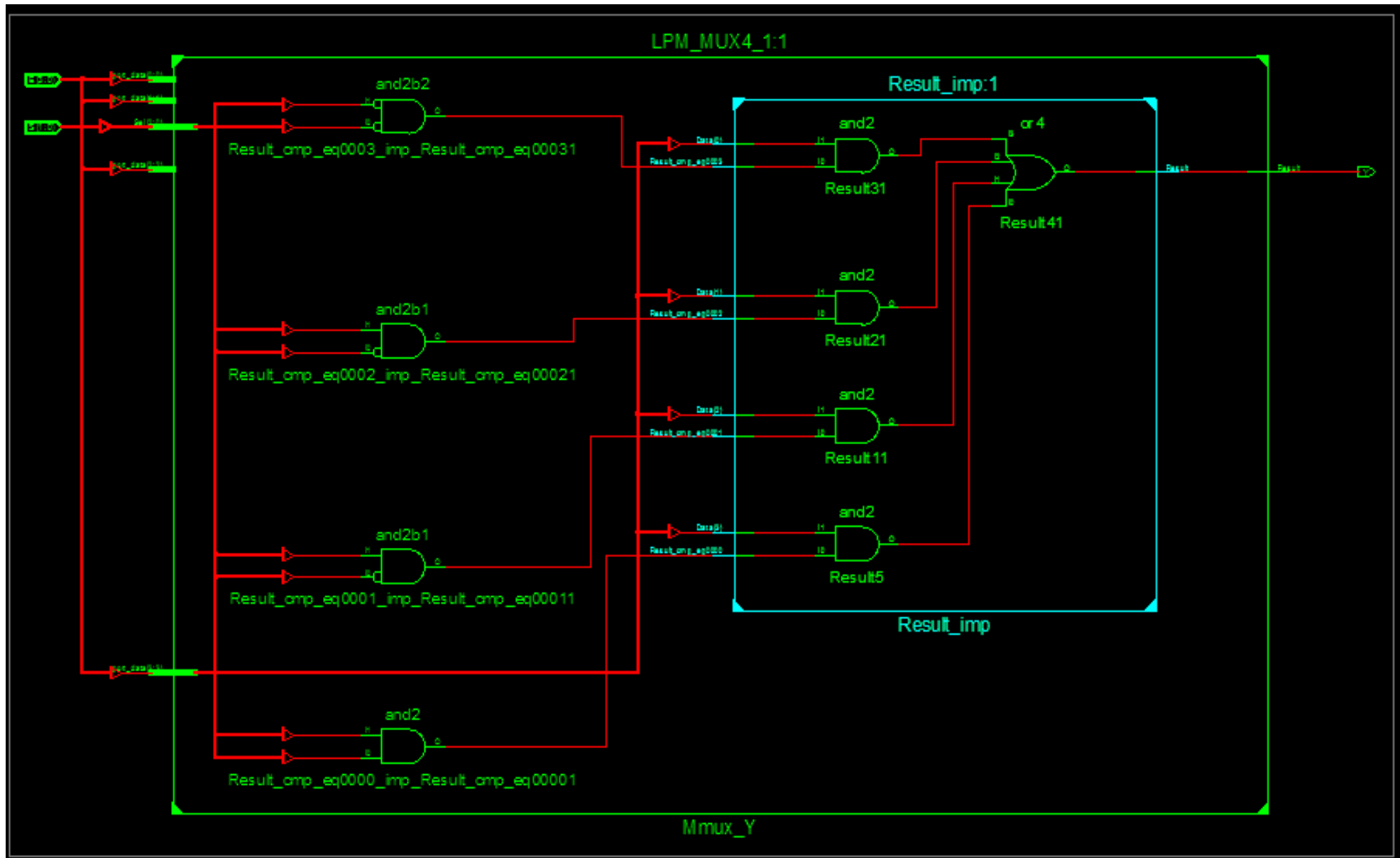
Syntheseergebnis (1/3)



Syntheseergebnis (2/3)



Syntheseergebnis (3/3)



Case - Anweisung

- Mehrfach-Verzweigung wie in Programmiersprachen
- Decodierung komplexer Codes (z.B. auf Bussen) bei Hardwareumsetzung

Syntax: `case expression is`
 `{when choices => sequential_statements}`
`end case;`

Möglichkeiten für choices

`value => sequential_statements`

genau ein Wert

`value1 | value2 ... => sequential_statements`

Aufzählung von Werten

`value1 to value2 => sequential_statements`

Wertebereich

`others => sequential_statements`

Voreinstellung

Beispiel - case

Beispiel: case BCD is

```
    when "0000" => LED := "1111110";
    when "0001" => LED := "1100000";
    when "0010" => LED := "1011011";
    when "0011" => LED := "1110011";
    when "0100" => LED := "1100101";
    when "0101" => LED := "0110111";
    when "0110" => LED := "0111111";
    when "0111" => LED := "1100010";
    when "1000" => LED := "1111111";
    when "1001" => LED := "1110111";
    when others => LED := "-----";
end case;
```

Decoder: BCD zu 7-Segment

don't care aus std_logic_1164

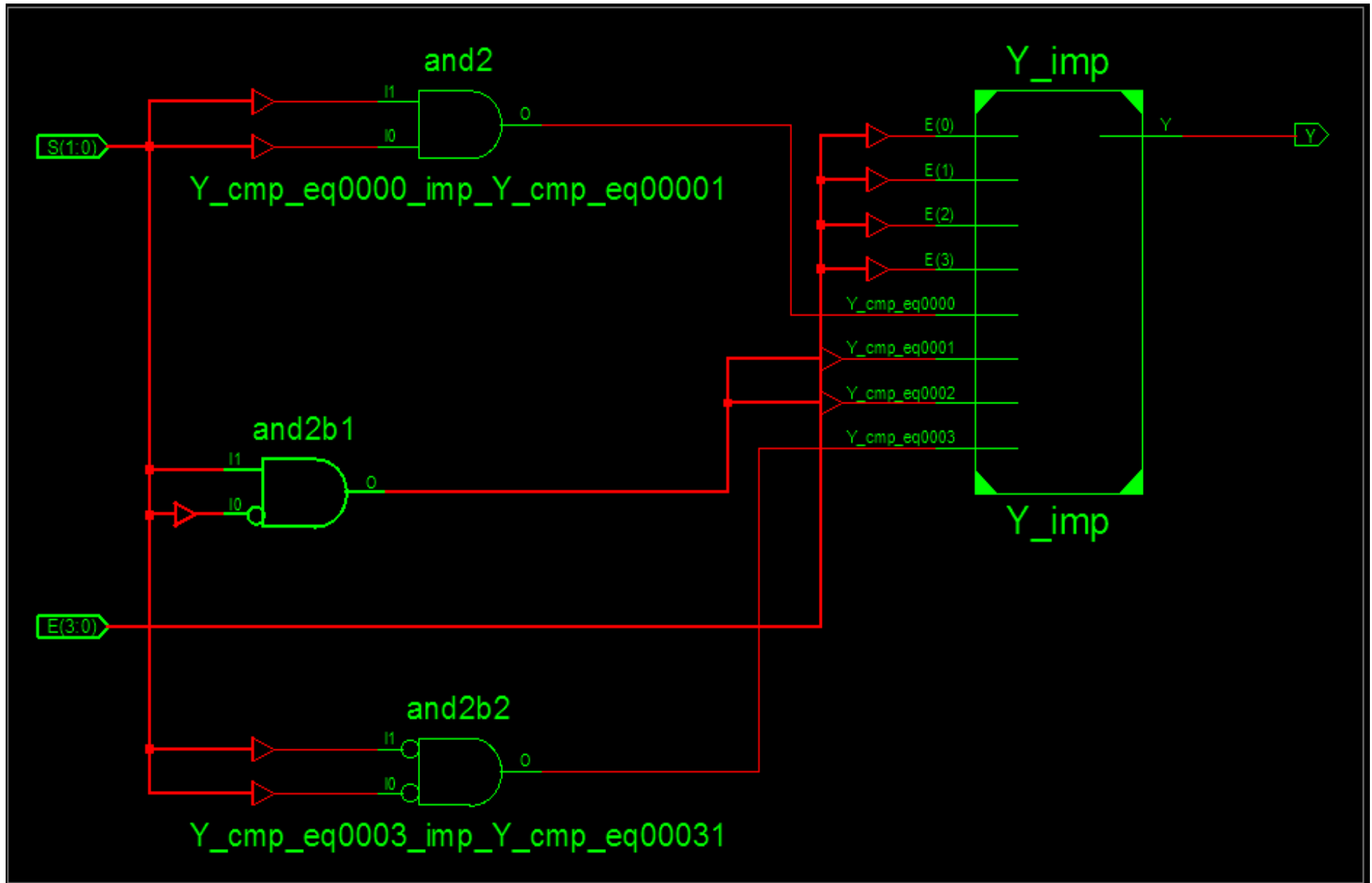
Gruppenübung

- **Implementieren Sie den 4x1 – Mux mittels der Case Anweisung**

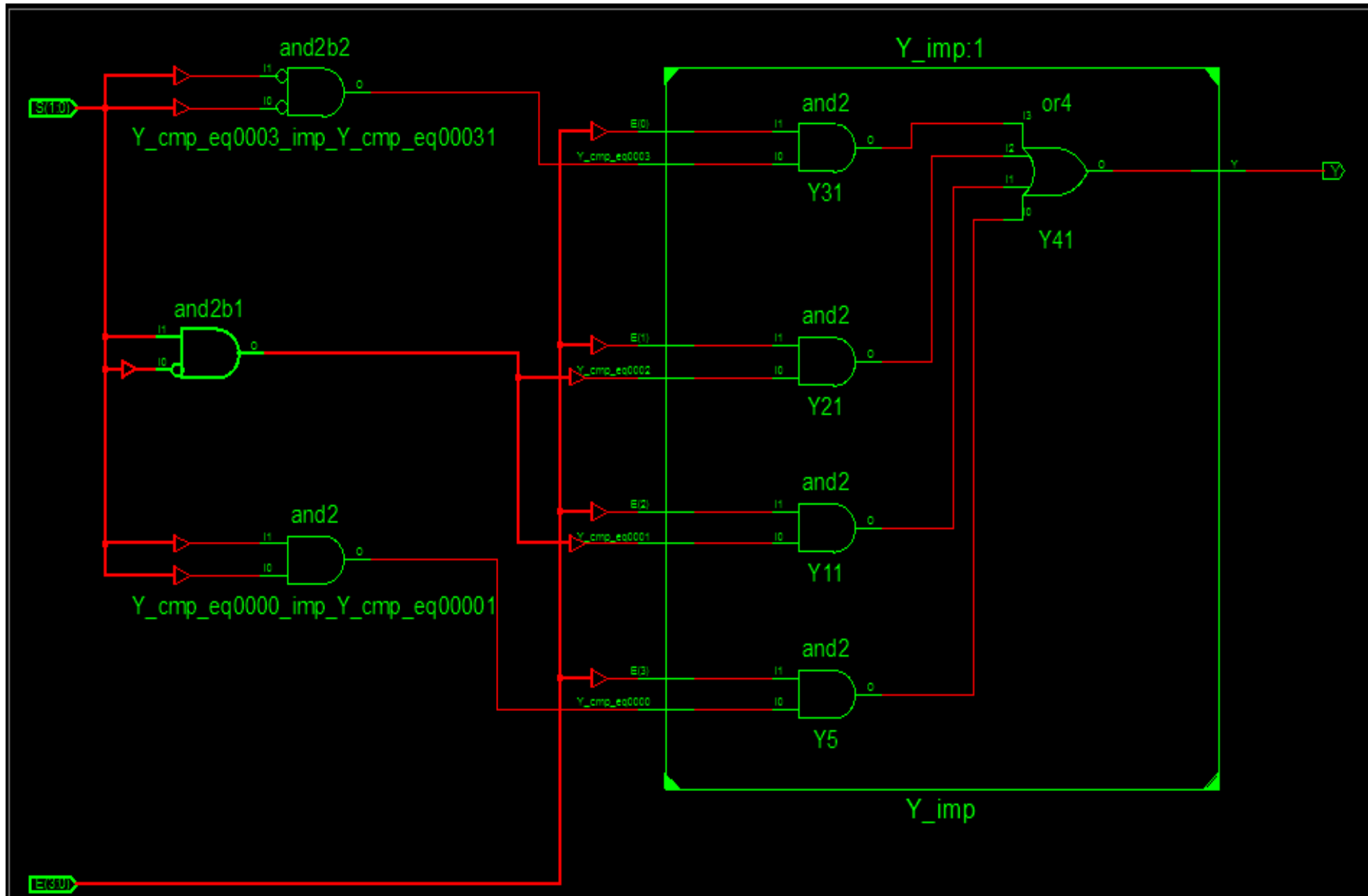
Lösung

```
20 library IEEE;
21 use IEEE.STD_LOGIC_1164.ALL;
22 use IEEE.STD_LOGIC_ARITH.ALL;
23 use IEEE.STD_LOGIC_UNSIGNED.ALL;
24
25 ---- Uncomment the following library declaration if instantiating
26 ---- any Xilinx primitives in this code.
27 --library UNISIM;
28 --use UNISIM.VComponents.all;
29
30 entity mux4x1_3_case is
31     Port ( S : in  STD_LOGIC_VECTOR (1 downto 0);
32           E : in  STD_LOGIC_VECTOR (3 downto 0);
33           Y : out STD_LOGIC);
34 end mux4x1_3_case;
35
36 architecture Behavioral of mux4x1_3_case is
37
38 begin
39
40 MuxProcess : process(S, E)
41
42     begin
43
44         case S is
45             when "00"    => Y <= E(0);
46             when "01"    => Y <= E(1);
47             when "10"    => Y <= E(2);
48             when others => Y <= E(3);
49         end case;
50
51     end process MuxProcess;
52
53 end Behavioral;
```

Syntheseergebnis (1/2)



Syntheseergebnis (2/2)



Funktionen

- Funktionen haben in VHDL die Aufgabe, immer wiederkehrende
 - Berechnungen
 - Zuweisungen
 - LUT
 - ...

Algorithmen zur Verfügung zu stellen.

- Funktionen können
 - keine
 - eine
 - mehrere

Übergabeparameter mitführen.

- Eine Funktion gibt IMMER einen Wert zurück
- Eine Funktion kann keine WAIT Anweisungen verarbeiten
 - in der Simulation konsumiert eine Funktion keine Simulationszeit

Funktion Syntax

- Die Funktion startet mit dem Funktionskopf, gefolgt von dem Funktionsbody
- Der Funktionskopf beinhaltet den Bezeichner *function*, gefolgt von dem Funktionsnamen
 - in Klammern folgen die Übergabeparameter und deren jeweiliger Typ
 - Für den Rückgabewert wird im Kopf nur der Typ angegeben
- Nach dem Kopf folgt die Definition von Konstanten und Variablen
- Zwischen *begin* und *end* findet die Implementierung statt
- Zum Schluss folgt die *return* Anweisung mit dem Rückgabewert

```
[pure|impure] function <function_name> (<parameter1_name> : <parameter1_type> := <default_value>;  
                                     <parameter2_name> : <parameter2_type> := <default_value>;  
                                     ... ) return <return_type> is  
  <constant_or_variable_declaration>  
begin  
  <code_performed_by_the_function>  
  return <value>  
end function;
```

Funktionsbeispiel (1/2)

- Die Funktion wird in die Architecture integriert

```
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity T21_TrafficLights is
generic(ClockFrequencyHz : natural);
port(
  Clk      : in std_logic;
  nRst     : in std_logic; -- Negative reset
  NorthRed : out std_logic;
  NorthYellow : out std_logic;
  NorthGreen : out std_logic;
  WestRed : out std_logic;
  WestYellow : out std_logic;
  WestGreen : out std_logic);
end entity;

architecture rtl of T21_TrafficLights is

  -- Enumerated type declaration and state signal declaration
  type t_State is (NorthNext, StartNorth, North, StopNorth,
                  WestNext, StartWest, West, StopWest);
  signal State : t_State;

  -- Calculate the number of clock cycles in minutes/seconds
  function CounterVal(Minutes : integer := 0;
                     Seconds : integer := 0) return integer is
    variable TotalSeconds : integer;
  begin
    TotalSeconds := Seconds + Minutes * 60;
    return TotalSeconds * ClockFrequencyHz - 1;
  end function;

  -- Counter for counting clock periods, 1 minute max
  signal Counter : integer range 0 to CounterVal(Minutes => 1) + 1;

begin

  process(Clk) is
  begin

    if rising_edge(Clk) then
      if nRst = '0' then
        -- Reset values
        NorthRed    <= '1';
        NorthYellow <= '0';

```

Quelle: <https://vhdlwhiz.com/function/>

Funktionsbeispiel (2/2)

- Ideal für arithmetische Berechnungen

```
33 :
34 ⊞ entity v_functiondev is
35 :     Port ( Reset      : in STD_LOGIC;
36 :           Clk         : in STD_LOGIC;
37 :           DataIn      : in STD_LOGIC_VECTOR (7 downto 0);
38 :           DataOut     : in STD_LOGIC_VECTOR (7 downto 0));
39 ⊞ end v_functiondev;
40 :
41 ⊞ architecture Behavioral of v_functiondev is
42 :
43 ⊞     function MittelWert(Value1 : Integer;
44 :                          Value2 : Integer;
45 :                          Value3 : Integer;
46 :                          Value4 : Integer) return Integer is
47 :
48 :         variable MittelWertValue : Integer := 0;
49 :         variable Summe           : Integer := 0;
50 :
51 :         begin
52 :
53 :             Summe := Value1 + Value2 + Value3 + Value4;
54 :
55 :             MittelWertValue := to_integer(shift_right(to_unsigned(Summe, 8), 2));
56 :
57 :             return(MittelWertValue);
58 :
59 ⊞     end function MittelWert;
60 :
61 :
62 :
63 : begin
```

Kapitel 5

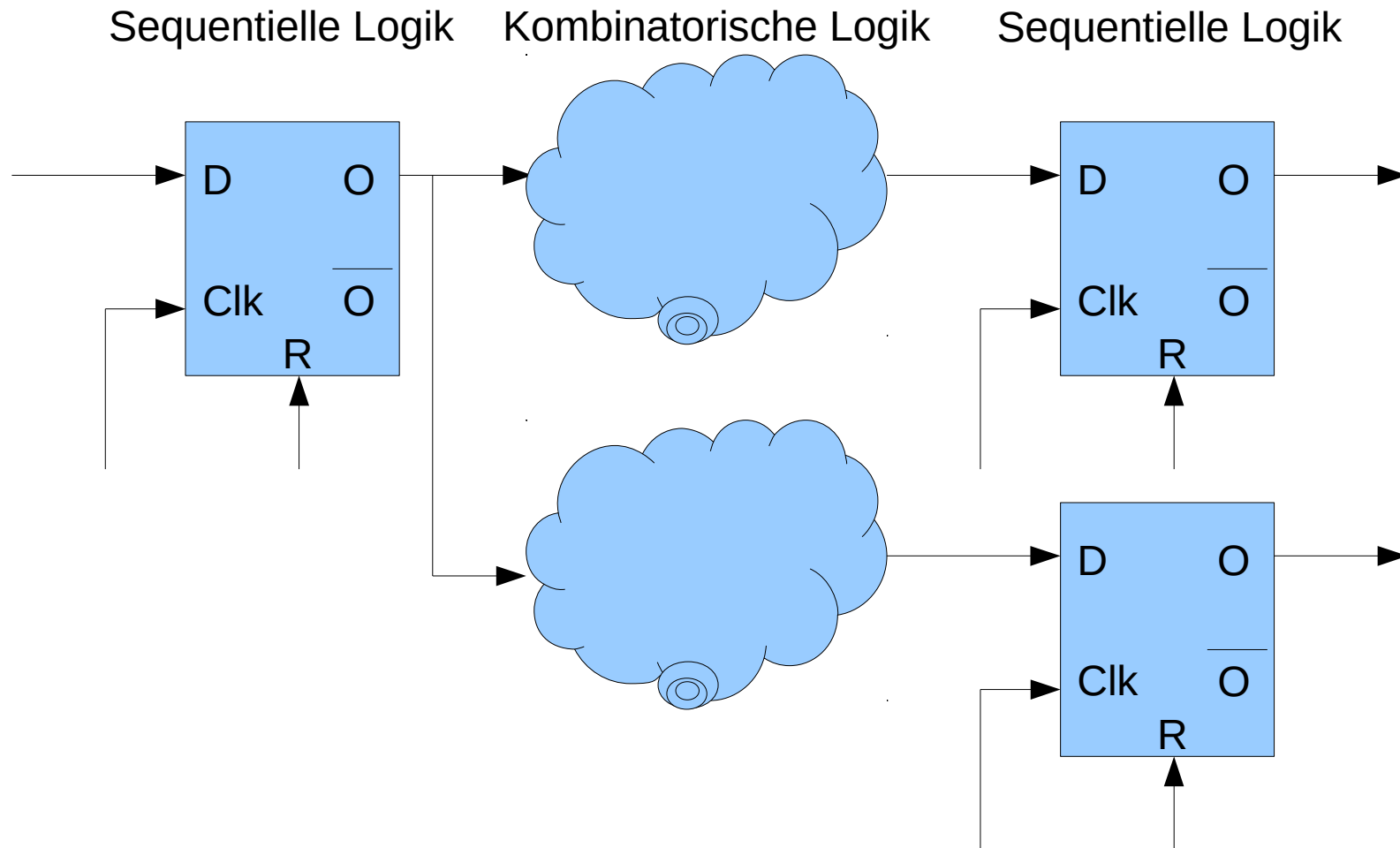
Register Transfer Level - RTL

Ziel von VHDL für ASIC und FPGA Designs

- Entwurf von
 - kombinatorischer
 - sequentieller

Logik für die Synthese in ASIC oder FPGA Bausteinen

wird als RTL Design bezeichnet – Register Transfer Level



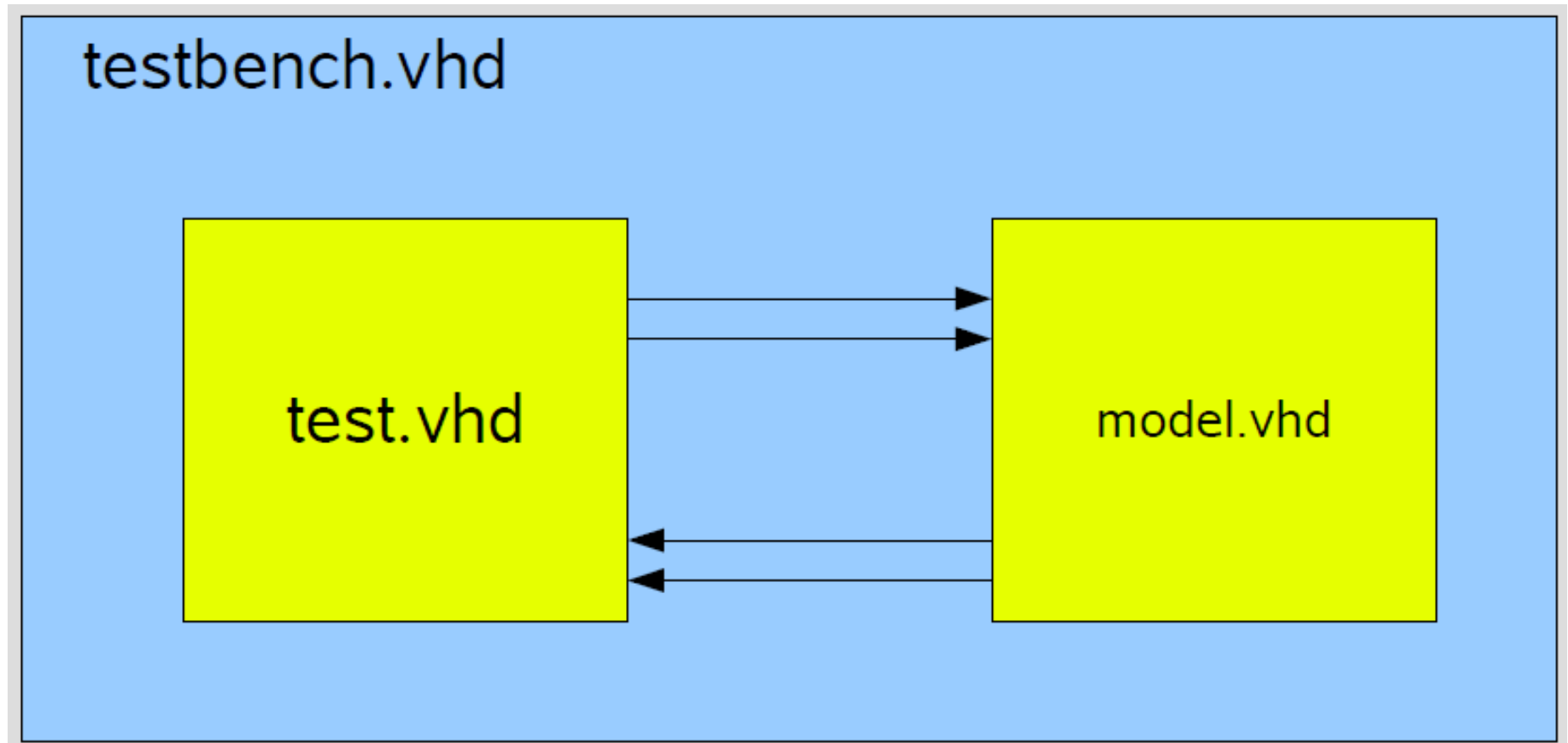
Kapitel 6

Testbench, Simulation und Synthese

Testen

- Um einen Entwurf zu simulieren, muss er mit wohldefinierten Stimuli versorgt werden
- Notwendigkeit einer Testbench (Testumgebung)
- Es werden zu definierten Eingangswerten die erwarteten Ausgangswerte angegeben

Testenbench



Beispiel – Testbench (1/3)

```
entity adder_tb is
end adder_tb;
architecture behav of adder_tb is
    component adder
        port (i0, i1 : in bit; ci : in bit; s : out bit; co : out bit);
    end component;
    for adder_0: adder use entity work.adder;
    signal i0, i1, ci, s, co : bit;
begin
    adder_0: adder port map (i0 => i0, i1 => i1, ci => ci, s =>
s, co => co);
    -- siehe nächste Folie
end behav;
```

Beispiel – Testbench (2/3)

```
process
  type pattern_type is record
    i0, i1, ci : bit;
    s, co : bit;
  end record;
  type pattern_array is array (natural range <>) of
pattern_type;
  constant patterns : pattern_array :=
    (('0', '0', '0', '0', '0'),
     ('0', '0', '1', '1', '0'),
     ('0', '1', '0', '1', '0'),
     ('0', '1', '1', '0', '1'),
     ('1', '0', '0', '1', '0'),
     ('1', '0', '1', '0', '1'),
     ('1', '1', '0', '0', '1'),
     ('1', '1', '1', '1', '1'));
```

Beispiel – Testbench (3/3)

```
begin
  for i in patterns'range loop
    i0 <= patterns(i).i0;
    i1 <= patterns(i).i1;
    ci <= patterns(i).ci;
    wait for 1 ns;
    assert s = patterns(i).s
      report "bad sum value" severity error;
    assert co = patterns(i).co
      report "bad carray out value" severity error;
  end loop;
  assert false report "end of test" severity note;
  wait;
end process;
```

Simulation von VHDL Code

VHDL Testbench

- Eine Testbench dient zur Überprüfung der VHDL code Funktionen
- Eine Testbench hat keine Ein- oder Ausgangsports
- Es werden Stimuli Signale erzeugt, die auf die Eingänge des DUT – Device Under Test gelegt werden
- Die Codes werden simuliert und die Ergebnisse im Simulator betrachtet
- Alternativ können zur grafischen Auswertung auch die erwarteten Ergebnisse mit den tatsächlichen Werten verglichen werden

Aufbau einer Testbench

- Leere Entity da keine Ein- oder Ausgänge
- Die zu testenden Komponente wird installiert
- Die Signale sollten genauso heißen wie die Ports des zu testenden Code-Blocks
- Die Signale und das DUT werden in der Port-Map zusammengeführt
- Der assert Befehl hilft bei der Ausgabe von Ergebnissen

Testbench Aufbau

```
28 LIBRARY ieee;
29 USE ieee.std_logic_1164.ALL;
30 USE ieee.std_logic_unsigned.all;
31 USE ieee.numeric_std.ALL;
32
33 ENTITY TB_Mux4_1 IS
34 END TB_Mux4_1;
35
36 ARCHITECTURE behavior OF TB_Mux4_1 IS
37
38     -- Component Declaration for the Unit Under Test (U
39
40     COMPONENT Mux4_1_Sync
41     PORT(
42         DataIn1      : IN  std_logic;
43         DataIn2      : IN  std_logic;
44         BitSelect    : IN  std_logic;
45         Clk          : IN  std_logic;
46         Reset        : IN  std_logic;
47         DataOut      : OUT std_logic
48     );
49     END COMPONENT;
50
51
52     --Inputs
53     signal DataIn1      : std_logic := '0';
54     signal DataIn2      : std_logic := '0';
55     signal BitSelect    : std_logic := '0';
56     signal Clk          : std_logic := '0';
57     signal Reset        : std_logic := '1';
58
59     --Outputs
60     signal DataOut      : std_logic;
```

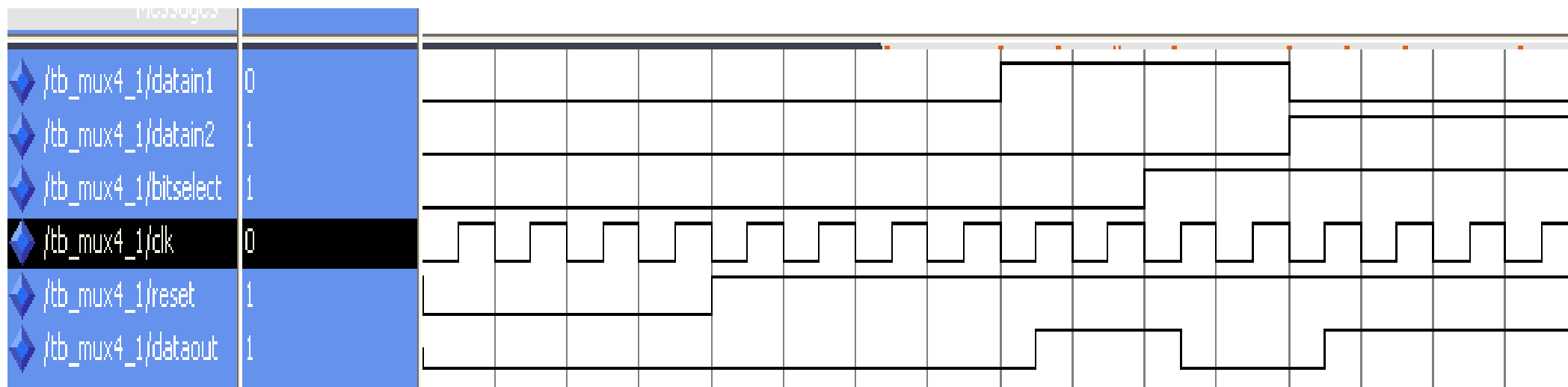

Testbench Aufbau

```
61
62     constant clk_period : time := 1ns;
63
64 BEGIN
65
66     -- Instantiate the Unit Under Test (UUT)
67     uut: Mux4_1_Sync PORT MAP (
68         DataIn1      => DataIn1,
69         DataIn2      => DataIn2,
70         BitSelect    => BitSelect,
71         Clk          => Clk,
72         Reset        => Reset,
73         DataOut      => DataOut
74     );
75
76     -- No clocks detected in port list. Replace <clock> below with
77     -- appropriate port name
78
79
80
81     clk_process : process
82
83     begin
84         clk <= '0';
85         wait for clk_period/2;
86         clk <= '1';
87         wait for clk_period/2;
88
89     end process;
90
```

Aufbau Testbench

```
90
91
92  -- Stimulus process
93  stim_proc: process
94  begin
95      -- hold reset state for 100ms.
96
97      Reset <= '0';
98      wait for clk_period * 4;
99
100     Reset <= '1';
101     wait for clk_period * 4;
102
103     -- insert stimulus here
104     DataIn1      <= '1';
105     DataIn2      <= '0';
106     BitSelect    <= '0';
107
108     wait for clk_period*2;
109
110     -- insert stimulus here
111     DataIn1      <= '1';
112     DataIn2      <= '0';
113     BitSelect    <= '1';
114
115     wait for clk_period*2;
116
117     -- insert stimulus here
118     DataIn1      <= '0';
119     DataIn2      <= '1';
120     BitSelect    <= '1';
121
122     wait;
123 end process;
124
125 END;
```

Beispiel Simulation



Kapitel 7

Zähler, Synchronisationsschaltungen, Endliche Automaten

Zähler

Überblick Zähler

- Synchrone Zähler
- Asynchrone Zähler

Einführung Zähler

- Zähler sind die einfachsten Formen von Schaltungen der sequentiellen Logik
- Wenn man mit n FF arbeitet, kann man 2^n verschiedene Ausgangszustände haben
 - Man kann also von $0 \dots 2^n - 1$ zählen
- Ein Zähler ist eine Schaltung, die nach einem Taktimpuls von einem Ausgangszustand in einen vorbestimmten anderen Zustand übergeht

Synchrone und Asynchrone Zähler

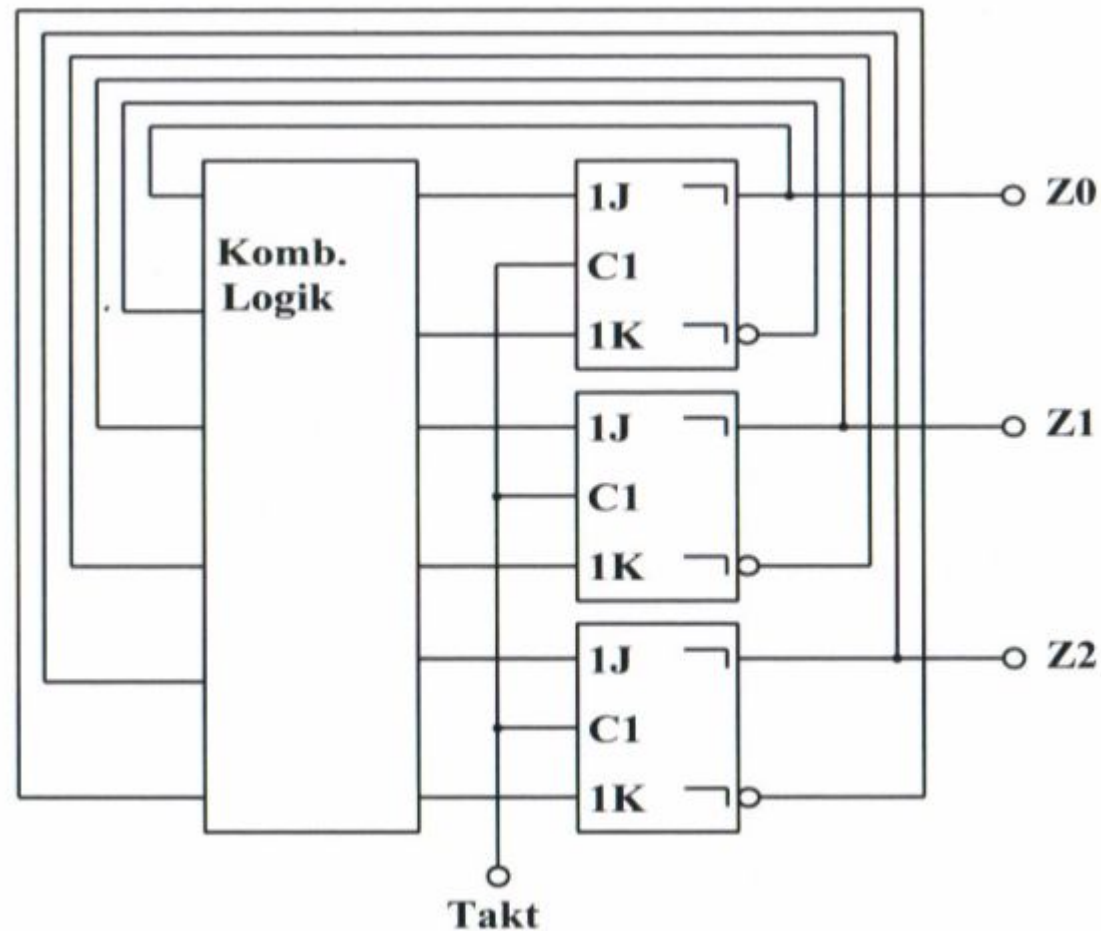
- Es gibt
 - synchrone
 - asynchrone

Zähler

- Bei synchronen Zählern erhalten alle FF den gleichen Taktimpuls
- Asynchrone Zähler verwenden für jedes FF einen eigenen Takt, der in der Regel vom Ausgang eines anderen FF abgeleitet wird
- Synchrone Zähler sind
 - Schneller
 - Zuverlässiger
 - höherer Designaufwand
- Asynchrone Systeme sind
 - nur sehr aufwändig zu testen
 - verbrauchen aber u.U. Weniger Strom

Synchrone Zähler

- Synchrone Zähler muss einen definierten Wechsel für den nächsten Zustand vorbereiten
- Aus dem Speicher werden die Signale benutzt, um den nächsten Zustand zu berechnen



Gruppenübung: Elektronischer Würfel

- Zähler mit hoher Taktfrequenz zählt im natürlichen Binärcode zyklisch von 1 ... 6
- Zu einem willkürlichen Zeitpunkt wird der Takt gestoppt
 - Damit steht das Würfelresultat fest
- **Gruppenübung:**
 - **Wieviele FF werden benötigt?**

Lösung

- Ziffern von 0 ... 7 können mit 3 Bit dargestellt werden → 3FF
- Die Art der FF ist nicht so wichtig
 - z.B. negativ flankengetriggerte JK FF

Wahrheitstabelle und Übergangstabelle

- Neben der Wahrheitstabelle kann eine Übergangstabelle zum Einsatz kommen, die für einen bestimmten Übergang die benötigten Werte von J und K enthält
- Die Übergangstabelle beschreibt ein JK FF beispielhaft

J	K	Q_{n+1}
0	0	Q_n
0	1	0
1	0	1
1	1	$\neg Q_n$



Übergang	J	K
$0 \rightarrow 0$	0	X
$0 \rightarrow 1$	1	X
$1 \rightarrow 1$	X	0
$1 \rightarrow 0$	X	1

Zählertabelle

W	Z2	Z1	Z0	J2	K2	J1	K1	J0	K0
1	0	0	1	0	X	1	X	X	1
2	0	1	0	0	X	X	0	1	X
3	0	1	1	1	X	X	1	X	1
4	1	0	0	X	0	0	X	1	X
5	1	0	1	X	0	1	X	X	1
6	1	1	0	X	1	X	1	1	X
7	0	0	1	← damit der Zyklus geschlossen bleibt					

- Zählertabelle ist gleichzeitig Wahrheitstabelle
 - 0 und 7 fehlen → unvollständige Wahrheitstabelle
 - Die Spalten J2 K2, J1 K1 und J0 K0 geben an, welches Bit das FF für den nächsten Zustand annimmt

Unzulässige Zustände

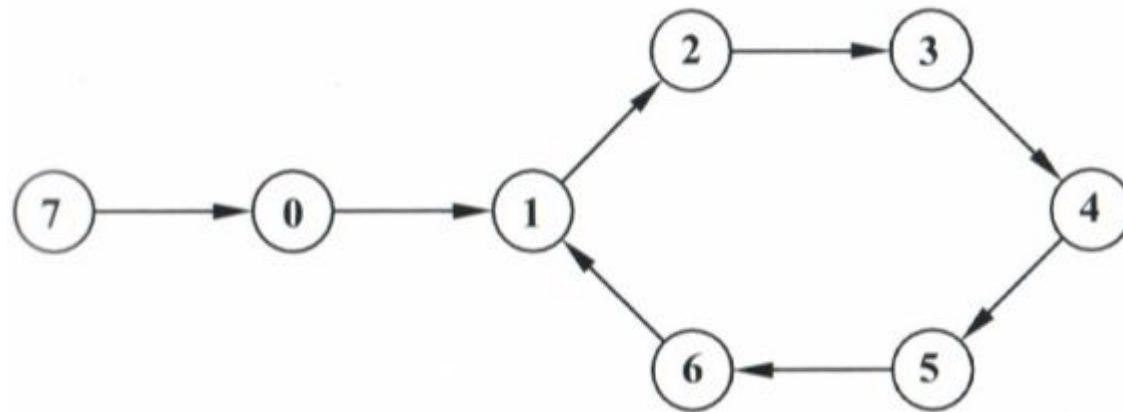
- Was passiert, wenn der Zähler z.B. nach dem Reset in einen nicht-vorhandenen Don't Care Zustand übergeht?
 - Zustand 0 = (0,0,0)
 - Zustand 7 = (1,1,1)

Z2	Z1	Z0	J2	K2	J1	K1	J0	K0	Z2'	Z1'	Z0'
0	0	0	0	0	0	0	1	1	0	0	1
1	1	1	1	1	1	1	1	1	0	0	0

- Von Zustand 0 würde in Zustand 1 gegangen
- Von Zustand 7 würde in Zustand 0 gegangen, danach in Zustand 1
- Antwort: Zähler kommt aus bei Don't Care Zuständen immer wieder in den gewollten Zustand 1
→ einfach den Zähler immer weiter zählen lassen

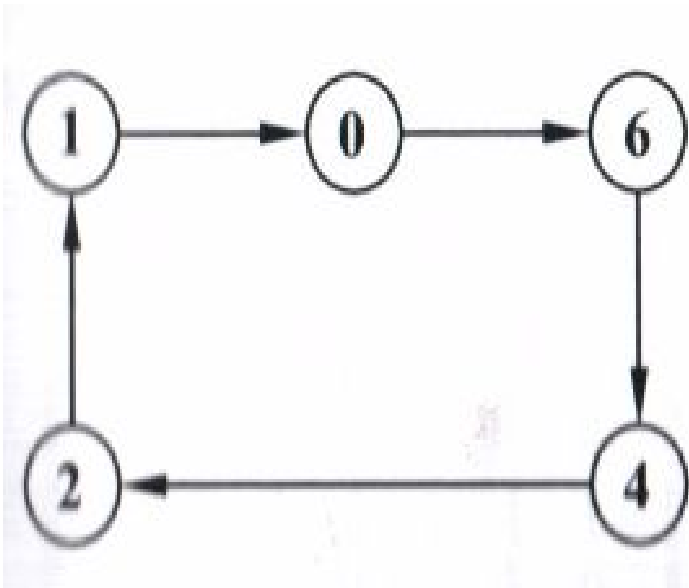
Zustandsdiagramm

- Hier wird gezeigt, dass die Zustände 7 und 0 kein Problem darstellen
 - Sie münden automatisch in den Zustand 1 → Hauptzyklus



Zustandsdiagramm

- Das Zustandsdiagramm darf weder noch
 - Parasitäre Zyklen
 - Keine Fixpunkteenthalten
- Hier fiktive Beispiele, die Zyklen und Fixpunkte enthalten



Hauptzyklus



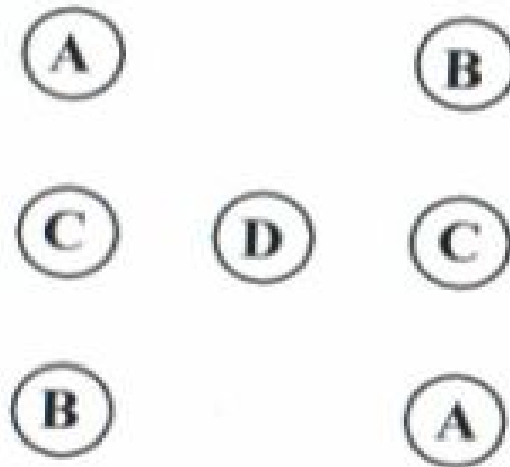
Parasitärer Zyklus



Fixpunkt

Anzeige des Ergebnisses

- Das Ergebnis soll in gewohntem Würfelformat angezeigt werden



- Einzelübung: Ordnen Sie den Würfelzahlen 1,2,3,4,5,6 die korrespondierenden Buchstaben und logischen Verknüpfungen zu

Lösung

- $1 = D$
 - $2 = A$
 - $3 = A \ \& \ D$
 - $4 = A \ \& \ B$
 - $5 = A \ \& \ B \ \& \ D$
 - $6 = A \ \& \ B \ \& \ C$
-
- **Einzelübung: Entwerfen Sie die Wahrheitstabelle mit aktive LEDs bei Pegel logisch 0**

Lösung

W	Z2	Z1	Z0	A	B	C	D
1	0	0	1	1	1	1	0
2	0	1	0	0	1	1	1
3	0	1	1	0	1	1	0
4	1	0	0	0	0	1	1
5	1	0	1	0	0	1	0
6	1	1	0	0	0	0	1

Dekadenzähler mit D FF

- Realisierung einer BCD-Zähldecade von 0 ... 9
- **Einzelübung: Realisieren Sie bitte die Zählertabelle**
 - **Wieviele D FF braucht man?**

Lösung Zählertabelle

- Schaltungen sind immer etwas komplizierter als bei anderen FF Typen → 4 D FF werden benötigt

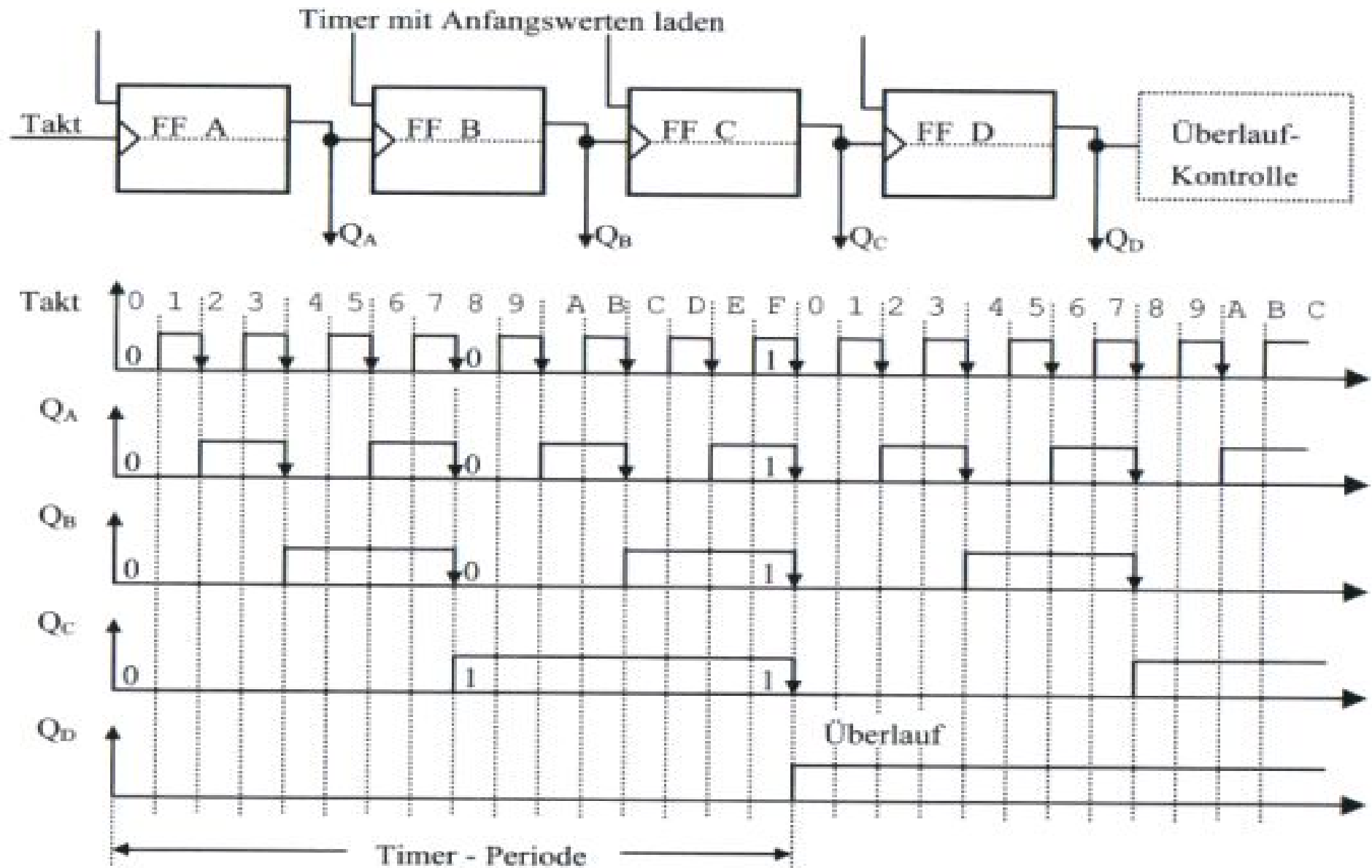
Wert	Zustand				Übergang			
w	z3	z2	z1	z0	D3	D2	D1	D0
0	0	0	0	0	0	0	0	1
1	0	0	0	1	0	0	1	0
2	0	0	1	0	0	0	1	1
3	0	0	1	1	0	1	0	0
4	0	1	0	0	0	1	0	1
5	0	1	0	1	0	1	1	0
6	0	1	1	0	0	1	1	1
7	0	1	1	1	1	0	0	0
8	1	0	0	0	1	0	0	1
9	1	0	0	1	0	0	0	0
10	1	0	1	0	DC	DC	DC	DC
11	1	0	1	1	DC	DC	DC	DC
12	1	1	0	0	DC	DC	DC	DC
13	1	1	0	1	DC	DC	DC	DC
14	1	1	1	0	DC	DC	DC	DC
15	1	1	1	1	DC	DC	DC	DC

Asynchrone Zähler

Asynchroner Zähler

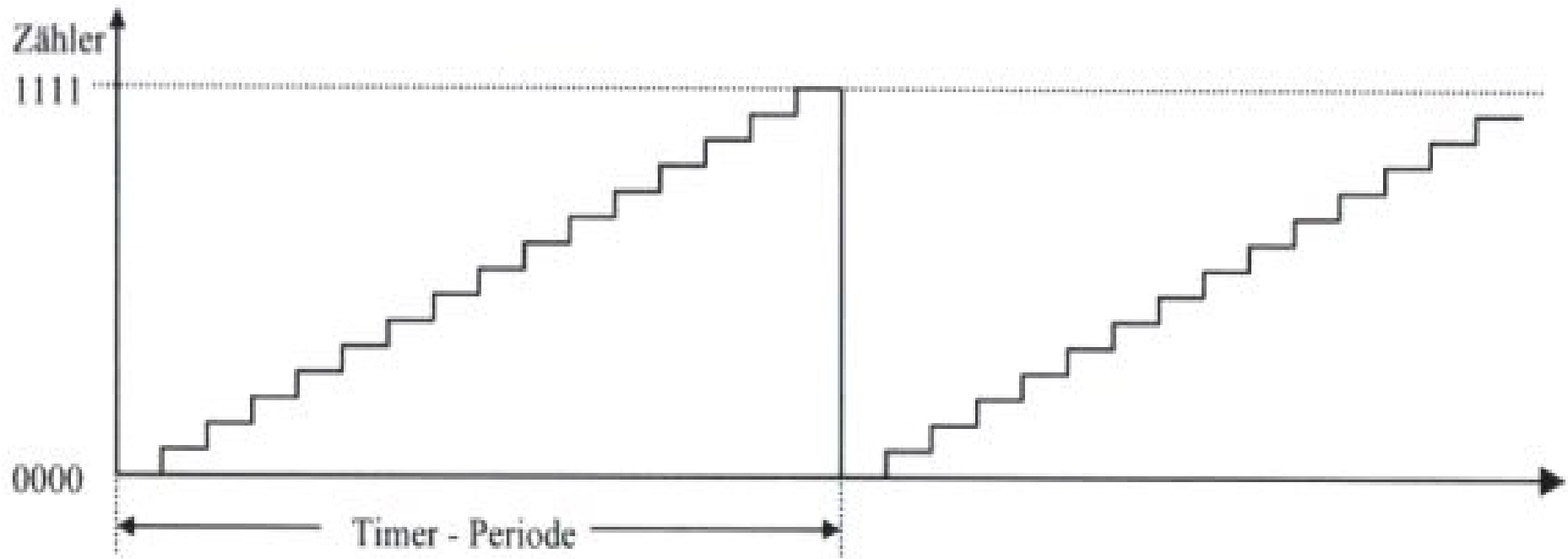
- Im Gegensatz zu den synchronen Schaltungen werden bei den asynchronen nicht alle FF vom gleichen externen Taktsignal angesteuert, sondern nur mindestens 1 FF
 - Die übrigen FF werden von den Ausgängen bzw. von Funktionen der Ausgänge der anderen FF angesteuert
- **Gruppenübung: Entwerfen Sie einen Zähler, der von 0 bis 15 zählt und nur das 1. FF von einem Takt angesteuert wird**

Lösung



4Bit Aufwärtzähler

- Aufwärtzähler von 0000 bis 1111
- Danach beginnt der Zähler wieder beim Anfangswert 0000
- Periode von 16 **Timertakten**



Übungsaufgaben

- Man realisiere einen synchronen 3-Bit-Zähler im natürlichen Binärcode unter Verwendung von RS-Master-Slave-FF
 - Es ist darauf zu achten, dass der Zustand $R = S = 1$ niemals auftreten darf
- Man entwerfe einen synchronen Zähler mit JK-FF für die Zählsequenz $0 - 1 - 2 - 3 - 4 - 5 - 0 - 1 \dots$ Verlangt wird die Zählerlogik und das Zustandsdiagramm

Johnsonzähler

Johnsonzähler

- Frequenzteiler
 - 1:2
 - 1:6
- Zähler mit 6 Zuständen nötig
- Zähler soll mit 3 Bit ausgegeben werden
- Frequenzteiler mit 2 Bit
 - MSB entspricht 2:1 Teilung
 - LSB entspricht 6:1 Teilung

Entity

```
19 -----
20 library IEEE;
21 use IEEE.STD_LOGIC_1164.ALL;
22 use IEEE.STD_LOGIC_ARITH.ALL;
23 use IEEE.STD_LOGIC_UNSIGNED.ALL;
24
25 ---- Uncomment the following library declaration if instantiating
26 ---- any Xilinx primitives in this code.
27 --library UNISIM;
28 --use UNISIM.VComponents.all;
29
30 entity johnson_arc is
31     Port ( Clk          : in  STD_LOGIC;
32           Reset         : in  STD_LOGIC;
33           Counter       : out  STD_LOGIC_VECTOR (2 downto 0);
34           FrequenzTeiler : out  STD_LOGIC_VECTOR (1 downto 0)
35     );
36 end johnson_arc;
```

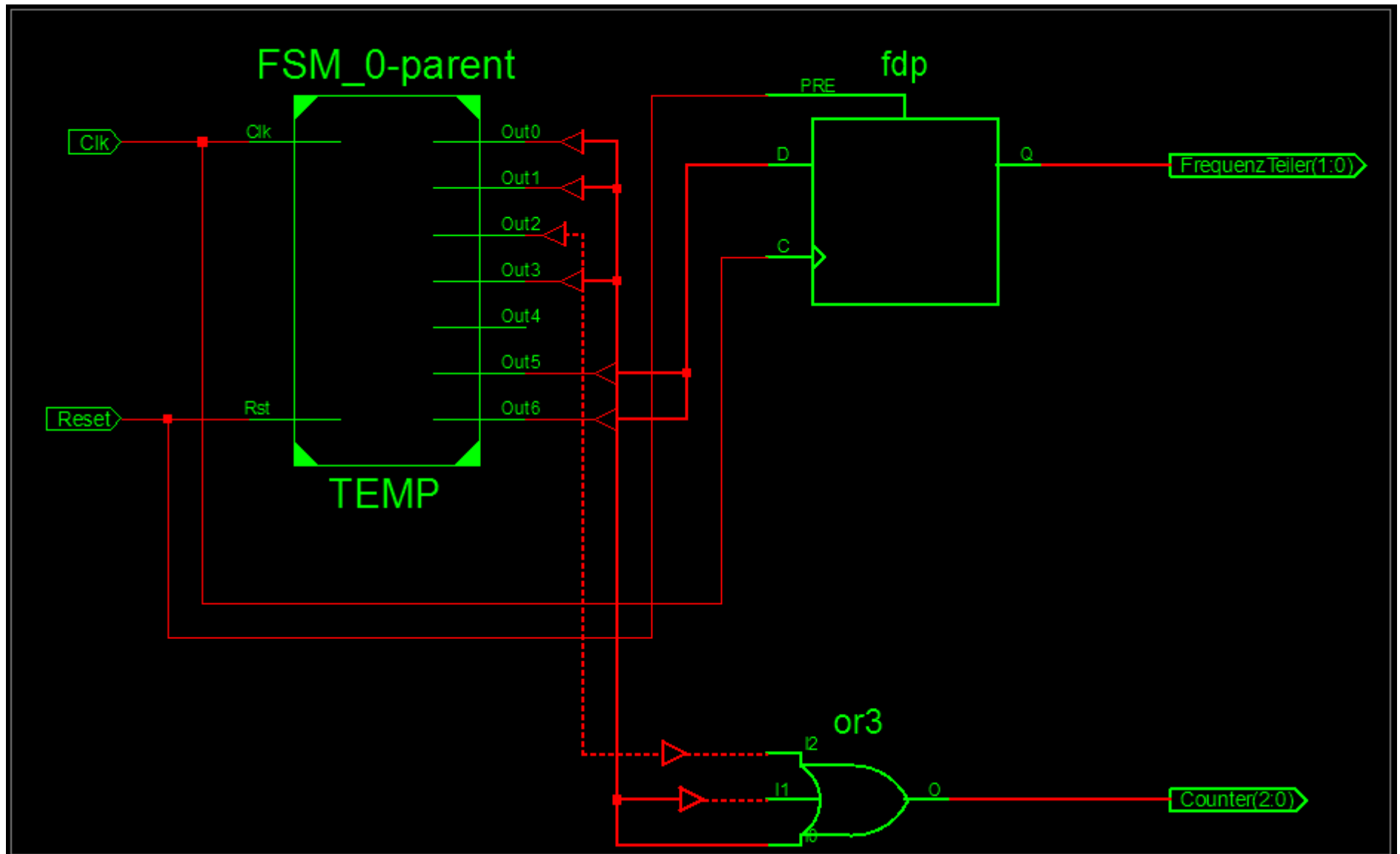
```

38 architecture Behavioral of johnson_arc is
39
40 signal TEMP : STD_LOGIC_VECTOR (2 downto 0);
41
42 begin
43
44 JohnsonProcess: process (Clk, Reset)
45 begin
46
47     if Reset = '1' then
48
49         TEMP <= "000";
50         FrequenzTeiler <= "11";
51
52     elsif Clk'event and Clk = '1' then
53
54         case TEMP is
55             when "000" => TEMP <= "001";
56                         FrequenzTeiler <= "01";
57             when "001" => TEMP <= "011";
58                         FrequenzTeiler <= "10";
59             when "011" => TEMP <= "111";
60                         FrequenzTeiler <= "00";
61             when "111" => TEMP <= "110";
62                         FrequenzTeiler <= "10";
63             when "110" => TEMP <= "100";
64                         FrequenzTeiler <= "00";
65             when "100" => TEMP <= "000";
66                         FrequenzTeiler <= "11";
67             when others => TEMP <= "111";
68                         FrequenzTeiler <= "11";
69         end case;
70     end if;
71 end process JohnsonProcess;
72
73 Counter <= TEMP;
74
75 end Behavioral;

```

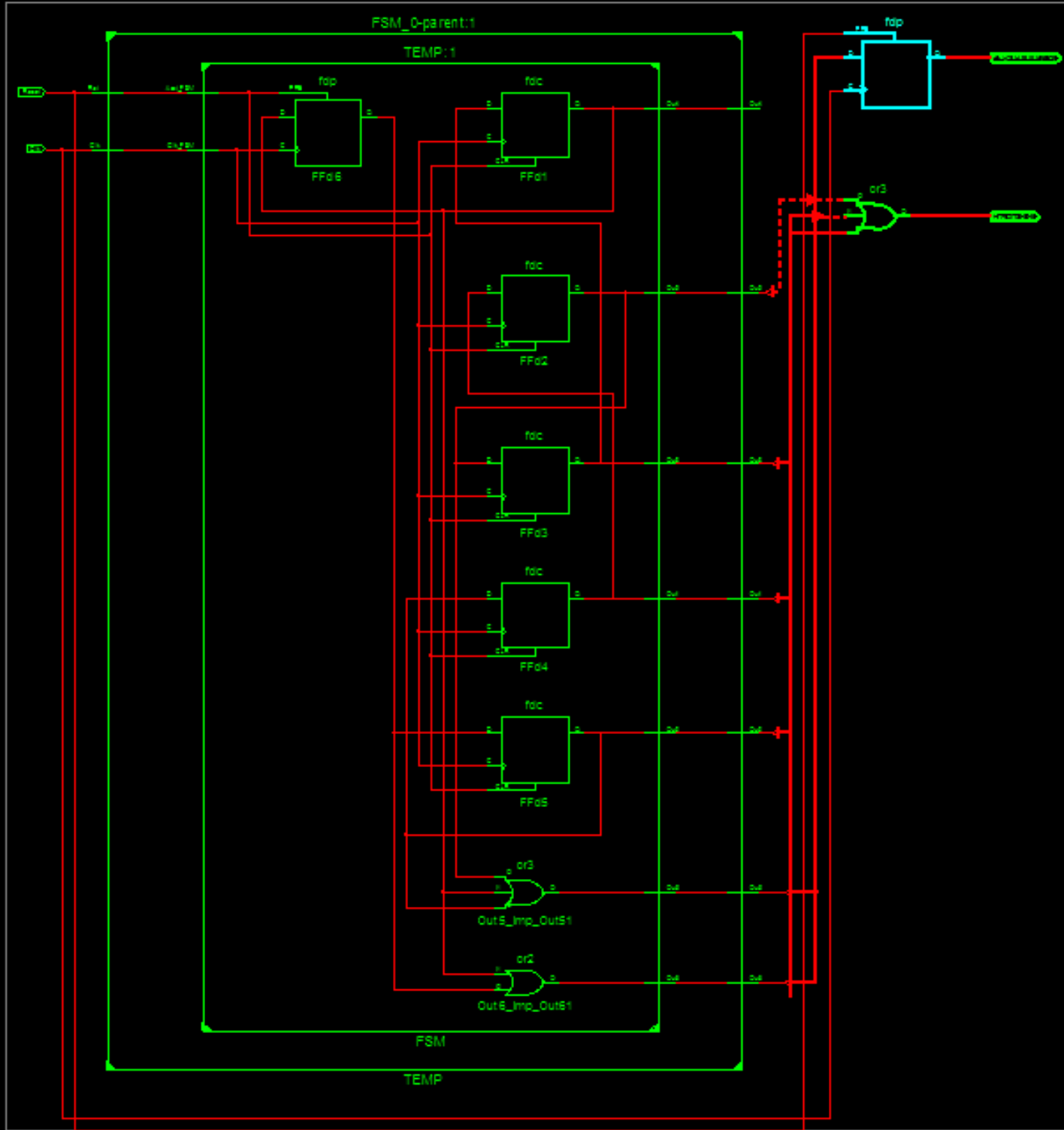
Architecture

Synthese(1/2)



Synthese(2/2)

- 2 D-FF mit Reset Eingang
- 6 Zustände
 - Others geht wieder zu TEMP = „111“



Register und Schieberegister

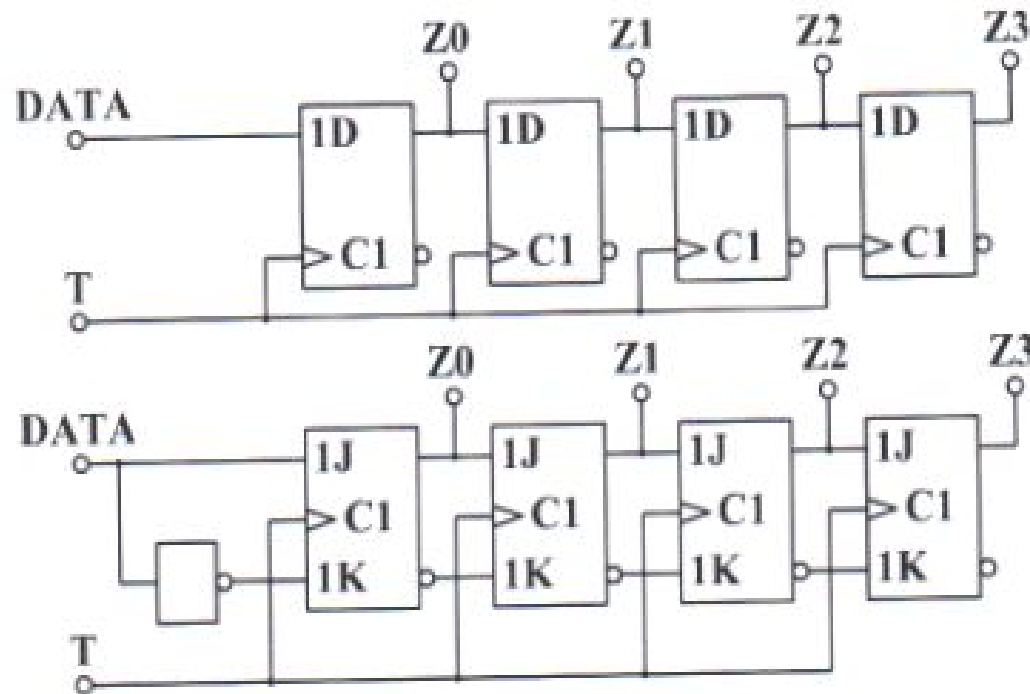
Register

- Register dienen dazu,
 - kurzfristig einige Bits zu speichern
 - als Pufferspeicher zwischen Geräten mit unterschiedlicher Art und Geschwindigkeit die Informationen zu verarbeiten
 - Seriell-zu-parallel Wandlung
 - Parallel-zu-seriell Wandlung
- Unterteilung in
 - Seriell
 - Parallelregister
- Parallelregister (oder kurz Register) liegen vor, wenn die Ein- und Ausgabe parallel erfolgt
- Alle anderen Register heißen Serien- und Schieberegister

Schieberegister

Grundschtaltung Schieberegister (1/3)

- Gruppenübung: Beschreiben Sie das Verhalten der folgenden Schaltung

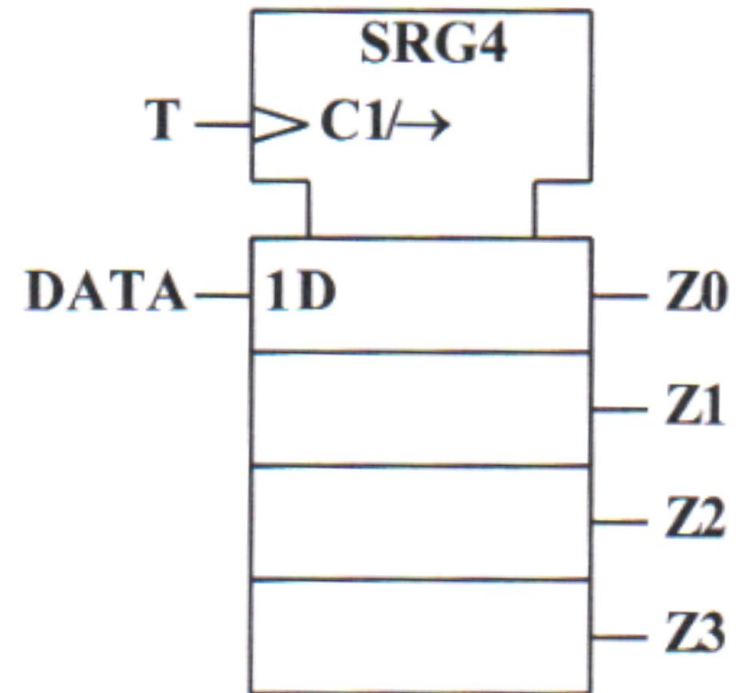


Grundschtaltung Schieberegister (2/3)

- Es kommen nicht-transparente FF zum Einsatz
 - Daten werden mit einer aktiven Taktflanke übernommen
 - Übernahme der Daten von DATA zu FF Z0
 - Gleichzeitige Übernahme der vorherigen Daten von Z0 zu Z1
 - usw
 - Die Information in Z3 zum Zeitpunkt vor der Taktflanke geht verloren
- Informationen werden also mit jeder Taktflanke um eine Position nach rechts verschoben
 - Z0 erhält eine neue Information
 - Z3 verliert die aktuelle Information
- Diese Art der Schaltung wird als Schieberegister bezeichnet

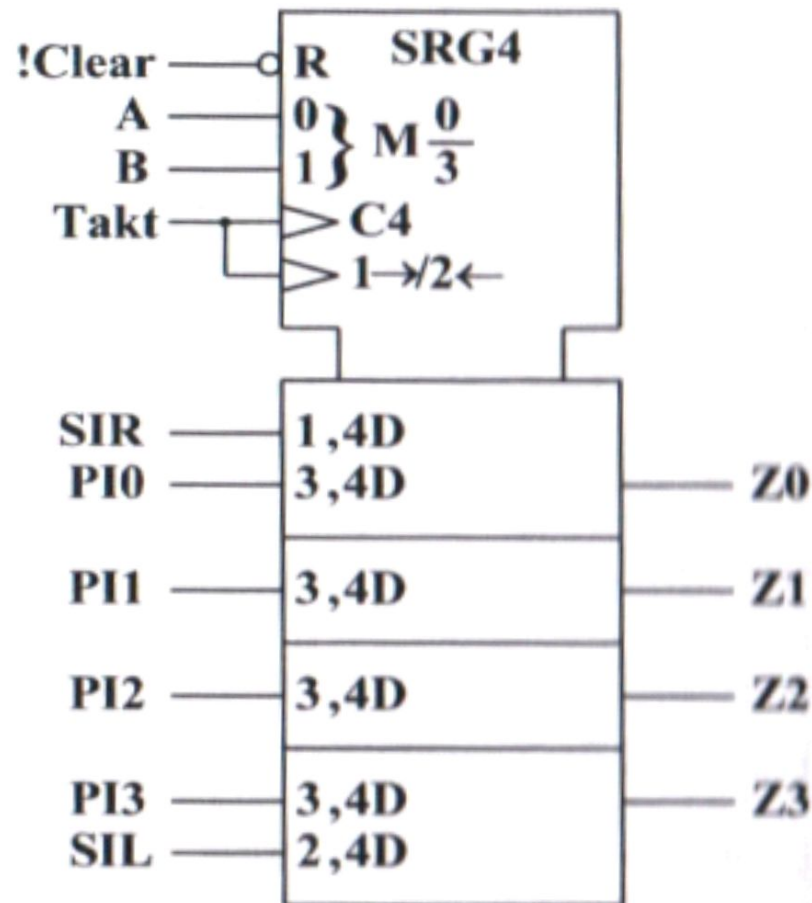
Grundschaltung Schieberegister (3/3)

- Seriell eingegebene Daten liegen später parallel an den Ausgängen Z0 ... Z3 an
- Die Länge von Schieberegistern kann beliebig groß sein
 - 4, 8, 16, 32, 64 oder 128 Bit sind typische Werte
 - Aber auch 158 Bit Register kommen vor
- Mögliches Symbol
 - SRG4 - Shift Register 4 Bit
 - Schieberichtung wird durch Pfeil am Takteingang angegeben
 - → nach rechts schiebendes Register – im Symbol bedeutet dies, dass von oben nach unten geschoben wird



Universal Schieberegister

- Gruppenübung: Beschreiben Sie das folgende Register



Universal-Schieberegister (1/2)

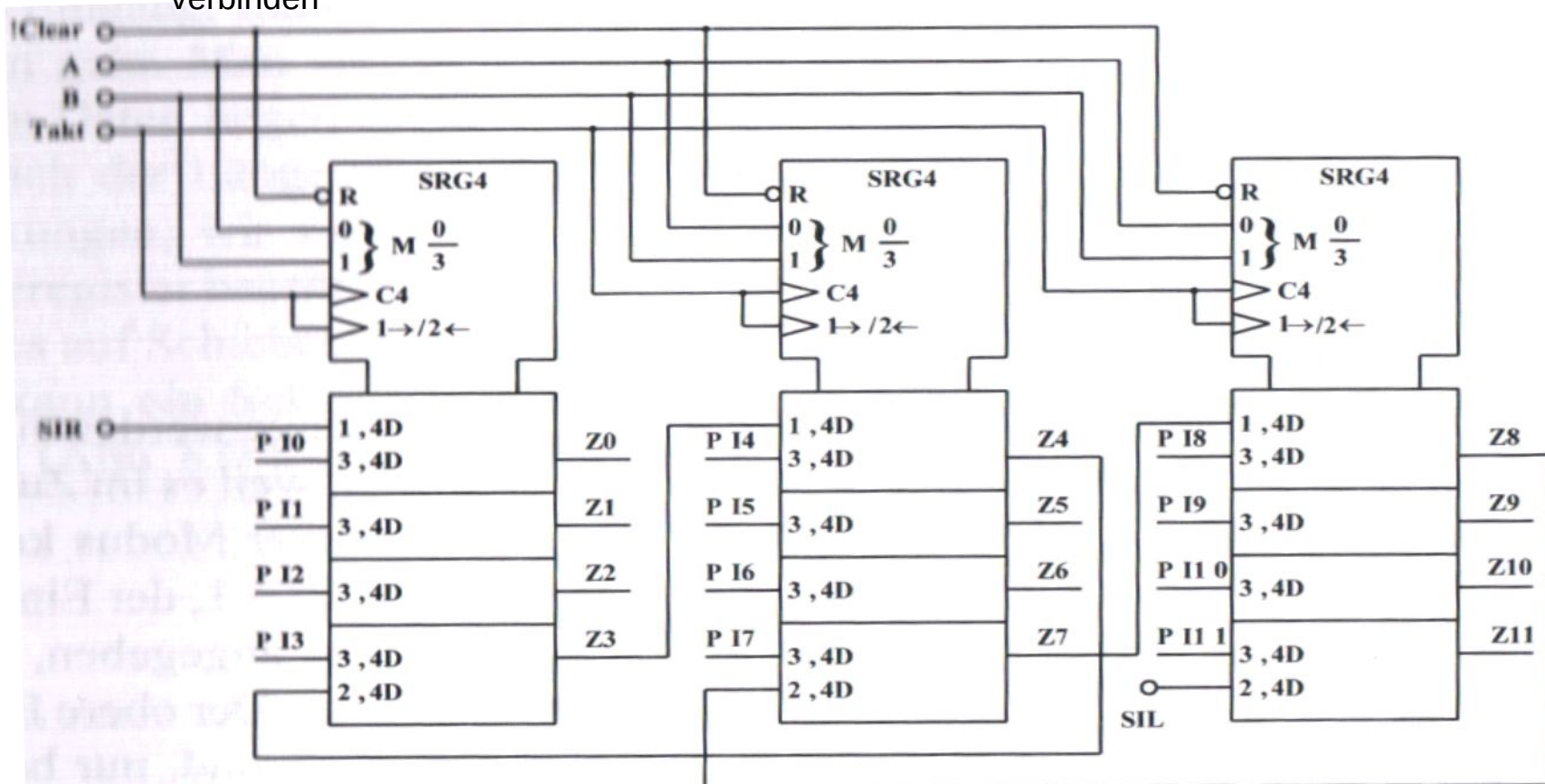
- Signal R bewirkt einen asynchronen Reset
 - !Clear → active low Reset
- A und B definieren einen Modus
 - A hat das Gewicht $2^0 = 1$
 - B hat das Gewicht $2^1 = 2$
 - Potenzen sind an den Eingängen angegeben
- Takteingang ist intern aufgesplittet
 - Oberer Eingang (C4) bedeutet, dass alle Operationen, die mit 4 gekennzeichnet sind, nur bei einer steigenden Taktflanke aktiv sind
 - Unterer Takteingang ($1 \rightarrow / 2 \leftarrow$) wird folgendes festgelegt
 - Modus 1 (A = 1, B = 0) Schieben nach rechts
 - Modus 2 (A = 0, B = 1) Schieben nach links

Universal Schieberegister (2/2)

- Erste und letzte Speicherzelle haben je zwei Eingänge
 - 3,4D Eingänge sind nur im Modus 3 ($A = 1$, $B = 1$) aktiv
- Im Modus 3 kann das Register mit Informationen von den Eingängen PI0 ... PI3 geladen werden
 - Ladevorgang ist synchron → taktflankengesteuert
- SIR (Serial Input Right) mit Kennzeichnung 1,4D
 - Nur im Modus 1 „Schieben nach Rechts“ aktiv
- SIL (Serial Input Left) mit Kennzeichnung 2,4D
 - Modus 2 „schieben nach Links“
- Modus 0, ($A = 0$, $B = 0$) – Schieberegister verbleibt in aktuellem Zustand

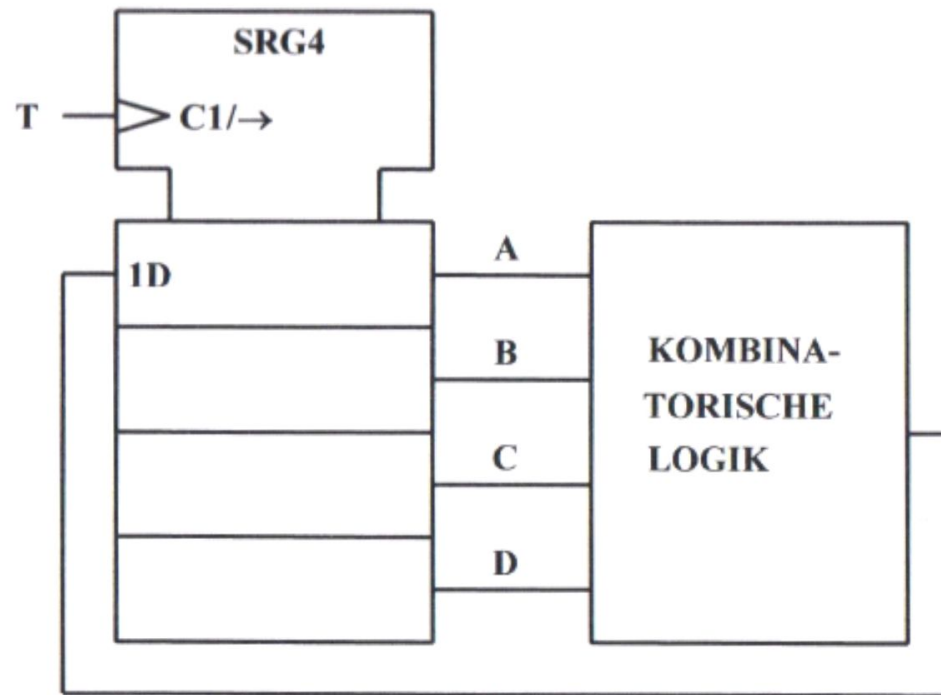
Kaskadierung

- Zur Realisierung längerer Schieberegister können diese auch kaskadiert werden
- 12-Bit Schieberegister
 - Rest-, Takt, und Modus-Eingänge aller Register werden verbunden
 - Schieben nach rechts: Ausgang aktuelles FF wird mit SIR Eingang des „nächsten“ verbunden
 - Schieben nach links: Ausgang des aktuellen FF mit SIL Eingang des „vorhergehenden“ Bausteins verbinden



Rückgekoppelte Schieberegister (1/2)

- Logische Funktion der Ausgangssignale des Schieberegisters wird auf den seriellen Eingang zurück gekoppelt
- Grundschriftung eines rückgekoppelten Schieberegisters



Rückgekoppelte Schieberegister (2/2)

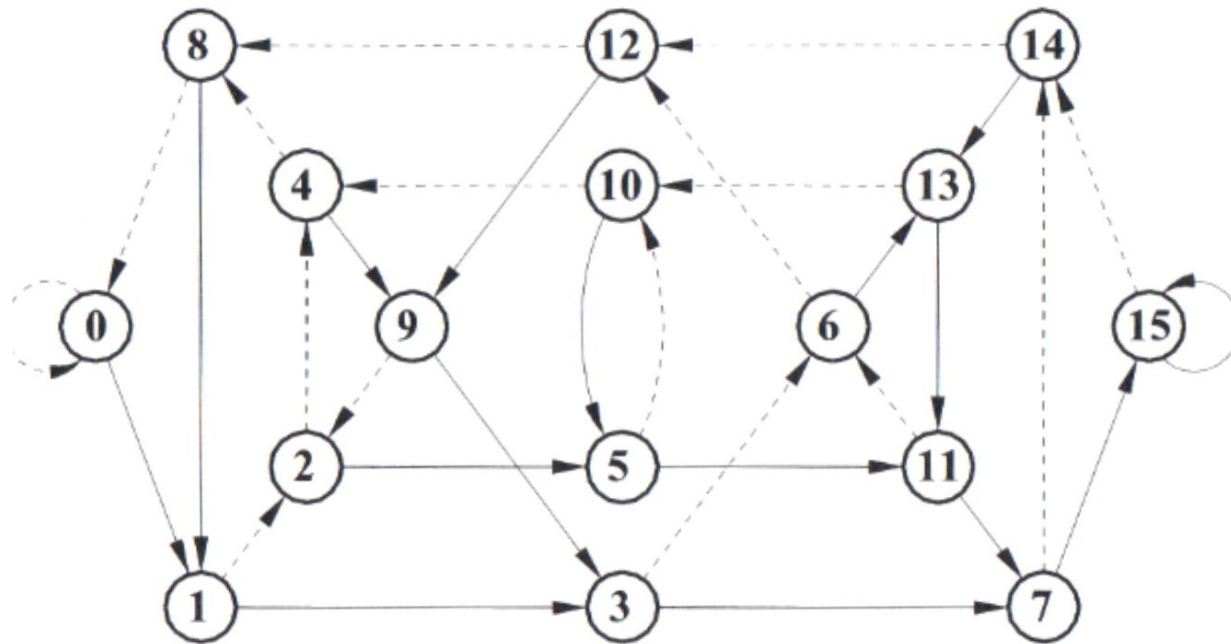
- 4Bit Schieberegister kann 16 verschiedene Zustände annehmen
- Man kann aber nur von einem bestimmten Zustand nur gerade in zwei neue Zustände gelangen
 - Es sind also nicht beliebige Zustandsfolgen möglich
- Aus dem Zustand ABCD kann man nur in einen der beiden Zustände 0ABC oder 1 ABC gelangen
 - A = LSB
 - D = MSB

je nachdem ob eine 0 oder eine 1 eingelesen wurde

- Alle anderen Daten werden nur um eine Stelle verschoben und bleiben im Übrigen unverändert

Allgemeines Zustandsdiagramm (1/2)

- Im allgemeinen Zustandsdiagramm sind alle möglichen Übergänge eingetragen
- „Durchgezogenen“ Pfeile entsprechen dabei den Übergängen, bei denen eine 1 eingeschoben wird
- „Gestrichelten“ Pfeile stehen für Übergänge mit einer eingeschobenen 0



- Dieses Zustandsdiagramm beschreibt kein konkretes System, sondern zeigt nur, welche Übergänge bei einem 4Bit Schieberegister überhaupt möglich sind

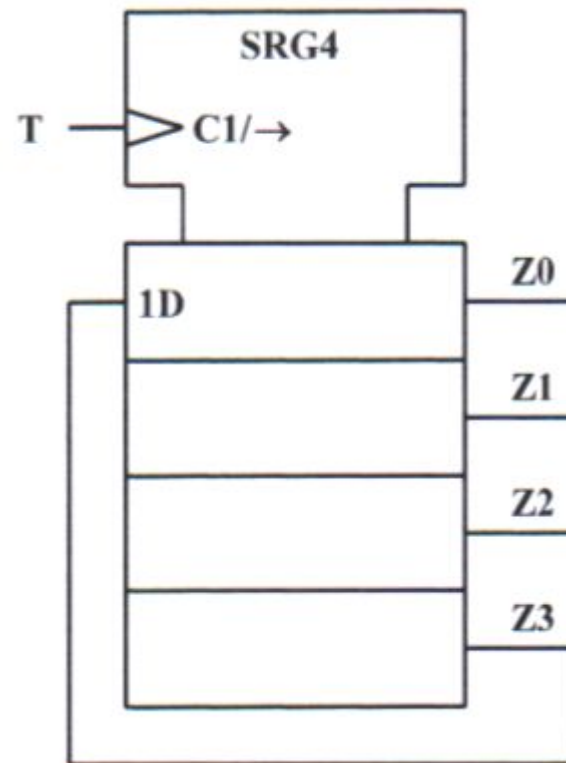
Allgemeines Zustandsdiagramm (2/2)

- Dieses Zustandsdiagramm beschreibt kein konkretes System, sondern zeigt nur, welche Übergänge bei einem 4Bit Schieberegister überhaupt möglich sind
- Ein direkter Übergang von Zustand 1 in den Zustand 5 wäre beispielsweise völlig unmöglich
- Wenn man nun eine konkrete Rückführungslogik auswählt, dann ist das resultierende Zustandsdiagramm ein Teilgraph dieses allgemeinen Zustandsgraphen

Ringzähler

- Serieller Eingang eines Schieberegisters wird mit dem in Schieberichtung letzten Ausgang verbunden
 - FF Kette wird also zu einem Ring verbunden
 - die im Register gespeicherte Information sollte also dauernd umlaufen

- Schema eines Ringzählers

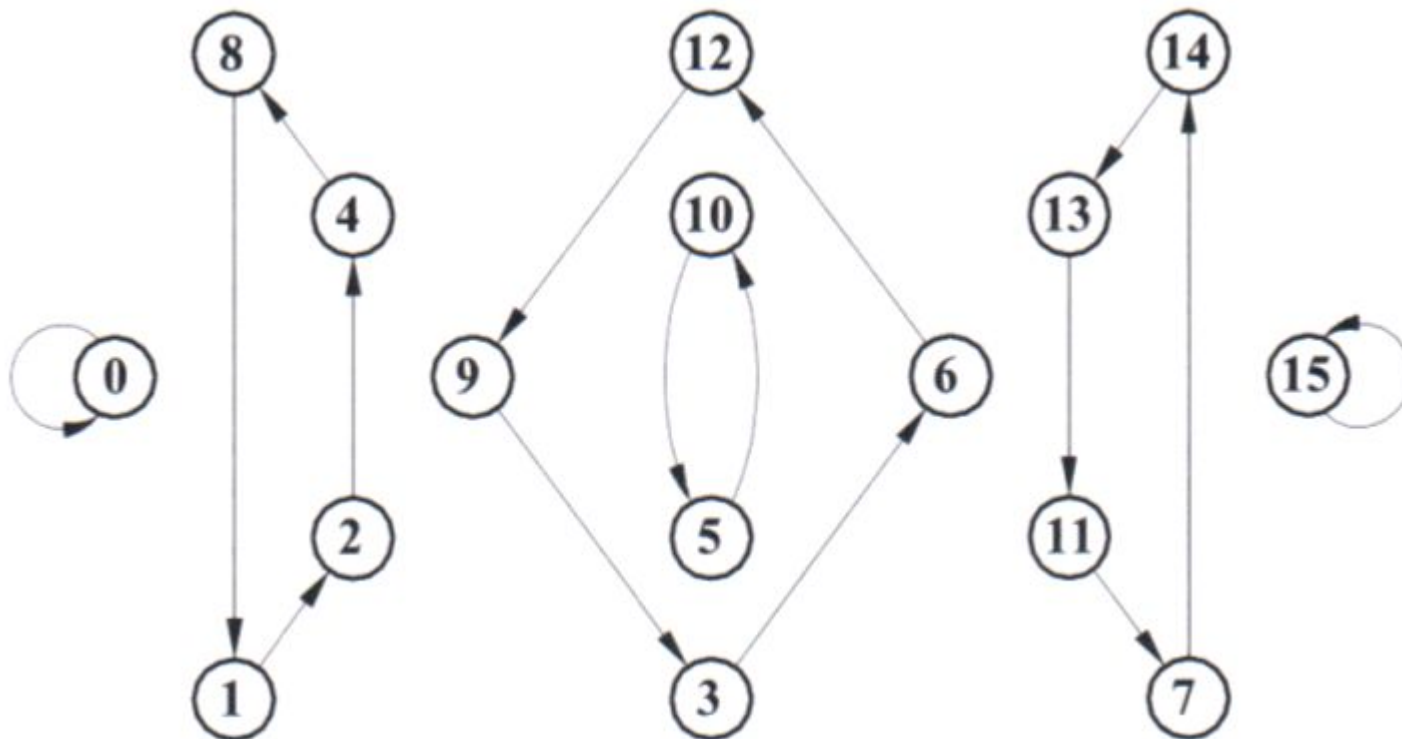


Gruppenübung

- Entwickeln Sie gemeinsam das Zustandsdiagramm eines Ringzählers

Zustandsdiagramm eines Ringzählers

- Zustandsdiagramm eines Ringzählers
 - Drei Zyklen der Länge 4
 - Ein Zyklus der Länge 2
 - Zwei Fixpunkte
- Damit das System sich in einem bestimmten Zyklus bewegt, muss ein **Zustand des gewünschten Zyklus beim Start geladen werden**



Einzelübung

- Laden Sie den 4Bit Ringzähler so vor, dass folgende Sequenzen entstehen
 - $1 - 2 - 4 - 8 - 1 - \dots$
 - $7 - 14 - 13 - 11 - 7 - \dots$

Mehrphasige Taktsignale

- Große praktische Bedeutung sind die beiden Zyklen
 - $1 - 2 - 4 - 8 - 1 - \dots$ Umlaufende 1
 - $7 - 14 - 13 - 11 - 7 - \dots$ Umlaufende 0
- Vorladung
 - 0001
 - 0111
- Auf diese Art können mehrphasige Taktsignale generiert werden

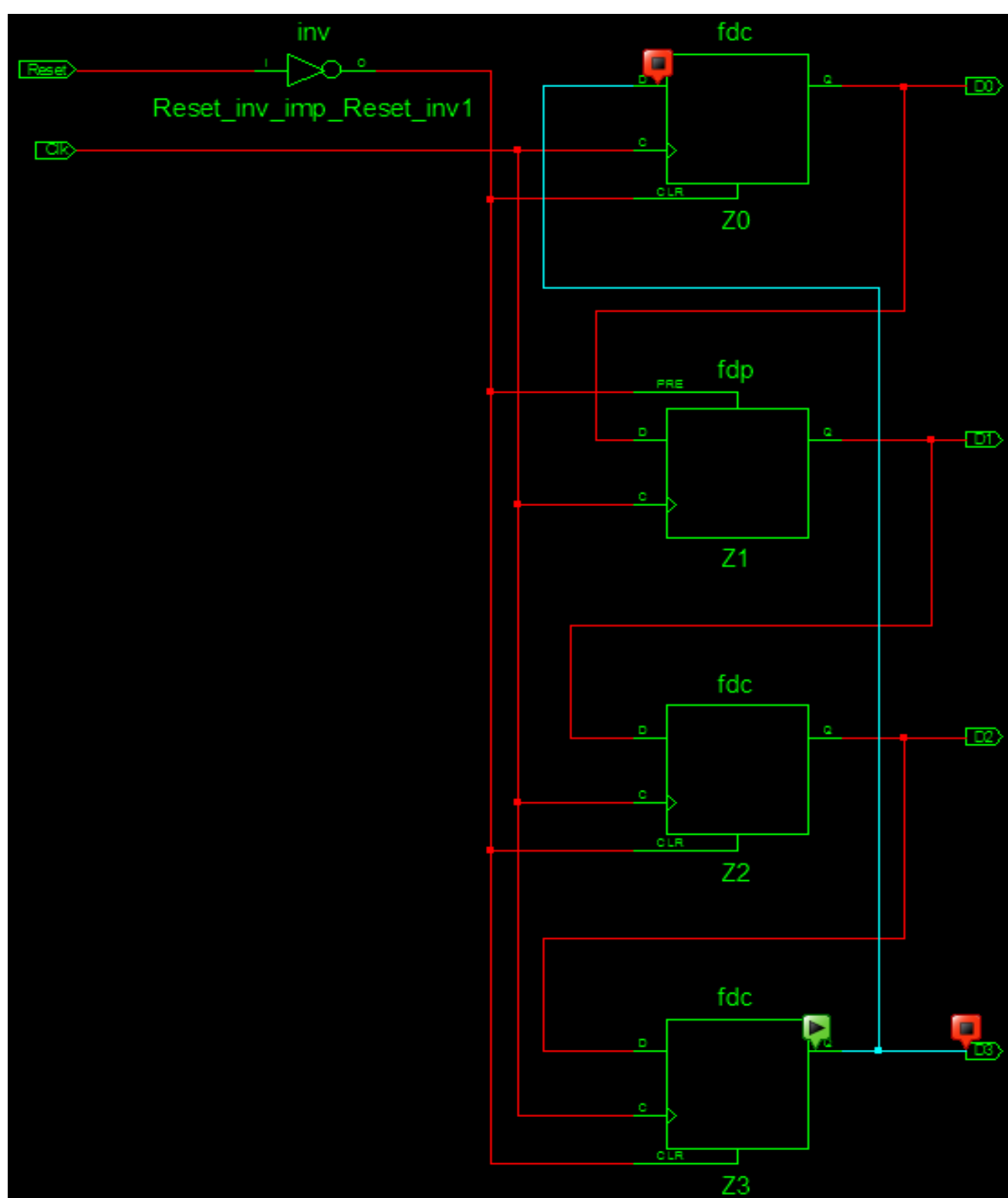
VHDL Code (1/2)

```
20 library IEEE;
21 use IEEE.STD_LOGIC_1164.ALL;
22 use IEEE.STD_LOGIC_ARITH.ALL;
23 use IEEE.STD_LOGIC_UNSIGNED.ALL;
24
25 ---- Uncomment the following library declaration if instantiating
26 ---- any Xilinx primitives in this code.
27 --library UNISIM;
28 --use UNISIM.VComponents.all;
29
30 entity ZaehlerEinfach is
31     Port ( Clk      : in  STD_LOGIC;
32           Reset    : in  STD_LOGIC;
33           D0       : out  STD_LOGIC;
34           D1       : out  STD_LOGIC;
35           D2       : out  STD_LOGIC;
36           D3       : out  STD_LOGIC);
37 end ZaehlerEinfach;
```

VHDL Code (2/2)

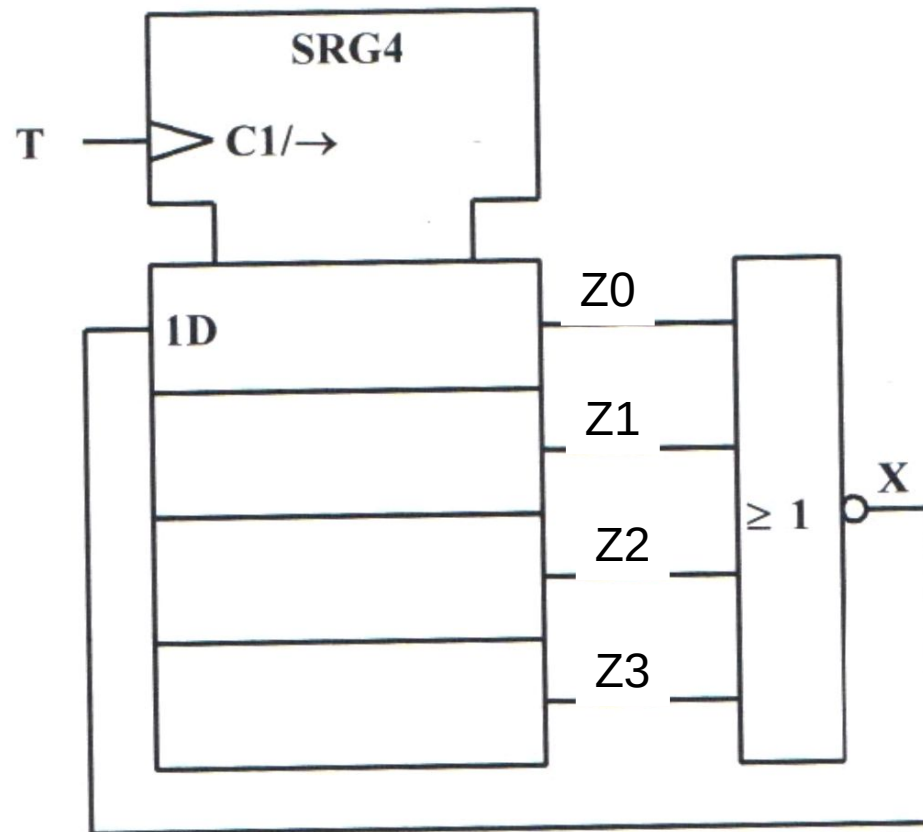
```
39 architecture Behavioral of ZaehlerEinfach is
40
41 signal Z0,Z1,Z2,Z3 : STD_LOGIC;
42
43 begin
44
45 P1_Synch: process(Clk,Reset)
46
47     begin
48
49         if Reset = '0' then
50
51             Z0 <= '0';
52             Z1 <= '1';
53             Z2 <= '0';
54             Z3 <= '0';
55
56         elsif Clk'event AND Clk = '1' then
57
58
59             Z0 <= Z3;  -- Rueckkopplung
60             Z1 <= Z0;  -- Shiften nach vorne
61             Z2 <= Z1;  -- Shiften
62             Z3 <= Z2;  -- Shiften
63
64         end if;
65
66     end process P1_Synch;
67
68 P2_ASynch : process(Z0,Z1,Z2,Z3)
69
70     begin
71
72         D0 <= Z0;
73         D1 <= Z1;
74         D2 <= Z2;
75         D3 <= Z3;
76
77
78
79     end process P2_ASynch;
80
81
82 end Behavioral;
```

Synthese



Korrigierter Ringzähler

- Das Laden des Ringzählers mit Startwerten ist keine dauerhafte Lösung
 - Bei der kleinsten Störung kann der Zähler in einen anderen Zyklus wechseln
- Die Rückführungsfunktion soll so ergänzt werden, dass ein selbst-korrigierender Ringzähler entsteht, bei dem alle Zustände nach einigen Taktimpulsen wieder in den Hauptzyklus einmünden → NOR Gatter



Gruppenübung

- Erzeugen Sie die Wahrheitstabelle
- Entwerfen Sie die VHDL Beschreibung

Lösung

- X bezeichnet den Ausgang des NOR-Gliedes
 - Wird aber interpretiert als Eingang vom Shiftregister
- X wird nicht für die Dezimalberechnung berücksichtigt

Z3	Z2	Z1	Z0	Dez	X

0	0	0	0	0	1
0	0	0	1	1	0
0	0	1	0	2	0
0	1	0	0	4	0
1	0	0	0	8	0
0	0	0	0	0	1
0	0	0	1	1	0
...					

VHDL (1/3)

```
20 library IEEE;
21 use IEEE.STD_LOGIC_1164.ALL;
22 use IEEE.STD_LOGIC_ARITH.ALL;
23 use IEEE.STD_LOGIC_UNSIGNED.ALL;
24
25 ---- Uncomment the following library declaration if instantiating
26 ---- any Xilinx primitives in this code.
27 --library UNISIM;
28 --use UNISIM.VComponents.all;
29
30 entity RingZaehlerKorrigiert is
31     Port ( Clk      : in  STD_LOGIC;
32           Reset    : in  STD_LOGIC;
33           D0       : out  STD_LOGIC;
34           D1       : out  STD_LOGIC;
35           D2       : out  STD_LOGIC;
36           D3       : out  STD_LOGIC);
37 end RingZaehlerKorrigiert;
```

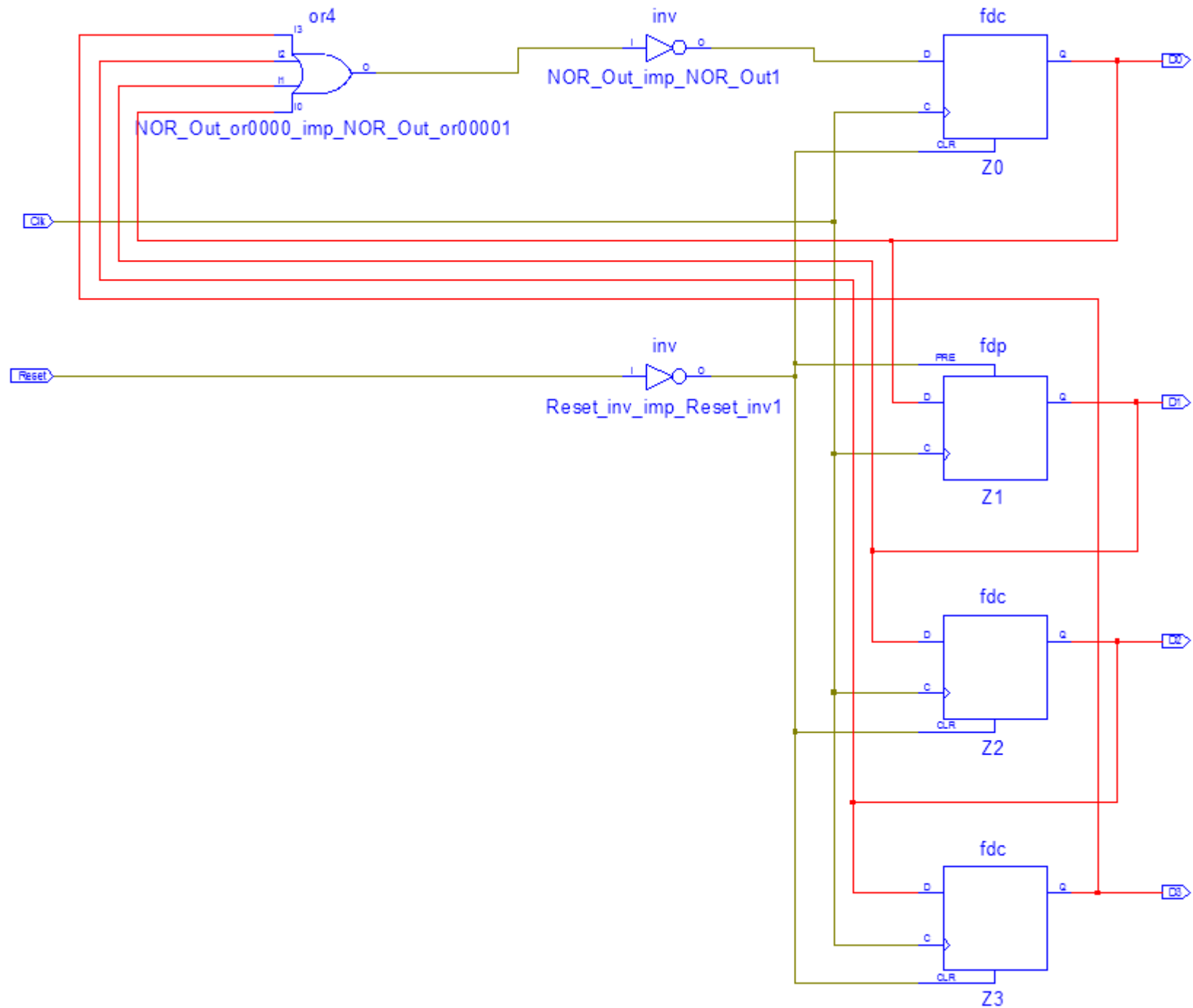
VHDL (2/3)

```
39 architecture Behavioral of RingZaehlerKorrigiert is
40
41 signal Z0,Z1,Z2,Z3 : STD_LOGIC;
42 signal NOR_Out : STD_LOGIC;
43
44 begin
45
46 -- SYNCHRONER Prozess
47 P1_Synch: process(Clk,Reset)
48
49 begin
50
51     if Reset = '0' then
52
53         -- fest vorgegebener Startwert
54         Z0 <= '0';
55         Z1 <= '1';
56         Z2 <= '0';
57         Z3 <= '0';
58
59     elsif Clk'event AND Clk = '1' then
60
61
62         Z0 <= NOR_Out; -- Rueckkopplung von dem NOR Gatter
63         Z1 <= Z0;      -- Shiften nach vorne
64         Z2 <= Z1;      -- Shiften
65         Z3 <= Z2;      -- Shiften
66
67     end if;
68
69 end process P1_Synch;
```

VHDL (3/3)

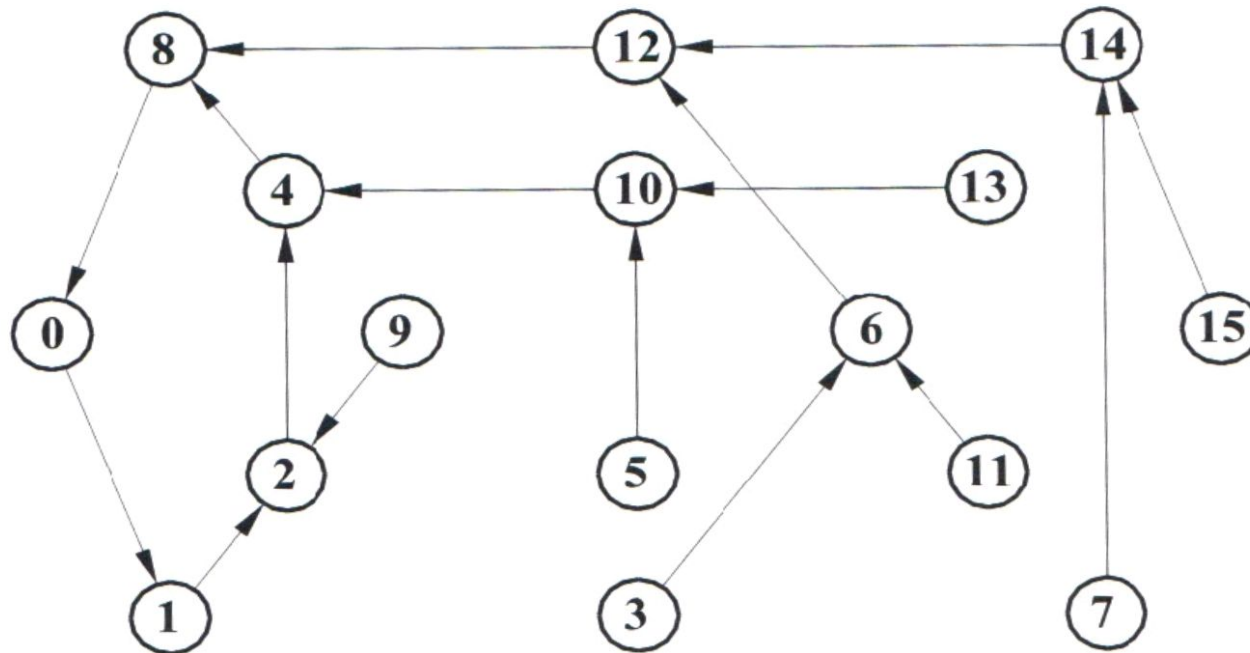
```
71  -- ASYNCHRONER Prozess -> kombinatorische Logik
72  P2_ASynch : process(Z0,Z1,Z2,Z3)
73
74      begin
75          -- Fliflop Inhalte auf den jeweiligen Ausgang legen
76          D0 <= Z0;
77          D1 <= Z1;
78          D2 <= Z2;
79          D3 <= Z3;
80
81          -- Alle Registerinhalte ODER verknuepfen und am Ende NEGIEREN
82          NOR_OUT <= NOT(Z0 OR Z1 OR Z2 OR Z3);
83
84
85
86      end process P2_ASynch;
87
88  end Behavioral;
```

Synthese



Zustandsdiagramm des korrigierenden Ringzählers

- Das NOR-Glied hat die Eigenschaft, dass der Ausgang nur dann auf 1 liegt, wenn alle Eingänge auf 0 liegen
- Am seriellen Eingang liegt also nur im Zustand 0 eine 1
 - In allen anderen Zuständen liegt eine 0 an



- **Gruppenübung: nach wie vielen Taktimpulsen mündet jeder Zustand spätestens in den Hauptzyklus**

Lösung

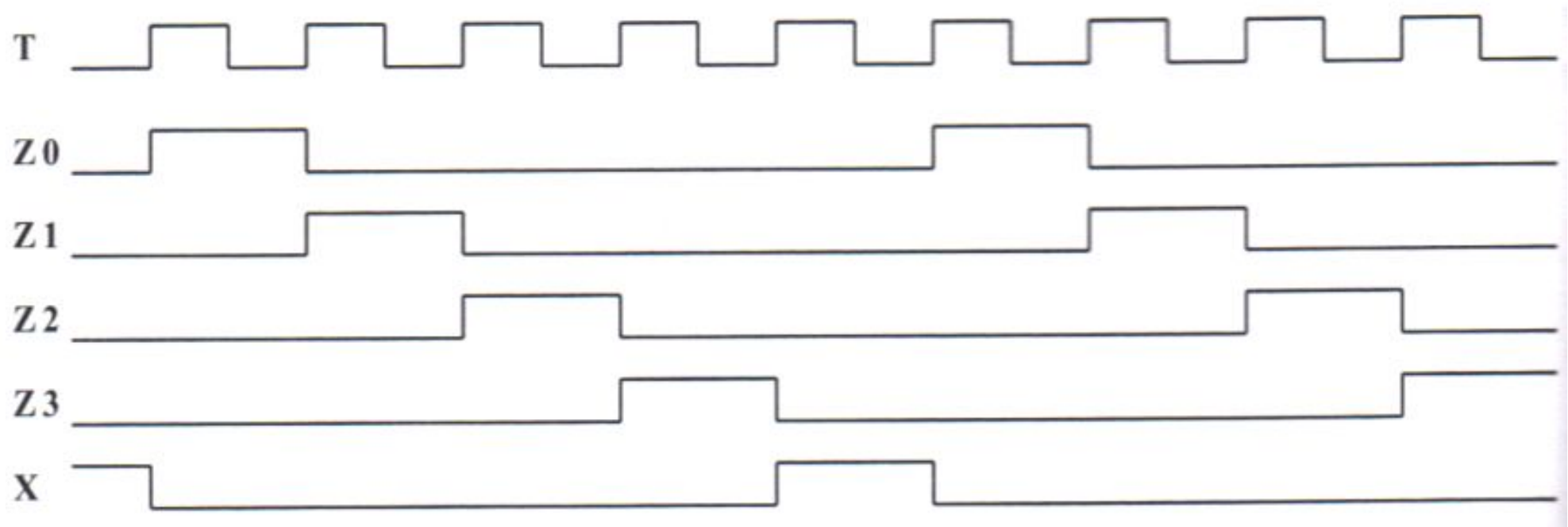
- Alle Zustände münden nach max. 3 Taktimpulsen wieder in den Hauptzyklus

Länge des Hauptzyklus

- Die Länge des Hauptzyklus hat zugenommen
 - Umfasst $N+1$ Zustände
- Da der Zustand 0 ebenfalls zum Zyklus gehört, haben wir nicht mehr das reine Bild einer umlaufenden 1
- Andererseits ist der Ausgang des NOR-Gliedes genau im Zustand 0 auf 1
- Wird der serielle Eingang des Schieberegisters = NOR-Ausgang als zusätzliches Ausgangssignal betrachtet, so erhält man eine umlaufende 1 mit 5 Signalen

Zeitdiagramm

- Signalverläufe beim korrigierten Ringzähler



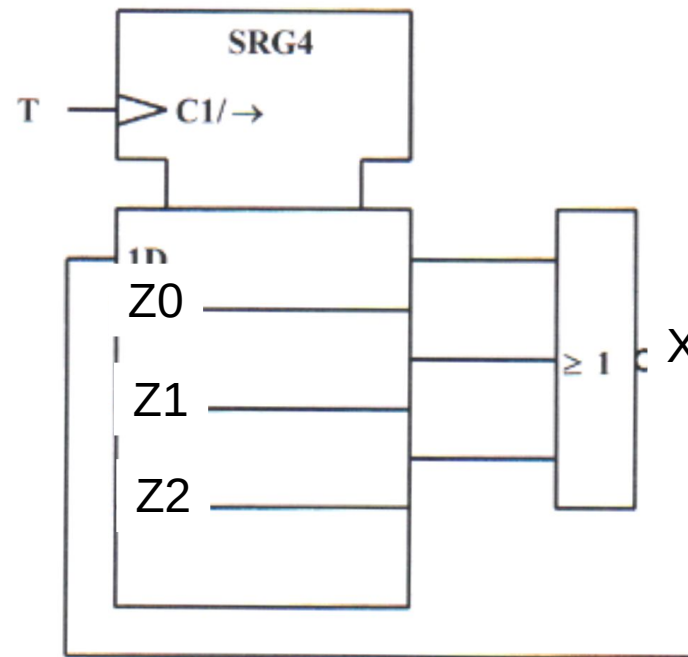
Verbesserung des Ringzählers durch den korrigierten Ringzähler

- Ausgangspunkt war ein Ringzähler mit 4Bit-Register
 - Es konnten folgende Sequenzen erzeugt werden
 - 1 – 2 – 4 – 8 – 1 – ... Umlaufende 1
 - 7 – 14 – 13 – 11 – 7 – ... Umlaufende 0
 - Allerdings nur mit dem Vorladen des Registers auf
 - 0001
 - 0111
- Nachteil: bei der kleinsten Störung kann der Zähler in einen anderen Zyklus wechseln
- Daher wurde der korrigierte Ringzähler eingeführt!
 - Umlaufende 1 wurde garantiert durch Nutzung der NOR-Gatters

Erzeugung eines Vierphasentaktes

Vierphasentakt

- Zur Realisierung eines Vierphasen Taktes müssen 3 FF verwendet werden
 - Rückkopplung erfolgt über ein NOR-Glied
- Schieberegister ließe sich auch über ein 4Bit Schieberegister realisieren, nur der letzte Ausgang bliebe unberücksichtigt



- **Gruppenübung: erstellen Sie die Bitsequenz / Wahrheitstabelle**
- **Entwerfen Sie den optimierten VHDL Code**

Lösung

- X bezeichnet den Ausgang des NOR-Gliedes
- X wird bei der Bestimmung der Dezimalzahl mitgerechnet

X	Z2	Z1	Z0	Dez

<u>1</u>	0	0	0	8
0	0	0	<u>1</u>	1
0	0	<u>1</u>	0	2
0	<u>1</u>	0	0	4
<u>1</u>	0	0	0	8
0	0	0	1	1
...				

- Vierphasen Takt: 8 - 1 - 2 - 4 - 8 - ... beinhaltet eine umlaufende 1

VHDL (1/3)

```
20 library IEEE;
21 use IEEE.STD_LOGIC_1164.ALL;
22 use IEEE.STD_LOGIC_ARITH.ALL;
23 use IEEE.STD_LOGIC_UNSIGNED.ALL;
24
25 ---- Uncomment the following library declaration if instantiating
26 ---- any Xilinx primitives in this code.
27 --library UNISIM;
28 --use UNISIM.VComponents.all;
29
30 entity VierPhasenTaskt is
31     Port ( Clk    : in  STD_LOGIC;
32           Reset   : in  STD_LOGIC;
33           D0      : out  STD_LOGIC;
34           D1      : out  STD_LOGIC;
35           D2      : out  STD_LOGIC;
36           D3      : out  STD_LOGIC);
37 end VierPhasenTaskt;
```

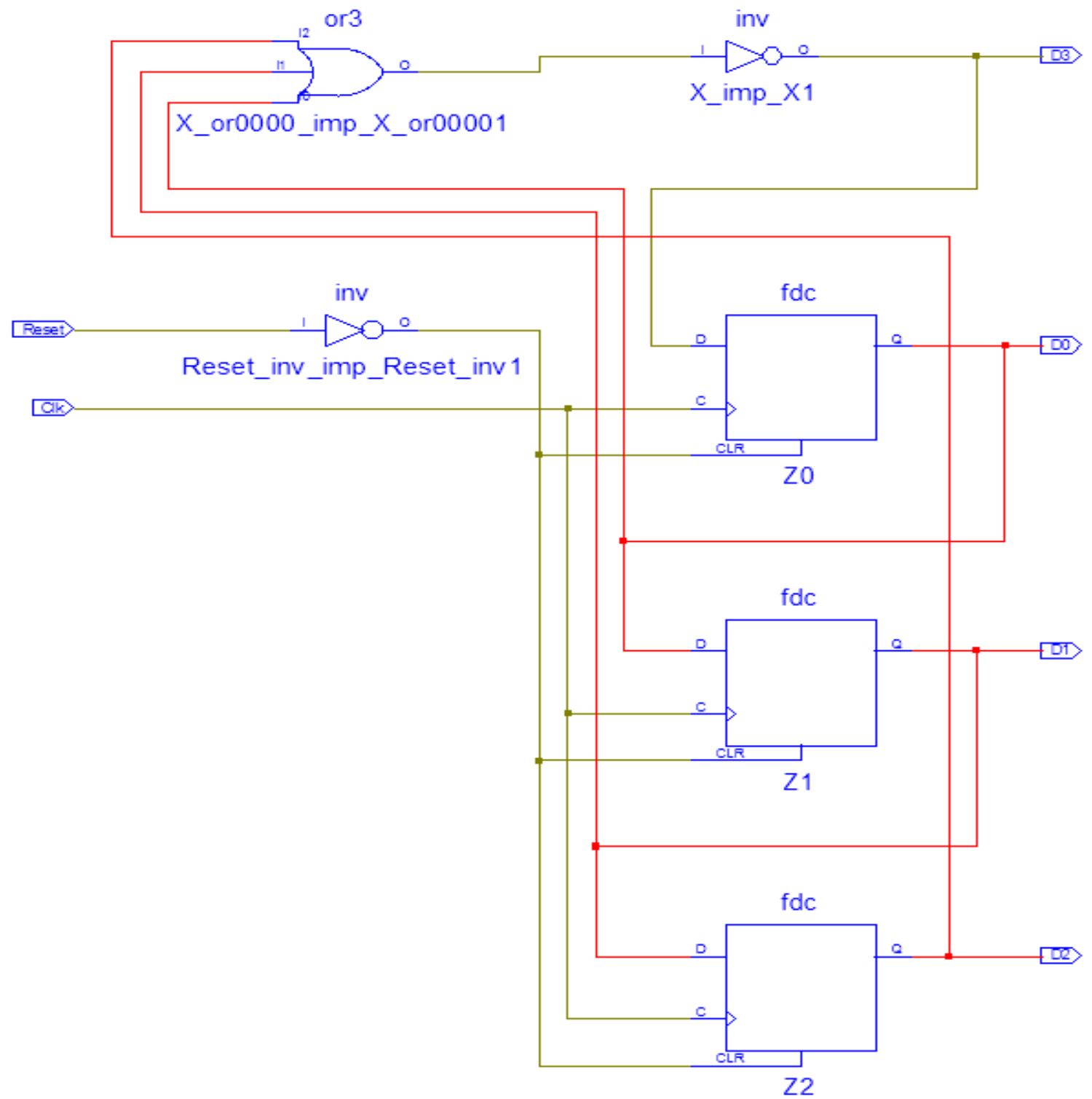
VHDL (2/3)

```
39 architecture Behavioral of VierPhasenTaskt is
40
41 signal Z0,Z1,Z2 : STD_LOGIC;
42 signal X : STD_LOGIC;
43
44 begin
45
46 -- SYNCHRONER Prozess
47 P1_Synch: process (Clk,Reset)
48
49     begin
50
51         if Reset = '0' then
52
53             -- fest vorgegebener Startwert
54             Z0 <= '0';
55             Z1 <= '0';
56             Z2 <= '0';
57
58         elsif Clk'event AND Clk = '1' then
59
60
61             Z0 <= X; -- Rueckkopplung von dem NOR Gatter
62             Z1 <= Z0; -- Shiften nach vorne
63             Z2 <= Z1; -- Shiften
64
65         end if;
66
67     end process P1_Synch;
```

VHDL (3/3)

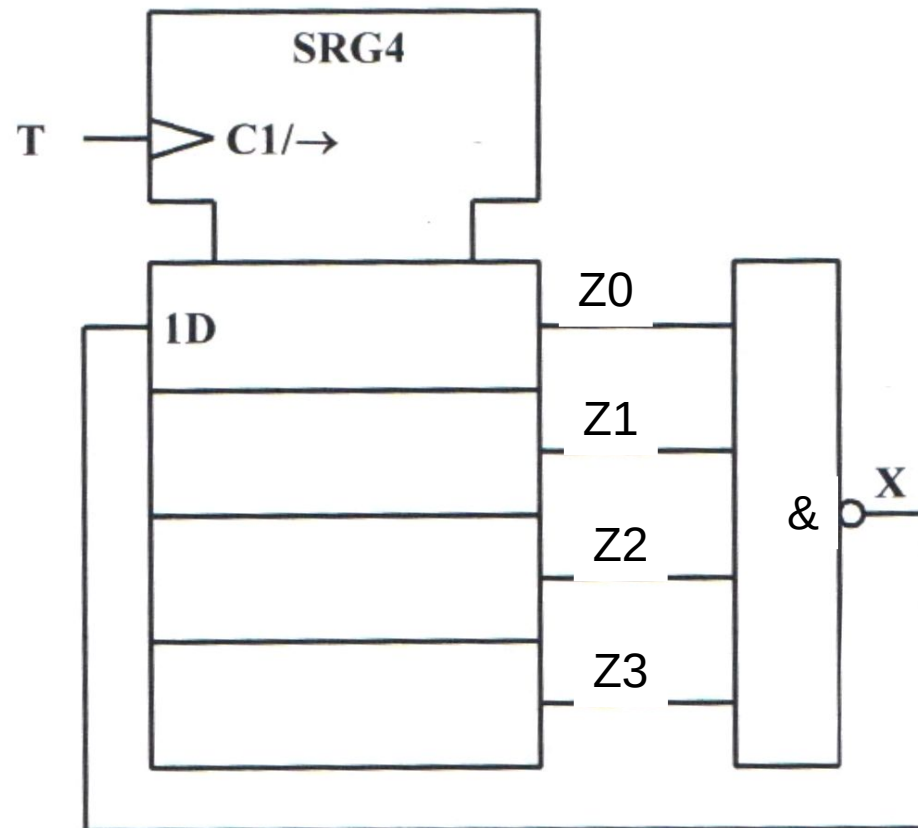
```
68
69 -- ASYNCHRONER Prozess -> kombinatorische Logik
70 ► P2_ASynch : process(Z0,Z1,Z2,X)
71
72     begin
73         -- FlIFlop Inhalte auf den jeweiligen Ausgang legen
74         D0 <= Z0;
75         D1 <= Z1;
76         D2 <= Z2;
77         D3 <= X;
78
79         -- Alle Registerinhalte ODER verknuepfen und am Ende NEGIEREN
80         X <= NOT(Z0 OR Z1 OR Z2);
81
82
83     end process P2_ASynch;
84
85 end Behavioral;
```


Synthese



Gruppenübung

- Ersetzen Sie das NOR-Glied durch eine NAND-Glied und erzeugen Sie die Wahrheitstabelle
- Entwerfen Sie den VHDL Code



Lösung

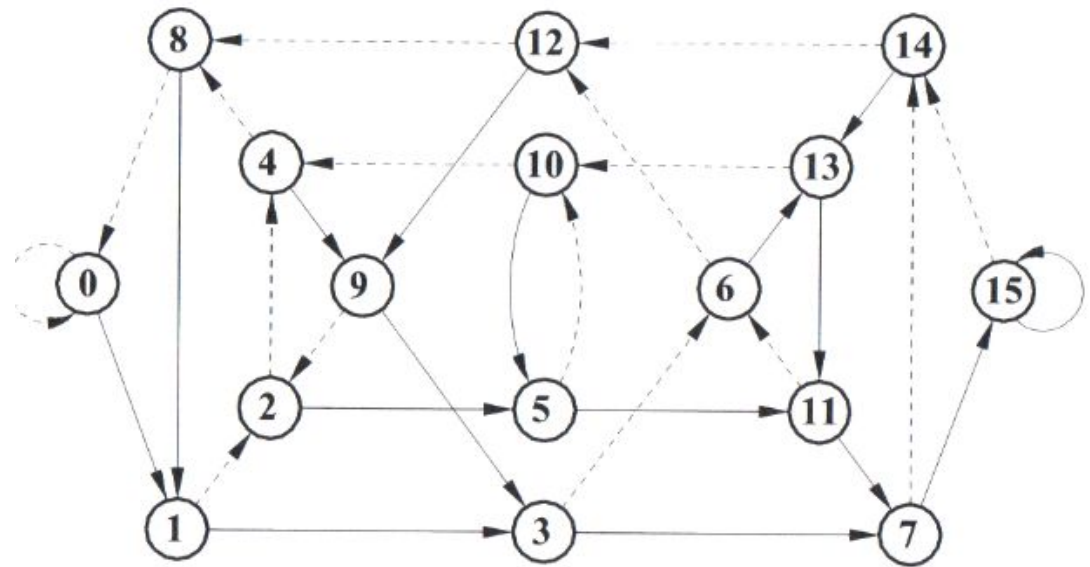
- X bezeichnet den Ausgang des NAND-Gliedes
- X wird nicht bei der Bestimmung der Dezimalzahl mitgerechnet

X	Z3	Z2	Z1	Z0	Dez
---	----	----	----	----	-----

1	0	0	0	0	0
1	0	0	0	1	1
1	0	0	1	1	3
1	<u>0</u>	1	1	1	7
<u>0</u>	1	1	1	1	15
1	1	1	1	<u>0</u>	14
1	1	1	<u>0</u>	1	13
1	1	<u>0</u>	1	1	11
1	<u>0</u>	1	1	1	7

...

- Fünfphasen Takt: 7 - 15 - 14 - 13 - 11 - 7 ... beinhaltet eine umlaufende 0



VHDL (1/3)

```
20 library IEEE;
21 use IEEE.STD_LOGIC_1164.ALL;
22 use IEEE.STD_LOGIC_ARITH.ALL;
23 use IEEE.STD_LOGIC_UNSIGNED.ALL;
24
25 ---- Uncomment the following library declaration if instantiating
26 ---- any Xilinx primitives in this code.
27 --library UNISIM;
28 --use UNISIM.VComponents.all;
29
30 entity FuenfPhasenTaskt is
31     Port ( Clk      : in  STD_LOGIC;
32           Reset    : in  STD_LOGIC;
33           D0       : out  STD_LOGIC;
34           D1       : out  STD_LOGIC;
35           D2       : out  STD_LOGIC;
36           D3       : out  STD_LOGIC);
37 end FuenfPhasenTaskt;
```

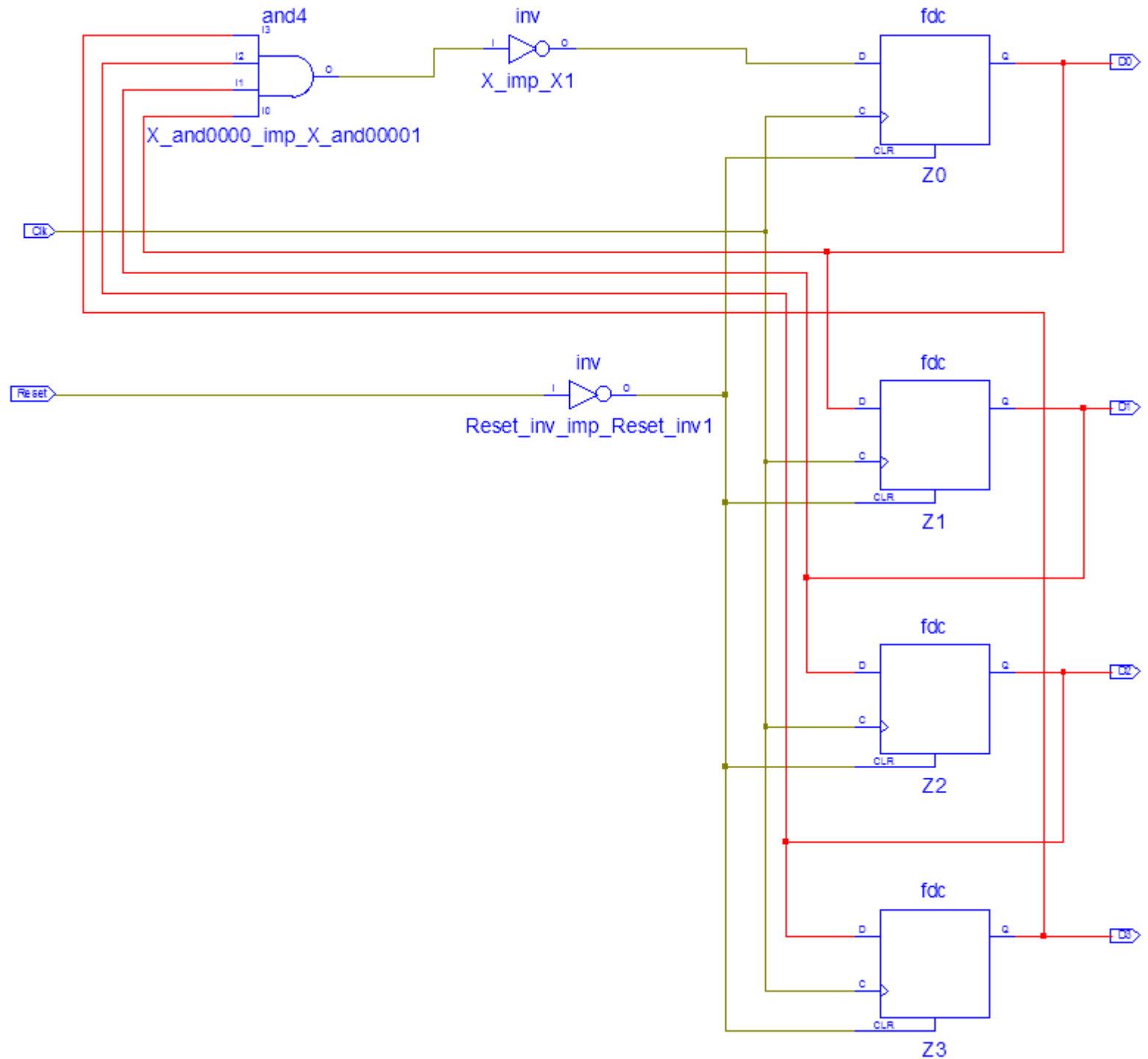
VHDL (2/3)

```
39 architecture Behavioral of FuenfPhasenTaskt is
40
41 signal Z0,Z1,Z2,Z3 : STD_LOGIC;
42 signal X : STD_LOGIC;
43
44 begin
45
46 -- SYNCHRONER Prozess
47 P1_Synch: process(Clk,Reset)
48
49 begin
50
51     if Reset = '0' then
52
53         -- fest vorgegebener Startwert
54         Z0 <= '0';
55         Z1 <= '0';
56         Z2 <= '0';
57         Z3 <= '0';
58
59     elsif Clk'event AND Clk = '1' then
60
61
62         Z0 <= X;  -- Rueckkopplung von dem NOR Gatter
63         Z1 <= Z0;  -- Shiften nach vorne
64         Z2 <= Z1;  -- Shiften
65         Z3 <= Z2;  -- Shiften
66
67     end if;
68
69 end process P1_Synch;
```

VHDL (3/3)

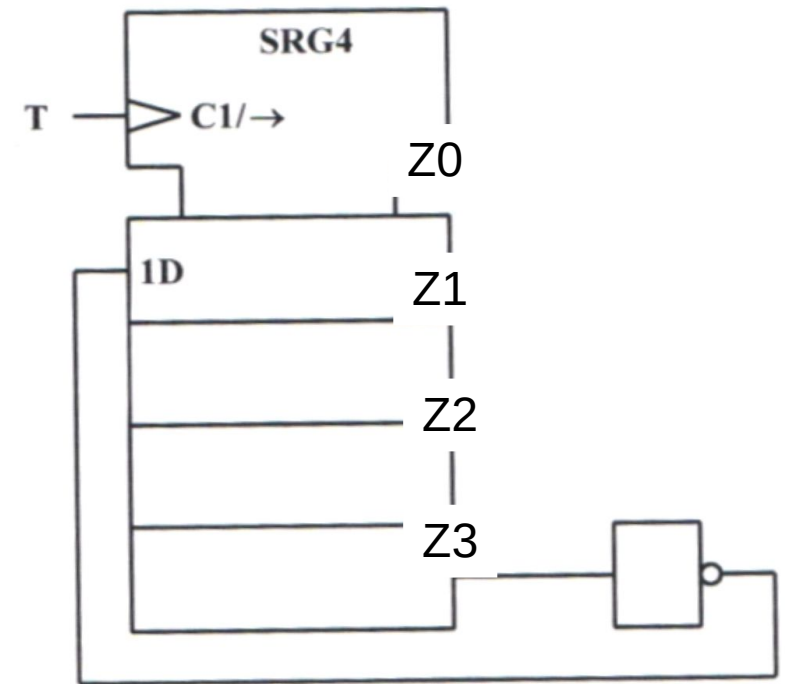
```
71  -- ASYNCHRONER Prozess -> kombinatorische Logik
72  P2_ASynch : process(Z0,Z1,Z2,Z3)
73
74      begin
75          -- FliFlop Inhalte auf den jeweiligen Ausgang legen
76          D0 <= Z0;
77          D1 <= Z1;
78          D2 <= Z2;
79          D3 <= Z3;
80
81          -- Alle Registerinhalte ODER verknuepfen und am Ende NEGIEREN
82          X <= NOT(Z0 AND Z1 AND Z2 AND Z3);
83
84
85      end process P2_ASynch;
86
87  end Behavioral;
```

Synthese



Johnson Zähler

- Beim Johnson-Zähler wird der invertierte Ausgang des in Schieberichtung letzten FF auf den seriellen Eingang zurückgeführt
- Johnson-Zähler werden häufig bei Frequenzmessern eingesetzt
- Andere Bezeichnungen
 - Möbius-Band
 - Pseudo-Ringzähler
- **Einzelübung:**
 - Erzeugen Sie bitte die Wahrheitstabelle
 - Erzeugen Sie bitte das Zustandsdiagramm
 - Schreiben Sie den VHDL Code



Wahrheitstabelle (1/2)

X	Z3	Z2	Z1	Z0	Dez

1	0	0	0	0	0
1	0	0	0	1	1
1	0	0	1	1	3
1	0	1	1	1	7
0	1	1	1	1	15
0	1	1	1	0	14
0	1	1	0	0	12
0	1	0	0	0	8
1	0	0	0	0	0
1	0	0	0	1	1
...					

X wird für die Dez. Berechnung
nicht mit benutzt

Wahrheitstabelle (2/2)

X	Z3	Z2	Z1	Z0	Dez
---	----	----	----	----	-----

1	0	0	1	0	2
---	---	---	---	---	---

1	0	1	0	1	5
---	---	---	---	---	---

0	1	0	1	1	11
---	---	---	---	---	----

1	0	1	1	0	6
---	---	---	---	---	---

0	1	1	0	1	13
---	---	---	---	---	----

0	1	0	1	0	10
---	---	---	---	---	----

1	0	1	0	0	4
---	---	---	---	---	---

0	1	0	0	1	9
---	---	---	---	---	---

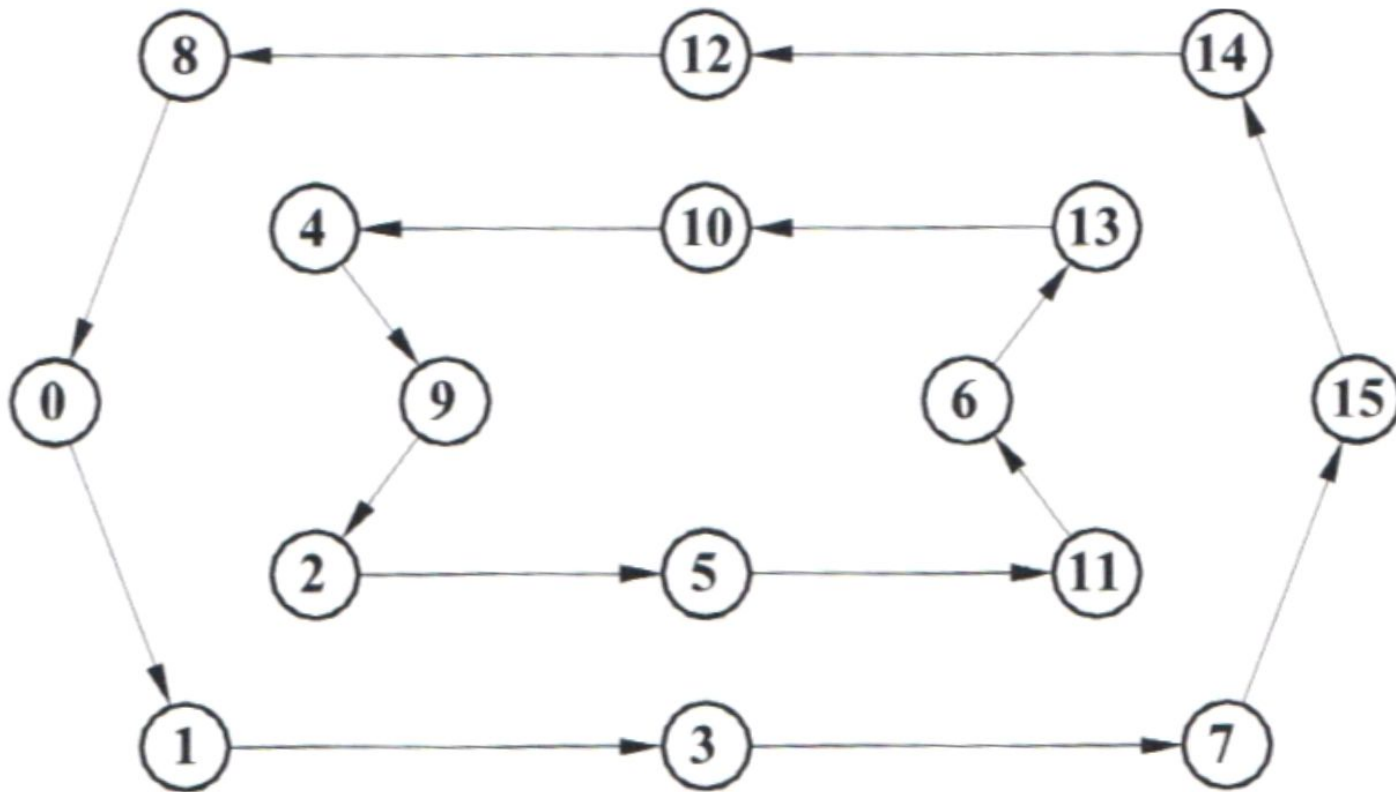
1	0	0	1	0	2
---	---	---	---	---	---

1	0	1	0	1	5
---	---	---	---	---	---

...

Lösung

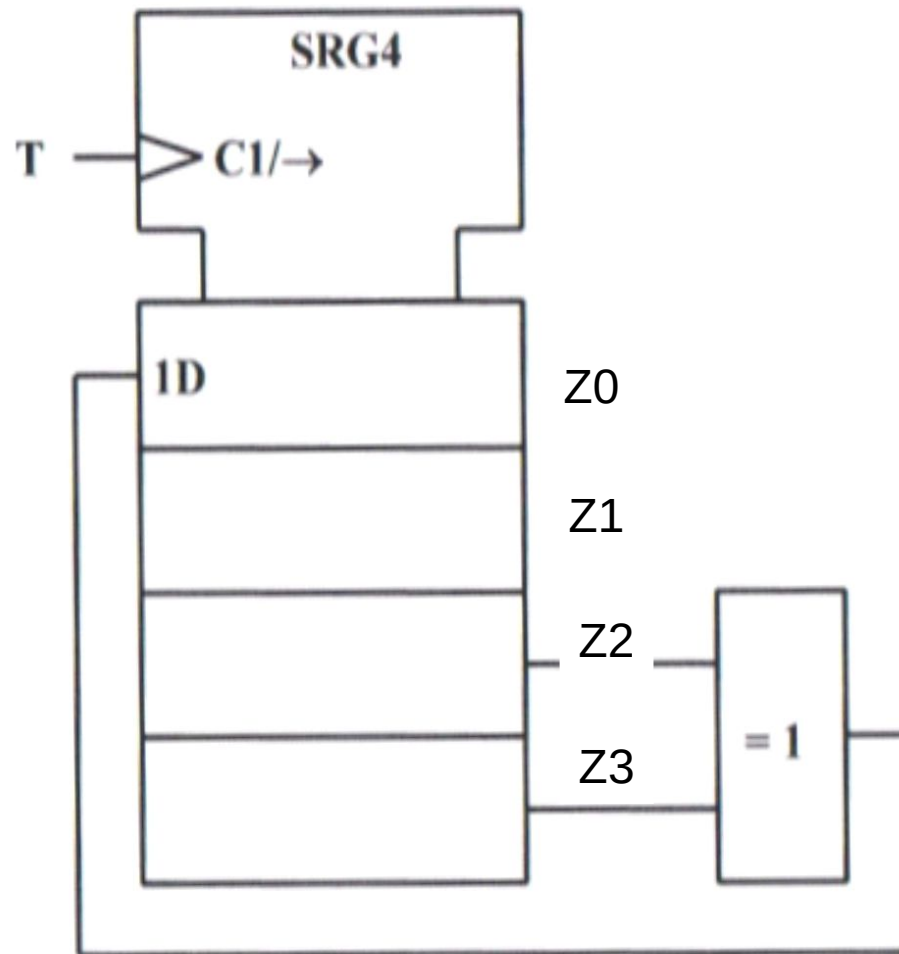
- Der Johnson-Zähler erzeugt also zwei Zyklen der Länge $2N$



Gruppenübung – Lineares Schieberegister

- Erzeugen Sie ein rückgekoppeltes Schieberegister mit 4 FF, wobei die letzten beiden FF EX-OR verknüpft sind und der EX-OR Ausgang auf das erste FF zurückgekoppelt ist
- **Erzeugen Sie bitte die Wahrheitstabelle**
- **Erzeugen Sie bitte das Zustandsdiagramm**
- **Schreiben Sie den VHDL Code**

Lösung



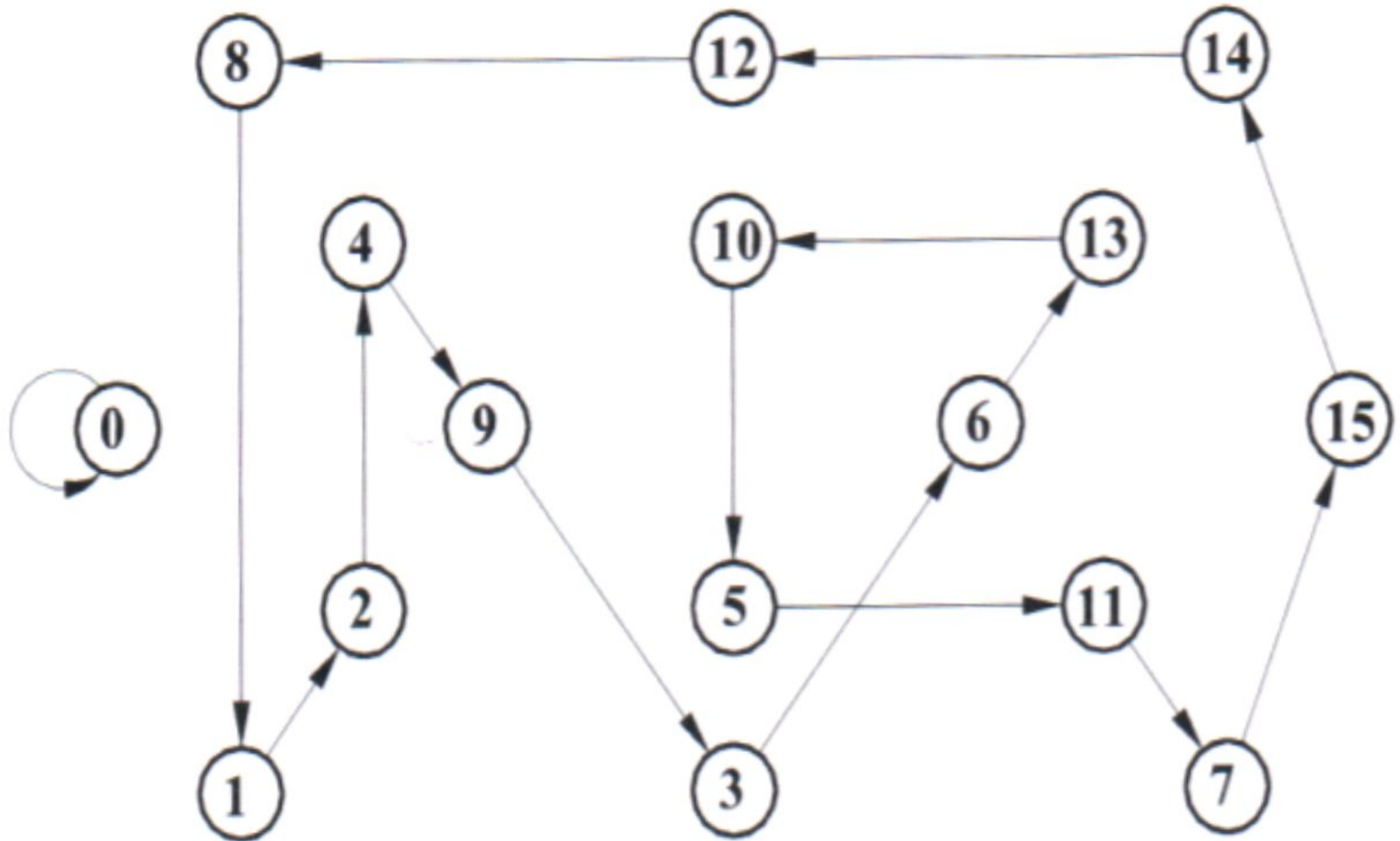
Wahrheitstabelle

X	Z3	Z2	Z1	Z0	Dez

0	0	0	0	0	0
0	0	0	0	1	1
0	0	0	1	0	2
1	0	1	0	0	4
1	1	0	0	1	9
0	0	0	1	1	3
1	0	1	1	0	6
0	1	1	0	1	13
1	1	0	1	0	10
1	0	1	0	1	5
1	1	0	1	1	11
1	0	1	1	1	7
0	1	1	1	1	15
0	1	1	1	0	14
0	1	1	0	0	12
1	1	0	0	0	8
0	0	0	0	1	1
0	0	0	1	0	2

...

Zustandsdiagramm

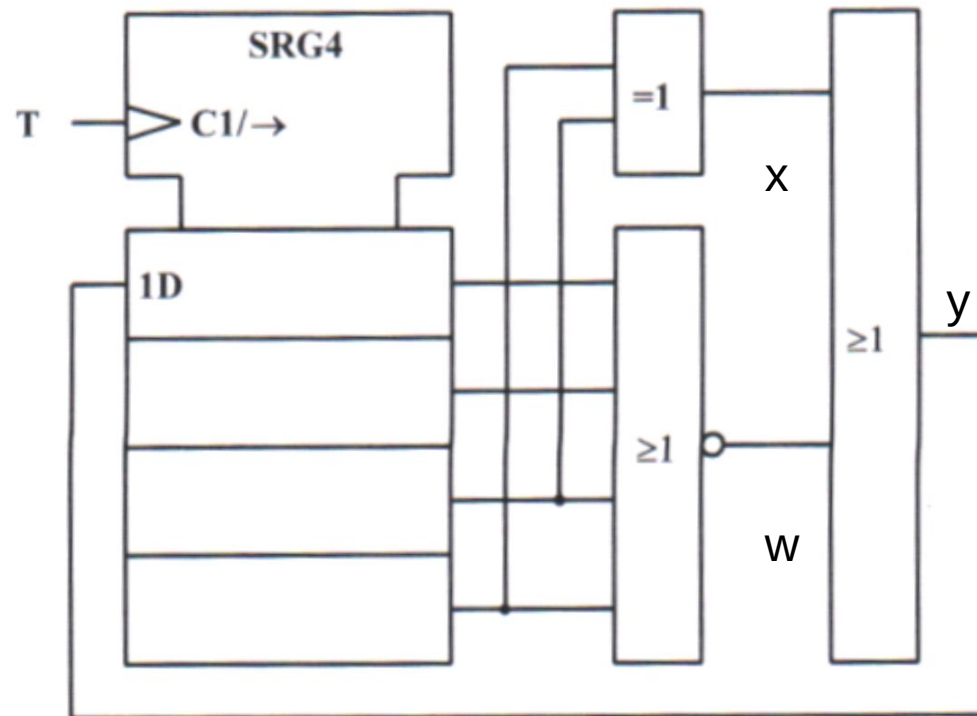


Lineares Schieberegister

- Es handelt sich um ein lineares Schieberegister, weil die EX-OR Funktion (Modulo-2 Addition) im Sinne der Algebra eine lineare Funktion ist
- Zyklenlänge $2^N - 1$
 - bei einem Schieberegister mit 20 FF sind das 1048575 Taktimpulse bis sich das Muster wiederholt
 - Auch bekannt als Pseudo-Zufallsregister

Korrigiertes Lineares Schieberegister

- EINZELÜBUNG: erklären Sie bitte die Funktionsweise des NOR Gatters
 - Erzeugen Sie bitte die Wahrheitstabelle
 - Schreiben Sie den VHDL Code



Z3 Z2 Z1 Z0 W X Y Dez

0	0	0	0	1	0	1	0
0	0	0	1	0	0	0	1
0	0	1	0	0	0	0	2
0	1	0	0	0	1	1	4
1	0	0	1	0	1	1	9
0	0	1	1	0	0	0	3
0	1	1	0	0	1	1	6
1	1	0	1	0	0	0	13
1	0	1	0	0	1	1	10
0	1	0	1	0	1	1	5
1	0	1	1	0	1	1	11
0	1	1	1	0	1	1	7
1	1	1	1	0	0	0	15
1	1	1	0	0	0	0	14
1	1	0	0	0	0	0	12
1	0	0	0	0	1	1	8
0	0	0	1	0	0	0	1
0	0	1	0	0	0	0	2

Wahrheitstabelle

Erklärung

- NOR: Im Zustand 0 eine 1 erzeugen → vom Zustand 0 in Zustand 1
 - NOR verändert nur im Zustand 0

VHDL(1/3)

```
20 library IEEE;
21 use IEEE.STD_LOGIC_1164.ALL;
22 use IEEE.STD_LOGIC_ARITH.ALL;
23 use IEEE.STD_LOGIC_UNSIGNED.ALL;
24
25 ---- Uncomment the following library declaration if instantiating
26 ---- any Xilinx primitives in this code.
27 --library UNISIM;
28 --use UNISIM.VComponents.all;
29
30 entity KorrLinReg is
31     Port ( Clk    : in  STD_LOGIC;
32           Reset  : in  STD_LOGIC;
33           D0     : out  STD_LOGIC;
34           D1     : out  STD_LOGIC;
35           D2     : out  STD_LOGIC;
36           D3     : out  STD_LOGIC);
37 end KorrLinReg;
```

VHDL(2/3)

```
39 architecture Behavioral of KorrlinReg is
40
41 signal Z0,Z1,Z2,Z3 : STD_LOGIC;
42 signal W,X,Y : STD_LOGIC;
43
44 begin
45
46 -- SYNCHRONER Prozess
47 P1_Synch: process(Clk,Reset)
48
49     begin
50
51         if Reset = '0' then
52
53             -- fest vorgegebener Startwert
54             Z0 <= '0';
55             Z1 <= '0';
56             Z2 <= '0';
57             Z3 <= '0';
58
59         elsif Clk'event AND Clk = '1' then
60
61
62             Z0 <= Y; -- Rueckkopplung von dem NOR Gatter
63             Z1 <= Z0; -- Shiften nach vorne
64             Z2 <= Z1; -- Shiften
65             Z3 <= Z2; -- Shiften
66
67         end if;
68
69     end process P1_Synch;
```

VHDL(3/3)

```
71  -- ASYNCHRONER Prozess -> kombinatorische Logik
72  P2_ASynch : process(Z0,Z1,Z2,Z3,W,X)
73
74      begin
75          -- Fliflop Inhalte auf den jeweiligen Ausgang legen
76          D0 <= Z0;
77          D1 <= Z1;
78          D2 <= Z2;
79          D3 <= Z3;
80
81          -- Alle Registerinhalte ODER verknuepfen und am Ende NEGIEREN
82          W <= NOT(Z0 OR Z1 OR Z2 OR Z3);
83
84          -- XOR Verknuepfung
85          X <= Z2 XOR Z3;
86
87          -- OR Verknuepfung von X und W
88          Y <= W OR X;
89
90      end process P2_ASynch;
91
92  end Behavioral;
```

Automaten

Einführung

- Automat ist ein sequentielles System
- Reaktion hängt vom
 - Aktuellen Zustand
 - Eingangsgrößeab
- **Gruppenübung**
 - Was ist der Unterschied zwischen einem
 - Klassischen Zähler
 - Automat

Lösung

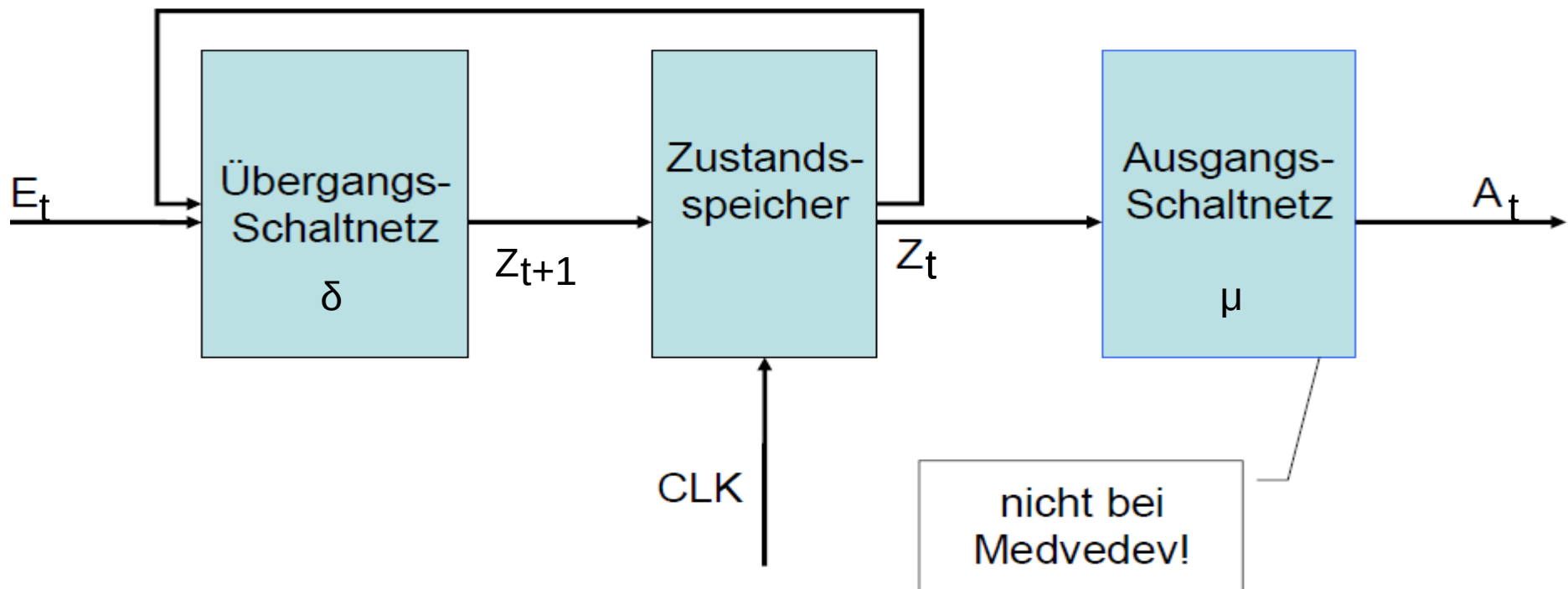
- Beim klassischen Zähler ist der nächste Zustand nur vom aktuellen Zustand abhängig
- Beim Automaten ist der nächste Zustand vom
 - Aktuellen Zustand
 - Eingangsgrößeabhängig
- Daher werden Zähler auch als *autonome Automaten* bezeichnet

Zwei Zustandsautomaten-Typen

- Allgemeine Automaten sind nur sehr schwer zu entwerfen
- Es haben sich in der Praxis zwei Automatentypen durchgesetzt
 - Moore
 - Mealy
 - Medvedev – Sonderfall vom Moore Automaten
- Kombinatorische Logik wird in zwei Blöcke aufgeteilt
 - Damit wird ein systematischer Entwurf möglich

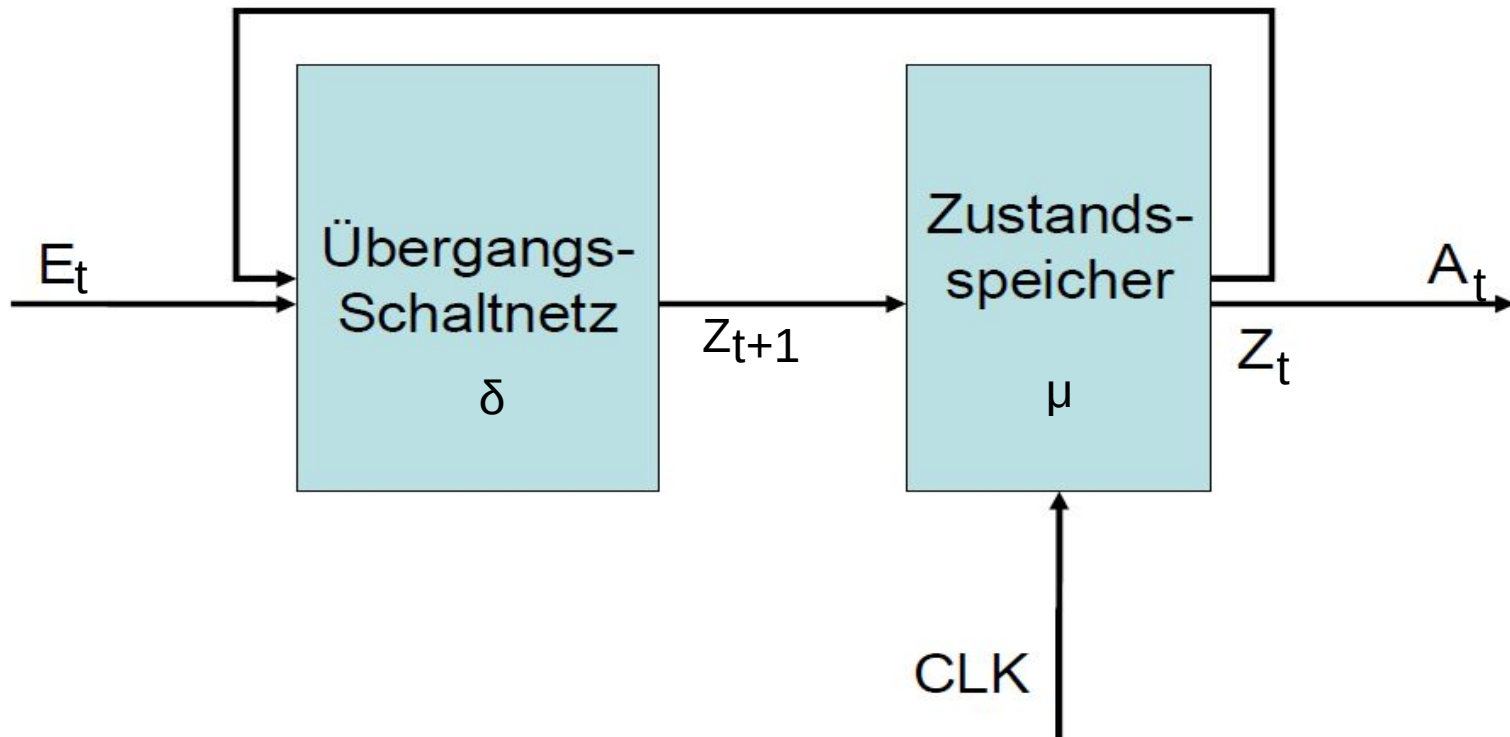
Moore Automat

- Kombinatorische Logik
 - Übergangsschaltnetz
 - Ausgangsschaltnetz
- Sequentielle Logik
 - Zustandsspeicher



Medvedev Automat

- Kein Ausgangsschaltnetz A vorhanden
- Vereinfachter Moore-Automat
 - Zustand Z ist gleich dem Ausgang A



Funktionsweise

- **Eingangsvektor $\mathbf{E} = \{E_0, E_1, \dots, E_{m-1}\}$** bezeichnet die Gesamtheit der Eingangsgrößen aus den Signalen $E_0, E_1, \dots, E_{m-1}$
 - Eingangsvektor $\mathbf{E}$ besteht also aus m -Bits
- Bezeichnung Vektor entspricht also einem Array
- **Zustandsvektor $\mathbf{Z} = \{Z_0, Z_1, Z_2, \dots, Z_{n-1}\}$** bezeichnet den inneren Zustand des Systems
 - N-Flip-Flops
- Zustände bleiben nach außen hin verborgen
 - nicht bei Medvedev $\rightarrow \mathbf{Z}$ ist der Ausgang
- **Ausgangsvektor $\mathbf{A} = \{A_0, A_1, A_2, \dots, A_{p-1}\}$** beschreibt die Ausgangsgrößen
 - Ausgangsvektor $\mathbf{P}$ besteht also aus p -Bits

Index t

- **Index t** beschreibt den aktuellen Zeitpunkt
- **Index t+1** bezeichnet die Werte, die einen Clock-Takt später anliegen
- Zustand Z_{t+1} beschreibt als den nächsten Zustand nach der aktiven Clock-Flanke

Zustandsübergangsfunktion

- Die Logik δ wird als
 - Zustandsübergangslogik
 - Steuerlogikbezeichnet
- δ legt die Abfolge der inneren Zustände fest

$$Z_{t+1} = \delta(Z_t, E_t)$$

- Der neue Zustand Z_{t+1} ist also abhängig von
 - Z_t - aktueller Zustand
 - E_t - aktuelle Eingangsgröße

Ausgangslogik

- Zweite kombinatorische Logik wird
 - Markierungsfunktion μ
 - Ausgangslogik

genannt

$$A_t = \mu(Z_t)$$

- Aktuelle Ausgangsgrößen sind also nur vom aktuellen Zustand abhängig
- Innerer Zustand Z kann also auch als Ausgang bezeichnet werden → Medvedev
- Damit reduziert sich der Entwurf eines Medvedev / Moore Automaten auf die Zustandsübergangslogik δ

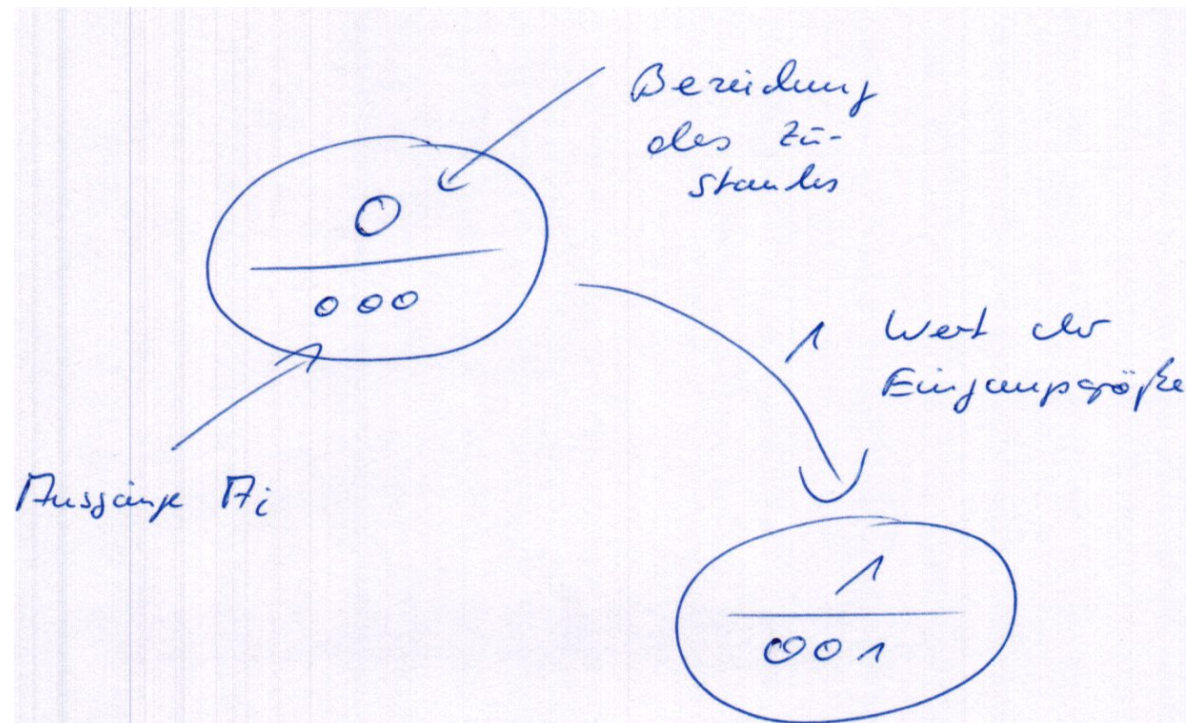
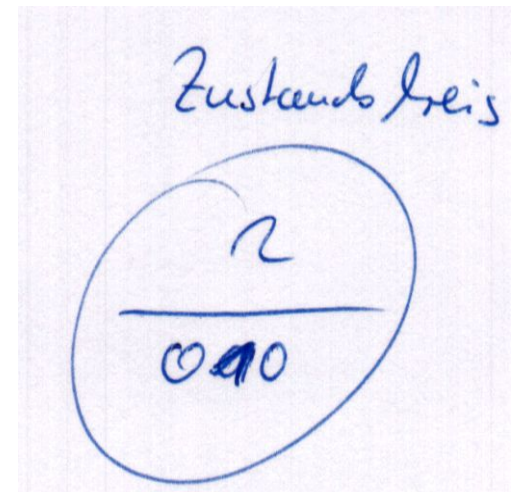
Zustandsfolgetabelle

- Zustandsfolgetabelle ist eine Wahrheitstabelle, die in einer Zeile folgende Daten aufträgt
 - Eingangsgröße E_t
 - Zustand Z_t
 - Folgezustand Z_{t+1} (nicht unbedingt wichtig für Wahrheitstabelle)
 - Flip-Flop Anregungsfunktionen D_i , J_i oder K_i

Eingänge		aktueller Zustand		Folgezustand		Anregungsfunktionen			
E_1	E_2	Z_1	Z_2	$Z_{1,t+1}$	$Z_{2,t+1}$	J_1	K_1	J_2	K_2
0	0	0	0	1	1	1	x	1	x
0	1	1	0	1	0	x	0	0	x
1	0	1	1	0	1	x	1	x	0

Zustandsdiagramm

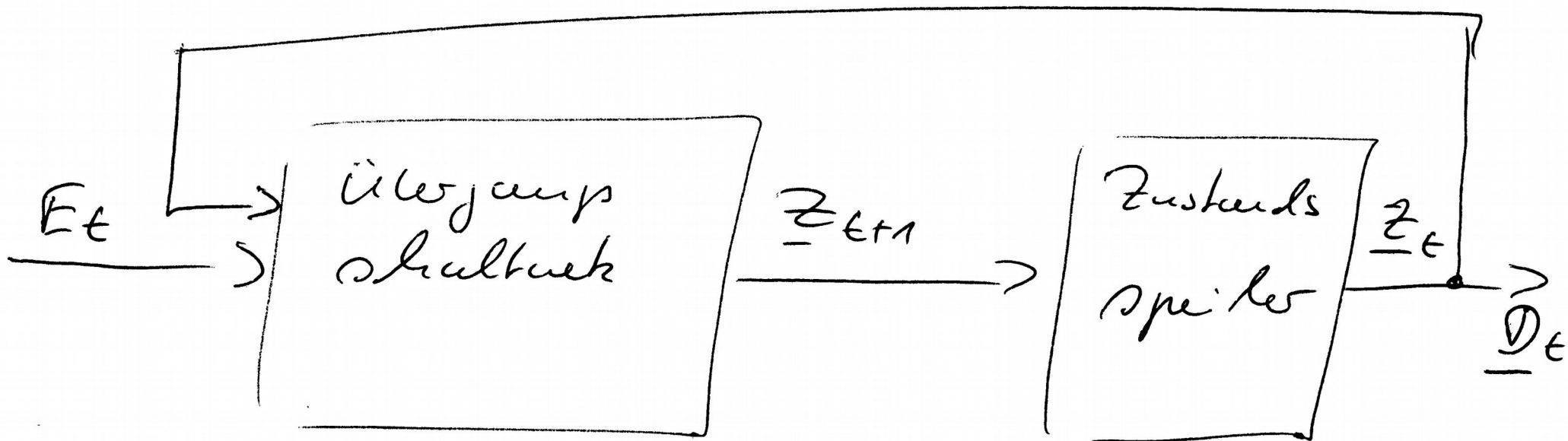
- Zustandskreis
- Oben: Bezeichnung des Zustandes
 - Willkürlicher Name
 - Dezimalzahl des Zählerwertes
- Unten: Ausgänge des Automaten
- Übergangspfeile
 - Werte der Eingangsgrößen E_t



Gruppenübung

- Ein 3-Bit Zähler soll
 - um 1 vorwärts zählen, wenn die Eingangsgröße $E = 0$ ist
 - um 2 vorwärts zählen, wenn die Eingangsgröße $E = 1$ ist
- Es werden D-Flip-Flops verwendet
 - $Z_2 = \text{MSB}$
 - $Z_0 = \text{LSB}$
- Entwerfen Sie
 - das Bild des Automaten
 - die Wahrheitstabelle
 - das Zustandsübergangsdiagramm
 - den VHDL Code

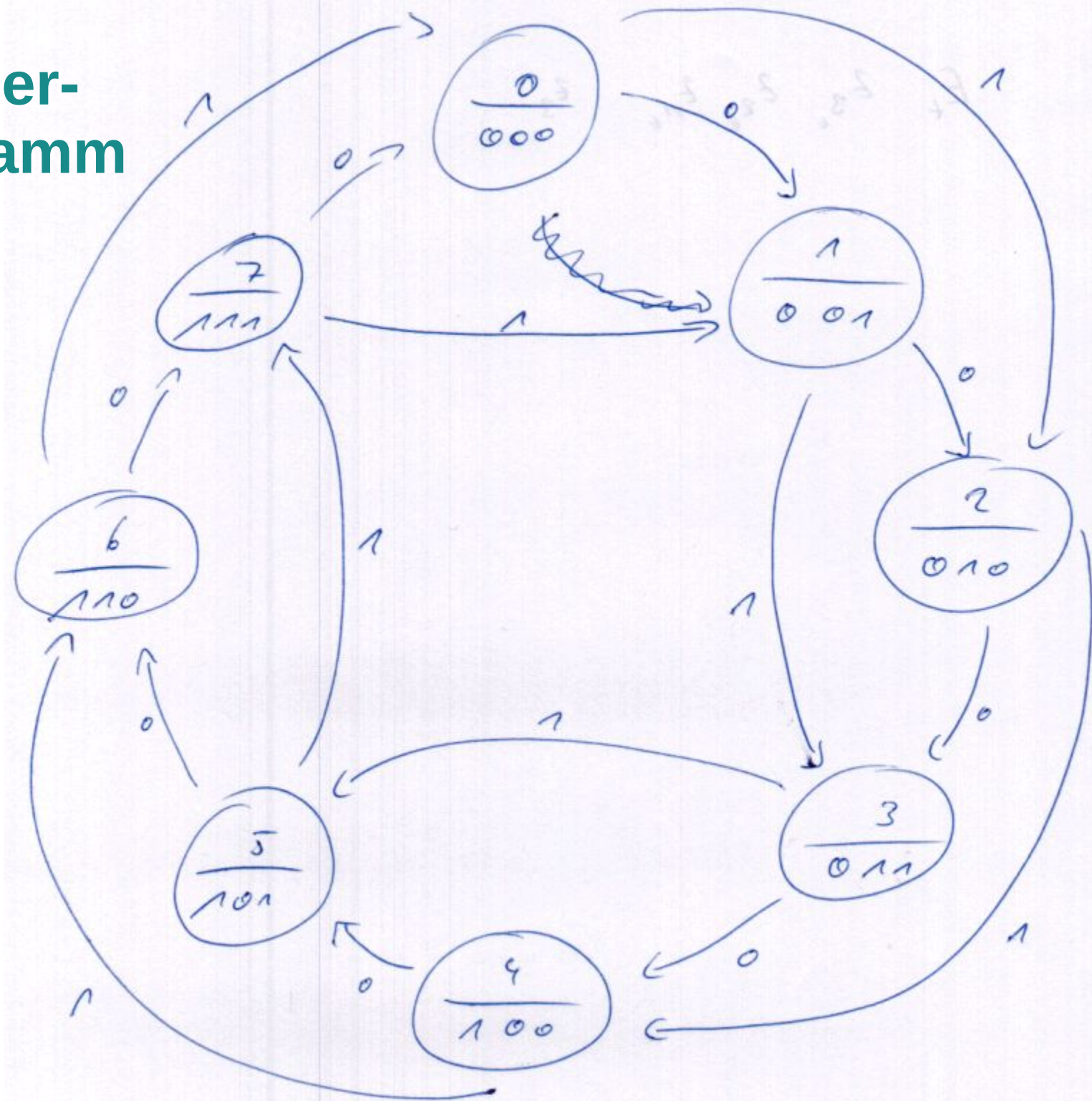
Automat



Wahrheitstabelle

Eingang	aktueller Zustand $\hat{= ①_t}$			Folge Zustand		
	z_{2t}	z_{1t}	z_{0t}	z_{2t+1}	z_{1t+1}	z_{0t+1}
0	0	0	0	0	0	1
0	0	0	1	0	1	0
0	0	1	0	0	1	1
0	0	1	1	1	0	0
0	1	0	0	1	0	1
0	1	0	1	1	1	0
0	1	1	0	1	1	1
0	1	1	1	0	0	0
1	0	0	0	0	1	0
1	0	0	1	0	1	1
1	0	1	0	1	0	0
1	0	1	1	1	0	1
1	1	0	0	1	1	0
1	1	0	1	1	1	1
1	1	1	0	0	0	0
1	1	1	1	0	0	1

Zustandsübergangsdiagramm



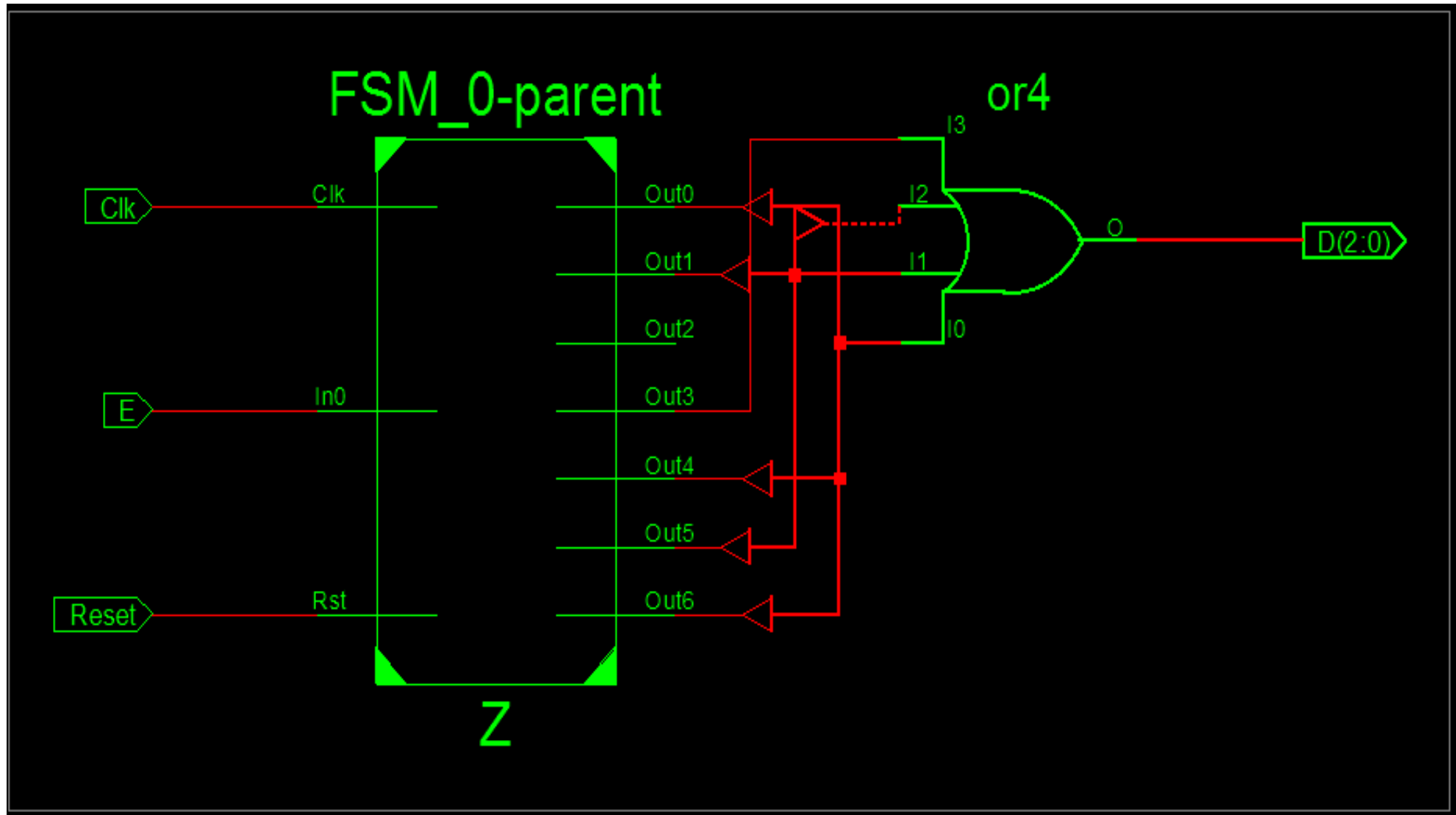
VHDL (1/2)

```
30 entity medvedev1 is
31     Port ( E      : in  STD_LOGIC;
32           Clk     : in  STD_LOGIC;
33           Reset   : in  STD_LOGIC;
34           D       : out STD_LOGIC_VECTOR (2 downto 0)
35           );
36 end medvedev1;
37
38 architecture Behavioral of medvedev1 is
39
40     signal Z : STD_LOGIC_VECTOR(2 downto 0) ;
41
42 begin
43
44     SM : process(Clk, Reset)
45
46     begin
47
48         if Reset = '0' then
49
50             Z <= "000";
51
52         elsif Clk'event and Clk = '1' then
53
54             if E = '0' then
55
56                 case Z is
57
58                     when "000" =>      Z <= "001" ;
59                     when "001" =>      Z <= "010" ;
60                     when "010" =>      Z <= "011" ;
61                     when "011" =>      Z <= "100" ;
62                     when "100" =>      Z <= "101" ;
63                     when "101" =>      Z <= "110" ;
64                     when "110" =>      Z <= "111" ;
65                     when others =>      Z <= "000" ;
66
67                 end case ;
```

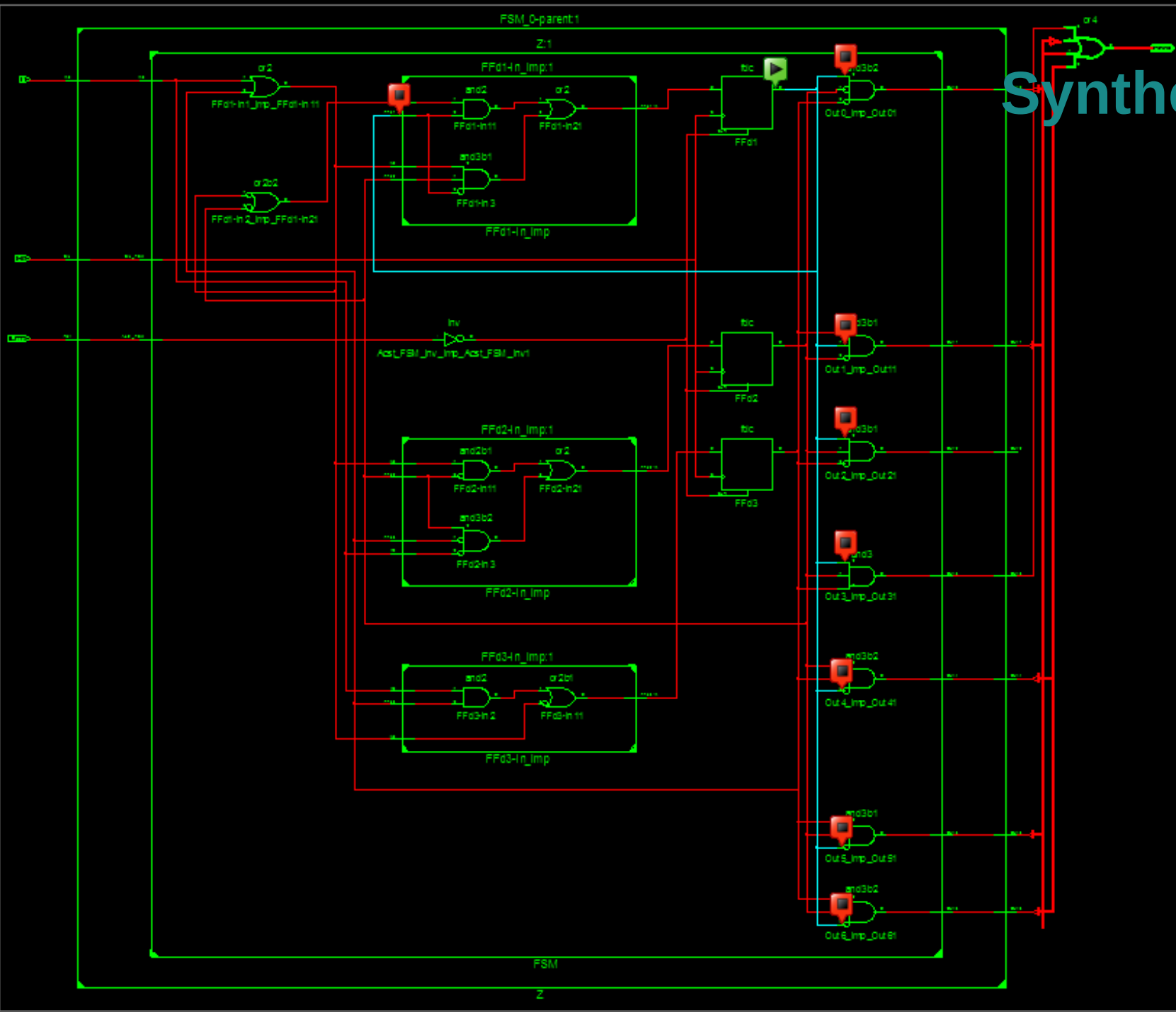
VHDL (2/2)

```
69         else
70
71             case Z is
72
73                 when "000" =>      Z <= "010" ;
74                 when "001" =>      Z <= "011" ;
75                 when "010" =>      Z <= "100" ;
76                 when "011" =>      Z <= "101" ;
77                 when "100" =>      Z <= "110" ;
78                 when "101" =>      Z <= "111" ;
79                 when "110" =>      Z <= "000" ;
80                 when others =>      Z <= "001" ;
81
82             end case ;
83
84         end if; -- E = '0'
85
86
87     end if; -- Reset
88
89
90 end process SM;|
91
92 -- link Z to output port D
93 D <= Z;
94
95 end Behavioral;
```


Synthese (1/2)



Synthese (2/2)



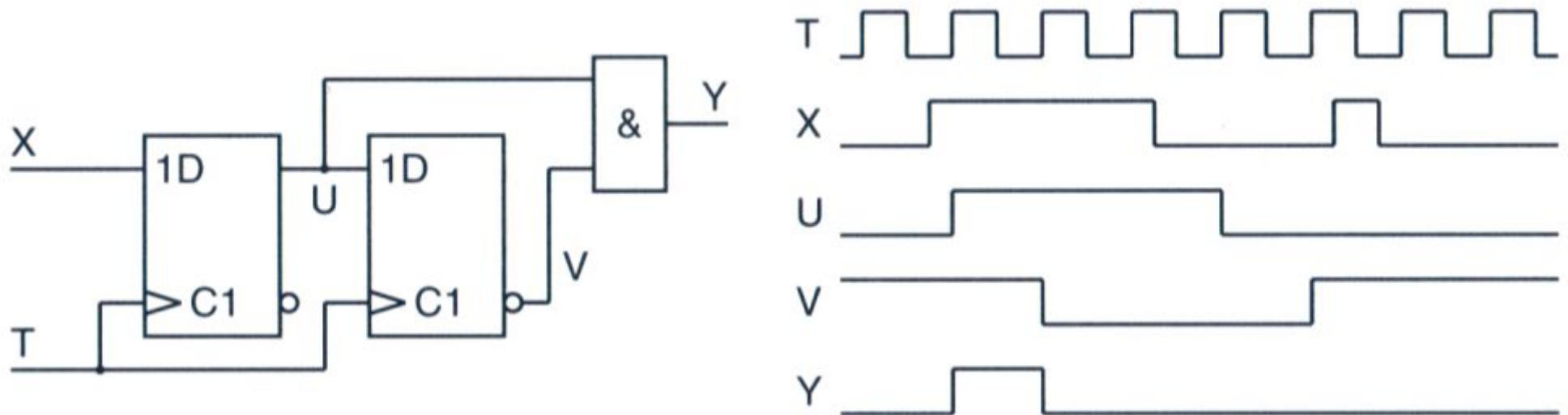
Synchronisierschaltung

Wozu Synchronisierschaltungen?

- Automaten bestehen aus synchron getakteten Flip Flops
- Eingangsgrößen können sich hingegen zu jedem Zeitpunkt ändern, da unsere Umwelt grundsätzlich asynchron ist
- Mögliche Beispiele
 - Einwurf von Münzen an einem Geldautomaten
 - Tastendruck beim Fahrkartenautomat
- Fällt der Wechsel einer Eingangsgröße zufällig ganz genau auf den Zeitpunkt der aktiven Flanke der Flip Flops, können unvorhergesehene zustände eintreten
- Lösung – die *internen Eingangsgrößen* des eigentlichen Automaten müssen stabil gehalten werden → Synchronisierschaltungen

Synchroner Monoflop

- Monostabiler Multivibrator
- Auf Eingangsimpuls beliebiger Länge wird mit einem Ausgangsimpuls definierter Länge reagiert
 - Ausgang eine Periode des Taktsignals



- **Gruppenübung:** Realisieren Sie die Synchronisierschaltung in VHDL

VHDL MonoFlop (1/2)

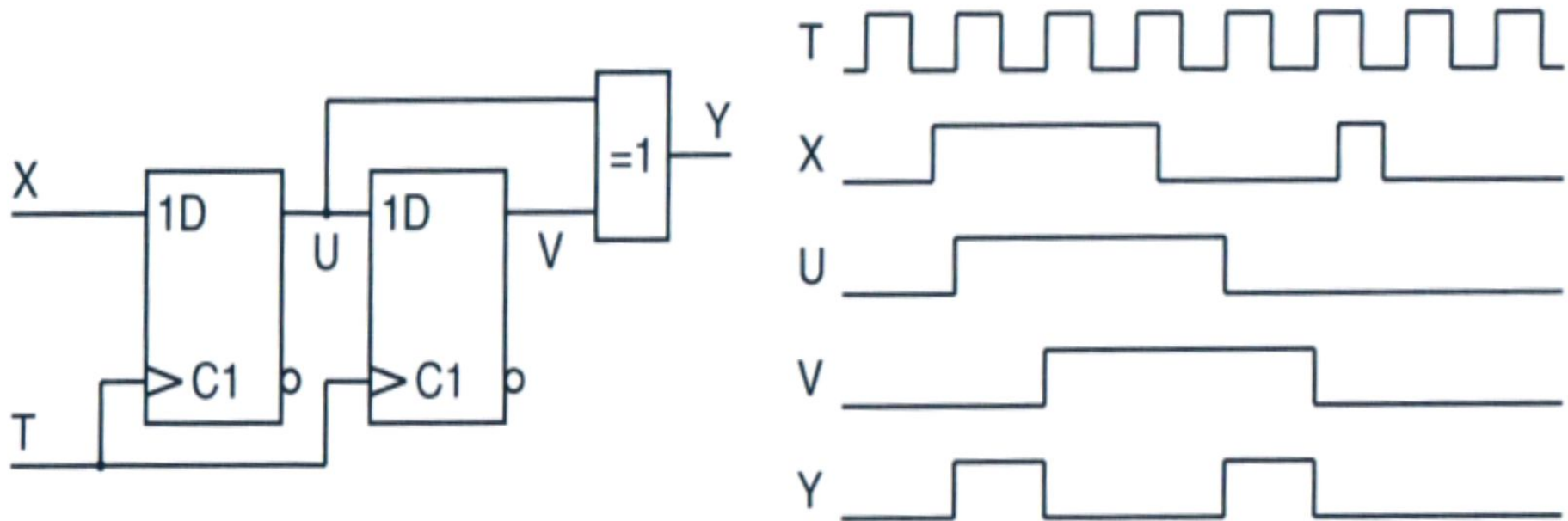
```
20 library IEEE;
21 use IEEE.STD_LOGIC_1164.ALL;
22 use IEEE.STD_LOGIC_ARITH.ALL;
23 use IEEE.STD_LOGIC_UNSIGNED.ALL;
24
25 ---- Uncomment the following library declaration if instantiating
26 ---- any Xilinx primitives in this code.
27 --library UNISIM;
28 --use UNISIM.VComponents.all;
29
30 entity SynchMonoFlopArch is
31     Port ( DataIn   : in  STD_LOGIC;
32           Clk       : in  STD_LOGIC;
33           Reset     : in  STD_LOGIC;
34           DataOut  : out  STD_LOGIC);
35 end SynchMonoFlopArch;
36
37 architecture Behavioral of SynchMonoFlopArch is
38
39     signal U : STD_LOGIC;
40     signal V : STD_LOGIC;
```

VHDL MonoFlop (2/2)

```
37 architecture Behavioral of SynchMonoFlopArch is
38
39     signal U : STD_LOGIC;
40     signal V : STD_LOGIC;
41
42     begin
43
44     P1 : process(Clk, Reset)
45
46         begin
47
48             if(Reset = '1') then
49
50                 U <= '0';
51                 V <= '0';
52
53             elsif(clk'event and Clk = '1') then
54
55                 -- first regsiter
56                 U <= DataIn;
57
58                 -- second register
59                 V <= not(U);
60
61             end if;
62
63         end process P1;
64
65         -- AND combination of both registers
66         DataOut <= U AND V;
67
68     end Behavioral;
```

Synchroner Änderungsdetektor

- Jede Änderung des Eingangssignals liefert einen synchronen Ausgangsimpuls der Länge einer Taktperiode



- Gruppenübung:** Realisieren Sie die Synchronisierschaltung in VHDL

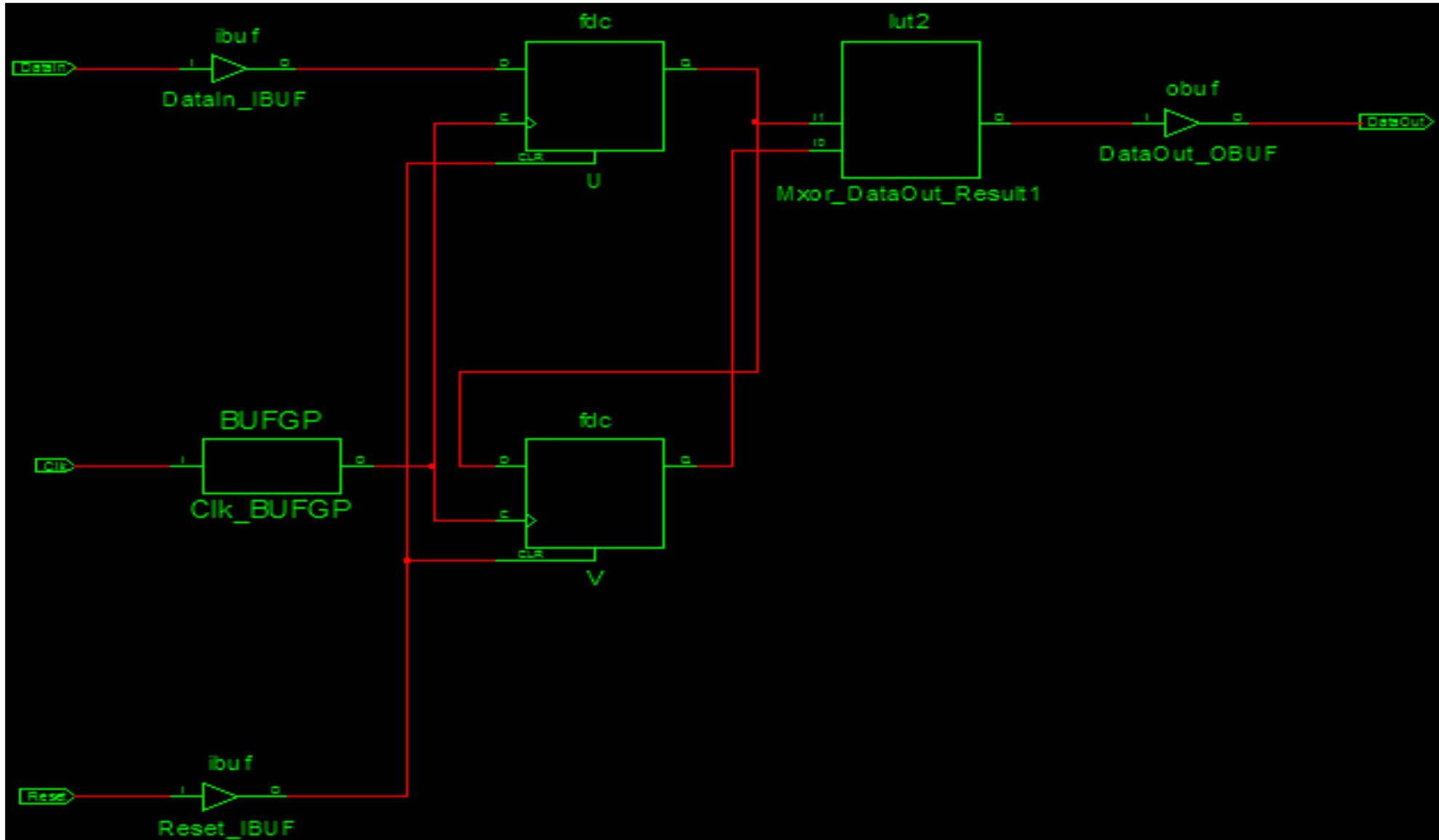
Synchroner Änderungsdetektor VHDL (1/2)

```
20 library IEEE;
21 use IEEE.STD_LOGIC_1164.ALL;
22 use IEEE.STD_LOGIC_ARITH.ALL;
23 use IEEE.STD_LOGIC_UNSIGNED.ALL;
24
25 ---- Uncomment the following library declaration if instantiating
26 ---- any Xilinx primitives in this code.
27 --library UNISIM;
28 --use UNISIM.VComponents.all;
29
30 entity SynchAendArch is
31     Port ( DataIn      : in   STD_LOGIC;
32           Clk          : in   STD_LOGIC;
33           Reset        : in   STD_LOGIC;
34           DataOut      : out  STD_LOGIC);
35 end SynchAendArch;
```

Synchroner Änderungsdetektor VHDL (2/2)

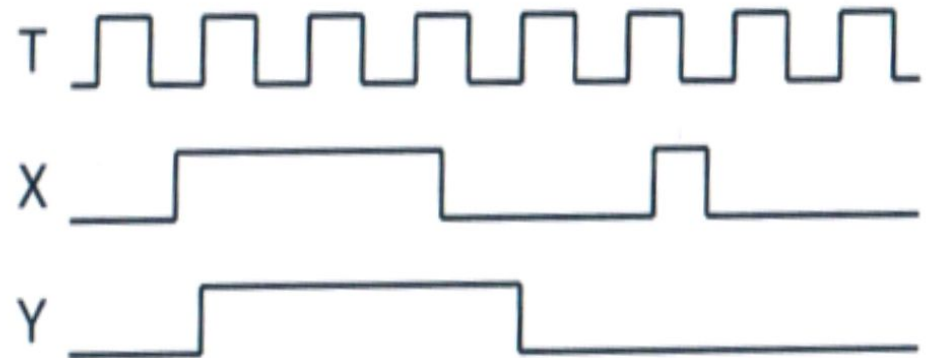
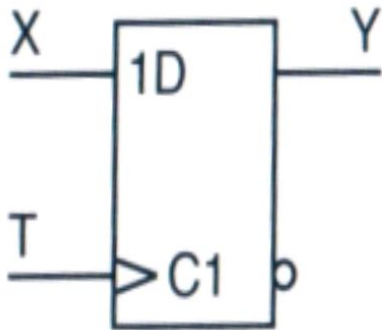
```
37 architecture Behavioral of SynchAendArch is
38
39     signal U : STD_LOGIC;
40     signal V : STD_LOGIC;
41
42     begin
43
44         P1 : process(Clk, Reset)
45
46             begin
47
48                 if(Reset = '1') then
49
50                     U <= '0';
51                     V <= '0';
52
53                 elsif (Clk'event and CLK = '1') then
54
55                     U <= DataIn;
56                     V <= U;
57
58                 end if;
59
60             end process P1;
61
62
63         DataOut <= (U XOR V);
64
65     end Behavioral;
```

Synchroner Änderungsdetektor



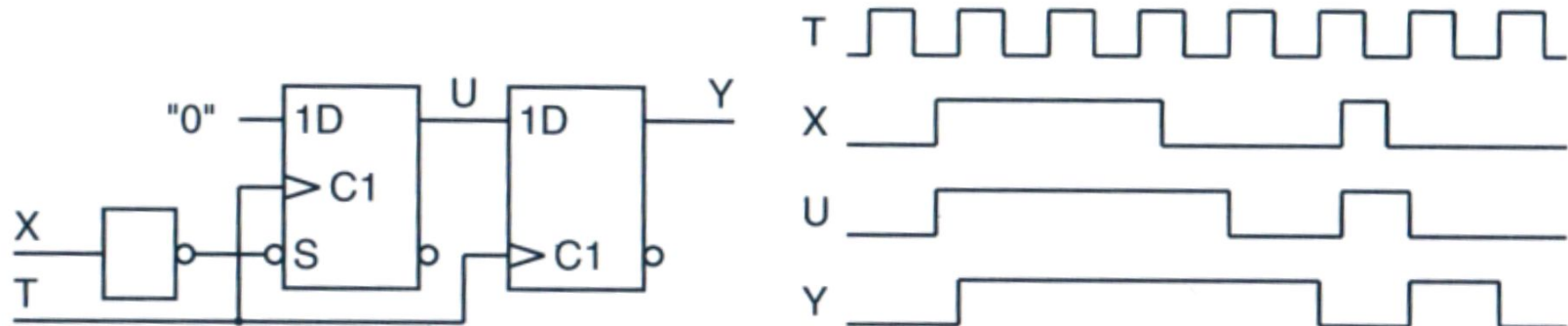
Impuls-Synchronisierung

- Kurzer Impuls auf der Eingangsleitung wird von keiner Taktflanke erfasst



Modifizierte Impuls-Synchronisierung

- Asynchroner Setzeingang im Flipflop
 - Geht sofort auf 1 falls Eingang auf 1
 - Geht bei der nächsten Taktflanke auf 0 falls Eingang auf 0



- **Gruppenübung:** Realisieren Sie die Synchronisierschaltung in VHDL

Entity

```
20 library IEEE;
21 use IEEE.STD_LOGIC_1164.ALL;
22 use IEEE.STD_LOGIC_ARITH.ALL;
23 use IEEE.STD_LOGIC_UNSIGNED.ALL;
24
25 ---- Uncomment the following library declaration if instantiating
26 ---- any Xilinx primitives in this code.
27 --library UNISIM;
28 --use UNISIM.VComponents.all;
29
30 entity synch_arch is
31     Port ( Clk      : in  STD_LOGIC;
32           Reset    : in  STD_LOGIC;
33           Set       : in  STD_LOGIC;
34           DataOut   : out STD_LOGIC);
35 end synch_arch;
36
37 architecture Behavioral of synch_arch is
38
39     signal U : STD_LOGIC;
40     signal Y : STD_LOGIC;
```

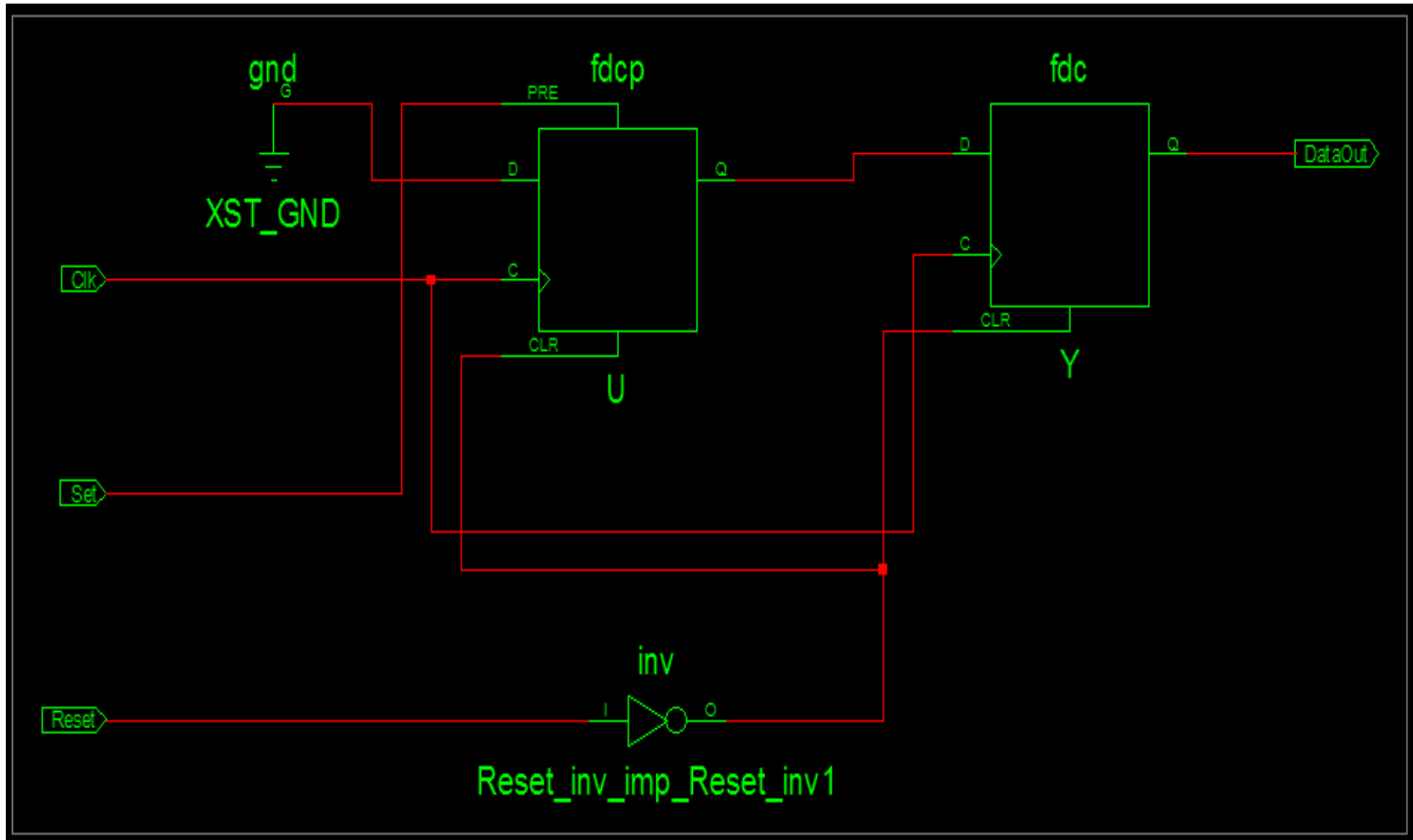
```

12 begin
43
44 SynchInProcess : process(Clk, Reset, Set)
45
46     begin
47
48         if Reset = '0' then
49
50             U <= '0';
51
52         elsif Set = '1' then
53
54             U <= '1' ;
55
56         elsif Clk'event and Clk = '1' then
57
58             U <= '0' ;
59
60         end if;
61
62     end process SynchInProcess;
63
64 SynchOutProcess : process(Clk, Reset)
65
66     begin
67
68         if Reset = '0' then
69
70             Y <= '0';
71
72         elsif Clk'event and Clk = '1' then
73
74             Y <= U ;
75
76         end if;
77     end process SynchOutProcess;
78
79 DataOut <= Y;
80
81 end Behavioral;

```

Architecture

Synthese



Synchronisierung

- Eingangsdaten des Automaten sollen mit der Synchronisationsschaltung synchronisiert werden
- **Gruppenübung:** Fügen Sie den Automaten und die Synchronisation des Eingangs aneinander

VHDL (1/5)

```
20 library IEEE;
21 use IEEE.STD_LOGIC_1164.ALL;
22 use IEEE.STD_LOGIC_ARITH.ALL;
23 use IEEE.STD_LOGIC_UNSIGNED.ALL;
24
25 ---- Uncomment the following library declaration if instantiating
26 ---- any Xilinx primitives in this code.
27 --library UNISIM;
28 --use UNISIM.VComponents.all;
29
30 entity autosynch is
31     Port ( E      : in  STD_LOGIC;
32           Clk     : in  STD_LOGIC;
33           Reset   : in  STD_LOGIC;
34           DataOut : out  STD_LOGIC_VECTOR (2 downto 0));
35 end autosynch;
```

VHDL (2/5)

```
37 architecture Behavioral of autosynch is
38
39     signal U : STD_LOGIC;
40     signal Y : STD_LOGIC;
41
42     signal Z : STD_LOGIC_VECTOR (2 downto 0);
43
44 begin -- architecture
45
46     PreSynch : process(Clk, Reset, E)
47
48     begin
49
50         if Reset = '0' then
51
52             U <= '0';
53
54         elsif E = '1' then
55
56             U <= '1';
57
58         elsif Clk'event AND Clk = '1' then
59
60             U <= E;
61
62         end if;
63
64     end process PreSynch;
```

VHDL (3/5)

```
67   FinalSynch : process(Clk, Reset)
68
69   begin
70
71       if Reset = '0' then
72
73           Y <= '0';
74
75       elsif Clk'event and Clk = '1' then
76
77           Y <= U;
78
79       end if;
80
81   end process FinalSynch;
```

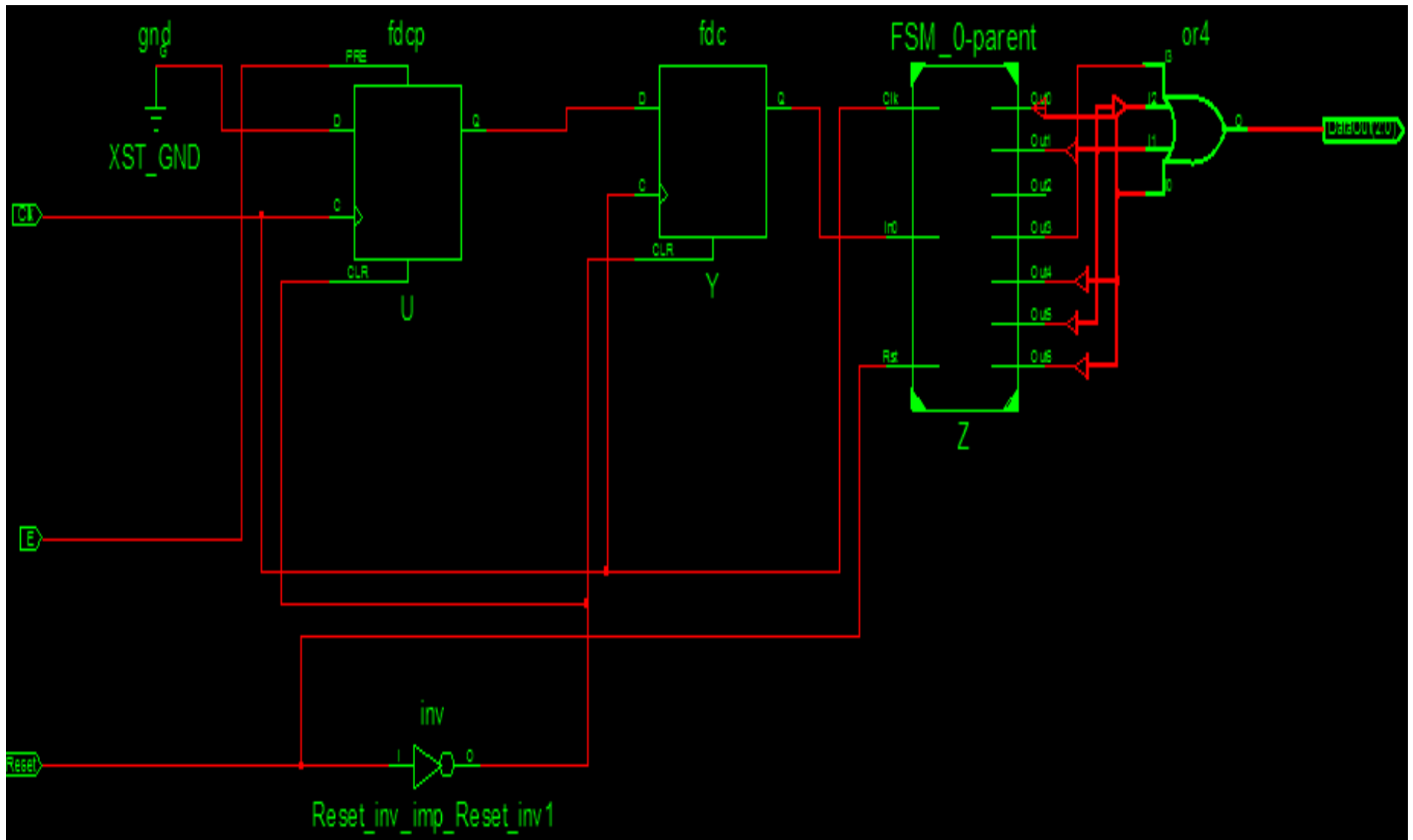
VHDL (4/5)

```
84     SM : process(Clk, Reset)
85
86     begin
87
88         if Reset = '0' then
89
90             Z <= "000";
91
92         elsif Clk'event and Clk = '1' then
93
94             if Y = '0' then
95
96                 case Z is
97
98                     when "000" => Z <= "001" ;
99                     when "001" => Z <= "010" ;
100                    when "010" => Z <= "011" ;
101                    when "011" => Z <= "100" ;
102                    when "100" => Z <= "101" ;
103                    when "101" => Z <= "110" ;
104                    when "110" => Z <= "111" ;
105                    when others => Z <= "000" ;
106
107                 end case ;
108
```

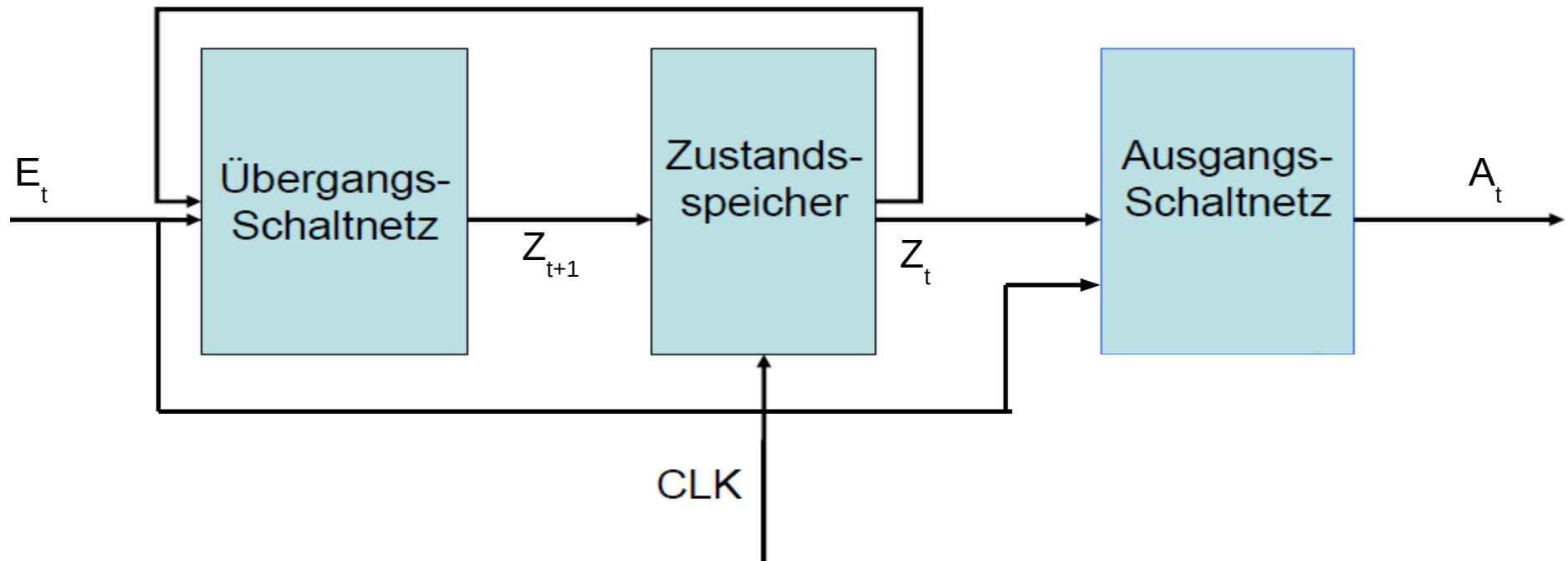
VHDL(5/5)

```
108
109         else
110
111             case Z is
112
113                 when "000" =>      Z <= "010" ;
114                 when "001" =>      Z <= "011" ;
115                 when "010" =>      Z <= "100" ;
116                 when "011" =>      Z <= "101" ;
117                 when "100" =>      Z <= "110" ;
118                 when "101" =>      Z <= "111" ;
119                 when "110" =>      Z <= "000" ;
120                 when others =>      Z <= "001" ;
121
122             end case ;
123
124         end if; -- E = '0'
125
126
127     end if; -- Reset
128
129
130 end process SM;
131
132
133 -- link Z to output port D
134 DataOut <= Z;
135
136 end Behavioral;
```

Synthese



Mealy Automat



Unterschied Mealy und Moore

- Moore
 - Ausgangszustand A ist vom inneren Zustand Z abhängig
- Mealy - Ausgangszustand A ist vom
 - inneren Zustand ZUND
 - Eingangsvektor Eabhängig

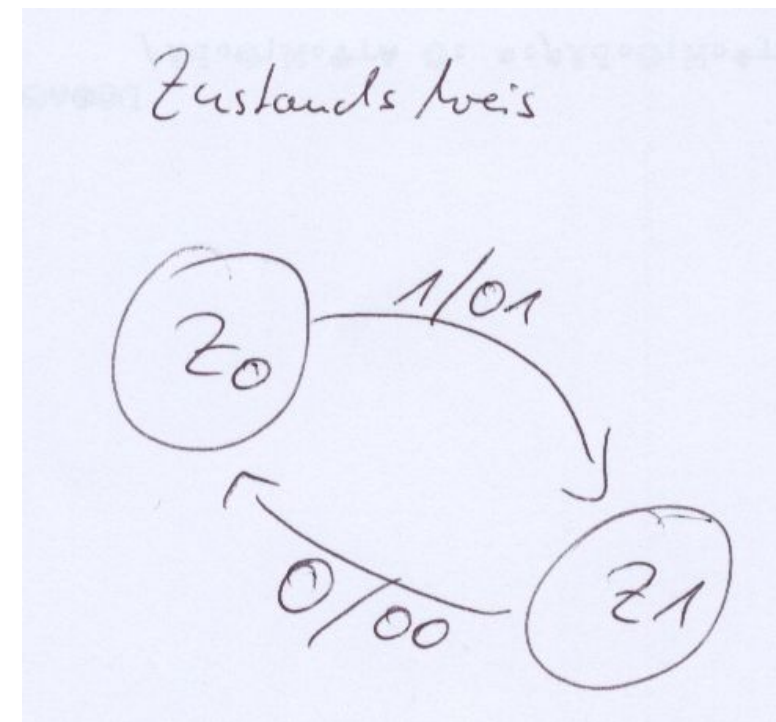
$$A_t = \omega(E_t, Z_t)$$

Zustandsdiagramm

- Die Kreise enthalten den Namen des Zustandes
- Pfeile enthalten Eingänge und Ausgänge des Automaten
 - Dies deshalb, weil die Ausgänge
 - sowohl vom aktuellen Zustand (Ausgangspunkt des Pfeiles)
 - als auch von den Eingangsgrößen

abhängig sind

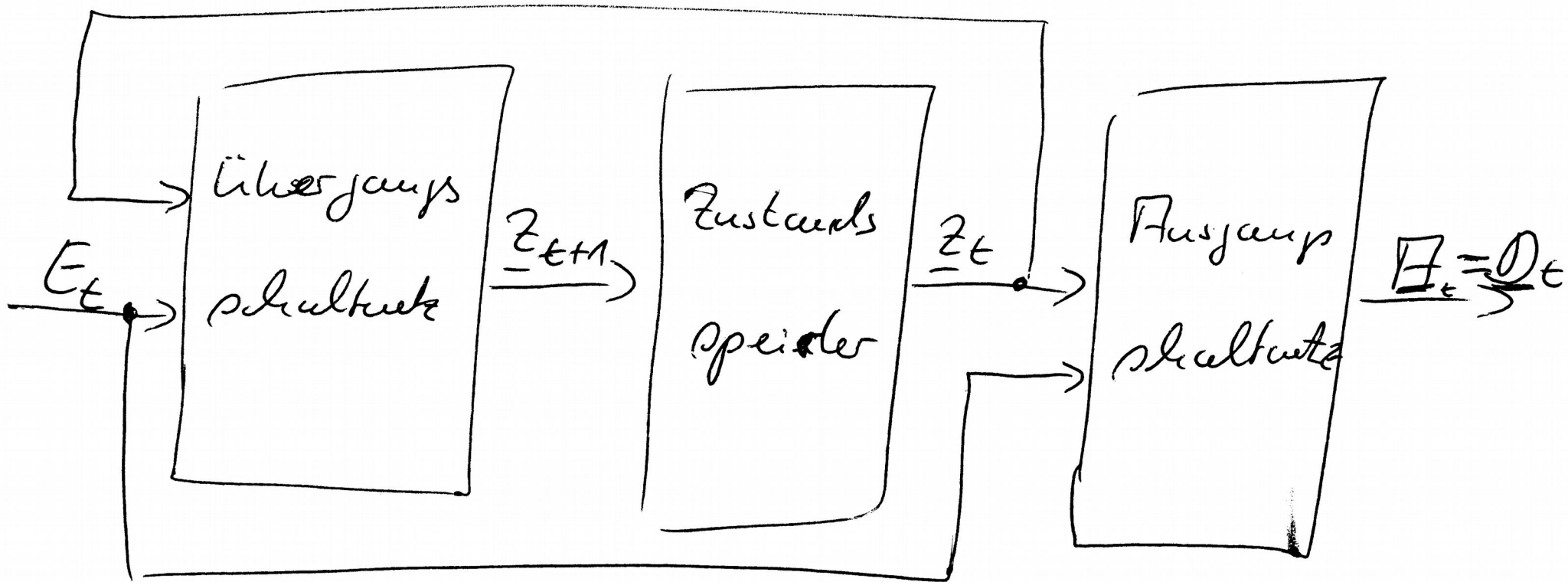
- Die Angabe erfolgt: EINGABE / AUSGABE
- Beim Übergang in den neuen Zustand Z_1 ändert sich auch die Ausgabe Z_t indem Z_t den Wert von Z_{t+1} übernimmt



Einzelübung(1/2) - Mealy

- Es soll ein Mealy Automat entworfen werden, der eine 1 oder eine 0 einliest, diesen Wert aber um einen Takt verzögert ausgibt
- Entwerfen Sie
 - das Bild des Automaten
 - die Wahrheitstabelle
 - das Zustandsübergangsdiagramm
 - den VHDL Code
- Tip: Versuchen Sie evtl. erst das Zustandsübergangsdiagramm und dann die Wahrheitstabelle zu erstellen.

Mealy - Automat



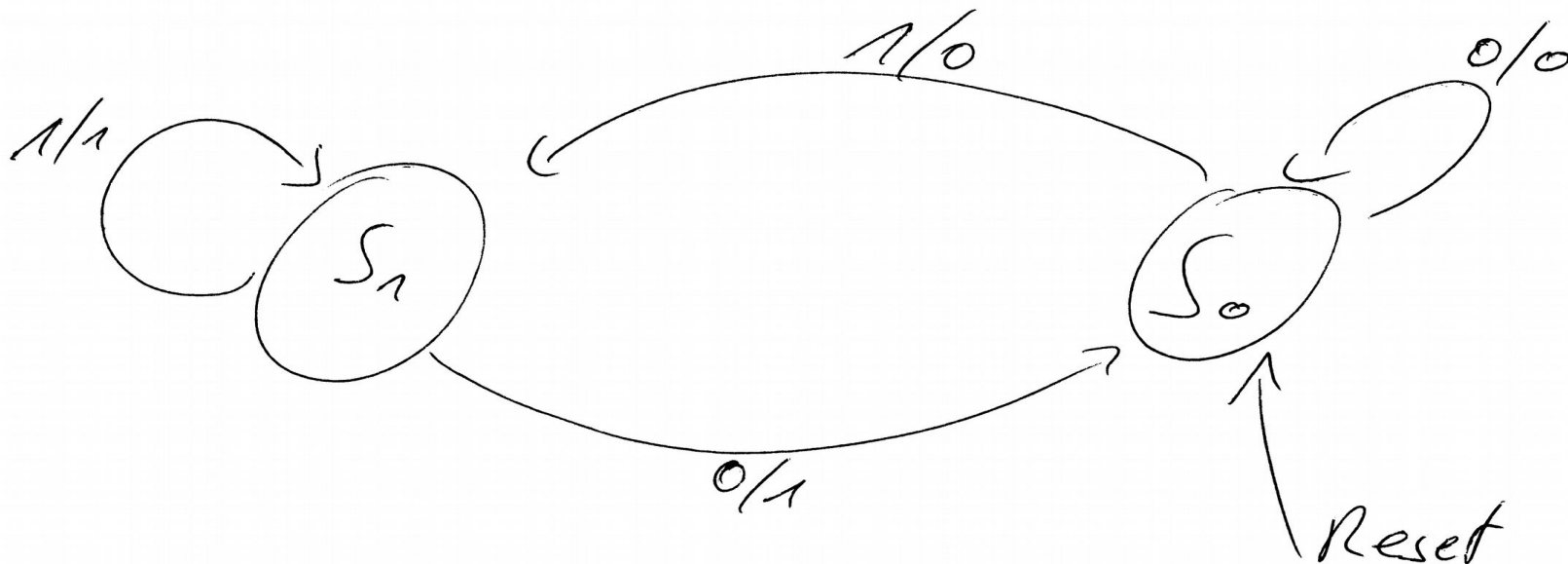
Mealy - Wahrheitstabelle

- $Z0_t$ ist der aktuelle Ausgang, der das Bit vom Takt zuvor ausgibt
- $Z0_{t+1}$ ist der zukünftige Ausgang, der das aktuelle Eingangbit E speichert und dann an $Z0_t$ übergibt

E	$Z0_t$	$Z0_{t+1}$	A
0	0	0	0
0	1	0	1
1	0	1	0
1	1	1	1

Mealy - Zustandsdiagramm

- Es werden nur 2 Zustände benötigt
 - Der innere Zustand S_0 repräsentiert die Eingabe von 0, unabhängig von der korrespondierenden Ausgabe
 - Der innere Zustand S_1 repräsentiert die Eingabe 1
- Die jeweiligen Ausgaben hängen nicht nur vom inneren Zustand ab, sondern auch vom Eingang



Mealy – VHDL Code (1/3)

```
20 library IEEE;
21 use IEEE.STD_LOGIC_1164.ALL;
22 use IEEE.STD_LOGIC_ARITH.ALL;
23 use IEEE.STD_LOGIC_UNSIGNED.ALL;
24
25 ---- Uncomment the following library declaration if instantiating
26 ---- any Xilinx primitives in this code.
27 --library UNISIM;
28 --use UNISIM.VComponents.all;
29
30 entity MealyDelay is
31     Port ( E      : in  STD_LOGIC;
32           Clk     : in  STD_LOGIC;
33           Reset   : in  STD_LOGIC;
34           A       : out STD_LOGIC);
35 end MealyDelay;
36
37 architecture Behavioral of MealyDelay is
38
39     signal Z0_t   : std_logic;
40     signal Z0_t1  : std_logic;
41
42     begin
```

Mealy – VHDL Code (2/3)

```
-- Asynchrones Uebergangsschaltnetz
Uebergangsschaltnetz : process(ZO_t, E)

begin

    if E = '0' AND ZO_t = '0' then

        ZO_t1 <= '0';

    elsif E = '0' AND ZO_t = '1' then

        ZO_t1 <= '0';

    elsif E = '1' AND ZO_t = '0' then

        ZO_t1 <= '1';

    else

        ZO_t1 <= '1';

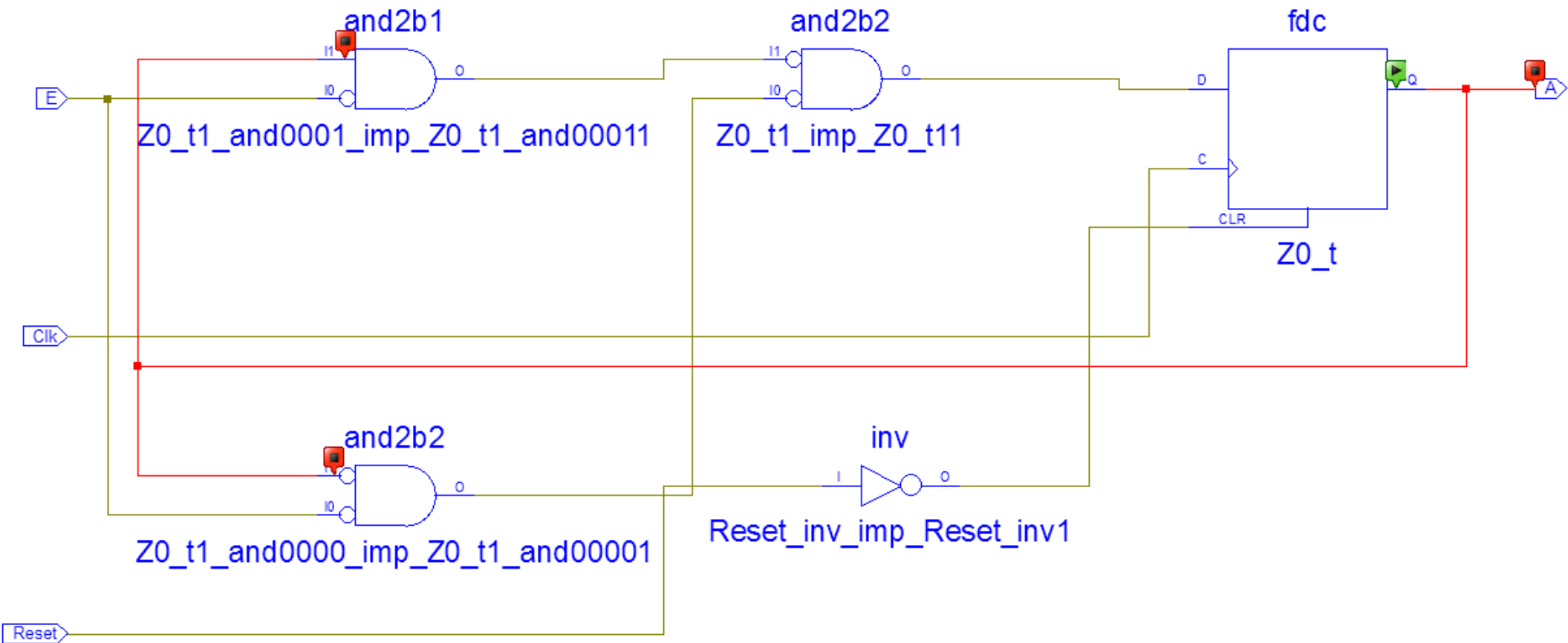
    end if;

end process Uebergangsschaltnetz ;
```


Mealy – VHDL Code (3/3)

```
68
69
70  -- Synchroner Zustandsspeicher
71  ZustandsSpeicher : process (Clk,Reset)
72
73      begin
74
75          if Reset = '0' then
76
77              Z0_t <= '0';
78
79          elsif Clk'event AND Clk = '1' then
80
81              Z0_t <= Z0_t1;
82
83          end if;
84
85      end process ZustandsSpeicher;
86
87
88  -- Asynchrones Ausgangsschaltnetz
89  -- hängt nur vom inneren Zustand Z0_t ab
90  AusgangsSchaltnetz : process (Z0_t)
91
92      begin
93
94          A <= Z0_t;
95
96      end process AusgangsSchaltnetz;
97
98
99  end Behavioral;
```

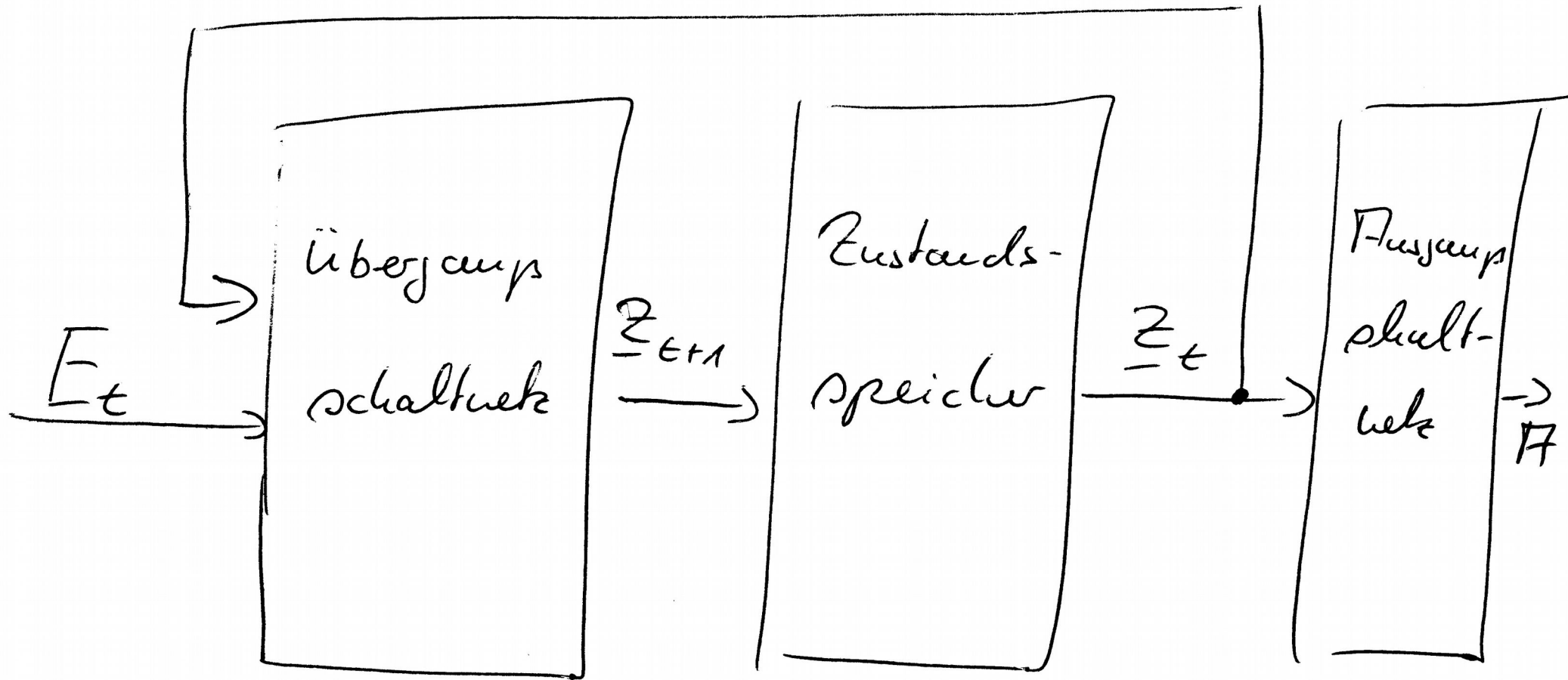
Mealy – Synthese



Einzelübung(2/2) - Moore

- Nun soll das gleiche System mittels Moore Automat entworfen werden
- Entwerfen Sie
 - das Bild des Automaten
 - die Wahrheitstabelle
 - das Zustandsübergangsdiagramm
 - den VHDL Code
- Tipp: Versuchen Sie evtl. erst das Zustandsübergangsdiagramm und dann die Wahrheitstabelle zu erstellen.

Moore - Automat



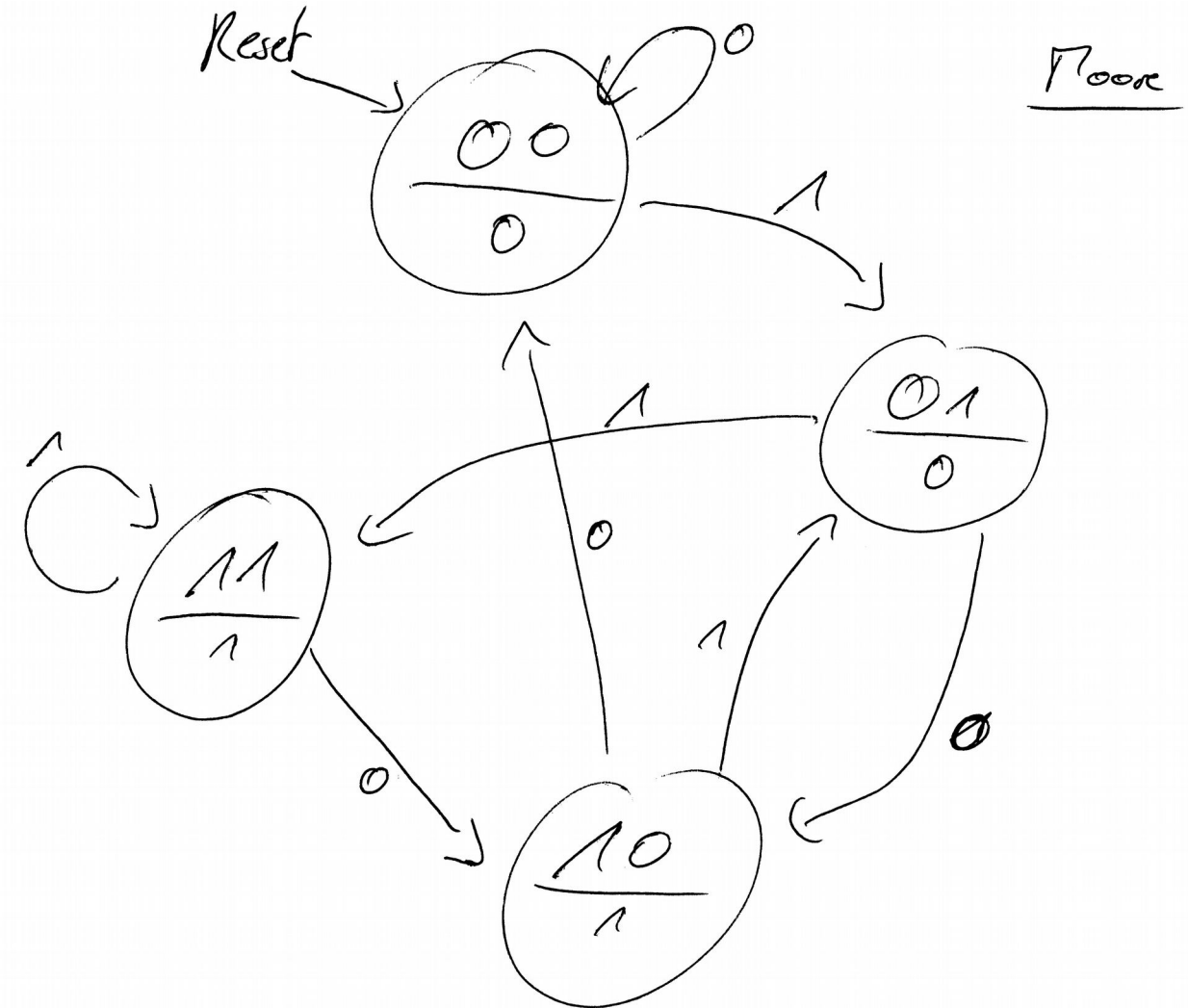
Moore - Wahrheitstabelle

- $Z1_t$ und $Z0_t$ bilden den aktuellen Zustand
- $Z0_t$ korrespondiert mit dem Bit der Ausgabe
- $Z1_{t+1}$ und $Z0_{t+1}$ bilden den nächsten Zustand
- Es gibt offensichtlich 4 Zustände

E	$Z1_t$	$Z0_t$	$Z1_{t+1}$	$Z0_{t+1}$	A
0	0	0	0	0	0
0	0	1	1	0	1
0	1	0	0	0	0
0	1	1	1	0	1
1	0	0	0	1	0
1	0	1	1	1	1
1	1	0	0	1	0
1	1	1	1	1	1

Moore - Zustandsdiagramm

- Es werden 4 Zustände benötigt
- 2 Zustände ergeben jeweils das gleiche Ausgangsbit aber speichern ein anderes Eingangsbit
- Die jeweiligen Ausgaben hängen nur vom inneren Zustand ab



Moore – VHDL Code (1/3)

```
20 library IEEE;
21 use IEEE.STD_LOGIC_1164.ALL;
22 use IEEE.STD_LOGIC_ARITH.ALL;
23 use IEEE.STD_LOGIC_UNSIGNED.ALL;
24
25 ---- Uncomment the following library declaration if instantiating
26 ---- any Xilinx primitives in this code.
27 --library UNISIM;
28 --use UNISIM.VComponents.all;
29
30 entity MooreDelay is
31     Port ( E      : in  STD_LOGIC;
32           Clk     : in  STD_LOGIC;
33           Reset   : in  STD_LOGIC;
34           A       : out STD_LOGIC);
35 end MooreDelay;
36
37 architecture Behavioral of MooreDelay is
38
39     signal Z_t   : std_logic_vector(1 downto 0);
40     signal Z_t1  : std_logic_vector(1 downto 0);
41
42     begin
```

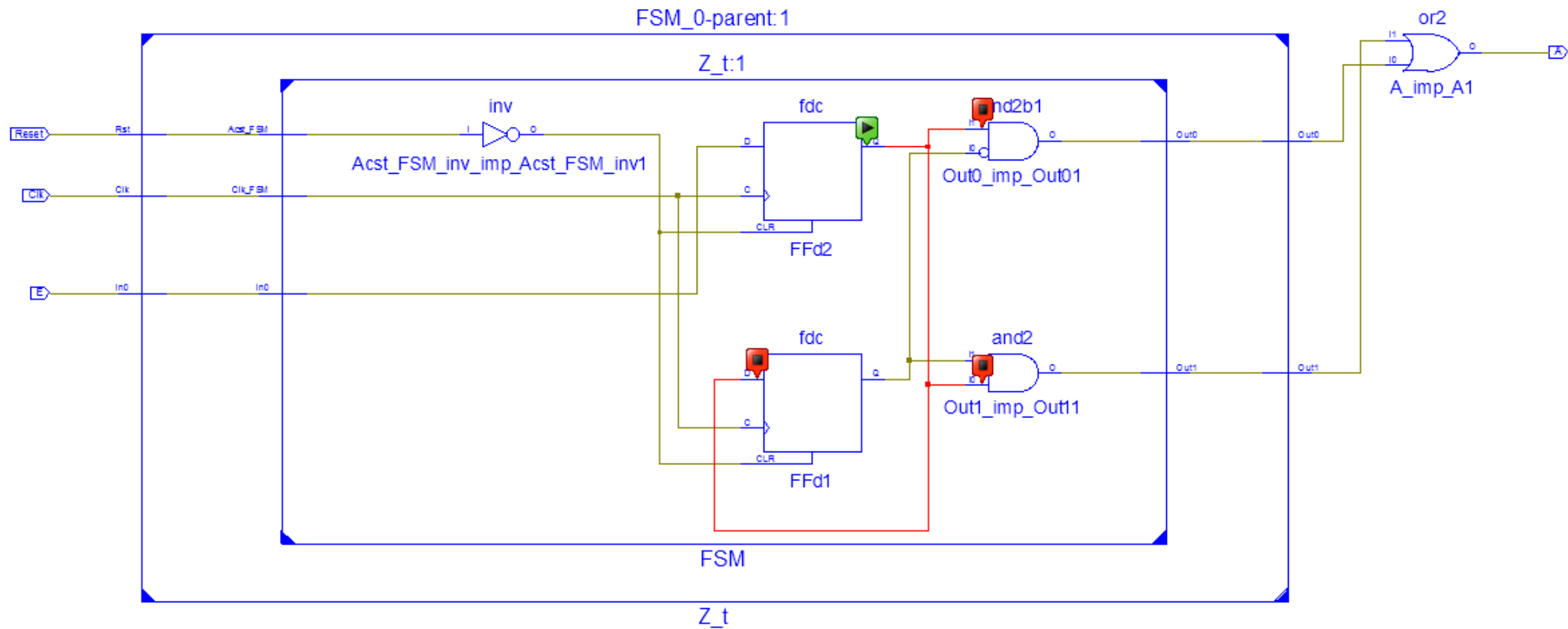
Moore – VHDL Code (2/3)

```
44 -- Asynchrones UebergangsSchaltnetz
45 UebergangsSchaltnetz : process(Z_t, E)
46
47 begin
48
49     if E = '0' AND Z_t = "00" then
50
51         Z_t1 <= "00";
52
53     elsif E = '0' AND Z_t = "01" then
54
55         Z_t1 <= "10";
56
57     elsif E = '0' AND Z_t = "10" then
58
59         Z_t1 <= "00";
60
61     elsif E = '0' AND Z_t = "11" then
62
63         Z_t1 <= "10";
64
65     elsif E = '1' AND Z_t = "00" then
66
67         Z_t1 <= "01";
68
69     elsif E = '1' AND Z_t = "01" then
70
71         Z_t1 <= "11";
72
73     elsif E = '1' AND Z_t = "10" then
74
75         Z_t1 <= "01";
76
77     else
78
79         Z_t1 <= "11";
80
81     end if;
82
83
84 end process UebergangsSchaltnetz ;
```


Moore – VHDL Code (3/3)

```
87  -- Synchroner Zustandsspeicher
88  ZustandsSpeicher : process (Clk,Reset)
89
90      begin
91
92          if Reset = '0' then
93
94              Z_t <= "00";
95
96          elsif Clk'event AND Clk = '1' then
97
98              Z_t <= Z_t1;
99
100         end if;
101
102
103     end process ZustandsSpeicher;
104
105
106  -- Asynchrones Ausgangsschaltnetz
107  -- hängt nur vom inneren Zustand Z_t(0) also dem LSB ab
108  AusgangsSchaltnetz : process (Z_t)
109
110      begin
111
112          A <= Z_t(0);
113
114      end process AusgangsSchaltnetz;
115
116
117  end Behavioral;
```

Moore – Synthese

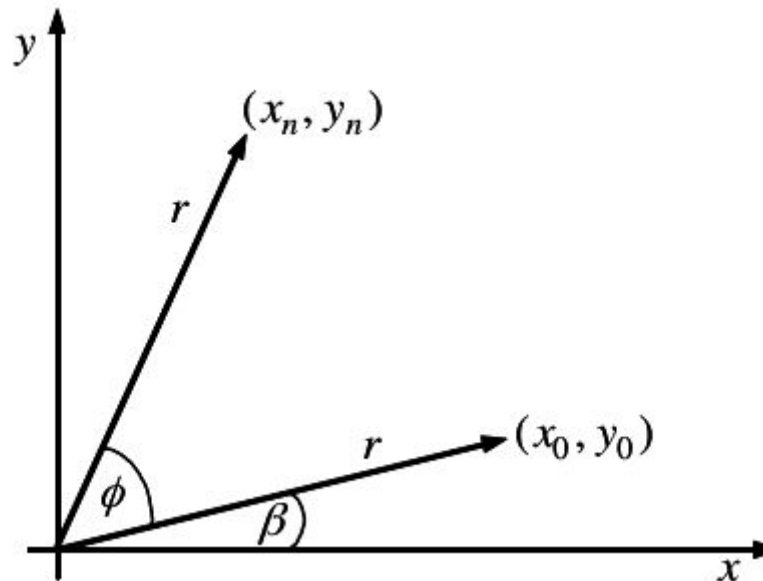


Kapitel 8

CORDIC

CORDIC - Coordinate Rotation Digital Computer

- Der CORDIC-Algorithmus wurde 1959 von J. E. Volder entwickelt
- 1971 von J. S. Walther auf die heute übliche Form erweitert
- CORDIC ist die Abkürzung für Coordinate Rotation Digital Computer
- Dieser Algorithmus ermöglicht es, elementare trigonometrische und hyperbolische Funktionen unter fast ausschließlicher Verwendung von schnellen Operationen effizient zu berechnen
- Mit schnellen Operationen sind hier Additionen (adds) und Multiplikationen mit Zweier-Potenzen (shifts) gemeint



Taylor-Polynome

- Taylor-Polynome werden häufig genutzt, Funktionen zu implementieren
- Beispiel: Taylor-Reihe einer Exponentialfunktion

$$e^x = 1 + \frac{x^1}{1!} + \frac{x^2}{2!} + \frac{x^3}{3!} + \frac{x^4}{4!} + \dots = \sum_{n=0}^{\infty} \frac{x^n}{n!}$$

- Die Konvergenz-Geschwindigkeit dieser Reihe ist vom Argument abhängig
 - Taylor-Reihe wird bei $n=8$ abgebrochen

$$e^{0.5} = 1.6487212707 \quad (\text{Taschenrechner})$$

$$e^{5.7} = 298.8674009671 \quad (\text{Taschenrechner})$$

$$e^{0.5} \approx \sum_{n=0}^8 \frac{0.5^n}{n!} = 1.6487212650$$

$$e^{5.7} \approx \sum_{n=0}^8 \frac{5.7^n}{n!} = 261.9926451175$$

CORDIC - Rechenarten

- CORDIC beherrscht folgende Rechenarten
 - Betriebsarten: Rotating und Vectoring
 - Modus: linear (m=0), zirkular (m=1), hyperbolisch (m=-1)

Mode m	Rotating $x_0 = 1, y_0 = 0, z_0 = \phi$ $z_n \rightarrow 0, \sigma_i = \text{sgn}(z_i)$	Vectoring $z_0 = 0, y_n \rightarrow 0$ $\sigma_i = -\text{sgn}(y_i)$
$m = 0$ $K_0 = 1$ $S(0, i) = \{0, 1, \dots, n-1\}$	$x_{i+1} = x_i$ mit: $y_0 = 0$ $y_{i+1} = y_i + \sigma_i \cdot 2^{-S(0, i)} \cdot x_i$ $0 \leftarrow z_{i+1} = z_i - \sigma_i \cdot 2^{-S(0, i)}$ $\Rightarrow x_n = x_0$ $y_n = x_0 \cdot z_0$	$x_{i+1} = x_i$ $0 \leftarrow y_{i+1} = y_i + \sigma_i \cdot 2^{-S(0, i)} \cdot x_i$ $z_{i+1} = z_i - \sigma_i \cdot 2^{-S(0, i)}$ $\Rightarrow x_n = x_0$ $z_n = y_0 / x_0$
$m = 1$ $K_1 = 0.607252935$ $S(1, i) = \{0, 1, \dots, n-1\}$	$x_{i+1} = x_i - \sigma_i \cdot 2^{-S(1, i)} \cdot y_i$ $y_{i+1} = y_i + \sigma_i \cdot 2^{-S(1, i)} \cdot x_i$ $0 \leftarrow z_{i+1} = z_i - \sigma_i \cdot \arctan(2^{-S(1, i)})$ $\Rightarrow x_n = K_1^{-1} \cdot \cos(\phi)$ $y_n = K_1^{-1} \cdot \sin(\phi)$	$x_{i+1} = x_i - \sigma_i \cdot 2^{-S(1, i)} \cdot y_i$ $0 \leftarrow y_{i+1} = y_i + \sigma_i \cdot 2^{-S(1, i)} \cdot x_i$ $z_{i+1} = z_i - \sigma_i \cdot \arctan(2^{-S(1, i)})$ $\Rightarrow x_n = K_1^{-1} \cdot \sqrt{x_0^2 + y_0^2}$ $z_n = \arctan(y_0 / x_0)$
$m = -1$ $K_{-1} = 1.207497068$ $S(-1, i) = \{1, 2, 3, 4, 4, 5, \dots, 13, 13, 14, \dots, n\}$	$x_{i+1} = x_i + \sigma_i \cdot 2^{-S(-1, i)} \cdot y_i$ $y_{i+1} = y_i + \sigma_i \cdot 2^{-S(-1, i)} \cdot x_i$ $0 \leftarrow z_{i+1} = z_i - \sigma_i \cdot \text{artanh}(2^{-S(-1, i)})$ $\Rightarrow x_n = K_{-1}^{-1} \cdot \cosh(\phi)$ $y_n = K_{-1}^{-1} \cdot \sinh(\phi)$	$x_{i+1} = x_i + \sigma_i \cdot 2^{-S(-1, i)} \cdot y_i$ $0 \leftarrow y_{i+1} = y_i + \sigma_i \cdot 2^{-S(-1, i)} \cdot x_i$ $z_{i+1} = z_i - \sigma_i \cdot \text{artanh}(2^{-S(-1, i)})$ $\Rightarrow x_n = K_{-1}^{-1} \cdot \sqrt{x_0^2 - y_0^2}$ $z_n = \text{artanh}(y_0 / x_0)$

Herleitung (1/3)

- Ziel: $\text{atan}(\varphi)$ zu berechnen $\rightarrow$ Winkelmessung bzw. Frequenzmessung bei oszillierenden Systemen $\rightarrow$ 50Hz im Stromnetz, Demodulation von digitalen Nachrichtensystemen, ...
- Ausgangspunkt für die Herleitung des CORDIC-Algorithmus ist die Drehung eines Vektors $[x_0, y_0]^T$ um den Winkel φ in einem kartesischen Koordinatensystem
- durch die Drehung wird der Betrag des Vektors nicht verändert

$$\cos(\beta) = \frac{x_0}{r} \Rightarrow x_0 = r \cdot \cos(\beta)$$

$$\sin(\beta) = \frac{y_0}{r} \Rightarrow y_0 = r \cdot \sin(\beta)$$

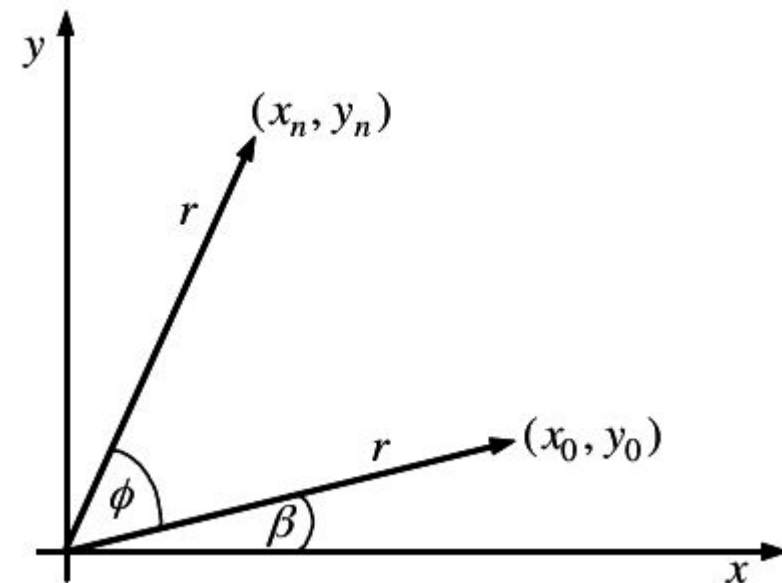
- Anwendung der Additionstheoreme führt zu

$$x_n = r \cdot \cos(\beta + \phi) = \underbrace{r \cdot \cos(\beta)}_{x_0} \cdot \cos(\phi) - \underbrace{r \cdot \sin(\beta)}_{y_0} \cdot \sin(\phi)$$

$$x_n = x_0 \cdot \cos(\phi) - y_0 \cdot \sin(\phi)$$

$$y_n = r \cdot \sin(\beta + \phi) = \underbrace{r \cdot \sin(\beta)}_{y_0} \cdot \cos(\phi) + \underbrace{r \cdot \cos(\beta)}_{x_0} \cdot \sin(\phi)$$

$$y_n = x_0 \cdot \sin(\phi) + y_0 \cdot \cos(\phi)$$



Herleitung (2/3)

- Die Gleichungen für x_n und y_n lassen sich zur sog. Drehmatrix zusammenfassen
- der Ergebnisvektor $[x_n, y_n]^T$ berechnet sich wie folgt

$$\begin{bmatrix} x_n \\ y_n \end{bmatrix} = \begin{bmatrix} \cos(\phi) & -\sin(\phi) \\ \sin(\phi) & \cos(\phi) \end{bmatrix} \cdot \begin{bmatrix} x_0 \\ y_0 \end{bmatrix}$$

- Durch Umformung gilt die folgende mathematische Beziehung

$$\cos(\phi) = \frac{1}{\sqrt{1 + \tan^2(\phi)}}$$

- Mit $\sin^2 + \cos^2 = 1$ folgt $\sin = \sqrt{1 - \cos^2}$ und $\tan() = \sin() / \cos()$ kann $\sin()$ und $\cos()$ ausgeklammert und durch $\tan()$ ersetzt werden

$$\begin{bmatrix} x_n \\ y_n \end{bmatrix} = \cos(\phi) \cdot \begin{bmatrix} 1 & -\tan(\phi) \\ \tan(\phi) & 1 \end{bmatrix} \cdot \begin{bmatrix} x_0 \\ y_0 \end{bmatrix} = \frac{1}{\sqrt{1 + \tan^2(\phi)}} \cdot \begin{bmatrix} 1 & -\tan(\phi) \\ \tan(\phi) & 1 \end{bmatrix} \cdot \begin{bmatrix} x_0 \\ y_0 \end{bmatrix}$$

Herleitung (3/3)

- Bei dem CORDIC-Algorithmus wird die Drehung um den Winkel ϕ durch mehrere Rotationen mit bekanntem, vorher vorab gespeicherten Teilwinkel α_i realisiert

$$\phi = \sum_{i=0}^{n-1} \sigma_i \cdot \alpha_i \quad \sigma_i \in \{-1, 1\}$$

- Durch festzulegende Vorzeichen σ_i wird die Summe der n Teilwinkel α_i den gesuchten Winkel ϕ annähern
- Winkel werden als eine Folge von Zweierpotenzen angenommen $\rightarrow$ Lookuptable in DEG

$$\tan(\alpha_i) = 2^{-i}$$

$$z_{i+1} = z_i - \sigma_i \cdot \arctan(2^{-i}) = z_i - \text{sgn}(z_i) \cdot \arctan(2^{-i})$$

$$\text{mit: } \sigma_i = \text{sgn}(z_i) = \begin{cases} +1 & z_i \geq 0 \\ -1 & z_i < 0 \end{cases}$$

Ausgabe des LookupTables		
LookUpTableWinkel:	Winkel 1.	45.000
LookUpTableWinkel:	Winkel 2.	26.565
LookUpTableWinkel:	Winkel 3.	14.036
LookUpTableWinkel:	Winkel 4.	7.125
LookUpTableWinkel:	Winkel 5.	3.576
LookUpTableWinkel:	Winkel 6.	1.790
LookUpTableWinkel:	Winkel 7.	0.895
LookUpTableWinkel:	Winkel 8.	0.448
LookUpTableWinkel:	Winkel 9.	0.224
LookUpTableWinkel:	Winkel 10.	0.112
LookUpTableWinkel:	Winkel 11.	0.056
LookUpTableWinkel:	Winkel 12.	0.028
LookUpTableWinkel:	Winkel 13.	0.014
LookUpTableWinkel:	Winkel 14.	0.007
LookUpTableWinkel:	Winkel 15.	0.003
LookUpTableWinkel:	Winkel 16.	0.002

Implementierung der Winkel- und Längenkalkulation

- Funktion in C zur Berechnung der zu speichernden Winkel und der zur Korrektur verwendeten Längenanpassungen
- Integerkonvertierung findet statt durch die Multiplikation mit 2er Potenzen

```
/* Einmaliges Füllen des LookUp Tables mit den Winkeln, um die der Vektor auf die x-Achse
 * gedreht werden soll. */
void precalcLookUpTable() {

    int i;

    /* Debugausgabe des LookUpTables */
    if(DEBUG == 1) {
        printf("*****\n");
        printf("Ausgabe des LookupTables \n");
    }

    /* Berechnung zweier LookupTable
     * LookUpTable: Winkel
     * es wird die Anzahl von ITERATIONEN Winkeln vorberechnet */
    for(i=0; i<ITERATIONEN; i++) {

        LookUpTableWinkel[i] = (float) atan(pow(2.0, (float) -i)) / M_PI * 180.0;
        LookUpTableWinkelInt[i] = (int) (atan(pow(2.0, (float) -i)) / M_PI * 180.0 * pow(2.0, ITERATIONEN));
    }

    if(DEBUG == 1) {
        for(i=0; i<ITERATIONEN; i++) {
            printf("LookUpTableWinkel Float: Winkel %d.\t\t%1.3f \n", i+1, LookUpTableWinkel[i]);
        }
        printf("\n");
        for(i=0; i<ITERATIONEN; i++) {
            printf("LookUpTableWinkel INTEGER: Winkel %d.\t\t%d \n", i+1, LookUpTableWinkelInt[i]);
        }
    }
    printf("*****\n\n\n");

    /* Berechnung zweier LookupTable
     * LookUpTable: LaengenSkalierung
     */
    LookUpTableLaenge[0] = 1.0 / sqrt(2.0);
    if(DEBUG == 1) {
        printf("LookUpTableLaengel: Laenge %d.\t\t%1.3f \n", 0, LookUpTableLaenge[0]);
    }
    for(i=1; i<ITERATIONEN; i++) {

        LookUpTableLaenge[i] = LookUpTableLaenge[i-1] * 1.0 / (sqrt(1.0 + pow(2.0, 2.0*(float) -i)));

        if(DEBUG == 1) {
            printf("LookUpTableLaengel: Laenge %d.\t\t%1.6f \n", i+1, LookUpTableLaenge[i]);
        }
    }
}
```

Vorkalkulierte Winkel als float und integer Werte

- Winkel werden von Float zu Integer konvertiert, indem mit $\text{pow}(2.0, \text{Iterations})$ multipliziert wird
- Die Vektorlänge muss je nach Anzahl der Iterationen noch angepasst werden, falls diese noch benötigt wird

```
*****
Ausgabe des LookupTables
LookupTableWinkel Float: Winkel 1.          45.000
LookupTableWinkel Float: Winkel 2.          26.565
LookupTableWinkel Float: Winkel 3.          14.036
LookupTableWinkel Float: Winkel 4.           7.125
LookupTableWinkel Float: Winkel 5.           3.576
LookupTableWinkel Float: Winkel 6.           1.790
LookupTableWinkel Float: Winkel 7.           0.895
LookupTableWinkel Float: Winkel 8.           0.448
LookupTableWinkel Float: Winkel 9.           0.224
LookupTableWinkel Float: Winkel 10.          0.112
LookupTableWinkel Float: Winkel 11.          0.056
LookupTableWinkel Float: Winkel 12.          0.028
LookupTableWinkel Float: Winkel 13.          0.014
LookupTableWinkel Float: Winkel 14.          0.007
LookupTableWinkel Float: Winkel 15.          0.003
LookupTableWinkel Float: Winkel 16.          0.002

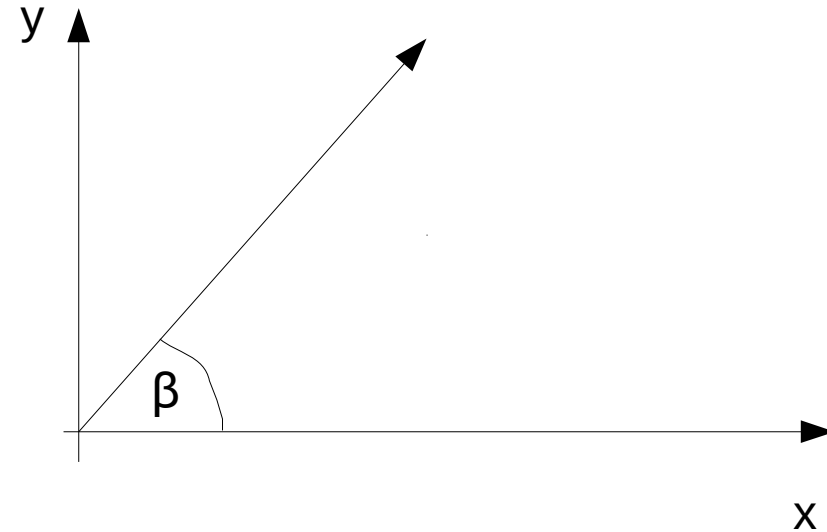
LookupTableWinkel INTEGER: Winkel 1.        46080
LookupTableWinkel INTEGER: Winkel 2.        27202
LookupTableWinkel INTEGER: Winkel 3.        14373
LookupTableWinkel INTEGER: Winkel 4.         7296
LookupTableWinkel INTEGER: Winkel 5.         3662
LookupTableWinkel INTEGER: Winkel 6.         1832
LookupTableWinkel INTEGER: Winkel 7.          916
LookupTableWinkel INTEGER: Winkel 8.          458
LookupTableWinkel INTEGER: Winkel 9.          229
LookupTableWinkel INTEGER: Winkel 10.         114
LookupTableWinkel INTEGER: Winkel 11.          57
LookupTableWinkel INTEGER: Winkel 12.          28
LookupTableWinkel INTEGER: Winkel 13.          14
LookupTableWinkel INTEGER: Winkel 14.           7
LookupTableWinkel INTEGER: Winkel 15.           3
LookupTableWinkel INTEGER: Winkel 16.           1
*****
```

```
LookupTableLaengel: Laenge 0.                0.707
LookupTableLaengel: Laenge 2.                0.632456
LookupTableLaengel: Laenge 3.                0.613572
LookupTableLaengel: Laenge 4.                0.608834
LookupTableLaengel: Laenge 5.                0.607648
LookupTableLaengel: Laenge 6.                0.607352
LookupTableLaengel: Laenge 7.                0.607278
LookupTableLaengel: Laenge 8.                0.607259
LookupTableLaengel: Laenge 9.                0.607254
LookupTableLaengel: Laenge 10.               0.607253
LookupTableLaengel: Laenge 11.               0.607253
LookupTableLaengel: Laenge 12.               0.607253
LookupTableLaengel: Laenge 13.               0.607253
LookupTableLaengel: Laenge 14.               0.607253
LookupTableLaengel: Laenge 15.               0.607253
LookupTableLaengel: Laenge 16.               0.607253
*****
```

atan() Funktion

- Funktion in C zur Berechnung des Arcus Tangens in der Einheit Grad
- Eingabe ist die x,y Koordinate eines Vektors im Koordinatensystem
- Für diesen Vektor soll der Winkel β zur x-Achse bestimmt werden

```
float FloatAtan(float x, float y) {  
    float Angle;  
    Angle = (float) atan((double) y/ (double) x) / M_PI * 180.0;  
    return(Angle);  
}
```



CORDIC als float Funktion

- Funktion in C zur Berechnung des Arcus Tangens in der Einheit Grad

```
float CordicFloat(float x, float y) {  
    float Sig = -1.0;  
    float xh, z;  
    int i;  
  
    z = 0.0;  
  
    if(DEBUG == 1) {  
        printf("x-Werte \t y-Werte \t Winkel \n");  
        printf("*****\n");  
    }  
  
    for(i=0; i<ITERATIONEN; i++) {  
        if(y < 0.0) {  
            Sig = +1.0;  
        }  
        else {  
            Sig = -1.0;  
        }  
  
        xh = x;  
        x = x - Sig * (float) pow(2.0, (float) -i) * y;  
        y = y + Sig * (float) pow(2.0, (float) -i) * xh;  
        z = z - Sig * LookUpTableWinkel[i];  
  
        if(DEBUG == 1) {  
            printf("%f \t %f \t %f \n", 1.0 / sqrt(2.0) * LookUpTableLaenge[ITERATIONEN-1] * x,y,z);  
        }  
    }  
    if(DEBUG == 1) {  
        printf("*****\n");  
    }  
    return(z);  
}
```

CORDIC als bittrue Funktion

- Funktion in C zur Berechnung des Arcus Tangens in der Einheit Grad
- Floating Point Werte werden durch Bittrue Limitierungen begrenzt

```
float CordicBittrue(float x, float y) {  
  
    float Sig = -1.0;  
    float xh, z;  
    int i;  
  
    z = 0.0;  
  
    if(DEBUG == 1) {  
        printf("x-Werte \t y-Werte \t Winkel \n");  
        printf("*****\n");  
    }  
  
    for(i=0; i<ITERATIONEN; i++) {  
  
        if(y < 0.0) {  
            Sig = +1.0;  
        }  
        else {  
            Sig = -1.0;  
        }  
  
        xh = ConvBittrue(x,8,5,1,1,1);  
        x = ConvBittrue(x,8,5,1,1,1) - Sig * (float) pow(2.0, (float) -i) * ConvBittrue(y,8,5,1,1,1);  
        y = ConvBittrue(y,8,5,1,1,1) + Sig * (float) pow(2.0, (float) -i) * xh;  
        z = ConvBittrue(z,8,5,1,1,1) - Sig * ConvBittrue(LookUpTableWinkel[i], 8,5,1,1,1);  
  
        if(DEBUG == 1) {  
            printf("%f \t %f \t %f \n",1.0 / sqrt(2.0) * LookUpTableLaenge[ITERATIONEN-1] * x,y,z);  
        }  
    }  
    if(DEBUG == 1) {  
        printf("*****\n");  
    }  
    return(z);  
}
```

CORDIC als Integer Funktion (1/6)

- Inputwerte auf -256 ... +255 begrenzen
- Wortbreiten vor dem Start der Iterationen überprüfen! → 17:Bit Signed
- Variable „Rot“ zeigt die Quadrantenzugehörigkeit an
→ später Nachverrechnung

```
float CordicInt(float x, float y) {

    int BitBreite = ITERATIONEN;

    int x_int, y_int;
    int xh, z_int;
    int i, Rot;

    float z_float;

    printf("%f %f \n\n", x, y);
    /* 9 Bit Signed Input von -256 ... + 255 */
    x = (float) ConvIntBittrue(x, 9, 0, 1);
    y = (float) ConvIntBittrue(y, 9, 0, 1);
    printf("%f %f \n\n", x, y);

    /* Shift der 9 Bit Signed Input von -256 ... + 255
     * auf 17 Bit Signed (16 -8) = 8 -> 9+8=17
     */
    x_int = (int) x << (BitBreite - 8);
    y_int = (int) y << (BitBreite - 8);

    printf("%d %d \n\n", x_int, y_int);
    x_int = ConvIntBittrue(x_int, 17, 0, 1);
    y_int = ConvIntBittrue(y_int, 17, 0, 1);
    printf("%d %d \n\n", x_int, y_int);

    z_int = 0;

    if(x_int < 0 && y_int >= 0) {
        x_int = -x_int;
        y_int = y_int;
        Rot = 1;
    }
    else if(x_int < 0 && y_int < 0) {
        x_int = -x_int;
        y_int = y_int;
        Rot = 2;
    }
    else {
        x_int = x_int;
        y_int = y_int;
        Rot = 0;
    }

    if(DEBUG == 1) {
        printf("x-Werte \t y-Werte \t Winkel \n");
        printf("*****\n");
    }
}
```

CORDIC als Integer Funktion (2/6)

- Wortbreiten während der Iterationen auf 19 Bit Signed begrenzen, aber nicht weniger, denn die Wortbreiten wachsen während der Iterationen an
- Ausgabe folgt dann mit nur 9 Bit Signed, womit Werte von -256 ... +256 dargestellt werden können
→ es sollen ja nur Ausgaben von -180° + 180° ermöglicht werden
- Die 16 Bit Unsigned interne Auflösung der Winkel und die 19 Bit Wortbreite von x,y dienen zur Rechengenauigkeit

```
for(i=0; i<ITERATIONEN; i++) {  
    xh = x_int;  
    if(y_int < 0) {  
        x_int = x_int - (y_int >> i);  
        y_int = y_int + (xh >> i);  
        z_int = z_int - LookUpTableWinkelInt[i];  
    }  
    else {  
        x_int = x_int + (y_int >> i);  
        y_int = y_int - (xh >> i);  
        z_int = z_int + LookUpTableWinkelInt[i];  
    }  
    /* 19 Bit, kein Clipping, Signed */  
    x_int = ConvIntBittrue(x_int, 19, 0, 1);  
    y_int = ConvIntBittrue(y_int, 19, 0, 1);  
    z_int = ConvIntBittrue(z_int, 19, 0, 1);  
}  
  
if(DEBUG == 1) {  
    if(y_int >= 10000 || -y_int > 1000) {  
        printf("%d \t \t %d \t %d \n", x_int, y_int, z_int);  
    }  
    else {  
        printf("%d \t \t %d \t \t %d \n", x_int, y_int, z_int);  
    }  
}  
}  
if(DEBUG == 1) {  
    printf("*****\n");  
}  
  
/* Umwandeln in Floatingpoint Winkel */  
/* Von 19 Bit um 16-6 = 10 Bit herunter auf 9 Bit Signed shiften */  
if(Rot == 0) {  
    z_float = (float) (z_int >> (BitBreite-6));  
}  
else if(Rot == 1) {  
    z_float = (float) (180.0 - (z_int >> (BitBreite-6)));  
}  
else {  
    z_float = (float) (-180.0 - (z_int >> (BitBreite-6)));  
}  
  
/* Finale Wortbreite mit 9 Bit festlegen -> zur Ueberpruefung */  
z_float = ConvIntBittrue(z_float, 9, 0, 1);  
return(z_float);  
}
```


CORDIC als Integer Funktion (3/6)

- Input mit $x = +255$ und $y = +255 \rightarrow$ Winkel 45°

```
255.000000 255.000000
255.000000 255.000000
65280 65280
65280 65280

x-Werte          y-Werte          Winkel
*****
130560            0            46080
130560          -65280          73282
146880          -32640          58909
150960          -14280          51613
151853          -4845           47951
152005          -100           46119
152007           2275           45203
152024           1088           45661
152028           495           45890
152028           199           46004
152028           51           46061
152028          -23           46089
152029           14           46075
152029          -4           46082
152030           5           46079
152030           1           46080
*****
*****
Cordic INTEGER:  x: 255.00      y:255.00      Winkel: 45.00
*****
```

CORDIC als Integer Funktion (4/6)

- Input mit $x = -255$ und $y = +255 \rightarrow$ Winkel 135°

```
-255.000000 255.000000
-255.000000 255.000000
-65280 65280
-65280 65280

x-Werte          y-Werte          Winkel
*****
130560            0            46080
130560          -65280          73282
146880          -32640          58909
150960          -14280          51613
151853           -4845          47951
152005           -100          46119
152007            2275          45203
152024            1088          45661
152028             495          45890
152028             199          46004
152028             51          46061
152028            -23          46089
152029             14          46075
152029             -4          46082
152030              5          46079
152030              1          46080
*****
*****
Cordic INTEGER:   x: -255.00      y:255.00      Winkel: 135.00
*****
```

CORDIC als Integer Funktion (5/6)

- Input mit $x = -255$ und $y = -255 \rightarrow$ Winkel -135°

```
-255.000000 -255.000000
-255.000000 -255.000000
-65280 -65280
-65280 -65280

x-Werte      y-Werte      Winkel
*****
130560      0      -46080
130560     -65280     -18878
146880     -32640     -33251
150960     -14280     -40547
151853     -4845      -44209
152005     -100      -46041
152007      2275      -46957
152024      1088      -46499
152028      495      -46270
152028      199      -46156
152028      51       -46099
152028     -23       -46071
152029      14       -46085
152029      -4       -46078
152030       5       -46081
152030       1       -46080
*****
*****
Cordic INTEGER:  x: -255.00      y: -255.00      Winkel: -135.00
*****
```

CORDIC als Integer Funktion (6/6)

- Input mit $x = 255$ und $y = -255 \rightarrow$ Winkel -45°

```
255.000000 -255.000000
255.000000 -255.000000
65280 -65280
65280 -65280

x-Werte      y-Werte      Winkel
*****
130560      0      -46080
130560     -65280     -18878
146880     -32640     -33251
150960     -14280     -40547
151853     -4845      -44209
152005      -100      -46041
152007      2275      -46957
152024      1088      -46499
152028      495       -46270
152028      199       -46156
152028       51       -46099
152028      -23       -46071
152029      14       -46085
152029      -4       -46078
152030       5       -46081
152030       1       -46080
*****
*****
Cordic INTEGER:  x: 255.00      y: -255.00      Winkel: -45.00
*****
```

CORDIC VHDL Code wie bei einer SW Entwicklung

- Debugging kaum möglich, da alle Iterationen in einem Schritt durchgeführt werden
- dor-Schleifen nicht sofort in der ersten Release verwenden

```
-- aktiver Prozessbereich
-- elsif reagiert nur auf eine steigende Flanke
ELSIF RISING_EDGE(Clk) THEN

    -- Zuweisung des Eingangsvektors x,y
    XVar := TO_INTEGER(SIGNED(XIn));
    YVar := TO_INTEGER(SIGNED(YIn));

    ZVar := 0;

    -- alle Iterationen in einer for Schleife
    FOR IVar IN 0 TO BitBreite-1 LOOP

        XHVar := XVar;

        IF(YVar < 0) THEN
            XVar := XVar - TO_INTEGER(SHIFT_RIGHT(TO_SIGNED(YVar, 16), IVar));
            YVar := YVar + TO_INTEGER(SHIFT_RIGHT(TO_SIGNED(XHVar, 16), IVar));
            ZVar := ZVar - PreDefinedAngles(IVar);
        ELSE
            XVar := XVar + TO_INTEGER(SHIFT_RIGHT(TO_SIGNED(YVar, 16), IVar));
            YVar := YVar - TO_INTEGER(SHIFT_RIGHT(TO_SIGNED(XHVar, 16), IVar));
            ZVar := ZVar + PreDefinedAngles(IVar);
        END IF;

        -- Datenausgabe
        DataOut <= STD_LOGIC_VECTOR(TO_SIGNED(ZVar, DataOut'LENGTH));

    END LOOP;

END IF;
```


Code (1/8)

Aufwand: ca. 8 Stunden

inkl. Testen

Aktuell funktionieren nur
Vektoren zwischen -45° - 45°

```
-- Einbinden der Libraries
library IEEE;
use IEEE.STD_LOGIC_1164.ALL; -- Bibliothek fuer Basisoperation mit STD_LOGIC Signalen
use IEEE.STD_LOGIC_UNSIGNED.ALL; -- Bibliothek fuer Basisoperation mit STD_LOGIC Unsigned Signalen
use IEEE.NUMERIC_STD.ALL; -- Bibliothek fuer numrische Berechnungen

-- Entity
-- Eingangs- und Ausgangsport deklarieren

ENTITY CordicE IS
    PORT (
        Reset : IN STD_LOGIC; -- Input Port der Clock
        Clk : IN STD_LOGIC; -- Input Port des Reset Signals
        ChipSelect : IN STD_LOGIC; -- Input Port damit der Chip aktiv wird
        XIn : IN STD_LOGIC_VECTOR(16 DOWNTO 0); -- Input Port des Datensignals X
        YIn : IN STD_LOGIC_VECTOR(16 DOWNTO 0); -- Input Port des Datensignals Y
        Aufloesung : IN STD_LOGIC_VECTOR(4 DOWNTO 0); -- geplante Iterationsgenauigkeit Max 16(Bit)
        Modus : IN STD_LOGIC_VECTOR(1 DOWNTO 0); -- Modus 0,1,2 fuer atan, tanh, cosh
        ChipReady : OUT STD_LOGIC; -- Berechnung ist fertig
        DataOut : OUT STD_LOGIC_VECTOR(17 DOWNTO 0) -- Ergebnis der atan, tanh oder cosh Berechnung
    );
END CordicE;

-- Architecture
-- Funktionalitaet in synchronen und asynchronen Prozessen definieren
-- synchrone Prozesse : Clk und Reset in der Sensitivity List
-- asynchrone Prozesse: Prozesseingangssignale im der Sensitivity List

ARCHITECTURE CordicArch OF CordicE IS
    TYPE Angles IS ARRAY (15 DOWNTO 0) OF INTEGER RANGE -65536 TO +65535; -- 17Bit SIGNED

    SIGNAL PreDefinedAngles : Angles;
    SIGNAL XVar : INTEGER RANGE -65536 TO +65535;
    SIGNAL YVar : INTEGER RANGE -65536 TO +65535;
    SIGNAL ZVar : INTEGER RANGE -65536 TO +65535;
    SIGNAL IVar : INTEGER RANGE -1 TO 17;
    SIGNAL ChipReadyReg : STD_LOGIC;
    SIGNAL RotSig : INTEGER RANGE 0 TO 3 := 0;

    BEGIN
        -- synchroner Prozess
        P1: PROCESS(Reset, Clk) IS

            VARIABLE XHVar : INTEGER RANGE -65536 TO +65535 := 0;
            VARIABLE XXVar : INTEGER RANGE -65536 TO +65535 := 0;
            VARIABLE YYVar : INTEGER RANGE -65536 TO +65535 := 0;
            VARIABLE BitBreite : INTEGER RANGE 0 TO 16 := 0;
```

Code (2/8)

```
BEGIN
-- Reset
-- allen Signalen und Ausgaengen einen Anfangswert zuweisen
-- nicht den Eingangssignalen
IF Reset = '0' THEN
    -- DataOut <= (OTHERS => '0');
    -- Vorkalkulierte Winkel speichern
    PreDefinedAngles(0) <= 0;
    PreDefinedAngles(1) <= 0;
    PreDefinedAngles(2) <= 0;
    PreDefinedAngles(3) <= 0;
    PreDefinedAngles(4) <= 0;
    PreDefinedAngles(5) <= 0;
    PreDefinedAngles(6) <= 0;
    PreDefinedAngles(7) <= 0;
    PreDefinedAngles(8) <= 0;
    PreDefinedAngles(9) <= 0;
    PreDefinedAngles(10) <= 0;
    PreDefinedAngles(11) <= 0;
    PreDefinedAngles(12) <= 0;
    PreDefinedAngles(13) <= 0;
    PreDefinedAngles(14) <= 0;
    PreDefinedAngles(15) <= 0;
    XVar <= 0;
    YVar <= 0;
    ZVar <= 0;
    IVar <= -1;
    ChipReadyReg <= '1';

    -- Iterationen auf 1.. 16 beschränken
    BitBreite := TO_INTEGER(UNSIGNED(Aufloesung));

    IF BitBreite > 16 THEN
        BitBreite := 16;
    ELSIF BitBreite < 1 THEN
        BitBreite := 1;
    END IF;
```

Code (3/8)

```
-- aktiver Prozessbereich
-- elsif reagiert nur auf eine steigende Flanke
ELSIF RISING_EDGE(Clk) THEN

-- Zustand: Initialisierung
IF IVar = -1 AND ChipSelect = '1' AND ChipReadyReg = '1' THEN

-- Zuweisung des Eingangsvektors x,y
XXVar := TO_INTEGER(SIGNED(XIn));
YYVar := TO_INTEGER(SIGNED(YIn));

-- Vektor im 1. Quadranten groesser 45°
-- Vektor um -90° in den 2. Quadranten drehen -> X und Y tauschen
IF YYVar > 0 AND XXVar > 0 AND YYVar > XXVar THEN
    XVar <= YYVar;
    YVar <= XXVar;
    RotSig <= 1;

-- X und -Y tauschen
ELSIF YYVar < 0 AND -YYVar > XXVar THEN
    XVar <= -YYVar;
    YVar <= XXVar;
    RotSig <= 2;

-- ELSIF YYVar > 0 AND XXVar > 0 AND YYVar > XXVar THEN
--     XVar <= YYVar;
--     YVar <= XXVar;
--     RotSig <= 3;

-- X und Y nicht tauschen
ELSE
    XVar <= XXVar;
    YVar <= YYVar;
    RotSig <= 0;
END IF;

ZVar <= 0;
```


Code (4/8)

```
-- atan Modus
IF Modus = "00" THEN

    -- Vorkalkulierte Winkel für die atan() Berechnung speichern
    PreDefinedAngles(0) <= 46080;
    PreDefinedAngles(1) <= 27202;
    PreDefinedAngles(2) <= 14373;
    PreDefinedAngles(3) <= 7296;
    PreDefinedAngles(4) <= 3662;
    PreDefinedAngles(5) <= 1832;
    PreDefinedAngles(6) <= 916;
    PreDefinedAngles(7) <= 458;
    PreDefinedAngles(8) <= 229;
    PreDefinedAngles(9) <= 114;
    PreDefinedAngles(10) <= 57;
    PreDefinedAngles(11) <= 28;
    PreDefinedAngles(12) <= 14;
    PreDefinedAngles(13) <= 7;
    PreDefinedAngles(14) <= 3;
    PreDefinedAngles(15) <= 1;

END IF; -- atan() Modus

IVar <= IVar + 1;

ChipReadyReg <= '0';

ELSIF IVar > -1 AND IVar < BitBreite AND ChipReadyReg = '0' THEN

    XHVar := XVar;
    IF(YVar < 0) THEN
        XVar <= XVar - TO_INTEGER(SHIFT_RIGHT(TO_SIGNED(YVar, 17), IVar));
        YVar <= YVar + TO_INTEGER(SHIFT_RIGHT(TO_SIGNED(XHVar, 17), IVar));
        ZVar <= ZVar - PreDefinedAngles(IVar);
    ELSE
        XVar <= XVar + TO_INTEGER(SHIFT_RIGHT(TO_SIGNED(YVar, 17), IVar));
        YVar <= YVar - TO_INTEGER(SHIFT_RIGHT(TO_SIGNED(XHVar, 17), IVar));
        ZVar <= ZVar + PreDefinedAngles(IVar);
    END IF;

    IVar <= IVar + 1;

    -- Berechnung ist fertig bzw. wurde noch gar nicht angefragt
ELSE
    ChipReadyReg <= '1';
    IVar <= -1;
    ZVar <= ZVar;
END IF;

END IF;

END PROCESS P1;
```

Code (5/8)

```
-- Datenausgabe
P2 : PROCESS(ZVar, ChipReadyReg, RotSig) IS

    VARIABLE ZZVar : INTEGER RANGE -131072 TO 131071 := 0;

    BEGIN

        IF RotSig = 1 THEN
            ZZVar := ((93160/2) - (ZVar/2)) * 2;    -- entspricht 90°
        ELSE
            ZZVar := ZVar;
        END IF;

        DataOut  <= STD_LOGIC_VECTOR(TO_SIGNED(ZZVar, DataOut'LENGTH));
        ChipReady <= ChipReadyReg;

    END PROCESS P2;

END CordicArch;
```

Code (6/8)

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_SIGNED.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;
-- Uncomment the following library declaration if using
-- arithmetic functions with Signed or Unsigned values
use IEEE.NUMERIC_STD.ALL;

-- Uncomment the following library declaration if instantiating
-- any Xilinx leaf cells in this code.
--library UNISIM;
--use UNISIM.VComponents.all;

entity v_simcortic is
end v_simcortic;

architecture Behavioral of v_simcortic is

COMPONENT CordicE PORT (
    Reset      : IN  STD_LOGIC;
    Clk        : IN  STD_LOGIC;
    ChipSelect  : IN  STD_LOGIC;
    XIn        : IN  STD_LOGIC_VECTOR(16 DOWNTO 0);
    YIn        : IN  STD_LOGIC_VECTOR(16 DOWNTO 0);
    Aufloesung  : IN  STD_LOGIC_VECTOR( 4 DOWNTO 0);
    Modus       : IN  STD_LOGIC_VECTOR( 1 DOWNTO 0);
    ChipReady   : OUT STD_LOGIC;
    DataOut     : OUT STD_LOGIC_VECTOR(17 DOWNTO 0)
);
END COMPONENT CordicE;

signal Reset      : STD_LOGIC := '0';
signal Clk        : STD_LOGIC := '0';
signal ChipSelectSig : STD_LOGIC := '0';
signal XInSig      : STD_LOGIC_VECTOR(16 DOWNTO 0) := (others => '0');
signal YInSig      : STD_LOGIC_VECTOR(16 DOWNTO 0) := (others => '0');
signal AufloesungSig : STD_LOGIC_VECTOR( 4 DOWNTO 0) := (others => '0');
signal ModusSig     : STD_LOGIC_VECTOR( 1 DOWNTO 0) := (others => '0');
signal DataOutSig   : STD_LOGIC_VECTOR(17 DOWNTO 0) := (others => '0');
signal ChipReadySig : STD_LOGIC := '0';
signal AngleReal    : REAL := 0.0;

constant clk_period : TIME := 1ns;

signal XValue : INTEGER RANGE -256 TO 255 := 100;
signal YValue : INTEGER RANGE -256 TO 255 := 200;
```

Code (7/8)

```
begin
dut : CordicE PORT MAP(Reset      => Reset,
                        Clk        => Clk,
                        ChipSelect => ChipSelectSig,
                        XIn        => XInSig,
                        YIn        => YInSig,
                        Aufloesung  => AufloesungSig,
                        Modus       => ModusSig,
                        ChipReady  => ChipReadySig,
                        DataOut    => DataOutSig);
-- Clock Erzeugung
clk_process : process
begin
    Clk <= '0';
    wait for clk_period/2;
    Clk <= '1';
    wait for clk_period/2;
end process clk_process;
reset_process : process
begin
    Reset <= '0';

    wait for clk_period * 4;

    Reset <= '1';
    wait;

end process reset_process;
```

Code (8/8)

```
stim_process : process
begin
    XInSig <= STD_LOGIC_VECTOR(SHIFT_LEFT(TO_SIGNED(XValue, 17), 8)) ;
    YInSig <= STD_LOGIC_VECTOR(SHIFT_LEFT(TO_SIGNED(YValue, 17), 8)) ;

    AufloesungSig <= "10000";
    ChipSelectSig <= '1';

    wait for clk_period * 20;

    XInSig <= "00000000000000001";
    YInSig <= "00000000000000010";

    AufloesungSig <= "10000";

    XValue <= XValue;
    YValue <= YValue - 10;

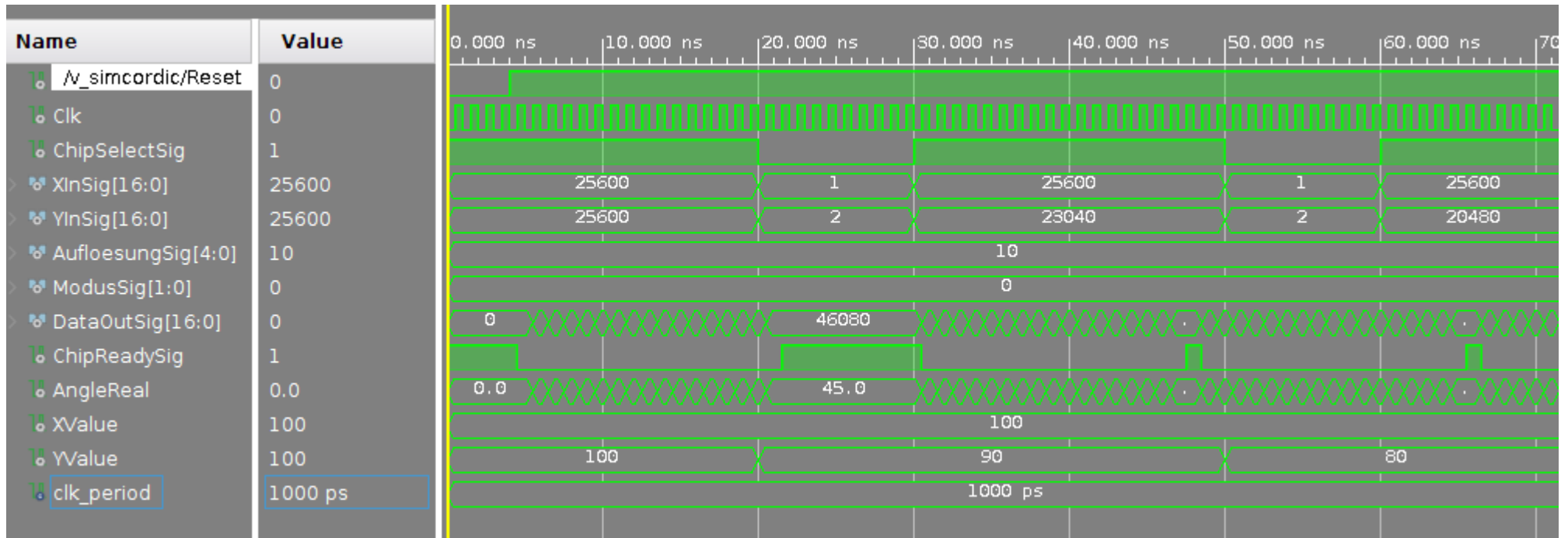
    ChipSelectSig <= '0';

    wait for clk_period * 10;
end process stim_process;

AngleReal <= REAL(TO_INTEGER(SIGNED(DataOutSig))) / 2**10;
end Behavioral;
```

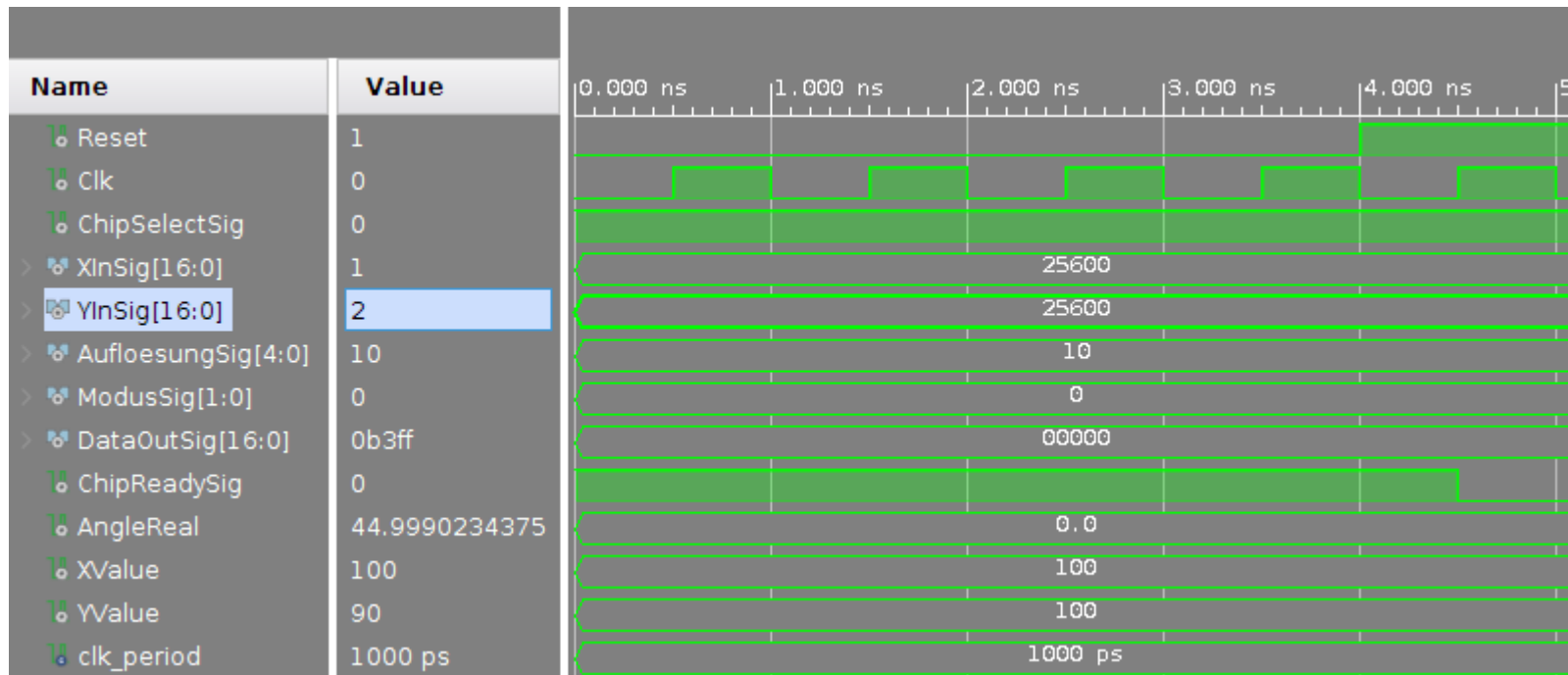

Simulation (1/8)

- XValue und YValue geben die Eingaben im Bereich -256 ... +255 vor
- XinSig und YinSig sind die Entity Signale die hier als Signed Integer dargestellt werden
- AngleReal gibt den floating Point Winkel aus der Testbench aus
- ChipSelektSig aktiviert die Berechnung, ChipReadySig zeigt an, dass das Ergebnis berechnet worden ist

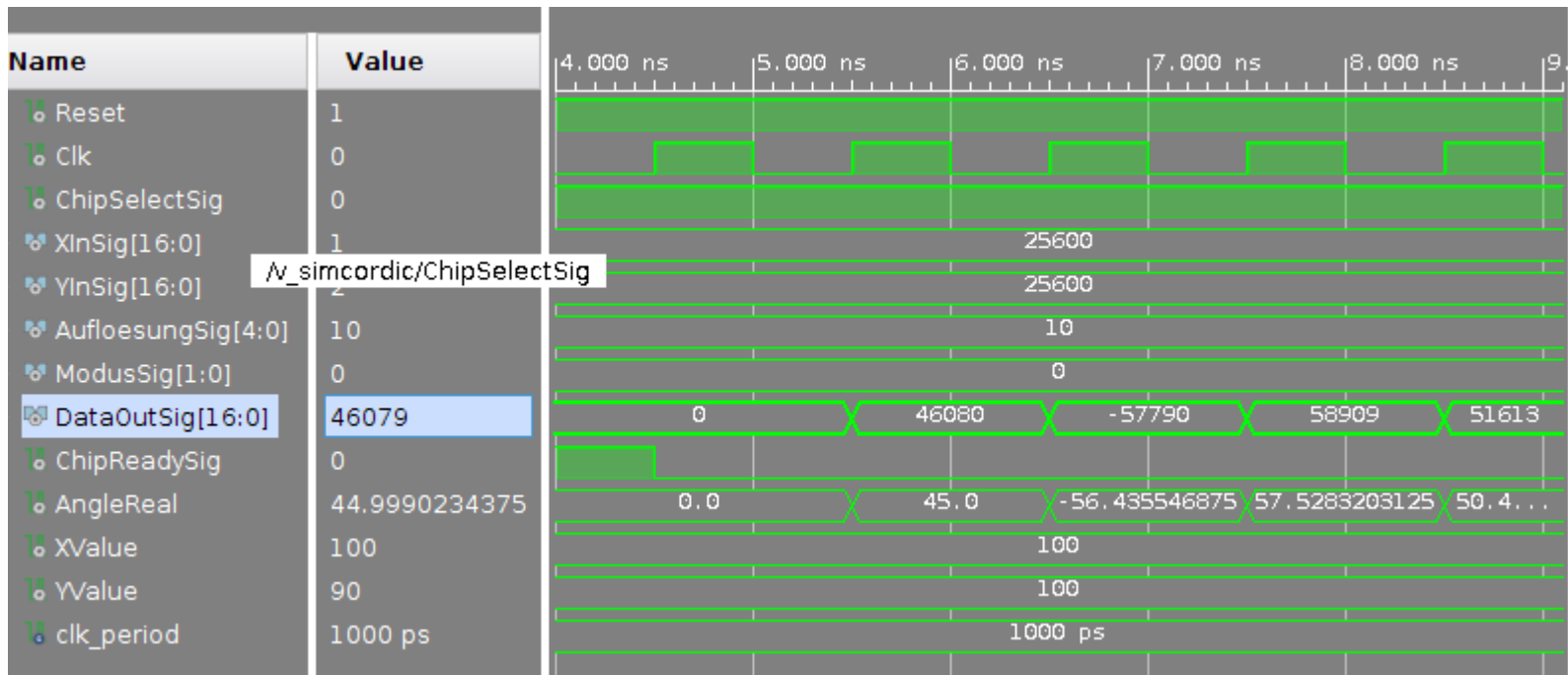


Simulation (2/8)

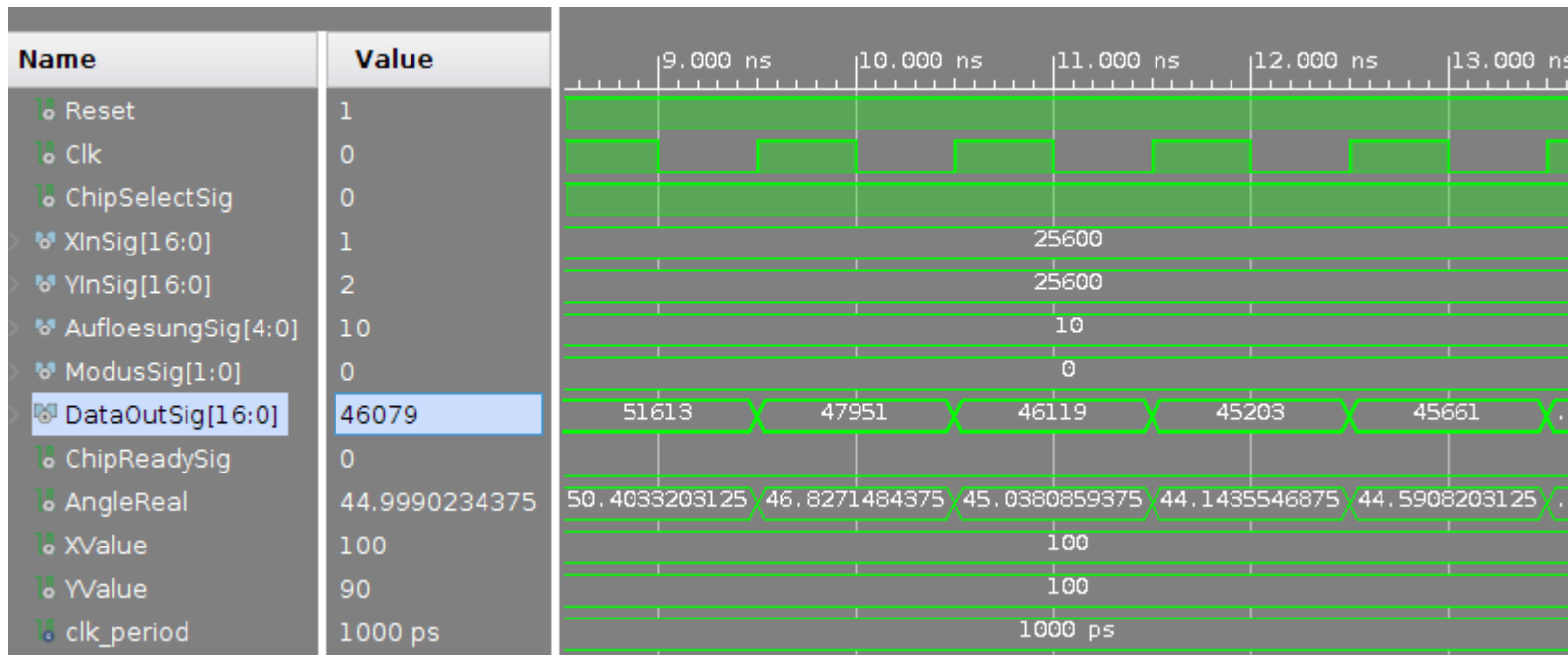
- XValue und YValue geben die Eingaben im Bereich -256 ... +255 vor
- XinSig und YinSig sind die Entity Signale die hier als Signed Integer dargestellt werden
- AngleReal gibt den floating Point Winkel aus der Testbench aus
- ChipSelektSig aktiviert die Berechnung, ChipReadySig zeigt an, dass das Ergebnis berechnet worden ist



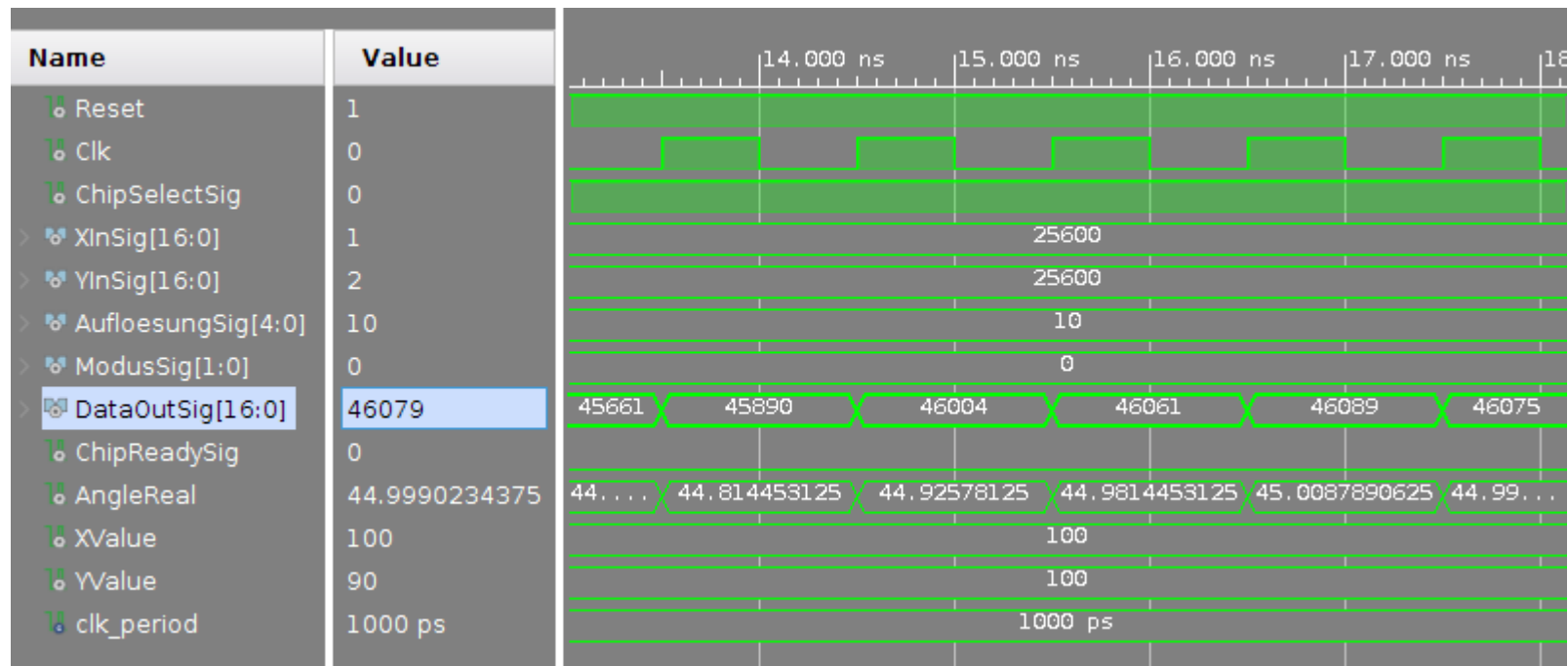
Simulation (2/6)



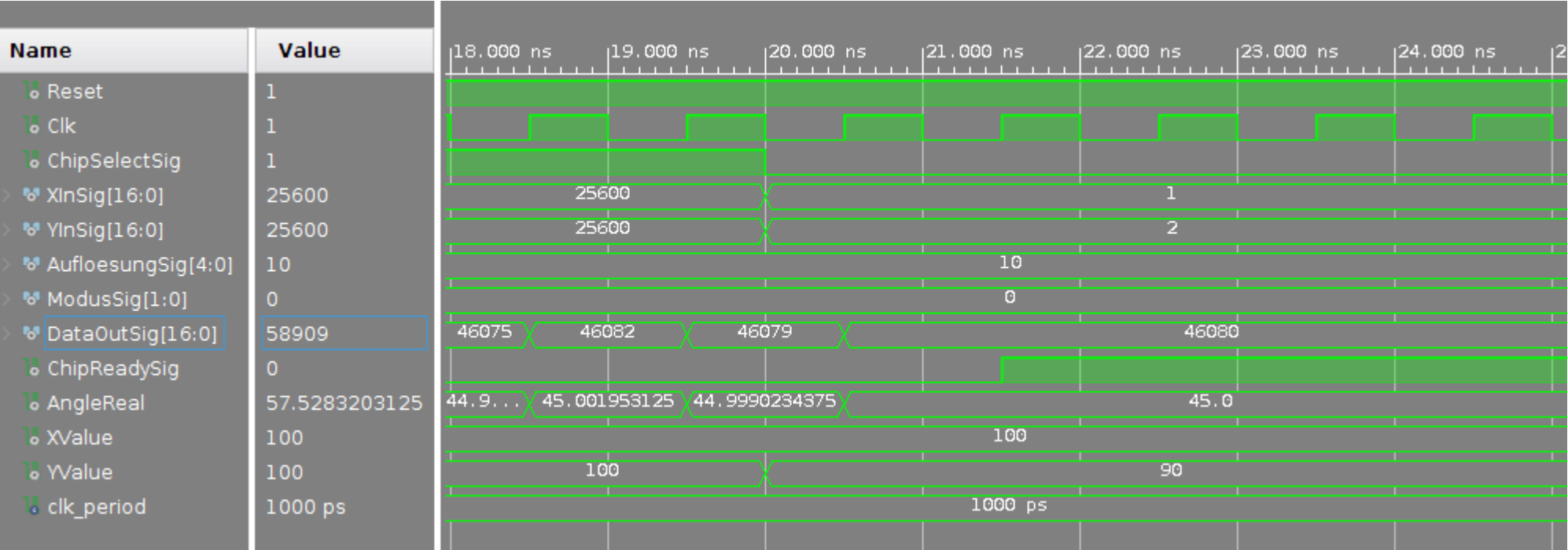
Simulation (4/8)



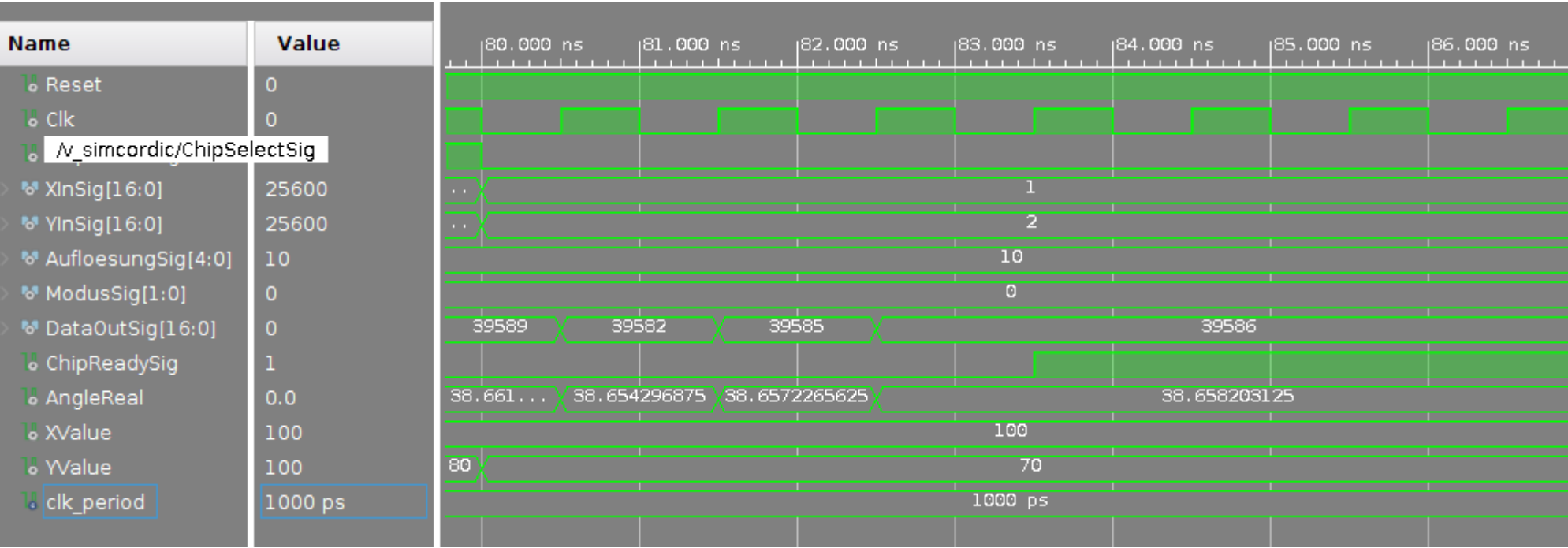
Simulation (5/8)



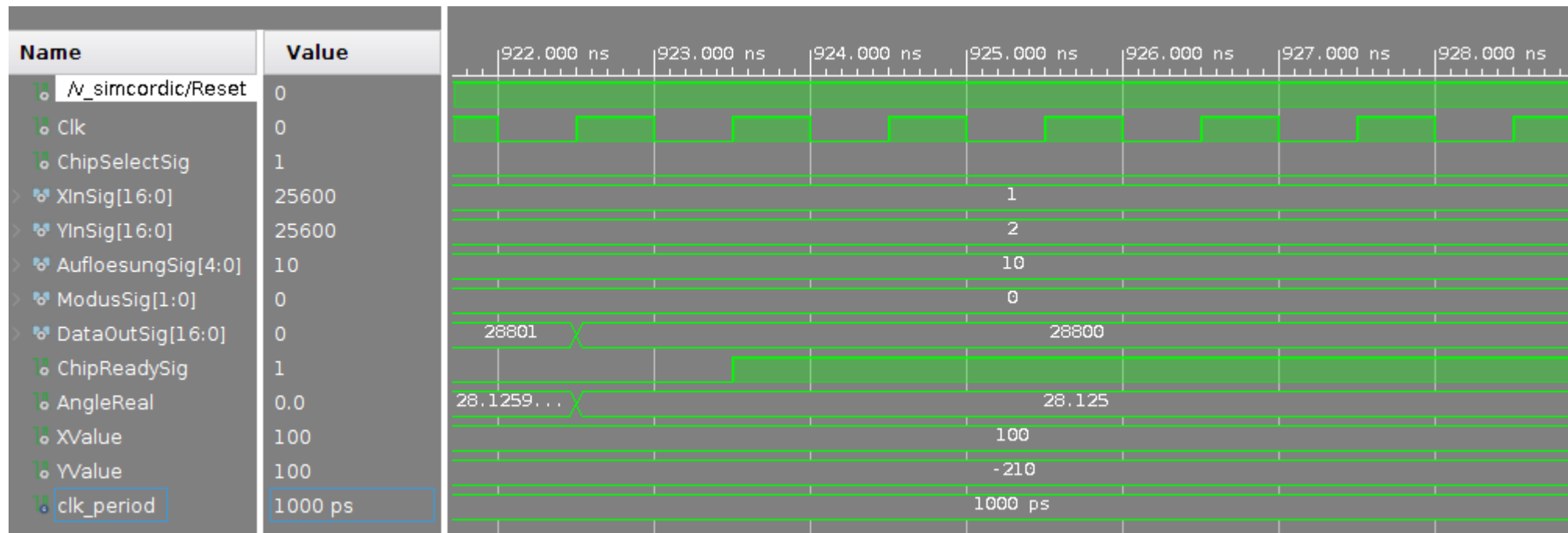
Simulation (6/8)



Simulation (7/8)



Simulation (8/8)



Kapitel 9

FIR und IIR Filter

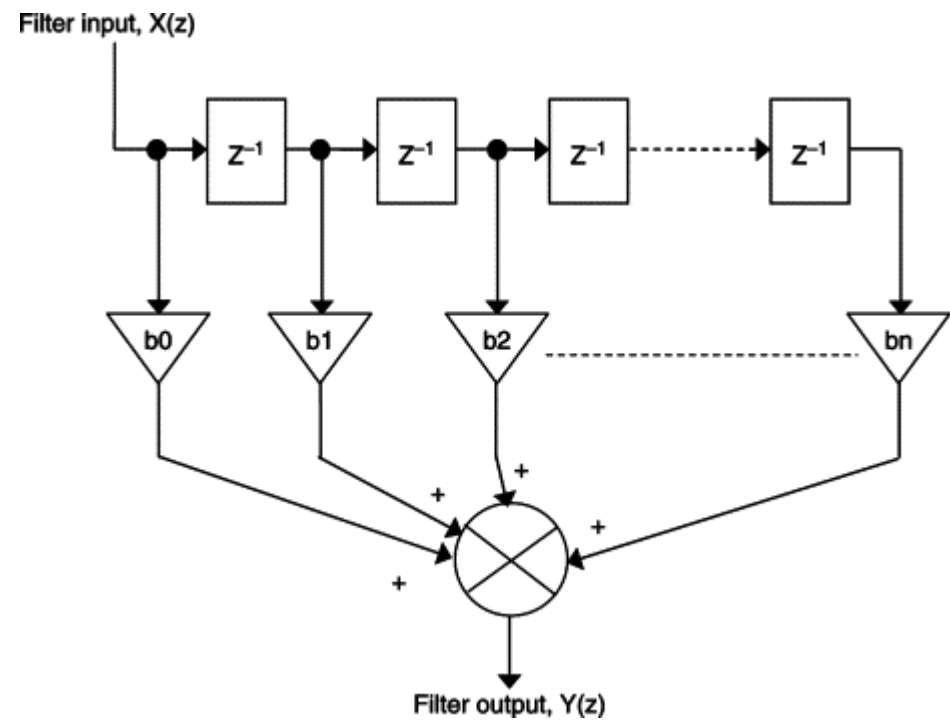
Zeitdiskretes System



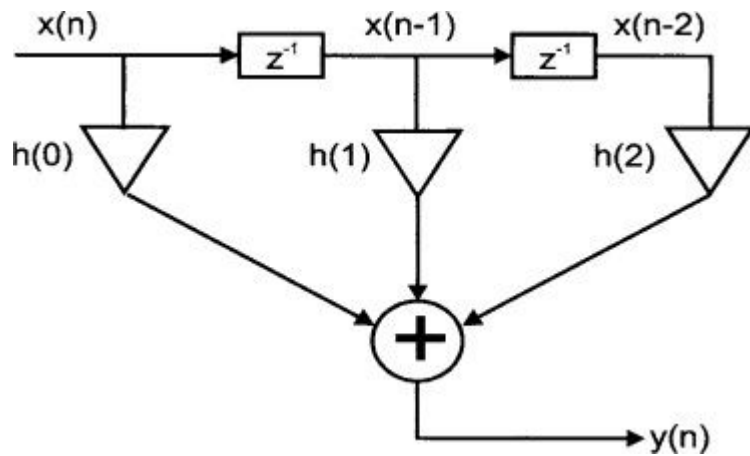
Filtergleichung für allgemeine FIR Filter

FIR – Finite Impuls Response Filter

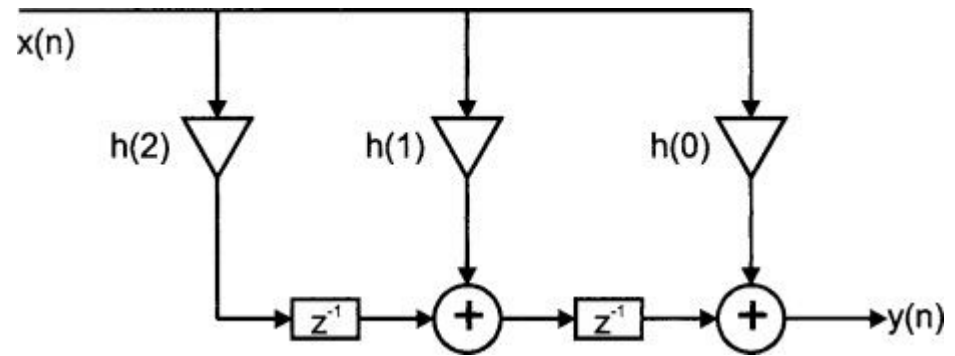
$$y[n] = \sum_{k=0}^M b_k x[n - k]$$



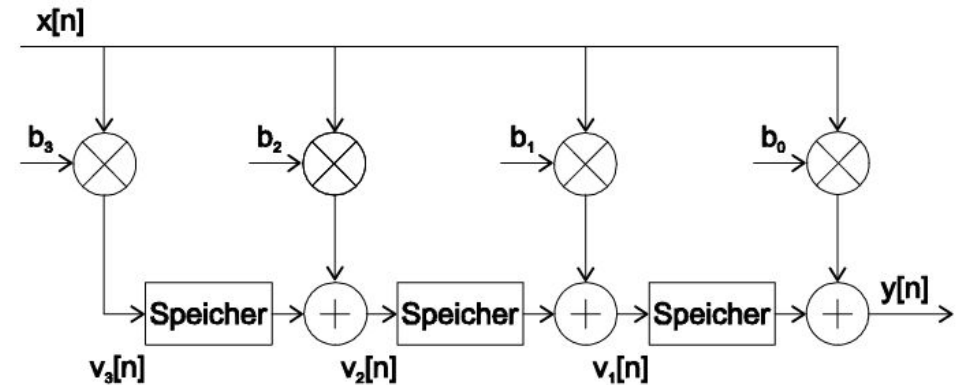
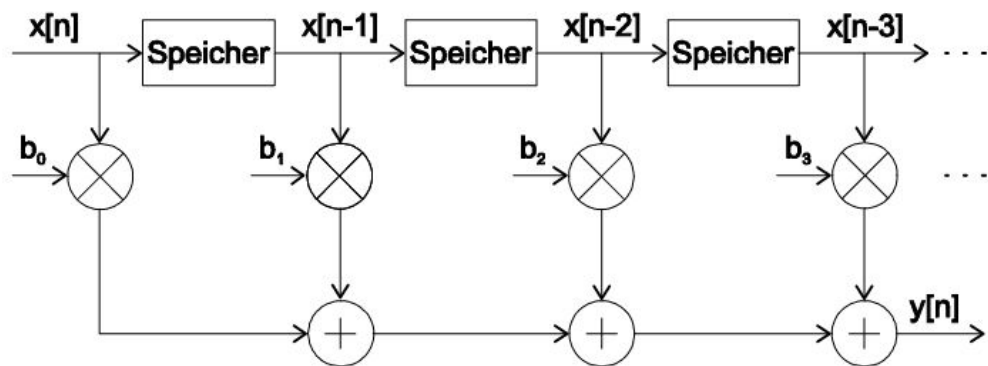
Direktform und transponiertes FIR Filter



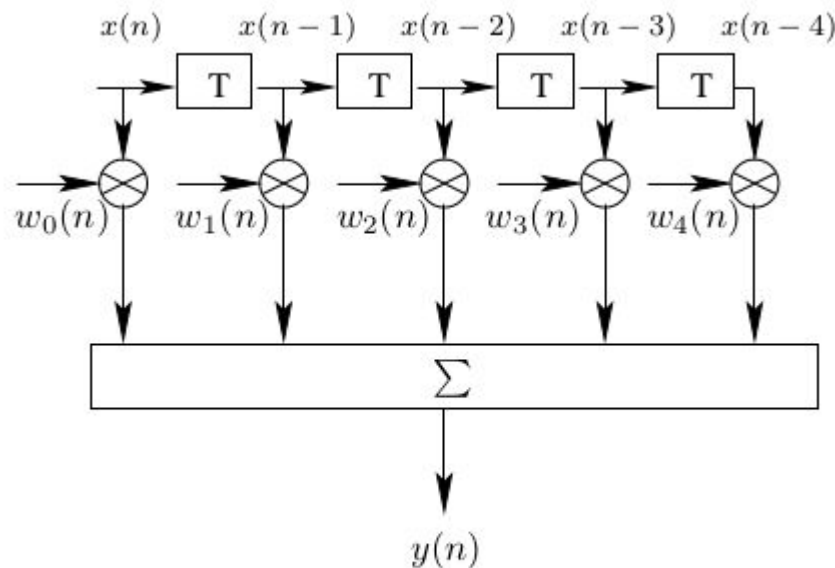
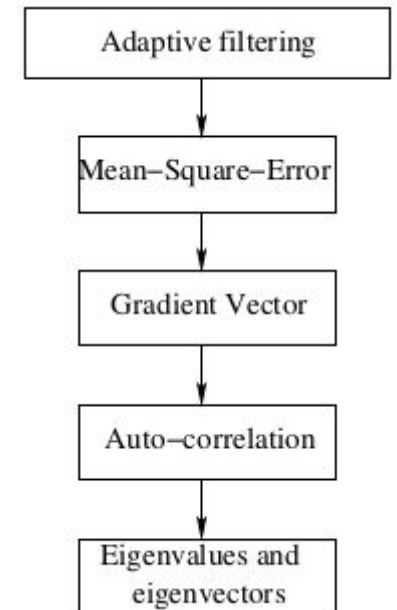
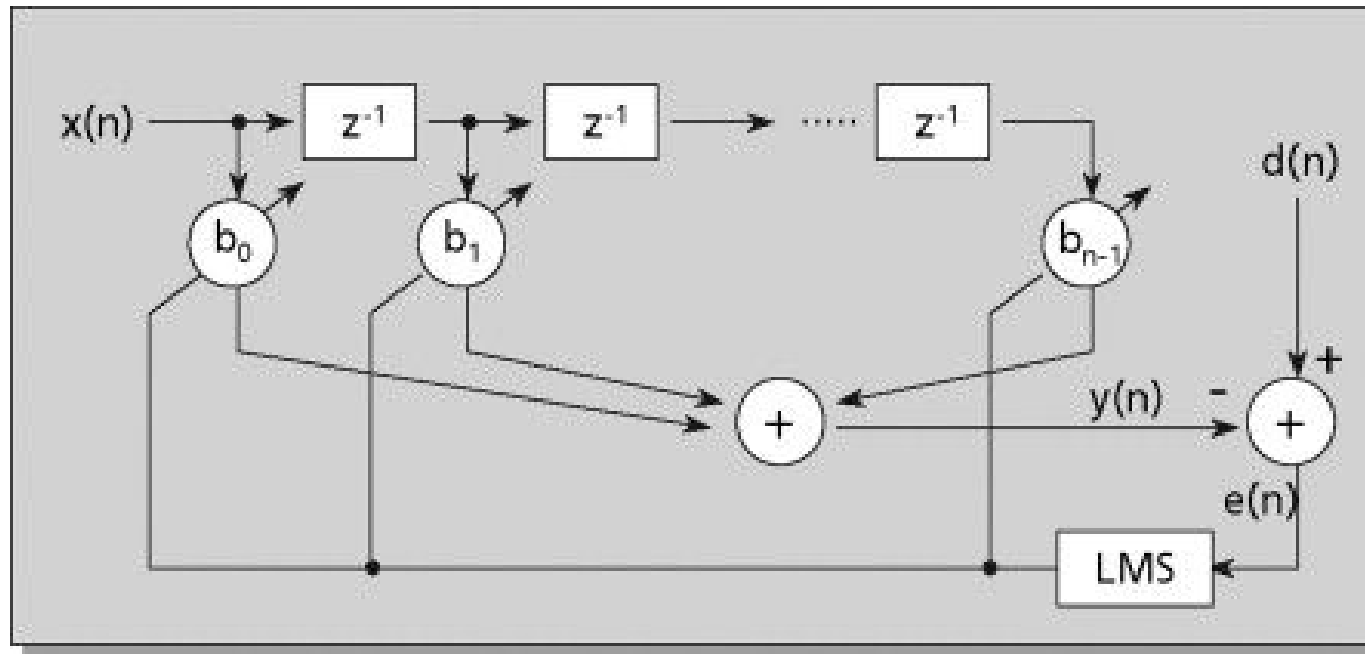
Direktform des FIR



Transponiertes FIR

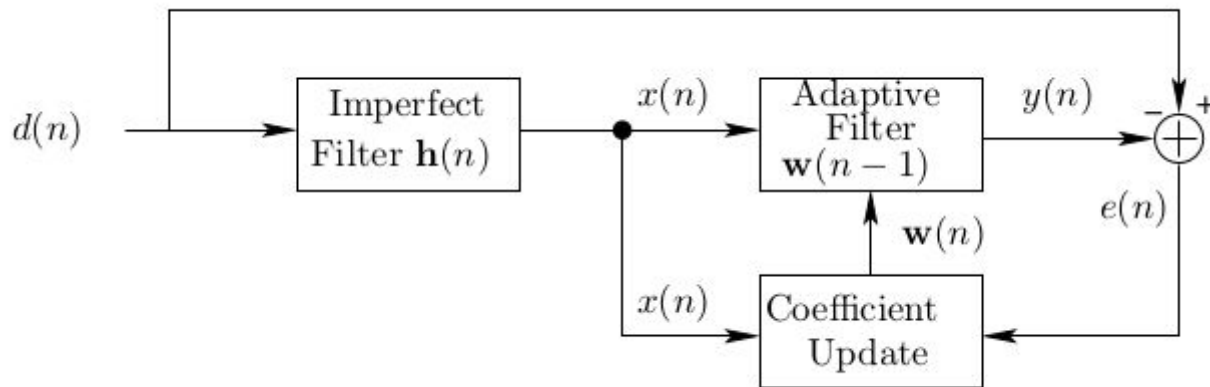


Least Mean Square



$$y(n) = \sum_{i=0}^{N-1} w_i(n) \cdot x(n-i) = \mathbf{w}^T(n) \cdot \mathbf{x}(n)$$

Least Mean Square



$$\mathbf{w}(n) = \mathbf{w}(n-1) + \mu \cdot e(n) \cdot \mathbf{x}(n)$$

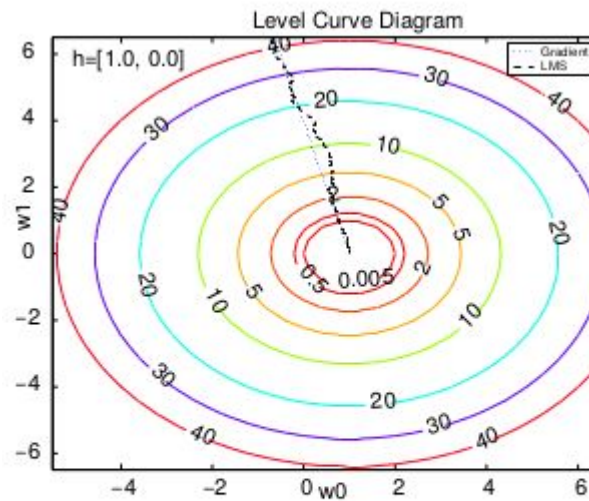


Figure 2.13: Level curve diagram with ideal transfer vector $\mathbf{h} = (1.0, 0.0)$.

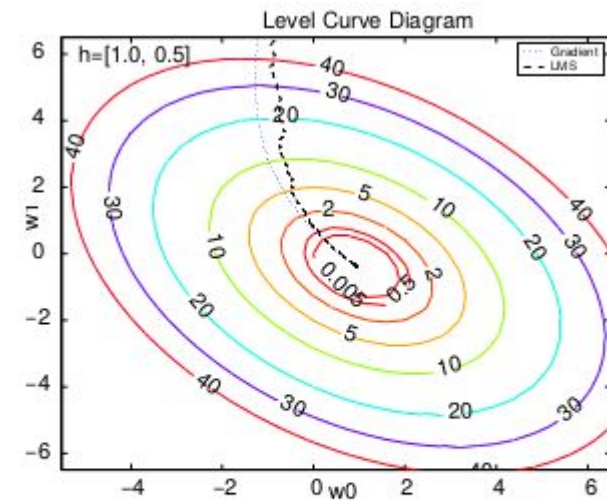
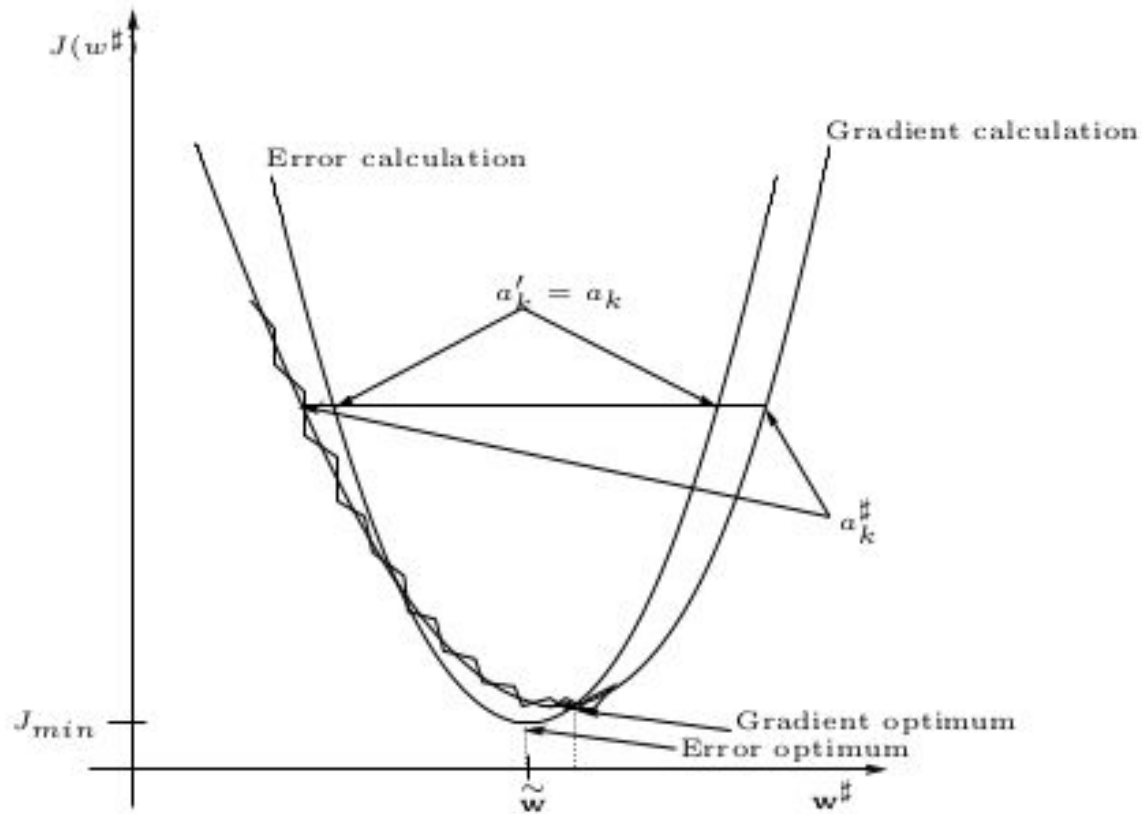


Figure 2.14: Level curve diagram with non-ideal transfer vector $\mathbf{h} = (1.0, 0.5)$.

Least Mean Square



21: Parabola for error calculation and gradient calculation.

Impulsantwort

- Die Impulsantwort eines FIR-Filters setzt sich aus dessen Filterkoeffizienten zusammen

- Einheitsimpuls $\delta_0[n] = \begin{cases} 1 & n = 0 \\ 0 & n \neq 0 \end{cases}$

- Impulsantwort $y[n] = h[n] = \sum_{k=0}^M b_k \delta_0[n - k] = \begin{cases} b_n & n = 0, 1, \dots, M \\ 0 & \text{sonst} \end{cases}$

- Das Anlegen eines Einheitspulses an ein FIR-Filter bewirkt, dass am Ausgang die Folge der Filterkoeffizienten b_k erscheint
- Da die Zahl der Koeffizienten endlich ist, ist auch die Dauer der Impulsantwort endlich, daher der Name Finite Impulse-Response Filter

$$y[n] = \sum_{k=0}^M h[k] x[n - k]$$

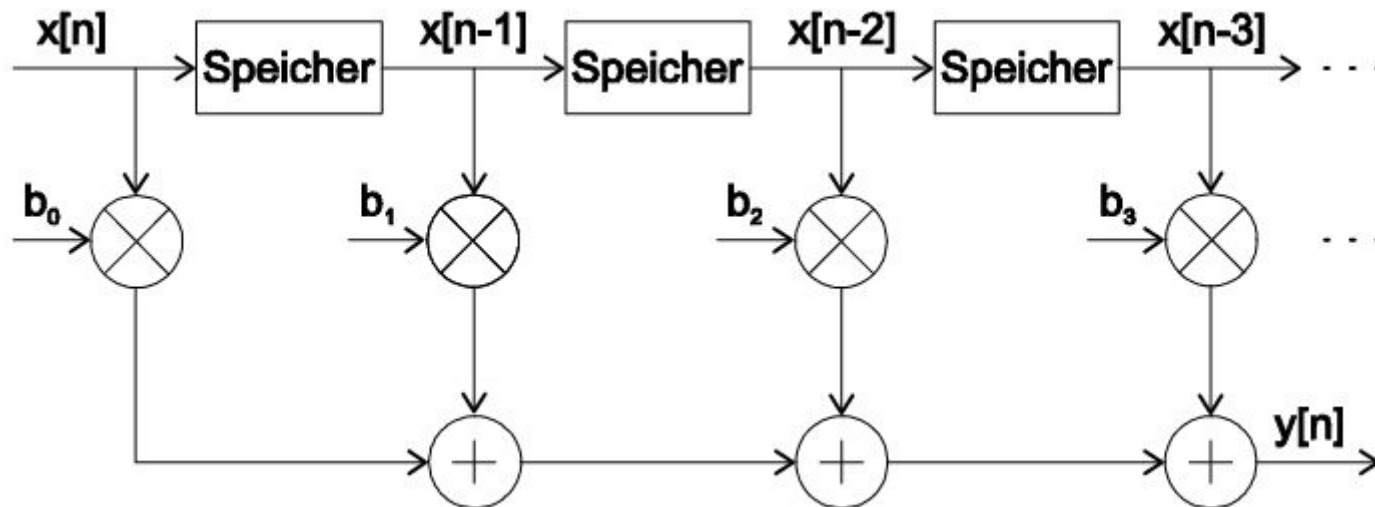
Faltungssumme

- Das Ausgangssignal entsteht durch Faltung des Eingangssignals mit der Impulsantwort

$$y[n] = \sum_{k=0}^M h[k]x[n-k]$$

- Kurzschreibweise

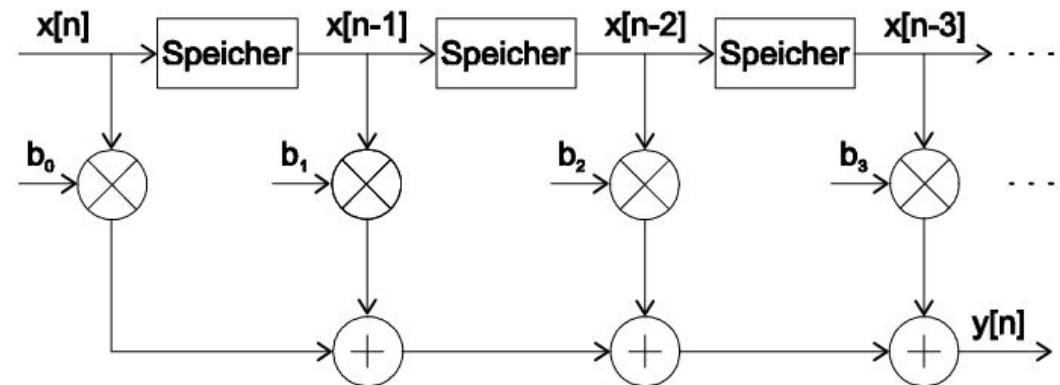
$$y[n] = x[n] * h[n]$$



Faltungssumme Beispiel

$$x = [0, 1, 2, 3, 4, 0, 0, 0, 0]$$

$$b = [1, -1, 2, -3]$$



$$y[0] = 0 * 1 + 0 * (-1) + 0 * 2 + 0 * (-3) = 0$$

$$y[1] = 1 * 1 + 0 * (-1) + 0 * 2 + 0 * (-3) = 1$$

$$y[2] = 2 * 1 + 1 * (-1) + 0 * 2 + 0 * (-3) = 2 - 1 = 1$$

$$y[3] = 3 * 1 + 2 * (-1) + 1 * 2 + 0 * (-3) = 3 - 2 + 2 = 3$$

$$y[4] = 4 * 1 + 3 * (-1) + 2 * 2 + 1 * (-3) = 4 - 3 + 4 - 3 = 2$$

$$y[5] = 0 * 1 + 4 * (-1) + 3 * 2 + 2 * (-3) = 0 - 4 + 6 - 6 = -4$$

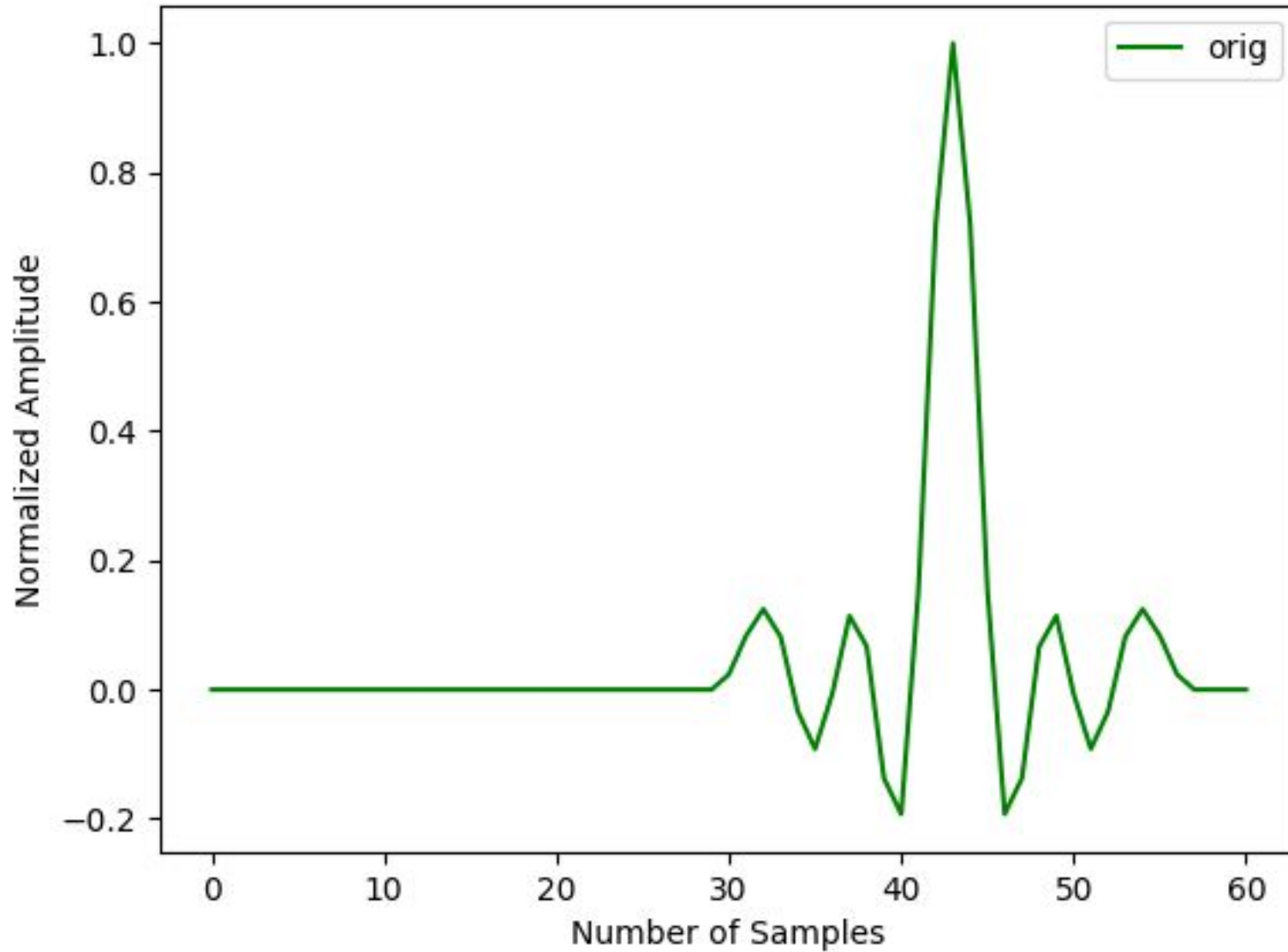
$$y[6] = 0 * 1 + 0 * (-1) + 4 * 2 + 3 * (-3) = 0 - 0 + 8 - 9 = -1$$

$$y[7] = 0 * 1 + 0 * (-1) + 0 * 2 + 4 * (-3) = 0 - 0 + 0 - 12 = -12$$

$$y[8] = 0 * 1 + 0 * (-1) + 0 * 2 + 0 * (-3) = 0 - 0 + 0 - 0 = 0$$

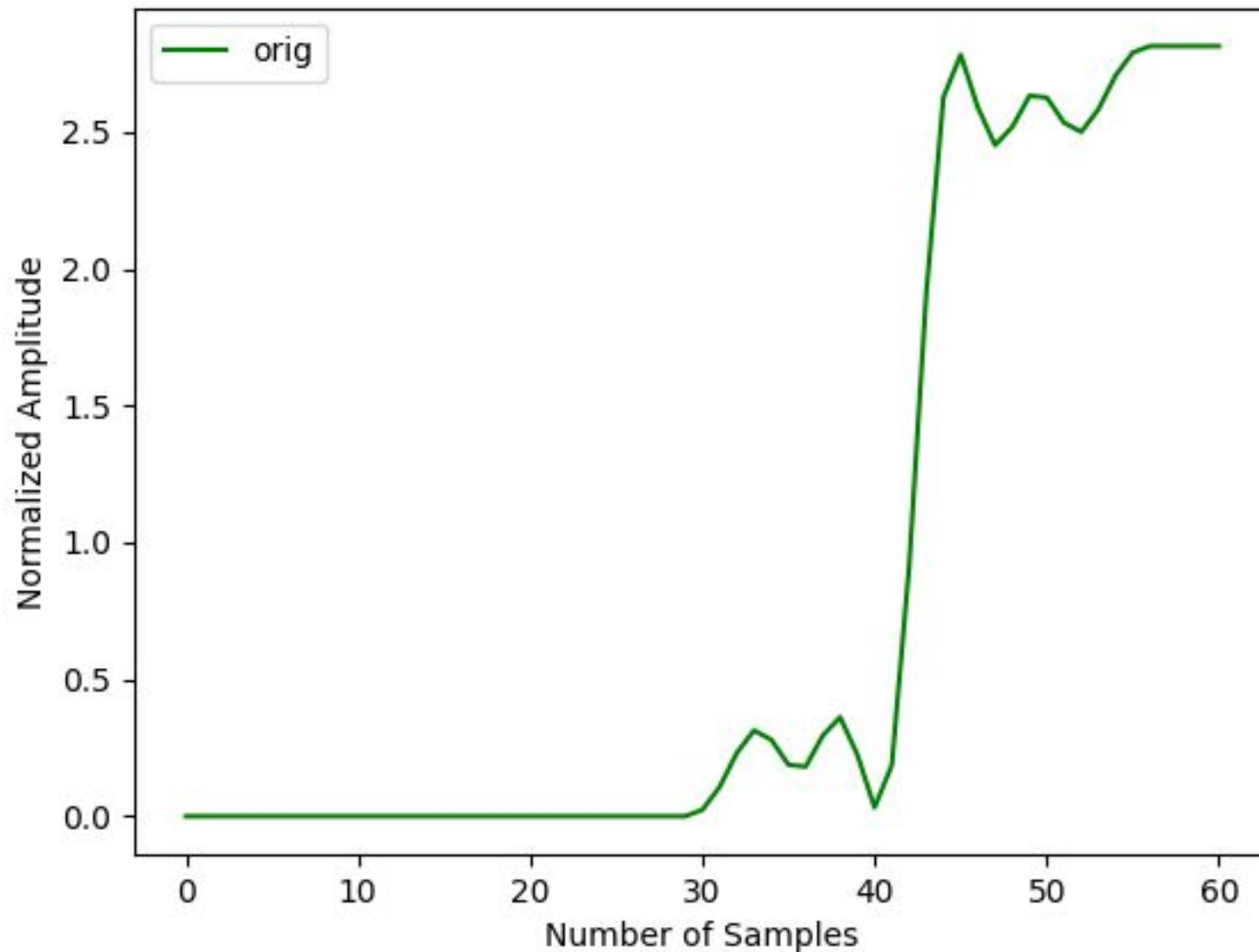
Impulsantwort

- Impulsantwort des Filters bei einem Dirac-Impuls 0 0 0 0 0 1 0 0 0 0



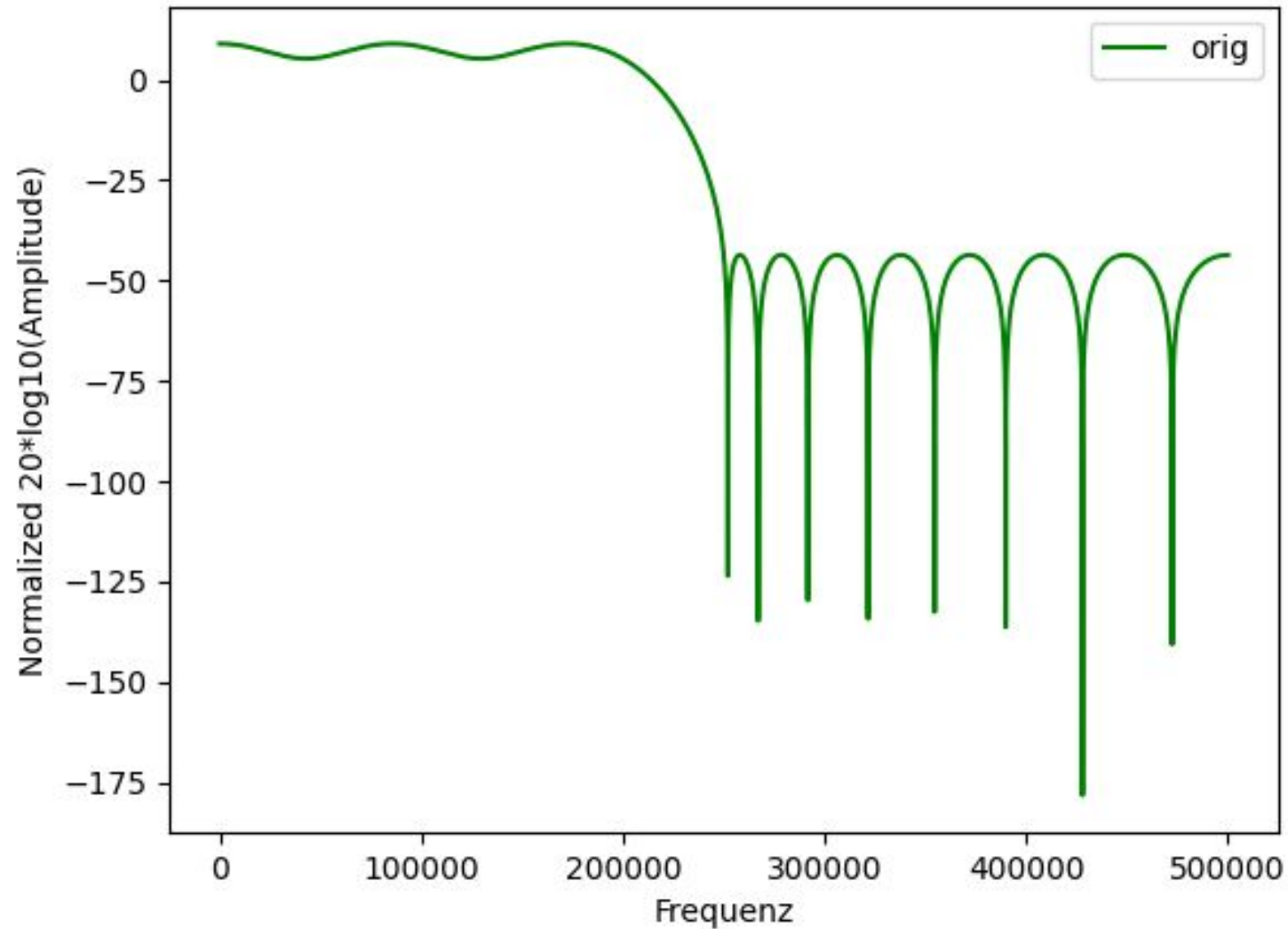
Sprungantwort

- Sprungantwort des Filters bei einem Dirac-Impuls 0 0 0 0 0 1 1 1 1 1 1 1 1 1 1 1 1 1 ...



Frequenzgang von FIR Filtern

- Erwünschte Dämpfung liegt bei ca. -50dB



Filterdesign - Impulsantwort

- Entwurf der folgenden Filterkoeffizienten

[+ add passband](#) [+ add stopband](#) !predefined ▼

from	to	gain	ripple/att.	act. rpl	
0 Hz	400 Hz	1	5 dB	3.73 dB	
500 Hz	1000 Hz	0	-50 dB	-50.95 dB	

sampling freq.	2000 Hz
desired #taps	minimum
actual #taps	27

DESIGN FILTER

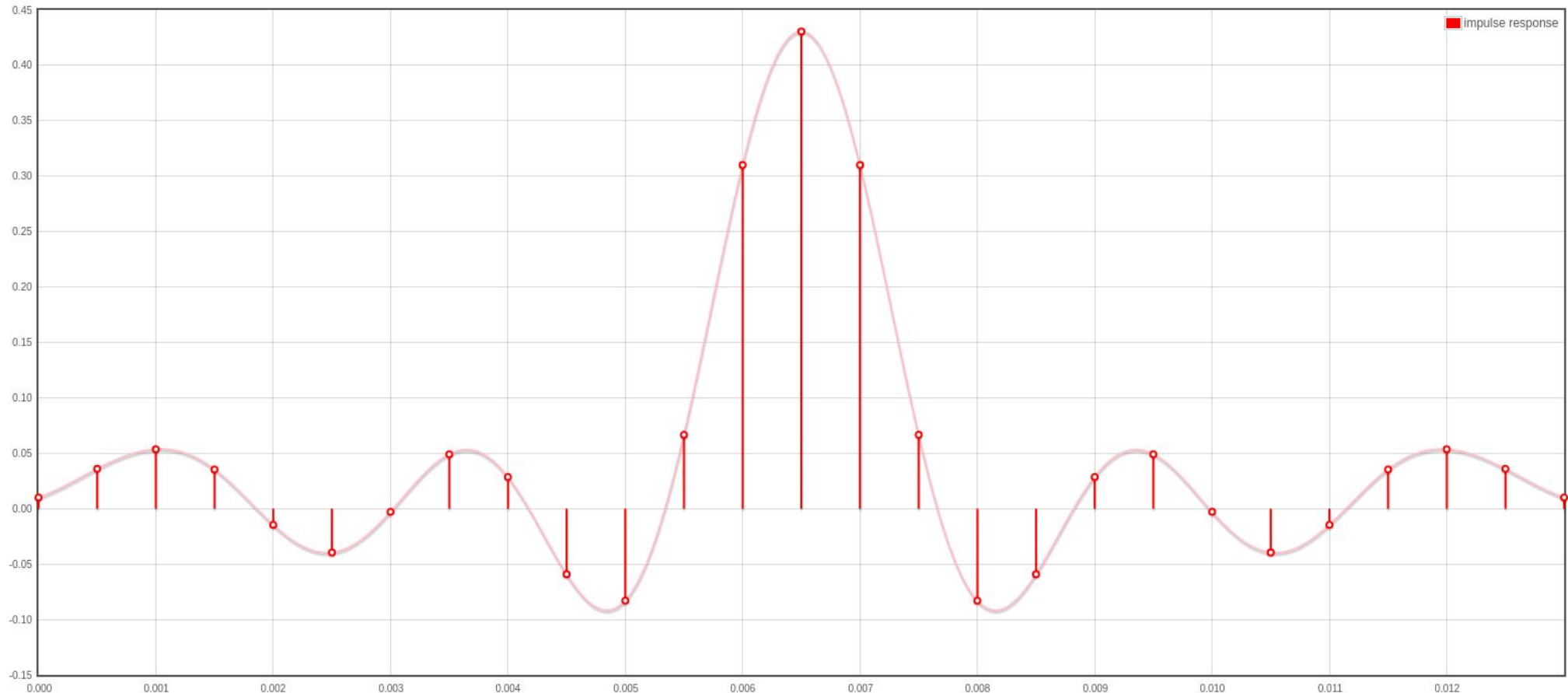
I am working on **TFilter2**. [Screenshot here](#). Features include CIC (Sinc) filters, effect of quantization, save/load/share, aliasing visualization, and signal chain.

If you want to **advertise here**, contact me at peterisza@gmail.com.

TFilter is being used by many tech companies and universities.

Gain vs. Frequency **Impulse Response** Source Code

[Feature Request](#) [Enterprise](#) [IIR](#)



Filterdesign - Frequenzgang

- Entwurf der folgenden Filterkoeffizienten

[+ add passband](#) [+ add stopband](#)

from	to	gain	ripple/att.	act. rpl	
0 Hz	400 Hz	1	5 dB	3.73 dB	
500 Hz	1000 Hz	0	-50 dB	-50.95 dB	

sampling freq.	2000 Hz
desired #taps	minimum
actual #taps	27

DESIGN FILTER

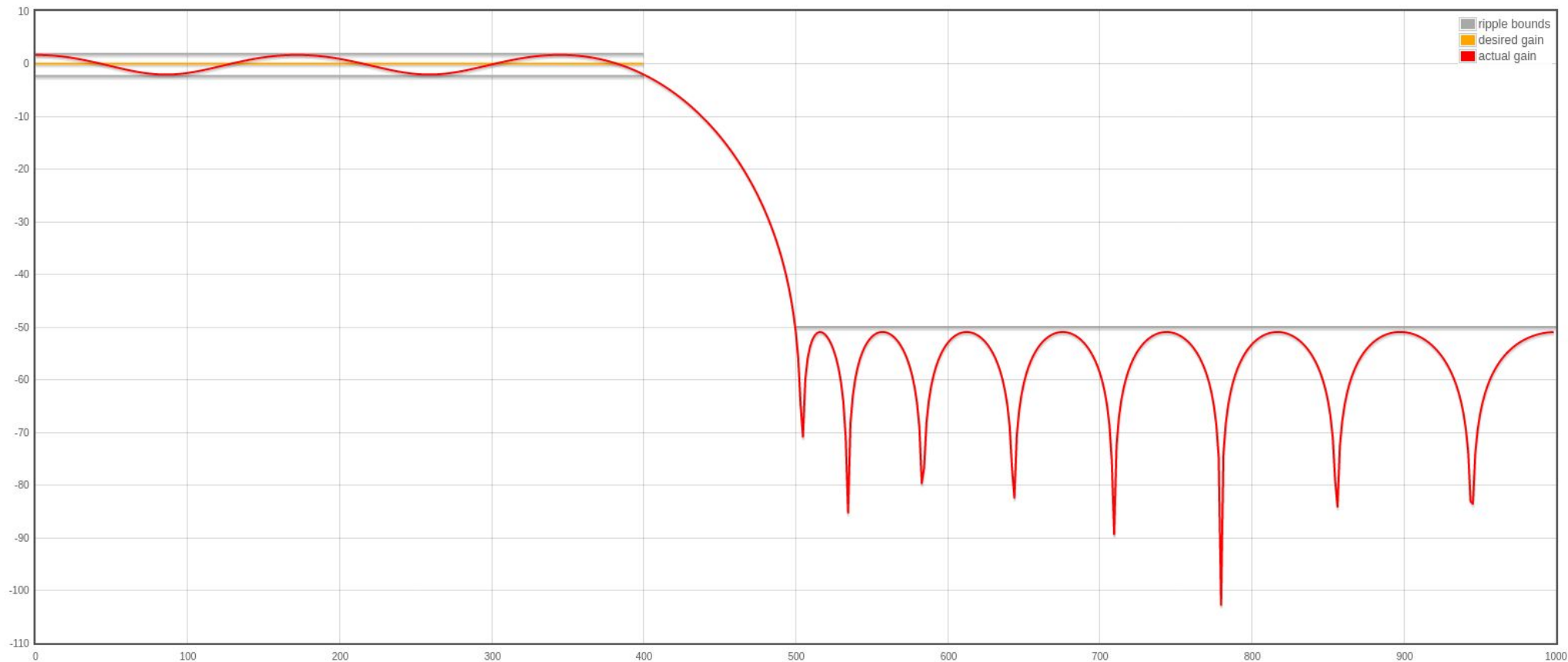
I am working on **TFilter2**. [Screenshot here](#). Features include CIC (Sinc) filters, effect of quantization, save/load/share, aliasing visualization, and signal chain.

If you want to **advertise here**, contact me at peterisza@gmail.com.

TFilter is being used by many tech companies and universities.

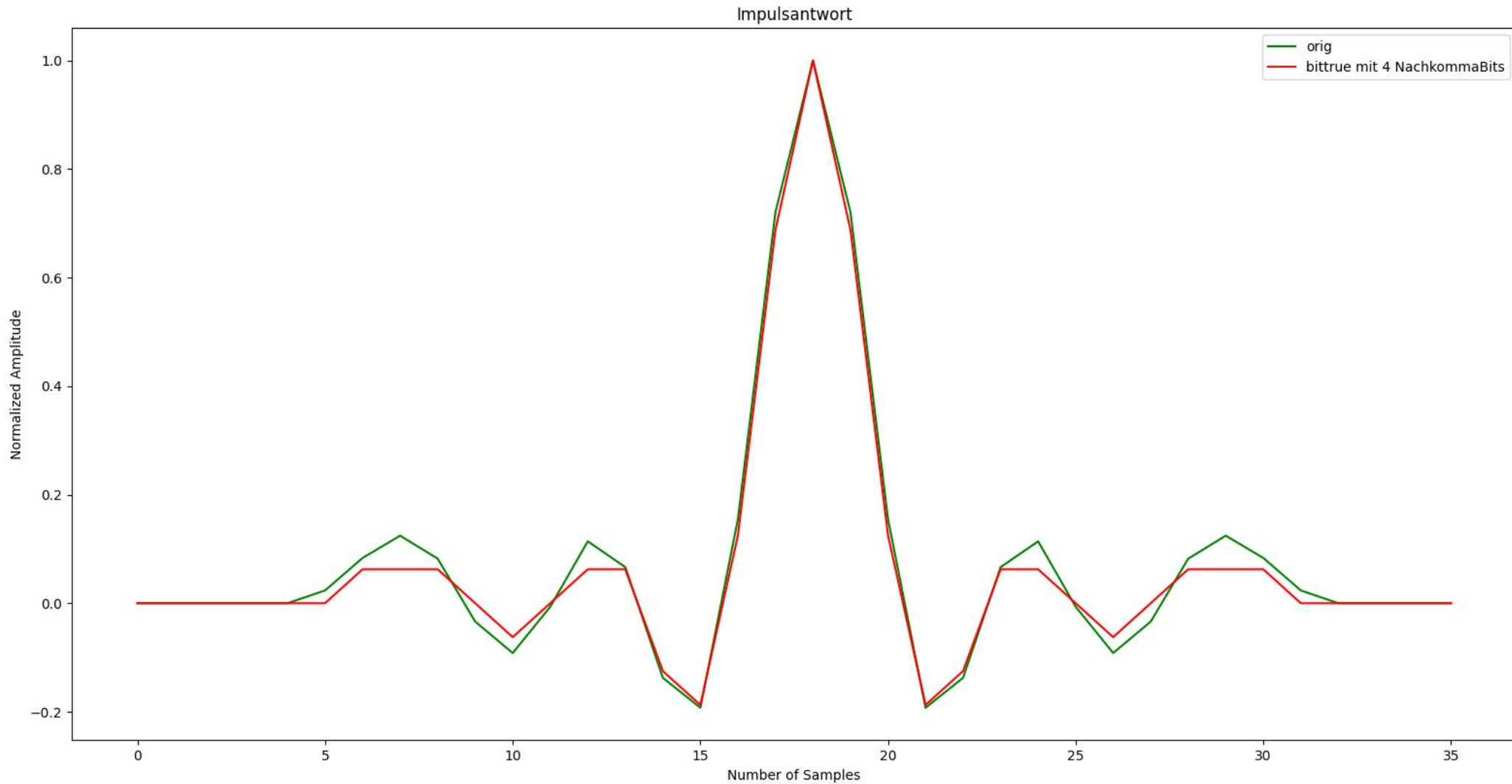
Gain vs. Frequency Impulse Response Source Code

[Feature Request](#) [Enterprise](#) [IIR](#)



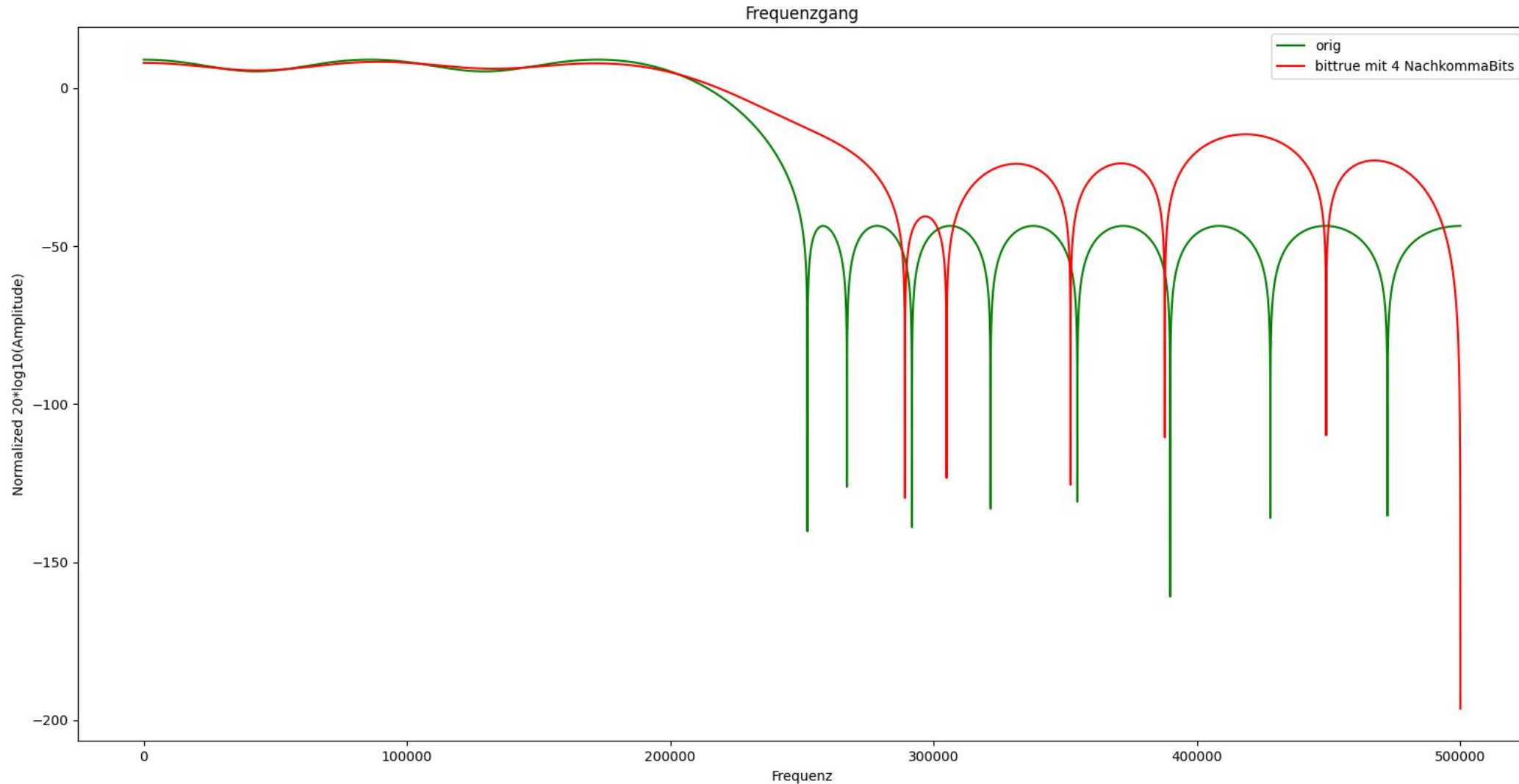
Filterdesign – Bittrue Modelling (1/12)

- Reduzierung der Fließkommastellen



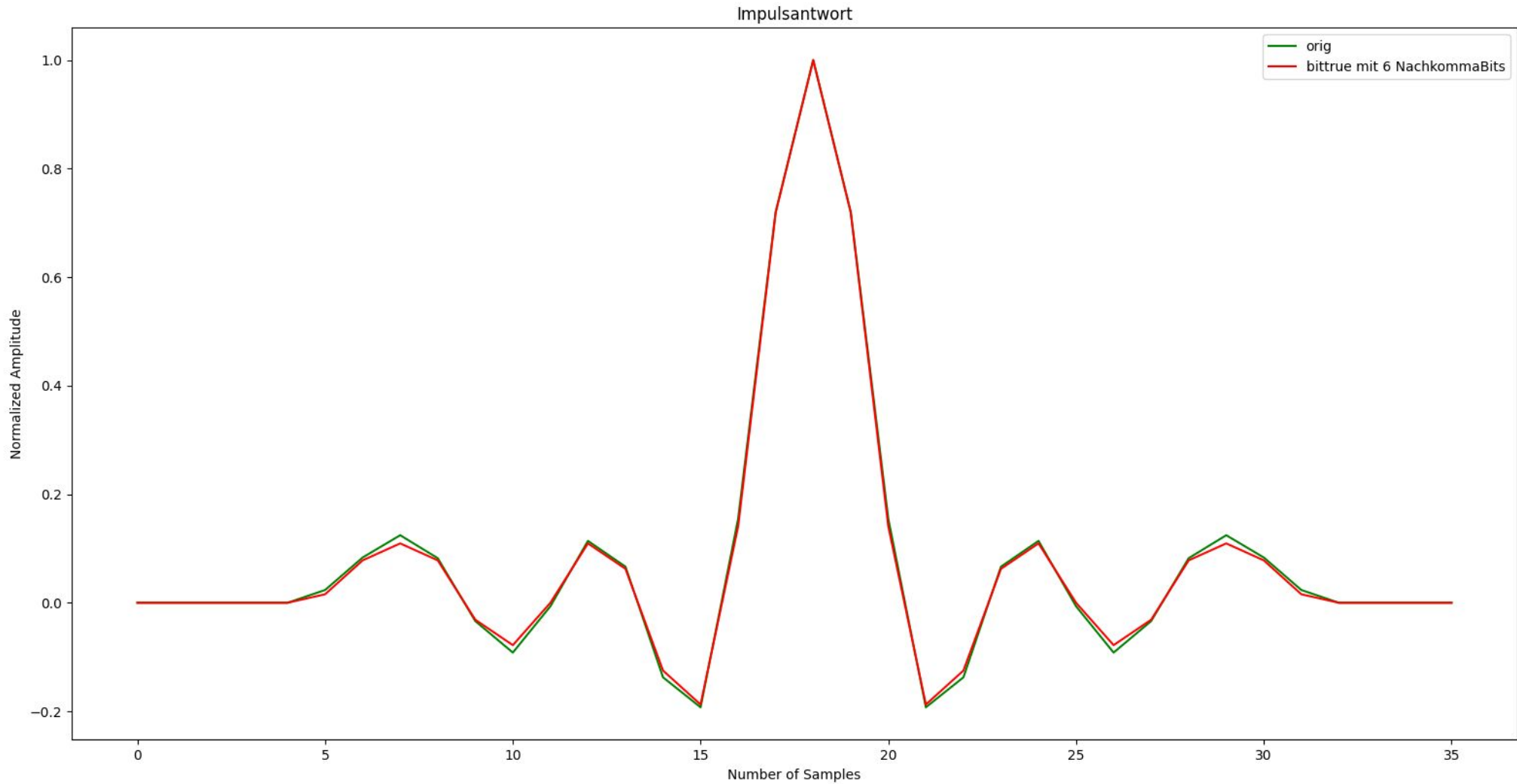
Filterdesign – Bittrue Modelling (2/12)

- Reduzierung der Fließsnachkommstellen



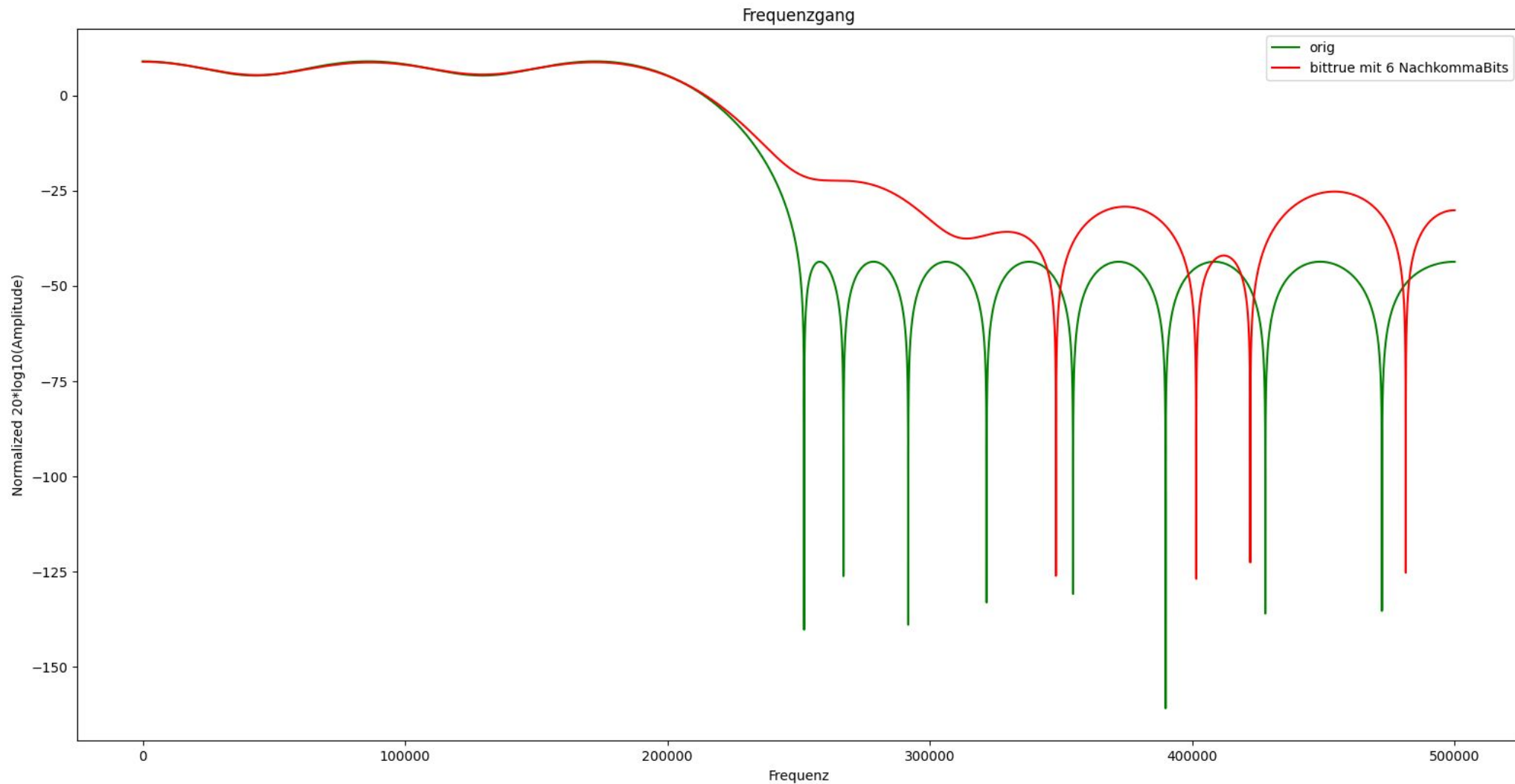
Filterdesign – Bittrue Modelling (3/12)

- Reduzierung der Fließsnachkommstellen



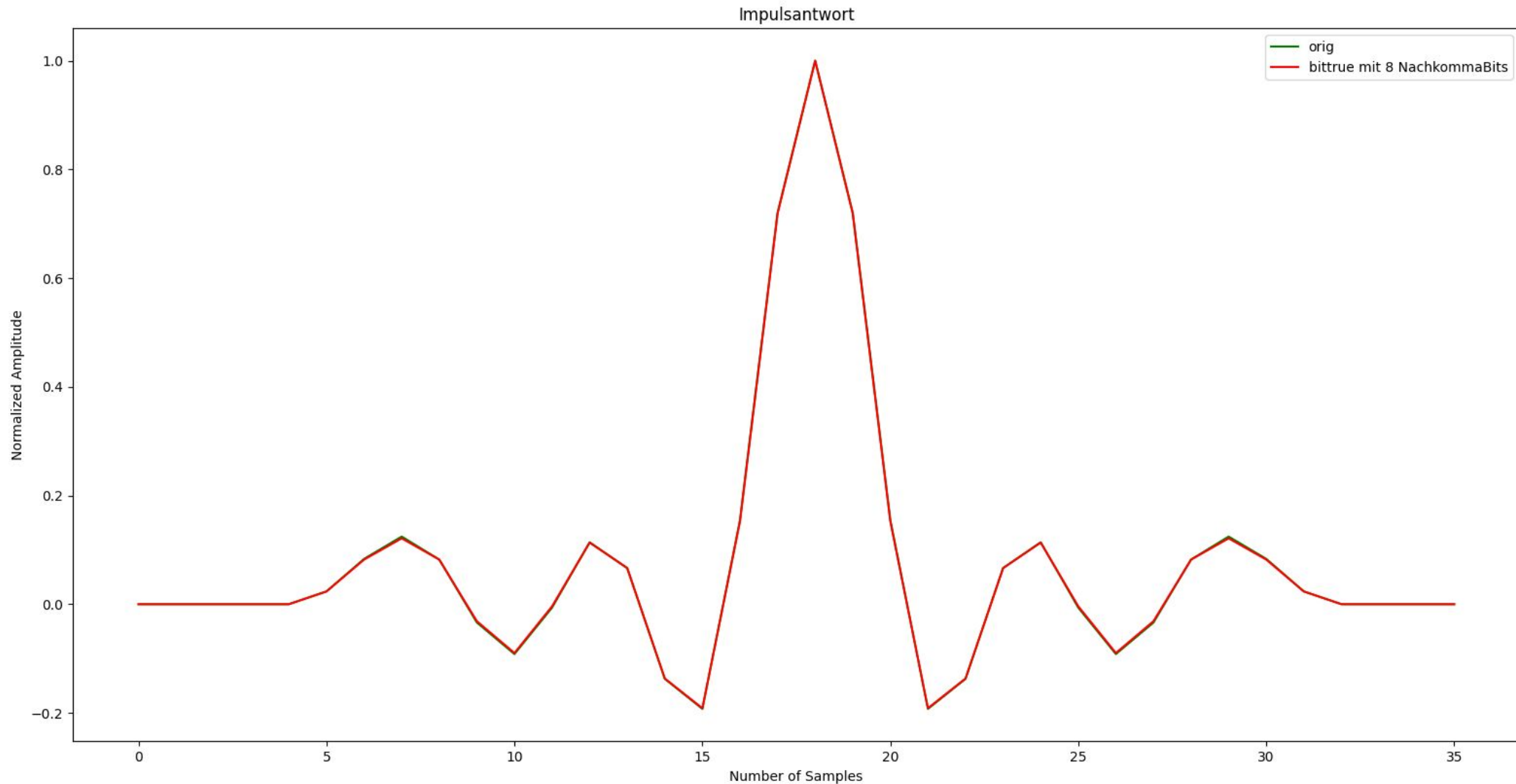
Filterdesign – Bittrue Modelling (4/12)

- Reduzierung der Fliessnachkommstellen



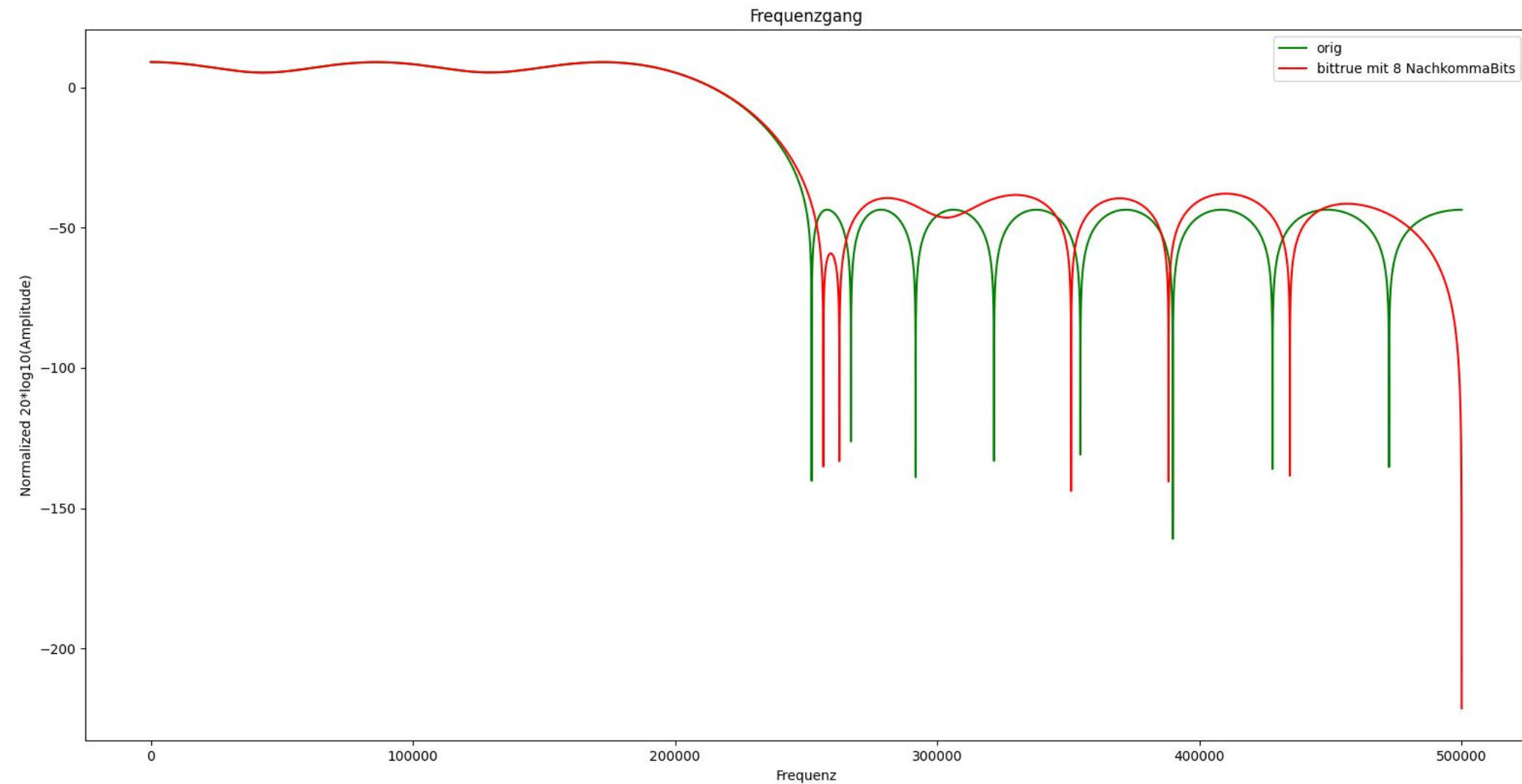
Filterdesign – Bittrue Modelling (5/12)

- Reduzierung der Fließsnachkommstellen



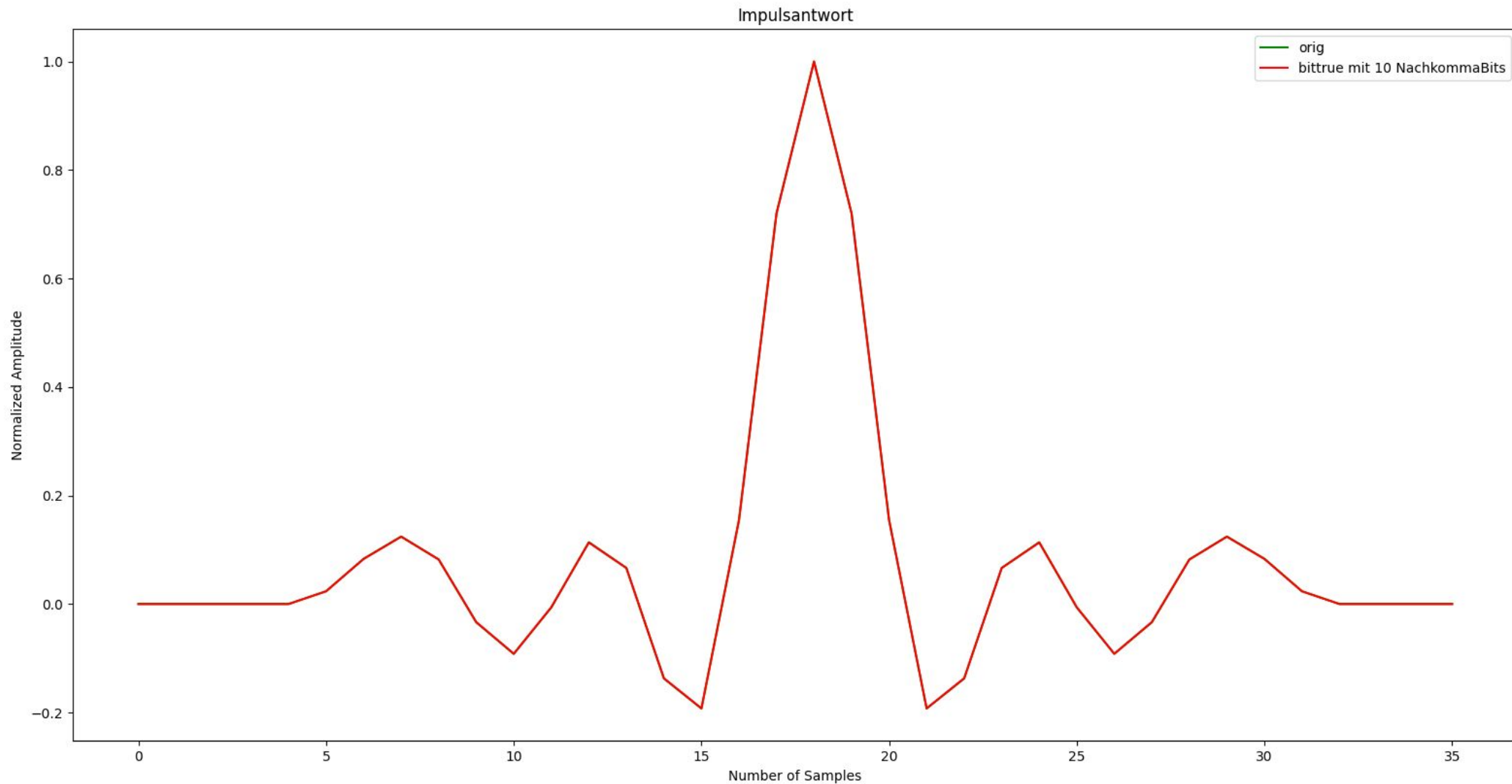
Filterdesign – Bittrue Modelling (6/12)

- Reduzierung der Fließkommastellen



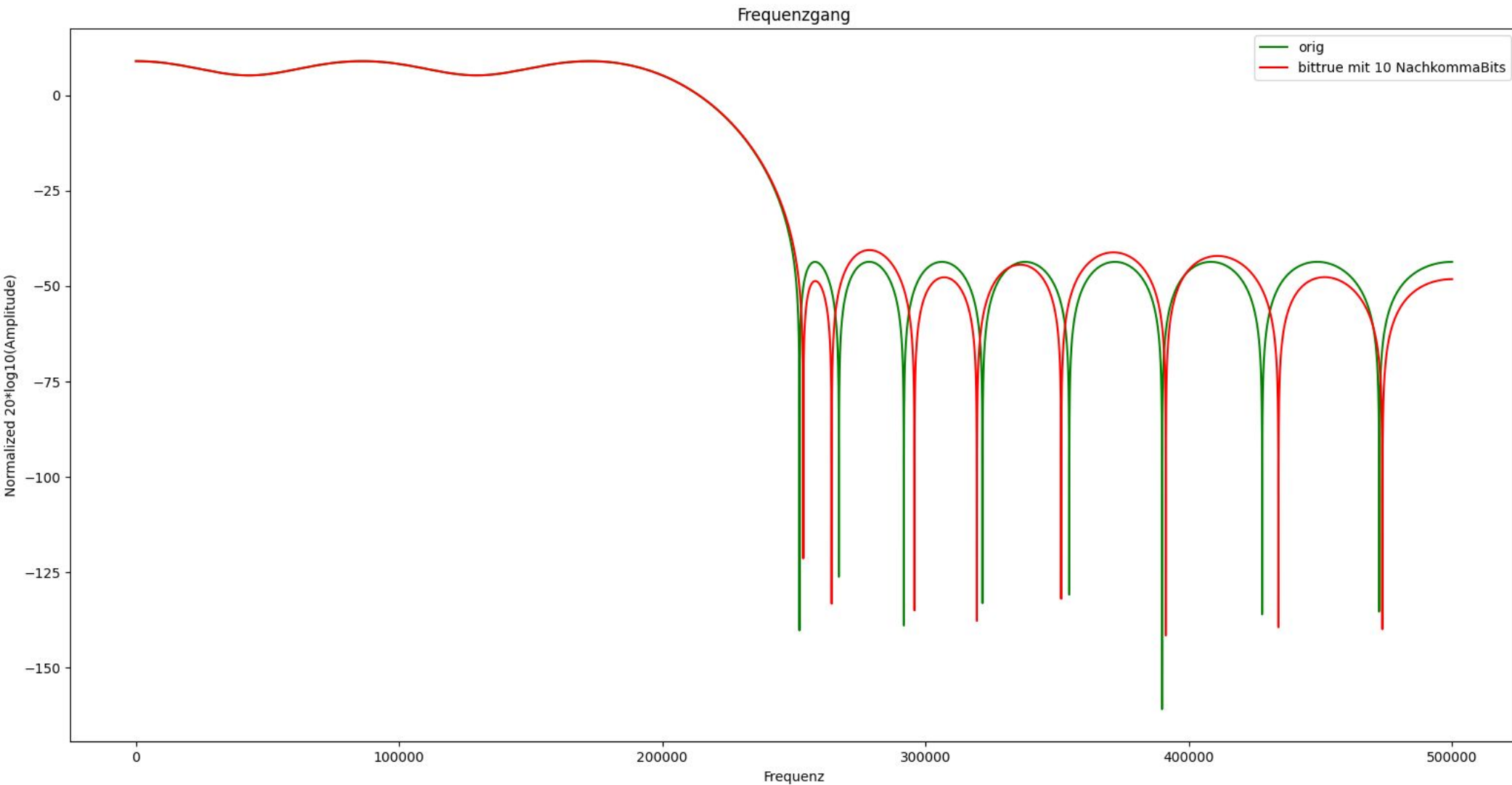
Filterdesign – Bittrue Modelling (7/12)

- Reduzierung der Fliessnachkommstellen



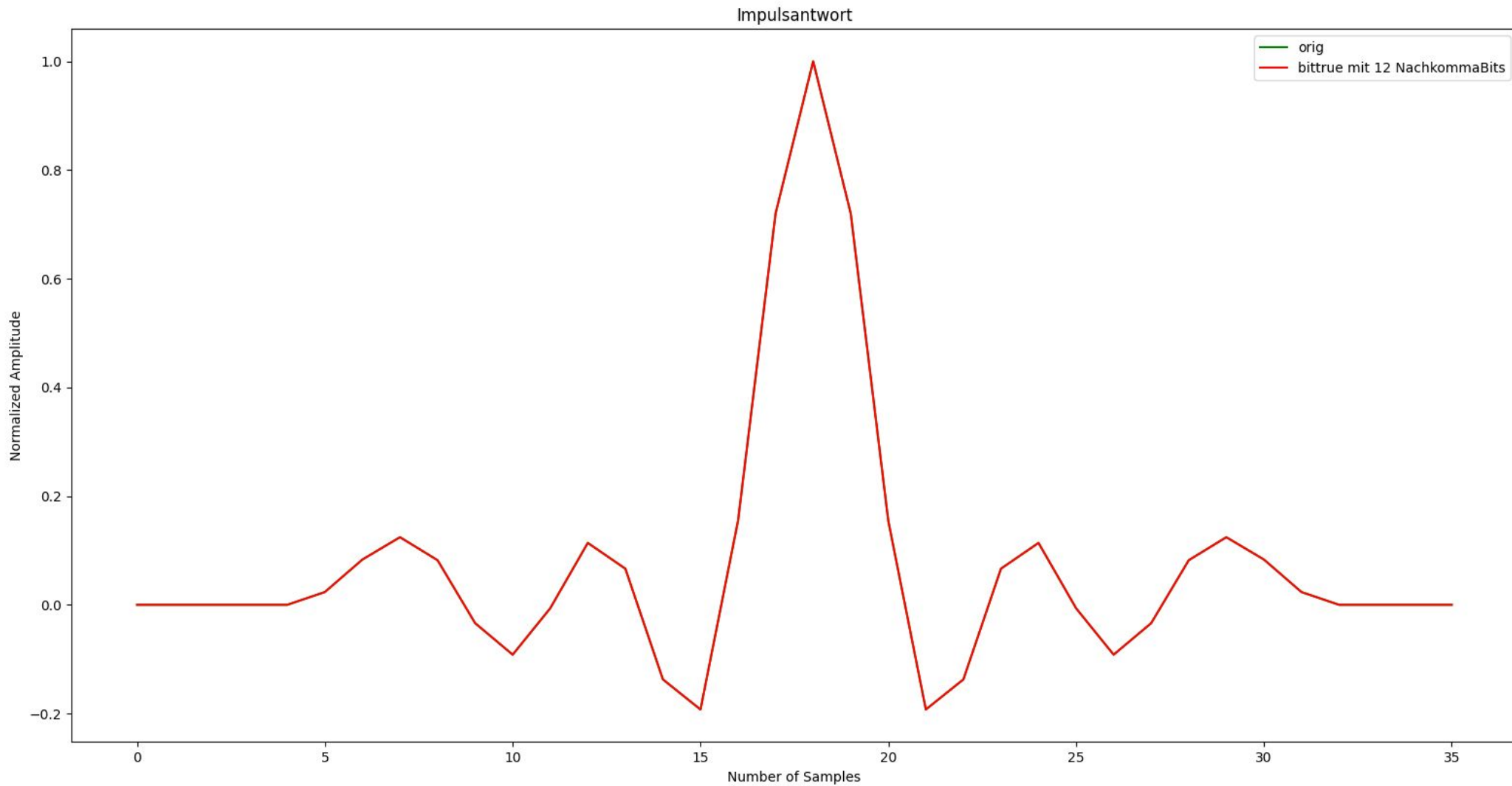
Filterdesign – Bittrue Modelling (8/12)

- Reduzierung der Fließsnachkommstellen



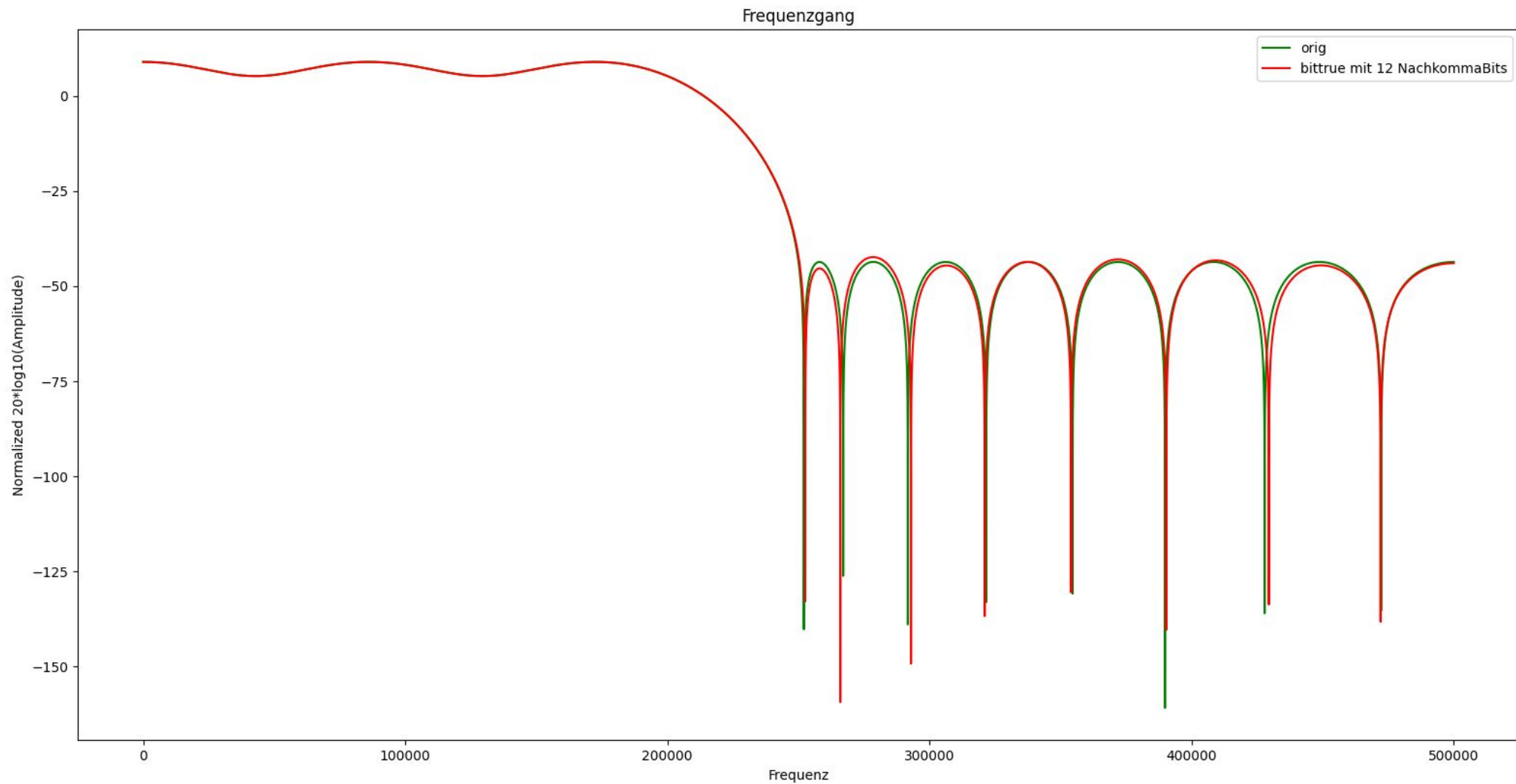
Filterdesign – Bittrue Modelling (9/12)

- Reduzierung der Fliessnachkommstellen



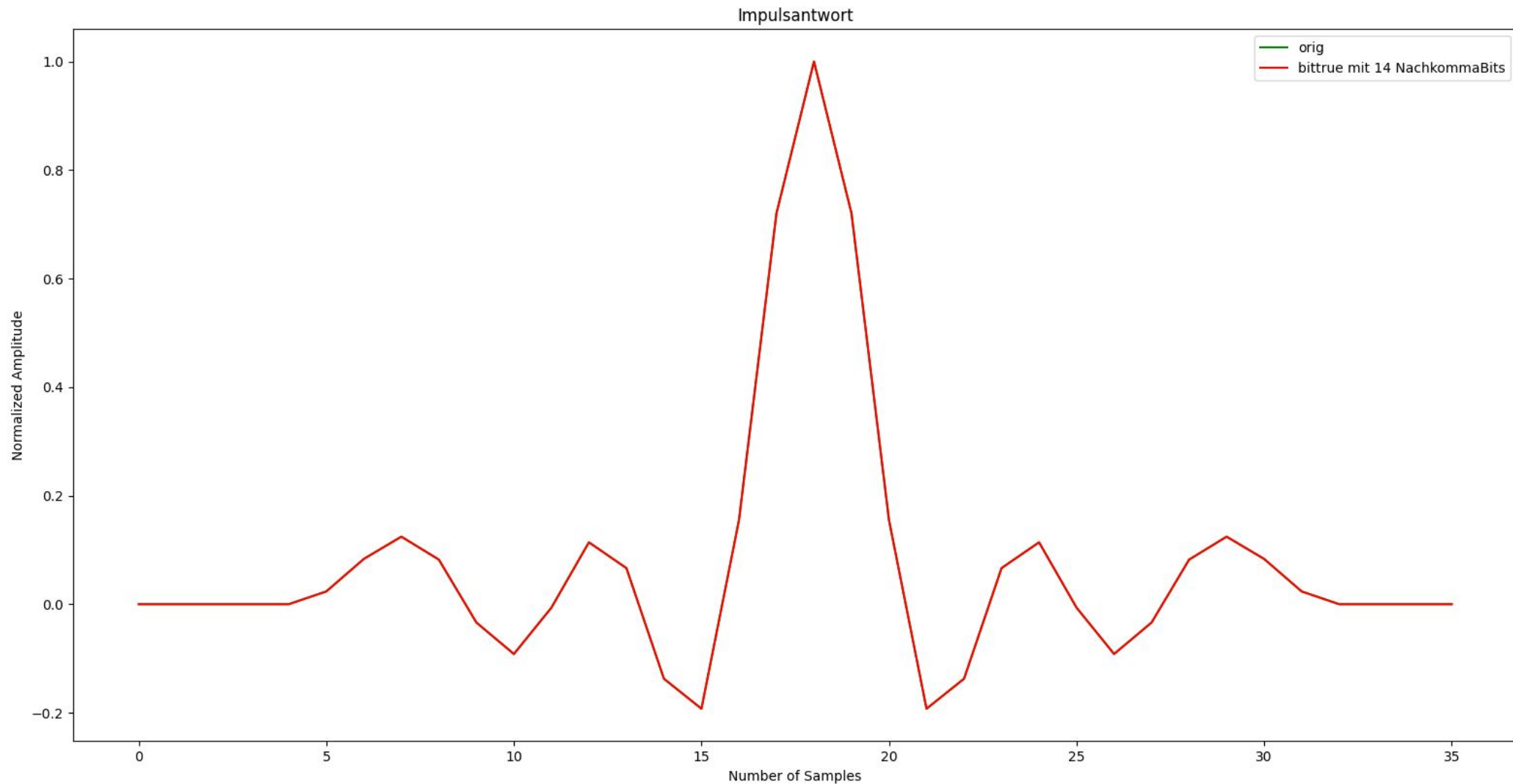
Filterdesign – Bittrue Modelling (10/12)

- Reduzierung der Fließsnachkommstellen



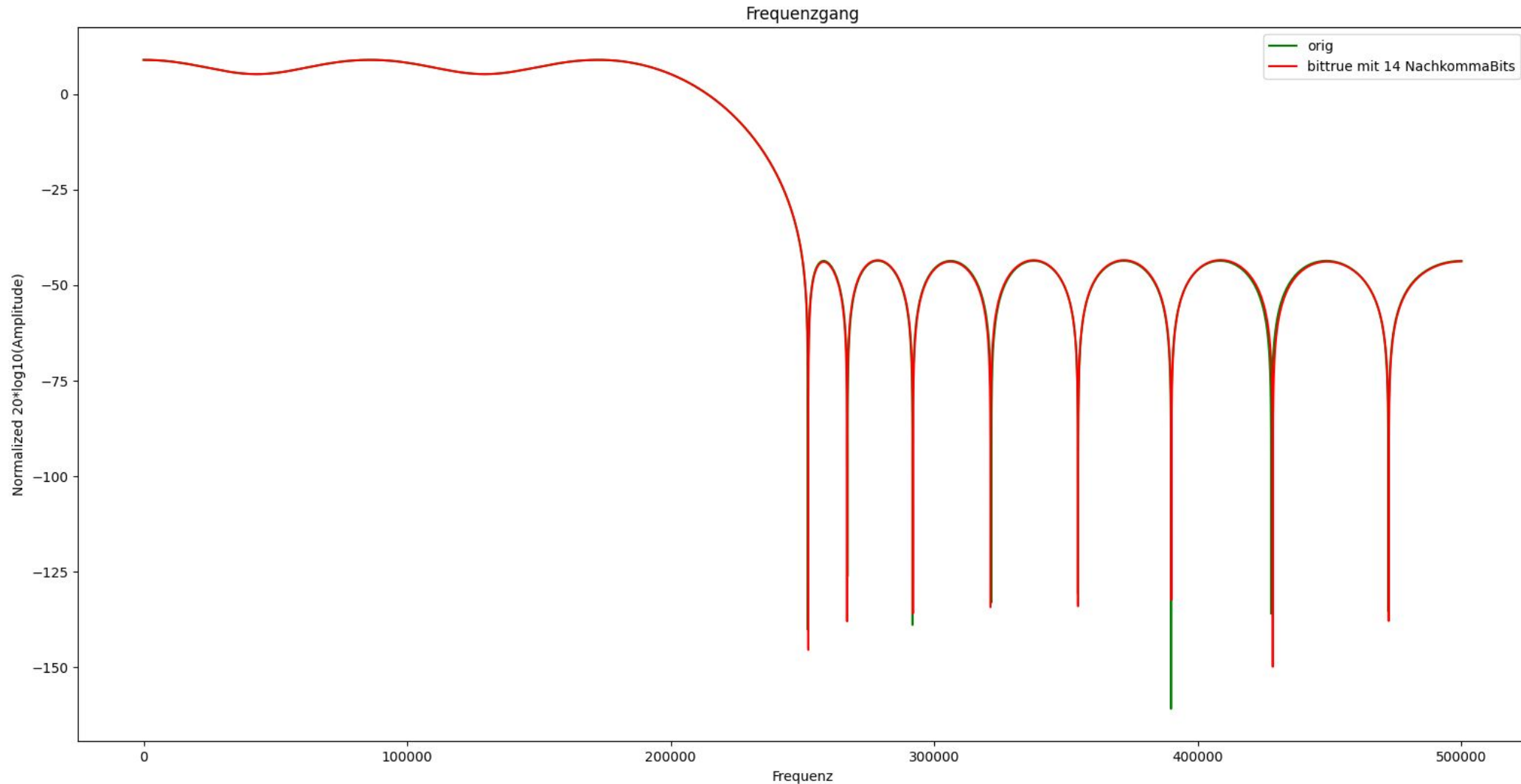
Filterdesign – Bittrue Modelling (11/12)

- Reduzierung der Fliessnachkommstellen



Filterdesign – Bittrue Modelling (12/12)

- Reduzierung der Fließsnachkommastellen



Kapitel 10

Künstliches Neuronales Netz

Kapitel 11

Stromeffizienter VHDL Code

Low Power Links

- <https://www.design-reuse.com/articles/20775/hdl-design-low-power.html>

Kapitel 12

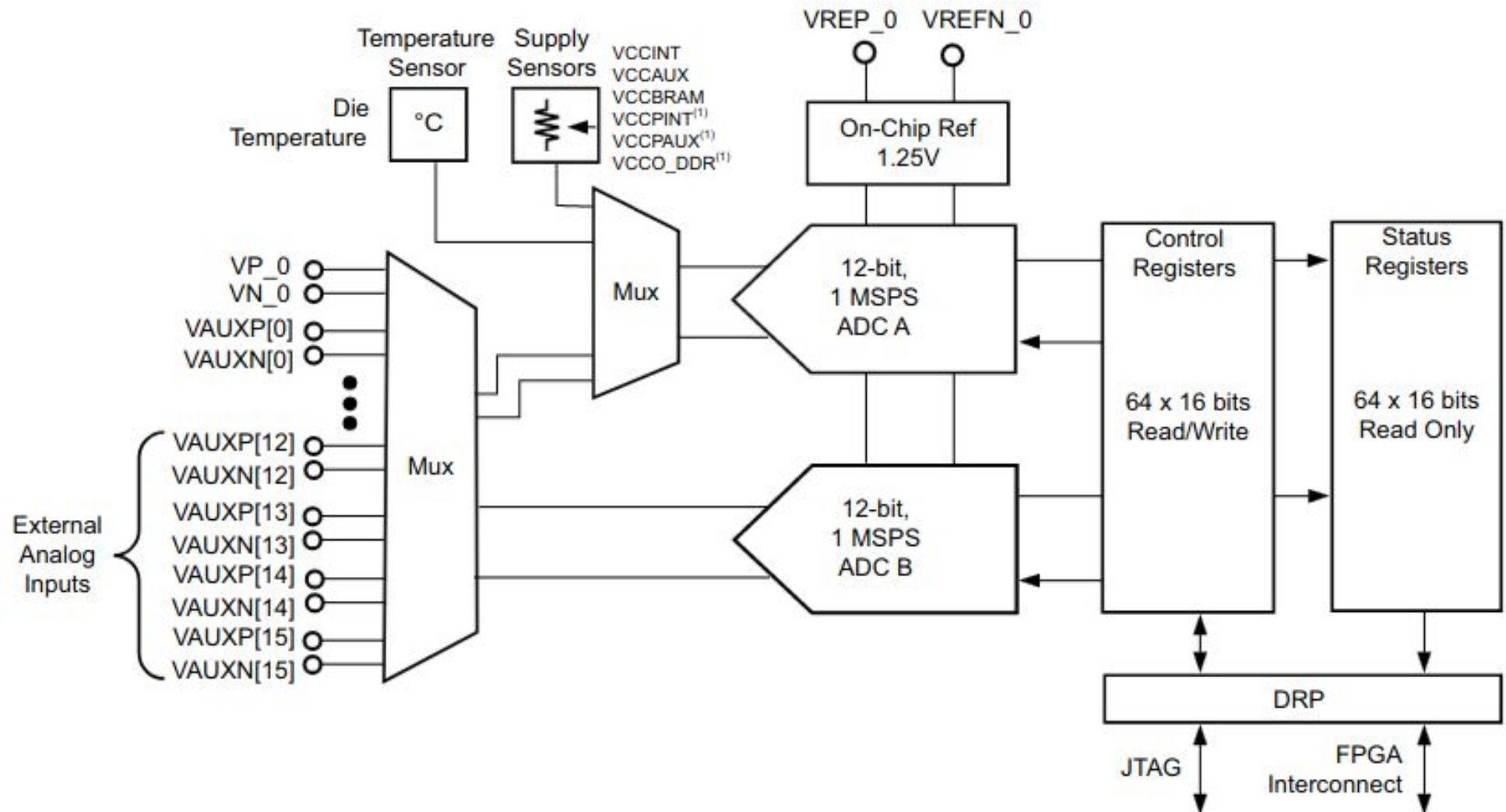
UART

Kapitel 13

ADC

XADC Blockdiagramm – Device Ebene

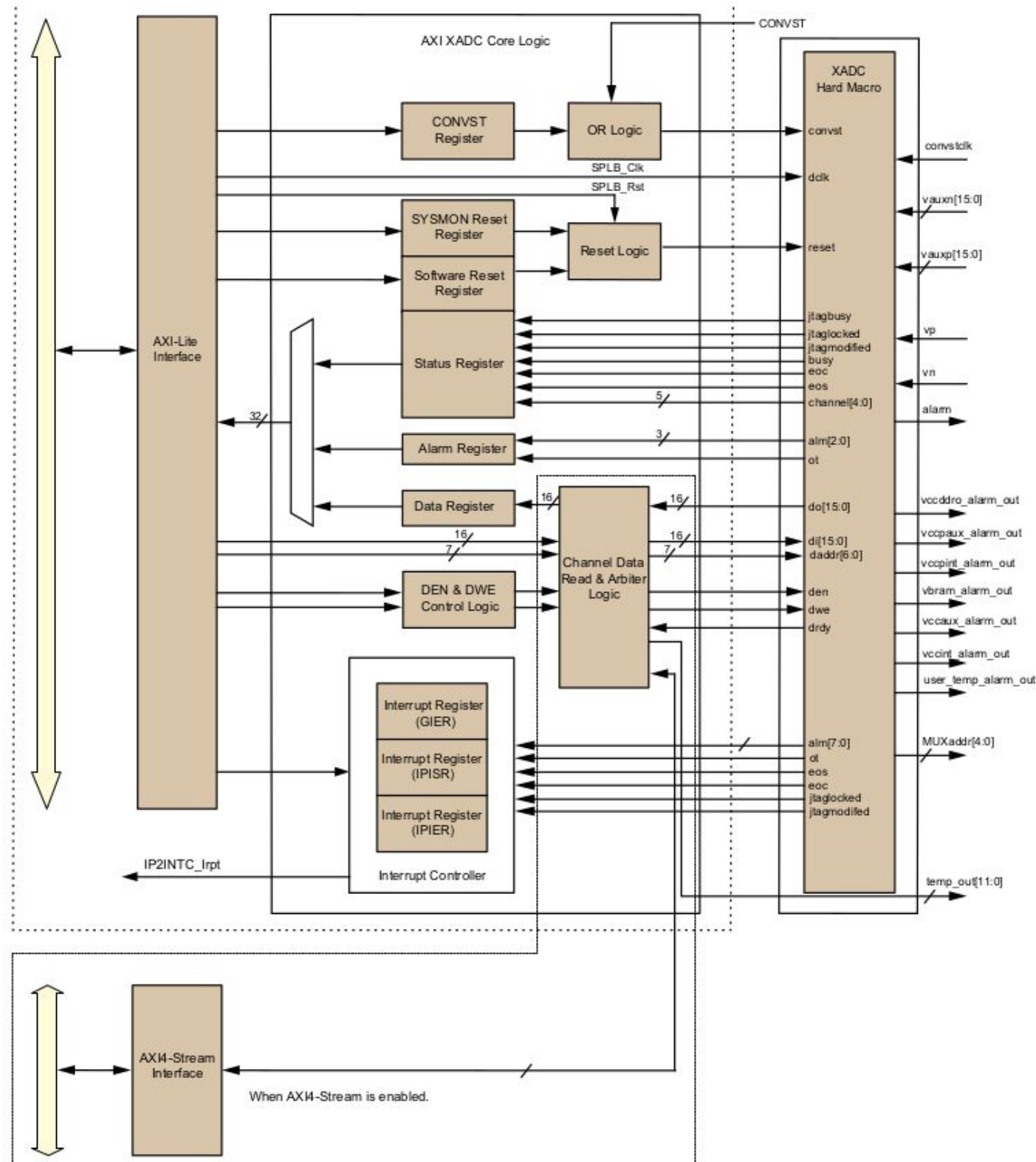
- Blockdiagramm des Analog zu Digital Konverter XADC, ohne FPGA Anbindung



XADC IP Core

Blockdiagramm

- FPGAs können auch analog Komponenten enthalten
- Analog zu Digital Konverter ADC
- Digital zu Analog Konverter DAC
- das Spartan S7 von AMD beinhaltet den XADC IP Core
- hier steht der IP Core mit seiner Anbindung an das FPGA im Mittelpunkt, nicht der XADC selber



XADC Features und IP Port Liste (1/4)

- ADC Konverter
 - 12Bit
 - 1MSPS
- FPGA Temperatur- und Spannungsmessung
- Alarm Mitteilungen einstellbar
- AXI4-Lite und AIX-Stream Interface von Advanced Microcontroller Bus Architecture (AMBA), eine freie Spezifikation von ARM https://en.wikipedia.org/wiki/Advanced_eXtensible_Interface
- Dynamic Reconfigurable Port (DRP)
- Auszug aus den IO Ports

Port	Direction	Description
DRP Ports		
di_in[15:0]	Input	Input data bus for the dynamic reconfiguration port (DRP).
do_out[15:0]	Output	Output data bus for the dynamic reconfiguration port.
daddr_in[6:0]	Input	Address bus for the dynamic reconfiguration port.
den_in	Input	Enable signal for the dynamic reconfiguration port.
dwe_in	Input	Write enable for the dynamic reconfiguration port.
dclk_in	Input	Clock input for the dynamic reconfiguration port.
drdy_out	Output	Data ready signal for the dynamic reconfiguration port.

XADC IO Ports - (2/4)

Port	Direction	Description
Control and Reset Ports		
reset_in	Input	Reset signal for the XADC control logic and maximum/minimum registers.
convst_in	Input	Convert start input. This input is used to control the sampling instant on the ADC input and is only used in Event Mode Timing (see Event-Driven Sampling in the <i>7 Series FPGAs XADC User Guide</i> (UG480) [Ref 3]).
convstclk_in	Input	Convert start input. This input is connected to a global clock input on the interconnect. Like CONVST, this input is used to control the sampling instant on the ADC inputs and is only used in Event Mode Timing.
External Analog Inputs		
vp_in vn_in	Input	One dedicated analog-input pair. The XADC has one pair of dedicated analog-input pins that provide a differential analog input.
vauxp15[15:0] vauxn15[15:0]	Inputs	16 auxiliary analog-input pairs. Also, the XADC uses 16 differential digital-input pairs as low-bandwidth differential analog inputs. These inputs are configured as analog during FPGA configuration.

XADC IO Ports - (2/4)

Alarm and Status Ports		
user_temp_alarm_out	Output	XADC temperature-sensor alarm output.
vccint_alarm_out	Output	XADC VCCINT-sensor alarm output.
vccaux_alarm_out	Output	XADC VCCAUX-sensor alarm output.
ot_out	Output	Over-Temperature alarm output.
channel_out[4:0]	Outputs	Channel selection outputs. The ADC input MUX channel selection for the current ADC conversion is placed on these outputs at the end of an ADC conversion.
eoc_out	Output	End of Conversion signal. This signal transitions to an active-High at the end of an ADC conversion when the measurement result is written to the Status registers. For detailed information, see the XADC Timing section in the <i>7 Series FPGAs XADC User Guide</i> (UG480) [Ref 3] .
eos_out	Output	End of Sequence. This signal transitions to an active-High when the measurement data from the last channel in the Channel Sequencer is written to the Status registers. For detailed information, see the XADC Timing section in the <i>7 Series FPGAs XADC User Guide</i> (UG480) [Ref 3] .
busy_out	Output	ADC busy signal. This signal transitions High during an ADC conversion. This signal transitions High for an extended period during calibration.
jtaglocked_out	Output	Used to indicate that drp port has been locked by the JTAG interface.
jtagmodified_out	Output	Used to indicate that a JTAG write to the drp has occurred
jtagbusy_out	Output	Used to indicate that a JTAG drp transaction is in progress
vbram_alarm_out	Output	XADC VBRAM sensor alarm output.
vccpint_alarm_out	Output	XADC VCCPINT sensor alarm output.

XADC IO Ports - (3/4)

Port	Direction	Description
vccpaux_alarm_out	Output	XADC VCCPAUX sensor alarm output.
vccddro_alarm_out	Output	XADC VCCDDRO sensor alarm output.
muxaddr_out[4:0]	Output	Use in external multiplexer mode to decode external MUX channel.
alarm_out	Output	Logic OR of alarms. Can be used to flag occurrence of any alarm.
temp_out[11:0]	Output	12-bit temperature output bus for the Memory Interface Generator (MIG). This should be connected to xadc_device_temp_i_pin of MIG.
AXI4-Lite Ports		
s_axi_aclk	Input	AXI clock connects to DCLK of XADC primitive.
s_axi_aresetn	Input	AXI Reset, Active-Low
s_axi_awaddr[10:0]	Input	AXI Write address. The write address bus gives the address of the write transaction.
s_axi_awvalid	Input	Write address valid. This signal indicates that a valid write address and control information are available.
s_axi_awready	Output	Write address ready. This signal indicates that the slave is ready to accept an address and associated control signals.
s_axi_wdata[31:0]	Input	Write data
s_axi_wstb[3:0]	Input	Write strobes. This signal indicates which byte lanes to update in memory.
s_axi_wvalid	Input	Write valid. This signal indicates that valid write data and strobes are available.
s_axi_wready	Output	Write ready. This signal indicates that the slave can accept the write data.
s_axi_bresp[1:0]	Output	Write response. This signal indicates the status of the write transaction 00 = OKAY (normal response) 10 = SLVERR (error condition) 11 = DECERR (not issued by core)
s_axi_bvalid	Output	Write response valid. This signal indicates that a valid write response is available.
s_axi_bready	Input	Response ready. This signal indicates that the master can accept the response information.
s_axi_araddr[10:0]	Input	Read address. The read address bus gives the address of a read transaction.
s_axi_arvalid	Input	Read address valid. This signal indicates, when High, that the read address and control information is valid and remains stable until the address acknowledgement signal, s_axi_arready, is High.
s_axi_arready	Output	Read address ready. This signal indicates that the slave is ready to accept an address and associated control signals.
s_axi_rdata[31:0]	Output	Read data

XADC IO Ports - (4/4)

Port	Direction	Description
s_axi_rresp[1:0]	Output	Read response. This signal indicates the status of the read transfer. 00 = OKAY (normal response) 10 = SLVERR (error condition) 11 = DECERR (not issued by core)
s_axi_rvalid	Output	Read valid. This signal indicates that the required read data is available and the read transfer can complete.
s_axi_rready	Input	Read ready. This signal indicates that the master can accept the read data and response information.
AXI4-Stream Ports		
m_axis_aclk	Input	The global clock signal. All signals until write interface of FIFO are sampled on the rising edge of m_axis_aclk.
s_axis_aclk	Input	The global clock signal. All streaming signals from Read interface of the FIFO are sampled on the rising edge of s_axis_aclk.
m_axis_resetsn	Input	The global reset signal. m_axis_resetsn is active-Low. Available when AXI4-Lite is not enabled
m_axis_tvalid	Output	Indicates that the master is driving a valid transfer. A transfer takes place when both TVALID and TREADY are asserted.
m_axis_tready	Input	Indicates that the slave can accept a transfer in the current cycle.
m_axis_tdata[15:0]	Output	The primary payload that is used to provide the data that is passing across the interface. The width of the data payload is two bytes.
m_axis_tid[4:0]	Output	The data stream identifier that indicates analog channel number corresponding to the different streams of data.
ip2intc_irpt	Output	Interrupt signal. When one of the selected interrupts mentioned in the Interrupt Enable Register occurs, this signal goes High.

Single Ended ADC vs. Differential Ended ADC

- Single Ended ADC sind mit ihrem GND und dem FPGA GND verbunden
 - das Eingangssignal kommt über eine einzige Signalleitung
 - Digitalwert ergibt sich zwischen 0 .. 2^n-1 als Integer unsigned Wert
 - 0V = 0 Integer
 - 3.3V = 4095 Integer bei einer $V_{REF} = 3.3V$

$$\text{Value}_{ADC} = U_{IN} / U_{REF} * \text{ADC}_{\text{Auflösung}}$$

$$2047 = 1.65V / 3.3V * 4095$$

Nur positive Eingangsspannungen

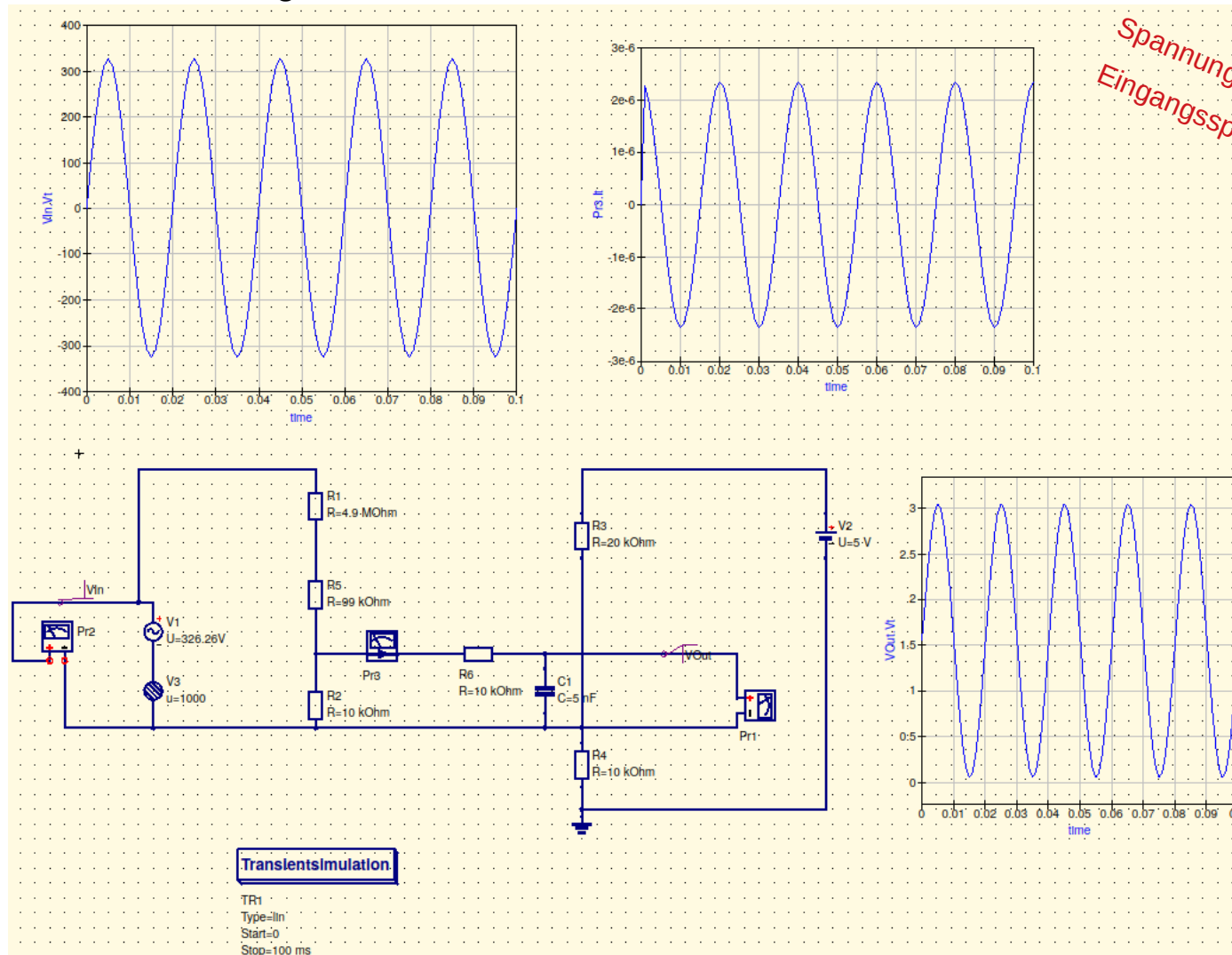
- Differential Ended ADC bedeutet nicht, dass jetzt negative Spannungen bezogen auf GND gemessen werden können, sondern, dass bei zwei Signaleingängen, die zusammen das Eingangssignal bilden, der negative Eingang größer ist als der positive Eingang
 - 1. Eingangssignal 2V, 2. Eingangssignal 2.5V → Differenz -0.5V
 - Beide Eingangssignale sind aber positiv

$$\text{Value}_{ADC} = (U_{IN1} - U_{IN2}) / U_{REF} * \text{ADC}_{\text{Auflösung}}$$

$$-310 = -0.5V / 3.3V * 2047$$

Messung eines negativen Signalanteils

- soll eine Signalspannung in dem Spannungsbereich von -1.5V - $+1.5\text{V}$ (z.B. Sinus-förmige Wechselfspannung) von einem Single Ended ADC aufgenommen werden, der in dem Bereich 0V - 3.3V arbeitet, muss das Wechselfspannungssignal über einen Spannungsteiler um 1.5V angehoben werden



Spannungsteiler aus R_3 und R_4 hebt die Eingangsspannung in den positiven Bereich an

Nachbau aus Sicherheitsgründen NICHT erlaubt!

Arty-S7 Board XADC Anbindung

- Der CORDIC-Algorithmus wurde 1959 von J. E. Volder entwickelt
-

Functionality

Applying a Voltage to the XADC Port

For this demo, A0-A5 are single-ended analog inputs while A6-A7, A8-A9, and V_P-V_N are differential analog input pairs.

Important



The Voltage range for differential analog input pairs is 0-1V and the range for single-ended analog inputs is 0-3.3V. For the pairs, A6, A8, and V_P are the positive inputs and A7, A9, and V_N are the negative inputs.

Quelle: https://digilent.com/reference/programmable-logic/arty-s7/demos/xadc?srsId=AfmBOoqTaMDVYfeb1rWmiy5ilB5rnT9VaymAHUmWcNp2qeKc_tIGX5Bc

Kapitel 14

Frequenzmessung eines analogen, periodischen Signals